

Operating Systems

Youjip Won

KAIST



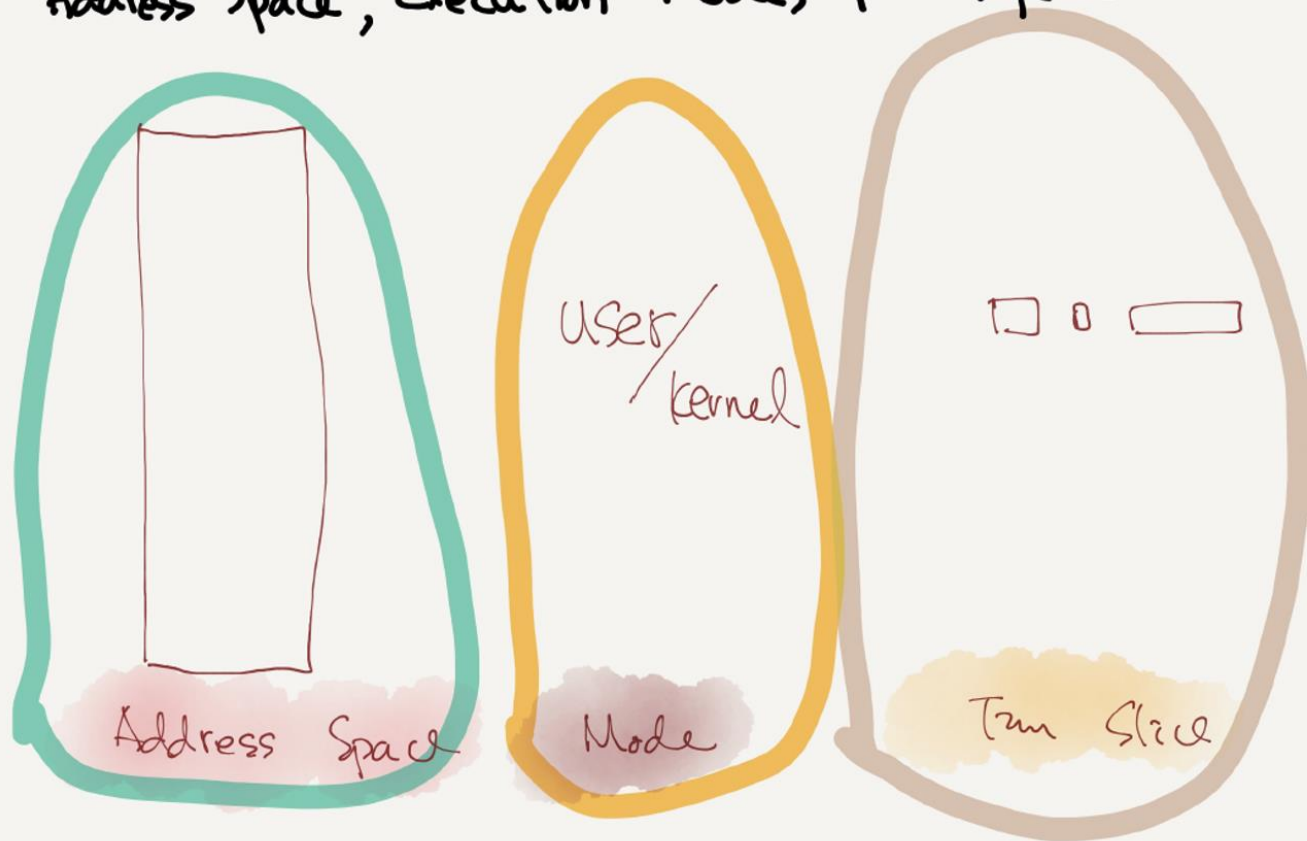
13. The Abstraction: Address Space

❓ What is **memory virtualization**?

- ◆ OS virtualizes its physical memory.
- ◆ OS provides an **illusion memory space** per each process.
- ◆ It seems to be seen like **each process uses the whole memory** .

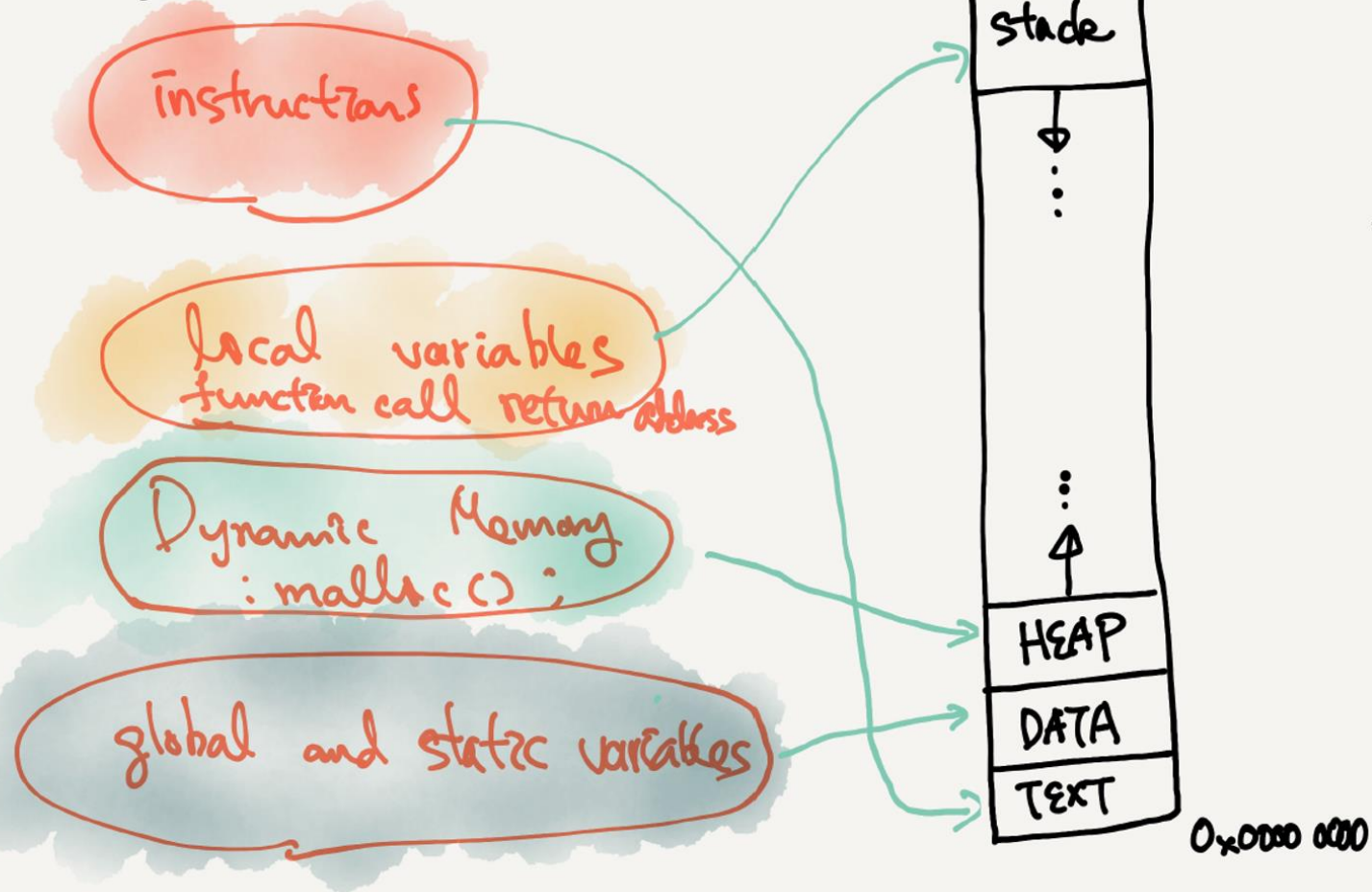
Process

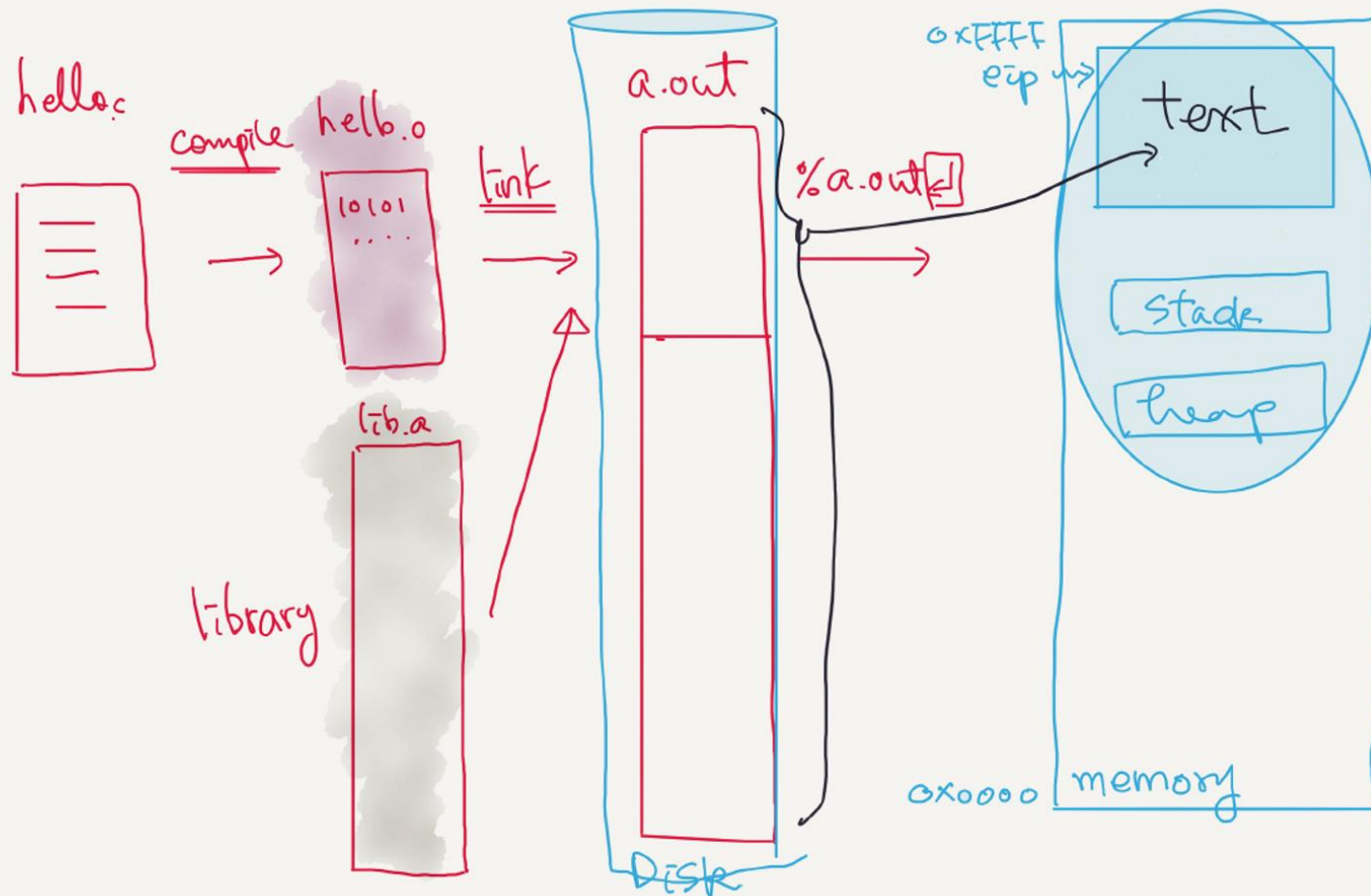
- Unit of Isolation
- Address Space, Execution Mode, CPU usage state



Address Space (User)

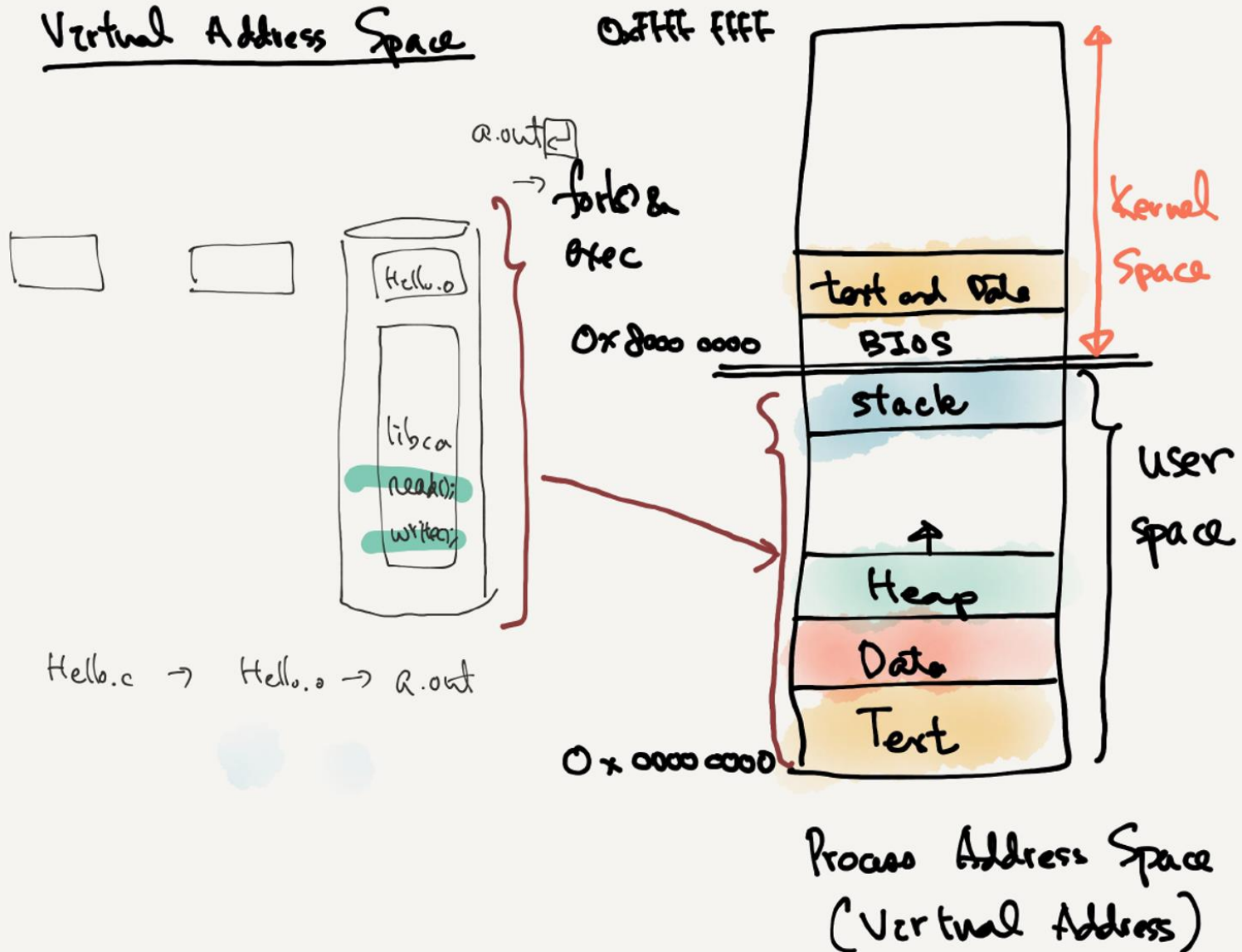
Objects in the address space





from lecture-1 but, where is kernel?

Virtual Address Space



Process structure: struct thread

pintos/src/threads/thread.h

```
struct thread
{
    /* Owned by thread.c. */
    tid_t tid;
    enum thread_status status;
    char name[16];
    uint8_t *stack;
    int priority;
    struct list_elem allelem;

    /* Shared between thread.c and synch.c. */
    struct list_elem elem;

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;
#endif

    /* Owned by thread.c. */
    unsigned magic;
};
```

```
enum thread_status
{
    THREAD_RUNNING,    /* Running thread. */
    THREAD_READY,      /* Not running but ready to run. */
    THREAD_BLOCKED,    /* Waiting for an event to trigger. */
    /*
    THREAD_DYING        /* About to be destroyed. */
};

/* Thread identifier. */
/* Thread state. */
/* Name (for debugging purposes). */
/* Saved stack pointer. */
/* Priority. */
/* List element for all threads list. */

/* List element. */

/* Page directory. */

/* Detects stack overflow. */
```

Process : struct thread

- page table

p->pagedir

- kernel stack

p->stack

- run state

p->status

process: unit of isolation

- page table

thread: unit of execution

- registers

- stacks

- local variables

- function call return address

User stack vs. kernel stack

Executing in User Mode

- user stack

executing in the kernel

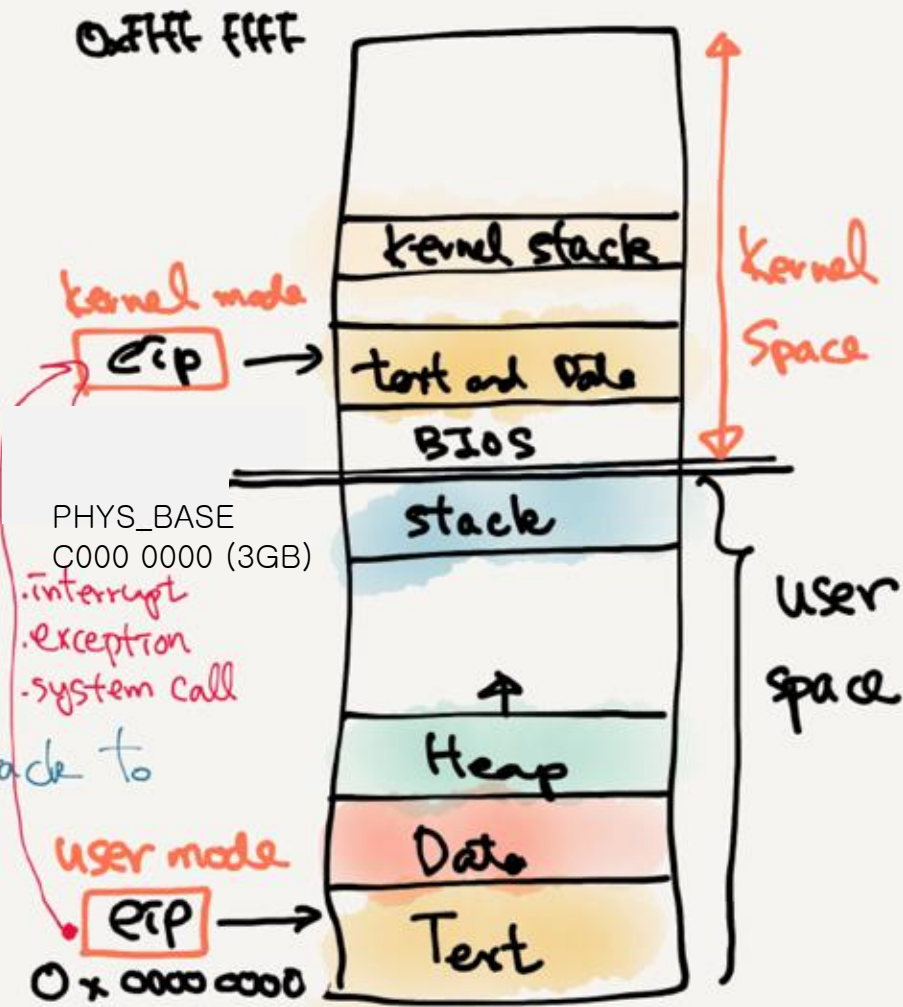
- use kernel stack

- System call

- entering the kernel

① switch from user stack to kernel stack

② raise privilege level

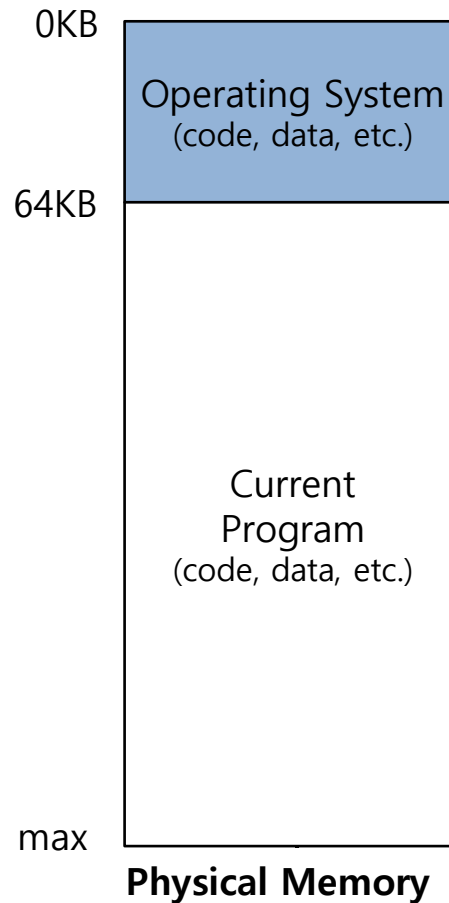


Benefit of Memory Virtualization

- ❑ Ease of use in programming
- ❑ Memory efficiency in terms of **times** and **space**
- ❑ The guarantee of isolation for processes as well as OS
 - ◆ Protection from **errant accesses** of other processes

OS in The Early System

- ❑ Load only one process in memory.
 - ◆ Poor utilization and efficiency



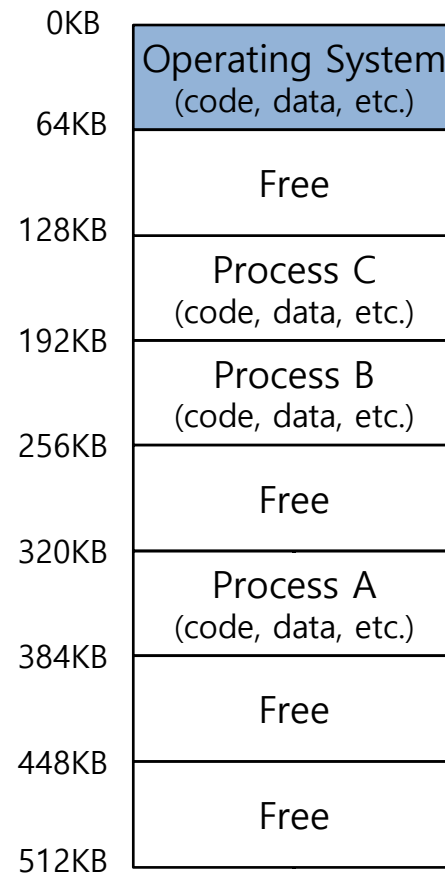
Multiprogramming and Time Sharing

❓ **Load multiple processes** in memory.

- ◆ Execute one for a short while.
- ◆ Switch processes between them in memory.
- ◆ Increase utilization and efficiency.

❓ Cause an important **protection issue**.

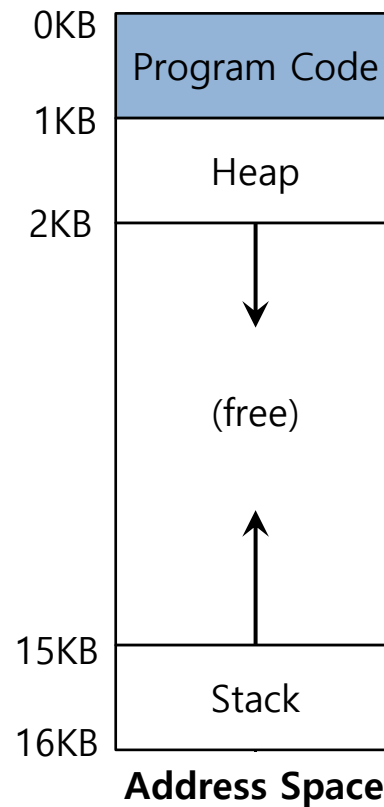
- ◆ Errant memory accesses from other processes



Physical Memory

Address Space

- OS creates an **abstraction** of physical memory.
 - ◆ The address space contains all about a running process.
 - ◆ That is consist of program code, heap, stack and etc.



Address Space(Cont.)

? Code

- ◆ Where instructions live

? Heap

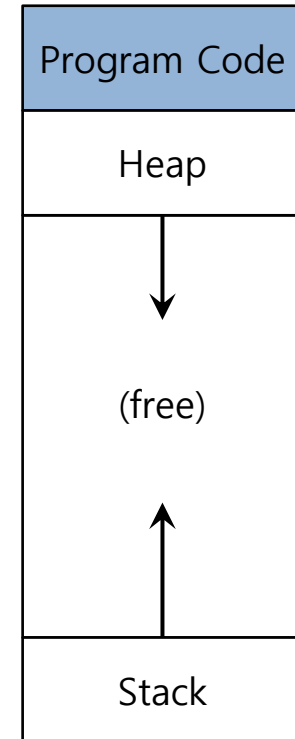
- ◆ Dynamically allocate memory.

- ? `malloc` in C language

- ? `new` in object-oriented language

? Stack

- ◆ Store return addresses or values.
- ◆ Contain local variables arguments to routines.



Address Space

Virtual Address

- ❓ **Every address** in a running program is virtual.
 - ◆ OS translates the virtual address to physical address

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]){

    printf("location of code   : %p\n", (void *) main);
    printf("location of heap   : %p\n", (void *) malloc(1));
    int x = 3;
    printf("location of stack  : %p\n", (void *) &x);

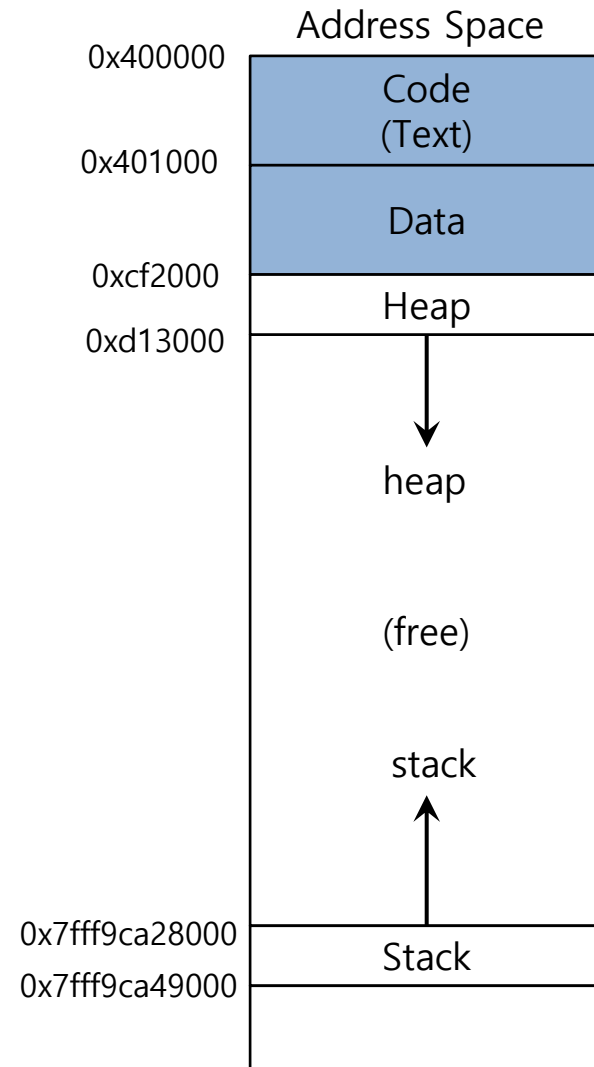
    return x;
}
```

A simple program that prints out addresses

Virtual Address(Cont.)

? The output in 64-bit Linux machine

```
location of code   : 0x40057d
location of heap   : 0xcf2010
location of stack  : 0x7fff9ca45fcc
```



Components of Virtual Address Space

```
#include <stdio.h>
#include <stdlib.h>

int InitializedGlobal[1024] = {0,};
int UnintGlobal[1024];

int main() {
    int localVar1;
    int localVar2;
    int *dynamicLocalVar1;
    int *dynamicLocalVar2;

    dynamicLocalVar1 = malloc(sizeof(int));
    dynamicLocalVar2 = malloc(sizeof(int));

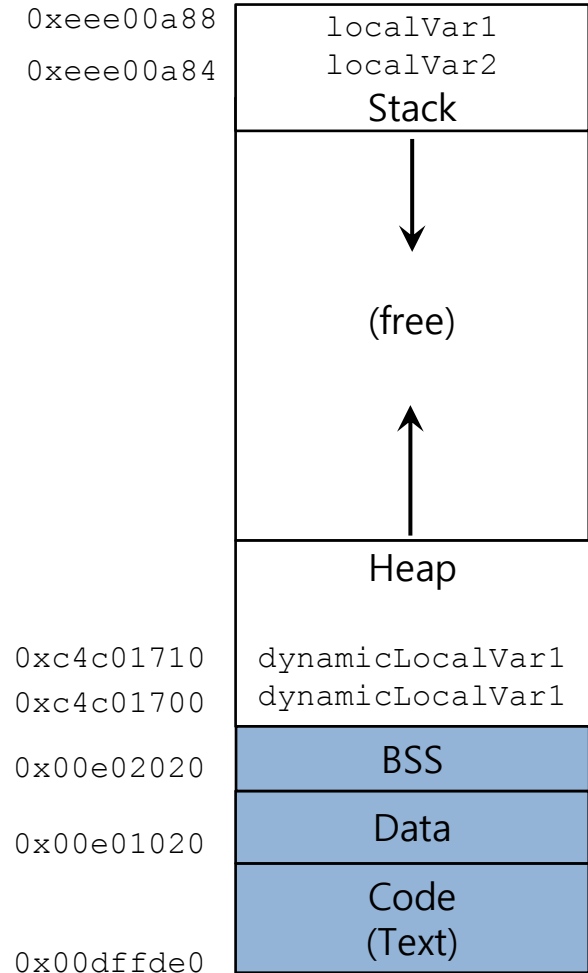
    printf("code           : 0x%x\n", main);
    printf("Data           : 0x%x\n", &InitializedGlobal);
    printf("BSS (Uninit Data)    : 0x%x\n", &UnintGlobal);

    printf("stack localVar1      : 0x%x\n", &localVar1);
    printf("stack localVar2      : 0x%x\n", &localVar2);
    printf("heap dynamicLocalVar1: 0x%x\n", dynamicLocalVar1);
    printf("heap dynamicLocalVar2: 0x%x\n", dynamicLocalVar2);
    return 0;
}
```

Components of Virtual Address Space

```
BulGok:~ yjwon$ ./a.out
code           : 0xdffde0
Data           : 0xe01020
BSS (Uninit Data) : 0xe02020
stack localVar1  : 0xee00a88
stack localVar2  : 0xee00a84
heap dynamicLocalVar1: 0xc4c01700
heap dynamicLocalVar2: 0xc4c01710
```

Minimum heap allocation unit: 16 Byte



Operating Systems

Youjip Won

KAIST

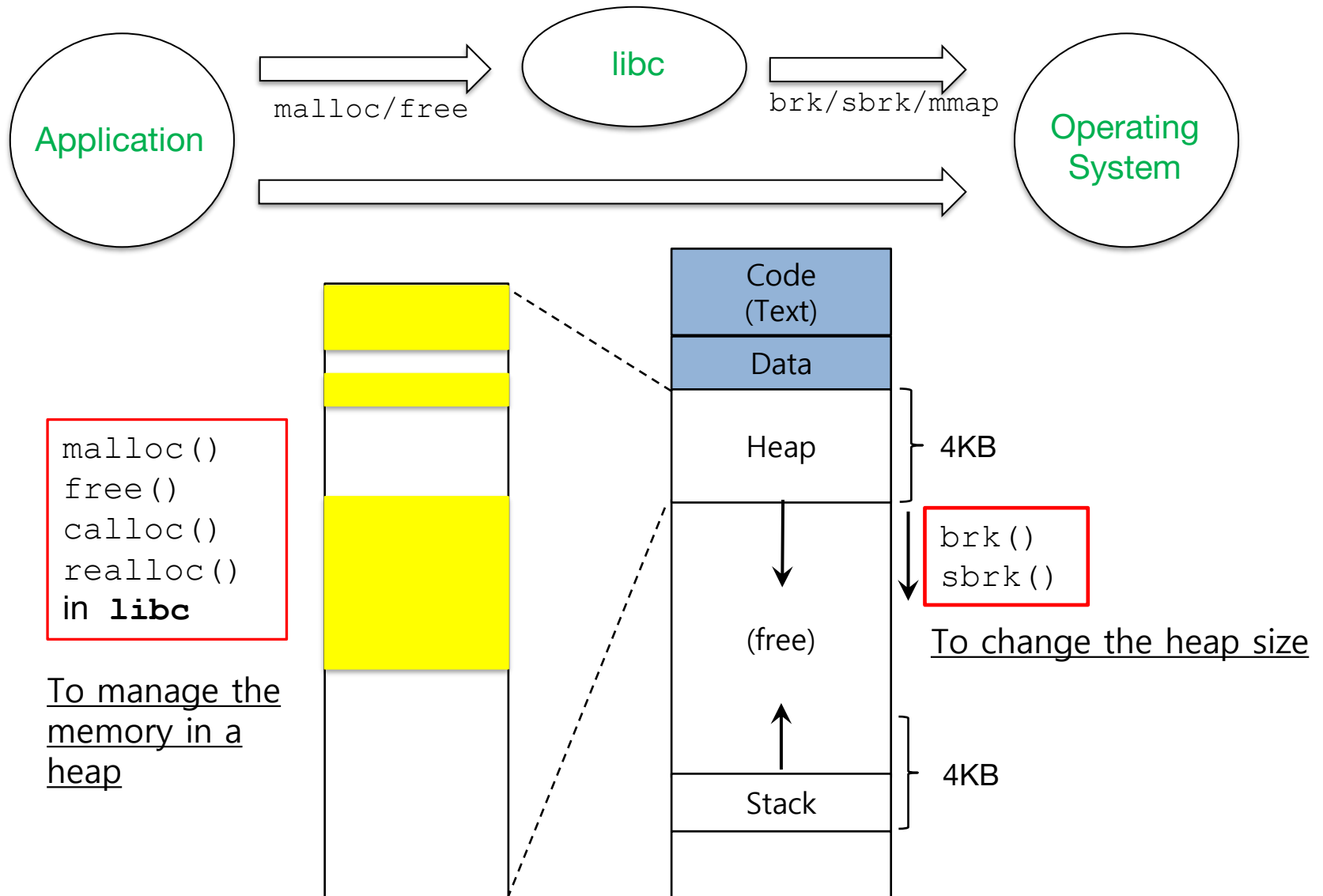


14. Memory API

Overview

- ❑ malloc/free
- ❑ calloc/realloc
- ❑ brk/sbrk
- ❑ mmap/munmap

Virtual Address Space



malloc()

```
#include <stdlib.h>

void* malloc(size_t size)
```

? Allocate a memory region on the heap.

◆ Argument

- ? `size_t size` : size of the memory block(in bytes)
- ? `size_t` is an unsigned integer type.

◆ Return

- ? Success : a void type pointer to the memory block allocated by `malloc`
- ? Fail : a null pointer

sizeof()

- ❓ Routines and macros are utilized for `size` in `malloc` instead typing in a number directly.
- ❓ Two types of results of `sizeof` with variables
 - ◆ The actual size of `'x'` is known at run-time.

```
int *x = malloc(10 * sizeof(int));  
printf("%d\n", sizeof(x));
```

4

- ◆ The actual size of `'x'` is known at compile-time.

```
int x[10];  
printf("%d\n", sizeof(x));
```

40

Memory API: `free()`

```
#include <stdlib.h>

void free(void* ptr)
```

? Free a memory region allocated by a call to `malloc`.

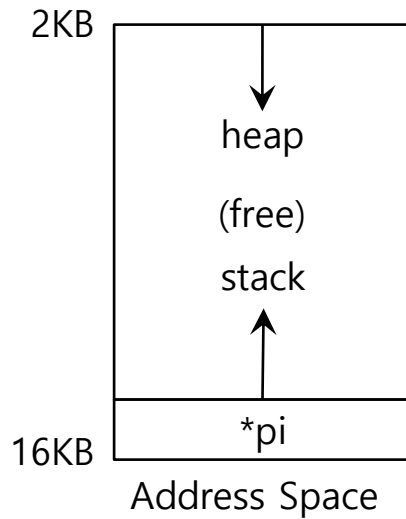
- ◆ Argument

- ? `void *ptr`: a pointer to a memory block allocated with `malloc`

- ◆ Return

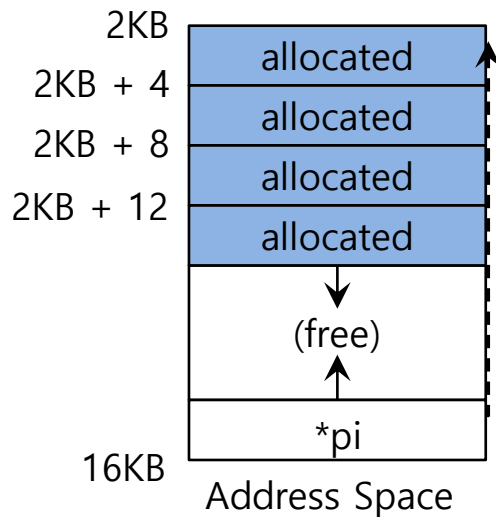
- ? none

Memory Allocating



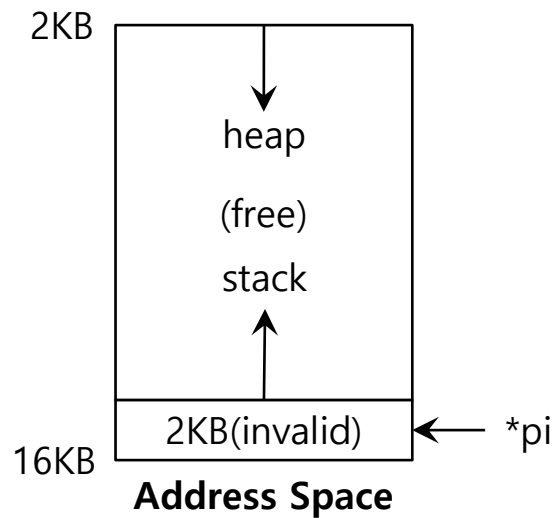
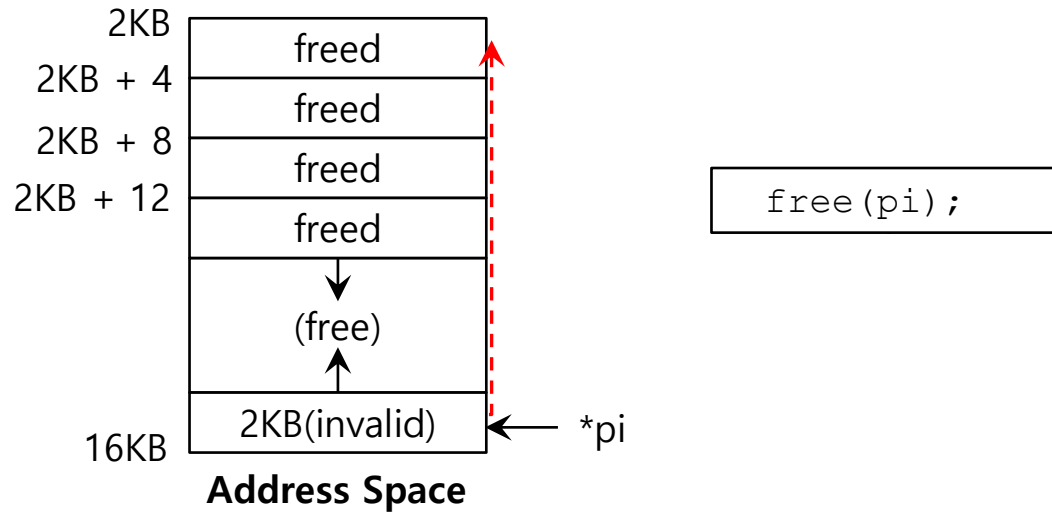
-----> pointer

```
int *pi; // local variable
```



```
pi = (int *)malloc(sizeof(int)*  
4);
```

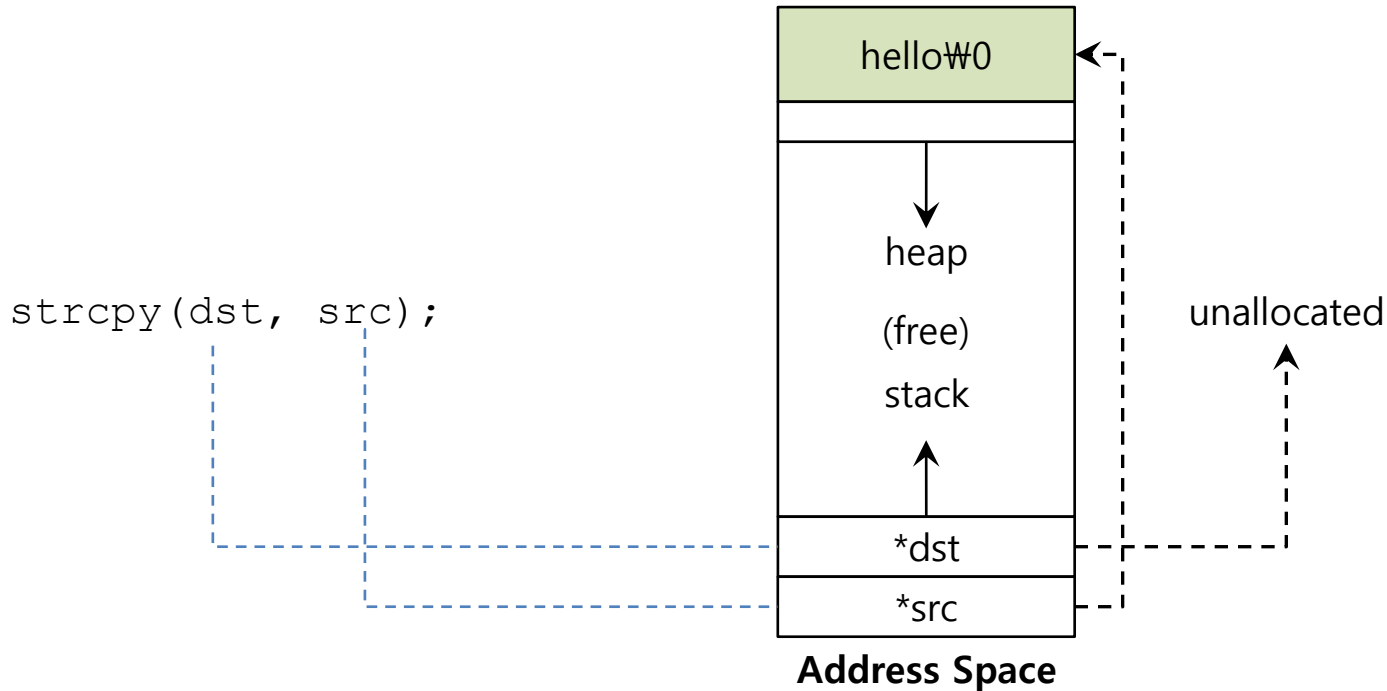
Memory Freeing



Forgetting To Allocate Memory

❓ Incorrect code

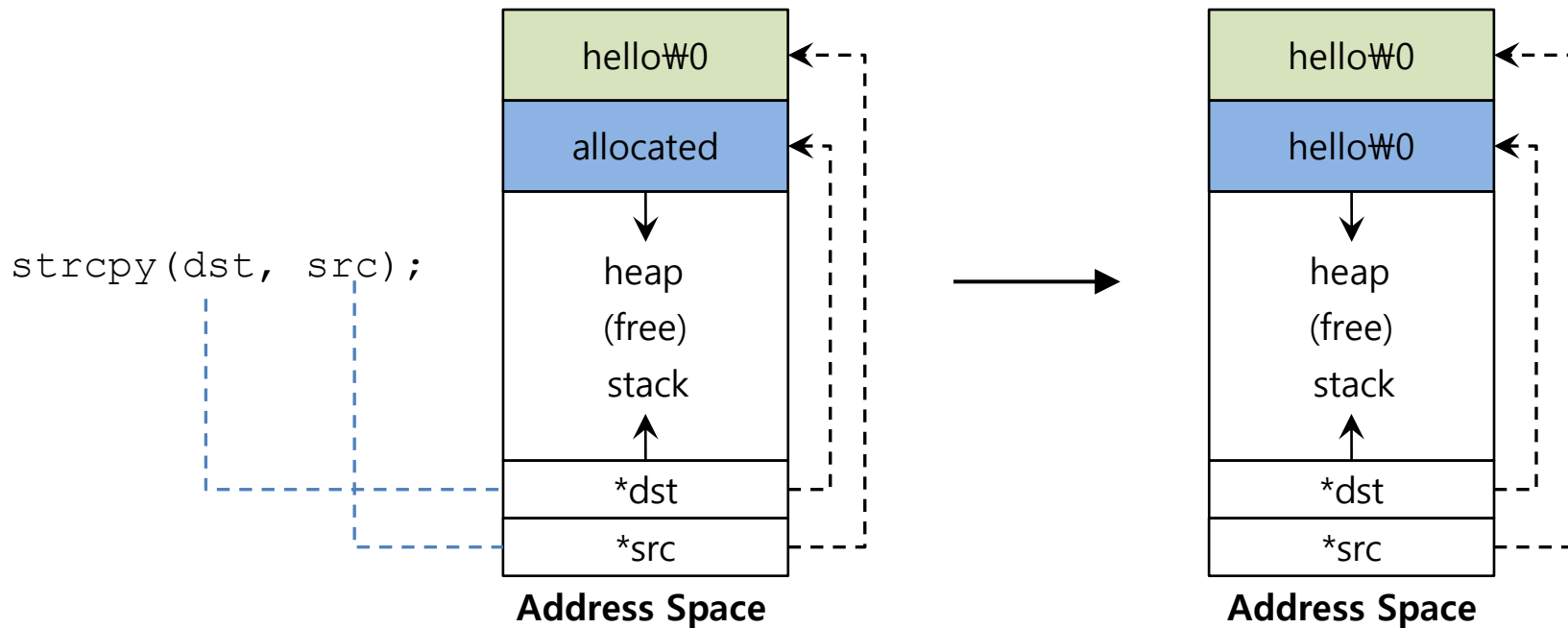
```
char *src = "hello"; //character string constant
char *dst;           //unallocated
strcpy(dst, src);    //segfault and die
```



Forgetting To Allocate Memory(Cont.)

❓ Correct code

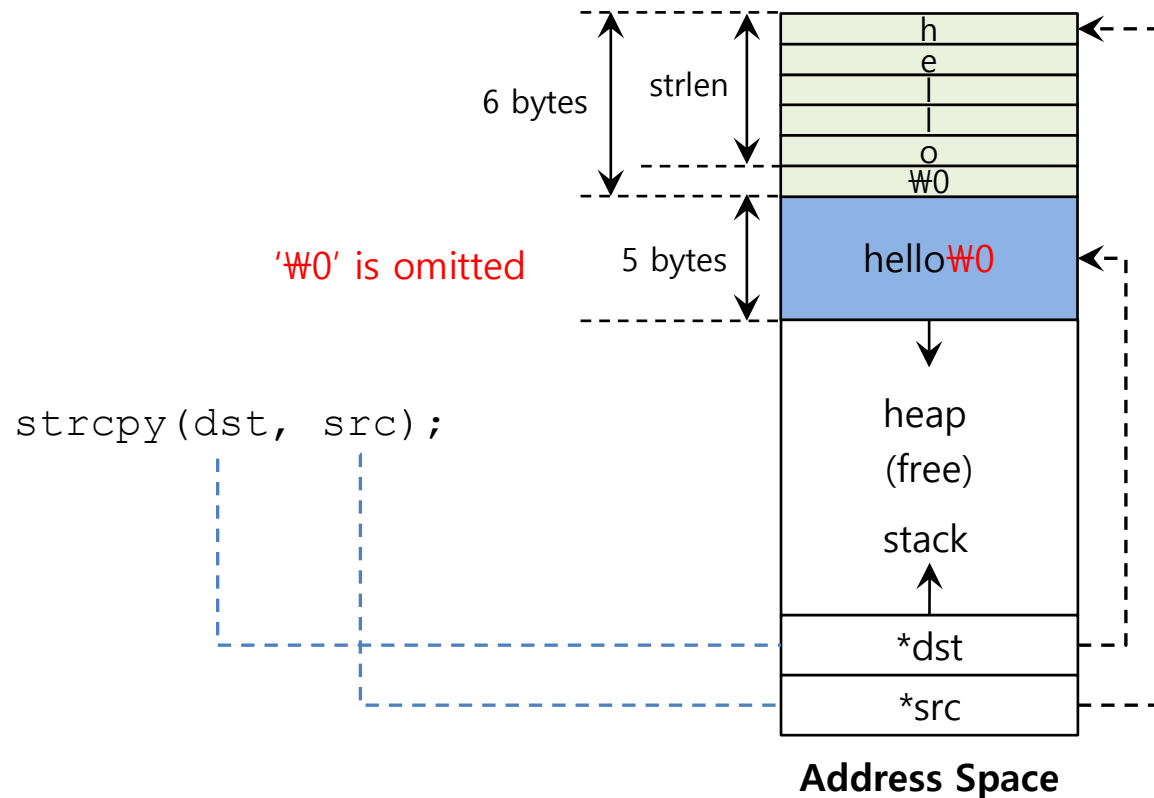
```
char *src = "hello";    //character string constant
char *dst (char *)malloc(strlen(src) + 1 ); // allocated
strcpy(dst, src);        //work properly
```



Not Allocating Enough Memory

Incorrect code, but work properly

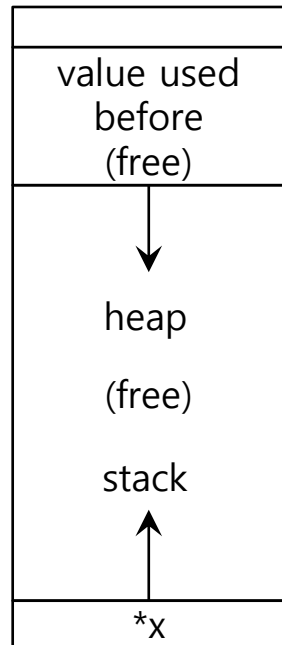
```
char *src = "hello"; //character string constant
char *dst (char *)malloc(strlen(src)); // too small
strcpy(dst, src);      //work properly
```



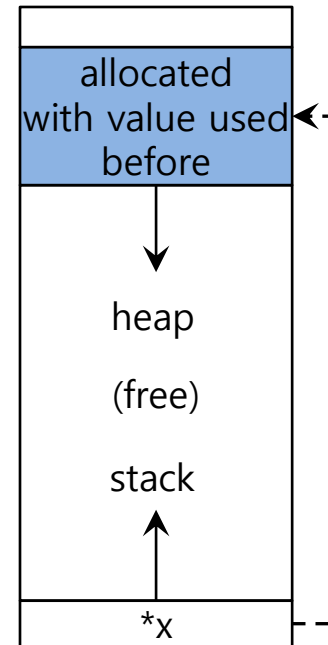
Forgetting to Initialize

❓ Encounter an uninitialized read

```
int *x = (int *)malloc(sizeof(int)); // allocated
printf("*x = %d\n", *x); // uninitialized memory access
```



Address Space



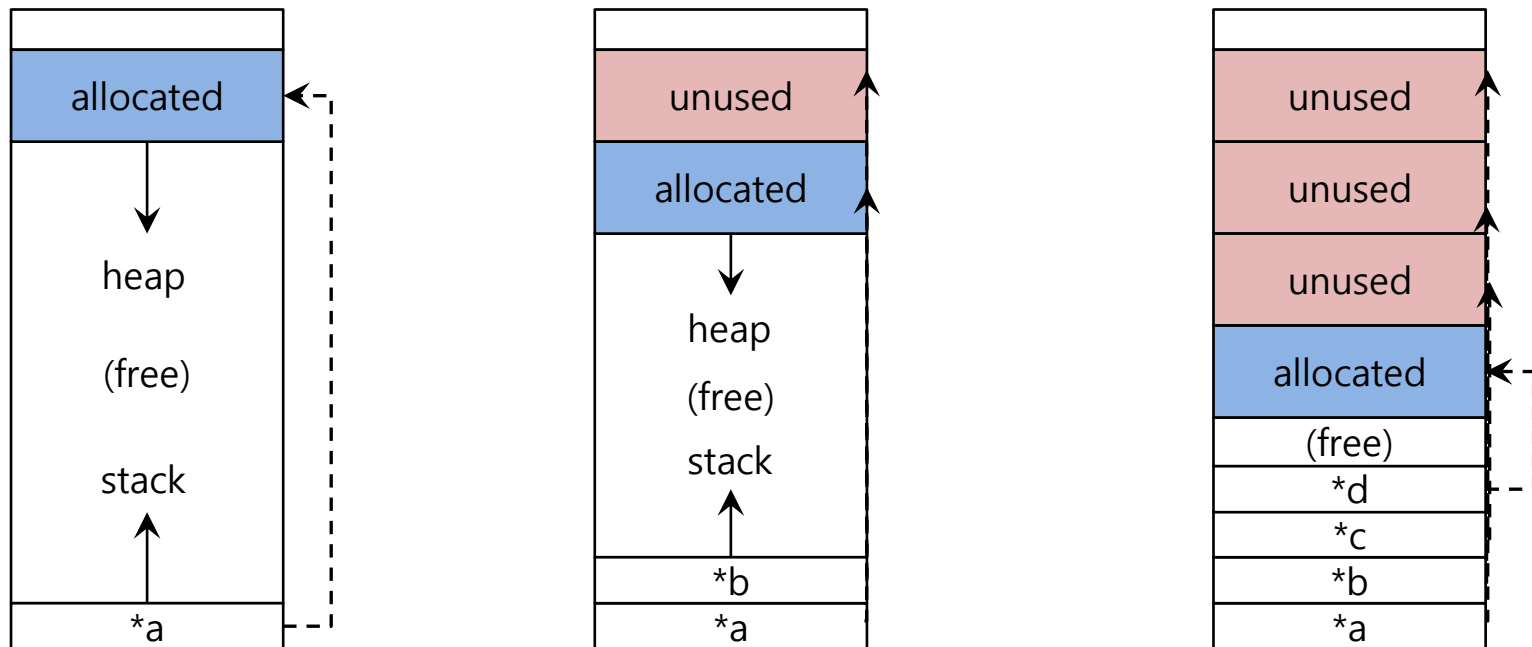
Address Space

Memory Leak

- ❓ A program keeps allocating memory without freeing it.
- ❓ A program runs out of memory and eventually is killed by OS.

```
while(1)
    malloc(4) ;
```

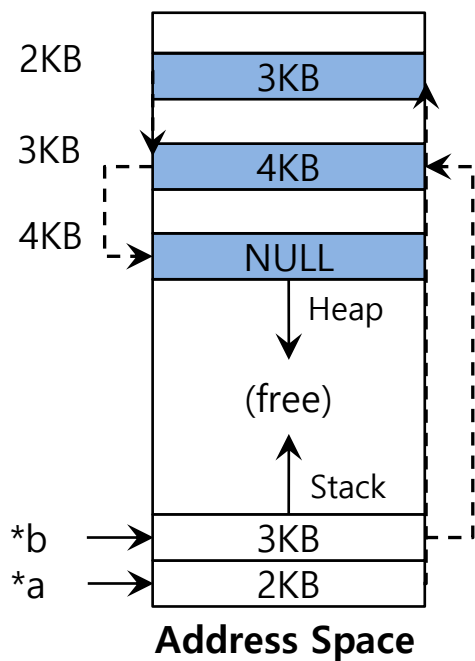
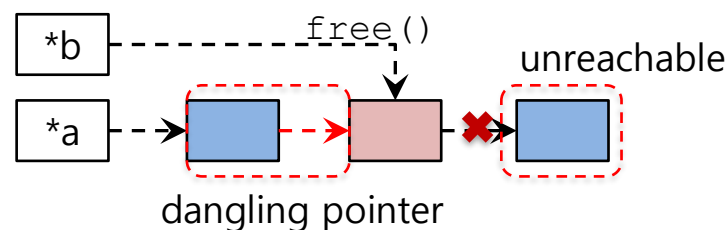
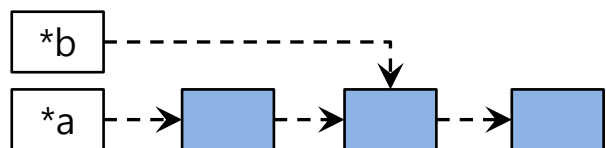
unused : allocated, but not freed



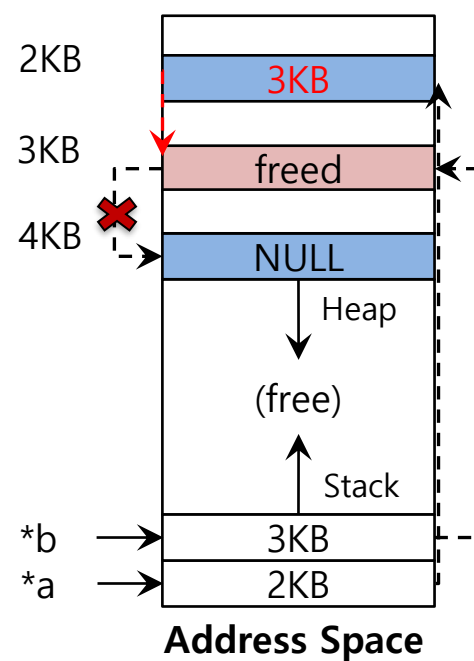
run out of memory

Dangling Pointer

- Freeing memory while it is being used.
 - A program accesses to memory with an invalid pointer



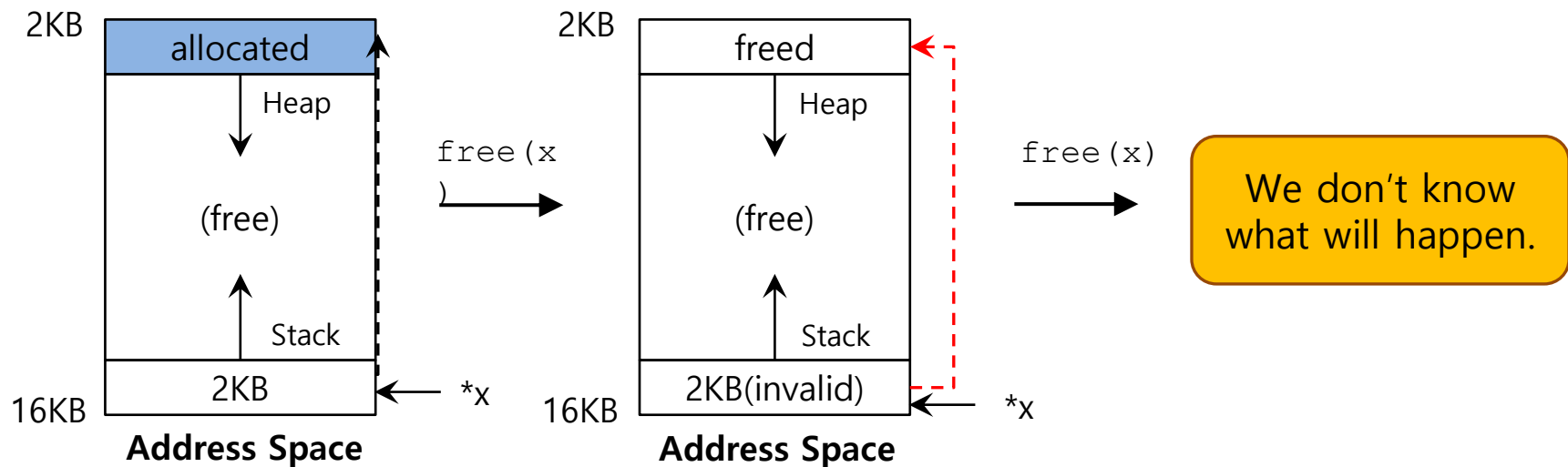
`free(b)`



Incorrect `free()`

- Free the memory that was freed already.

```
int *x = (int *)malloc(sizeof(int)); // allocated
free(x); // free memory
free(x); // free repeatedly
```



- Free the memory that was not allocated via `malloc()`.

```
int *x = (int *)malloc(sizeof(int)); // allocated
free(x+12); // free memory
```

Other Memory APIs: `calloc()` and `realloc()`

```
#include <stdlib.h>

void *calloc(size_t num, size_t size)
```

❓ Allocate memory and zeroes it before returning.

- ◆ `size_t num` : the number of objects to allocate
- ◆ `size_t size` : size of an object (in bytes)

```
#include <stdlib.h>

void *realloc(void *ptr, size_t size)
```

❓ Change the size of memory block.

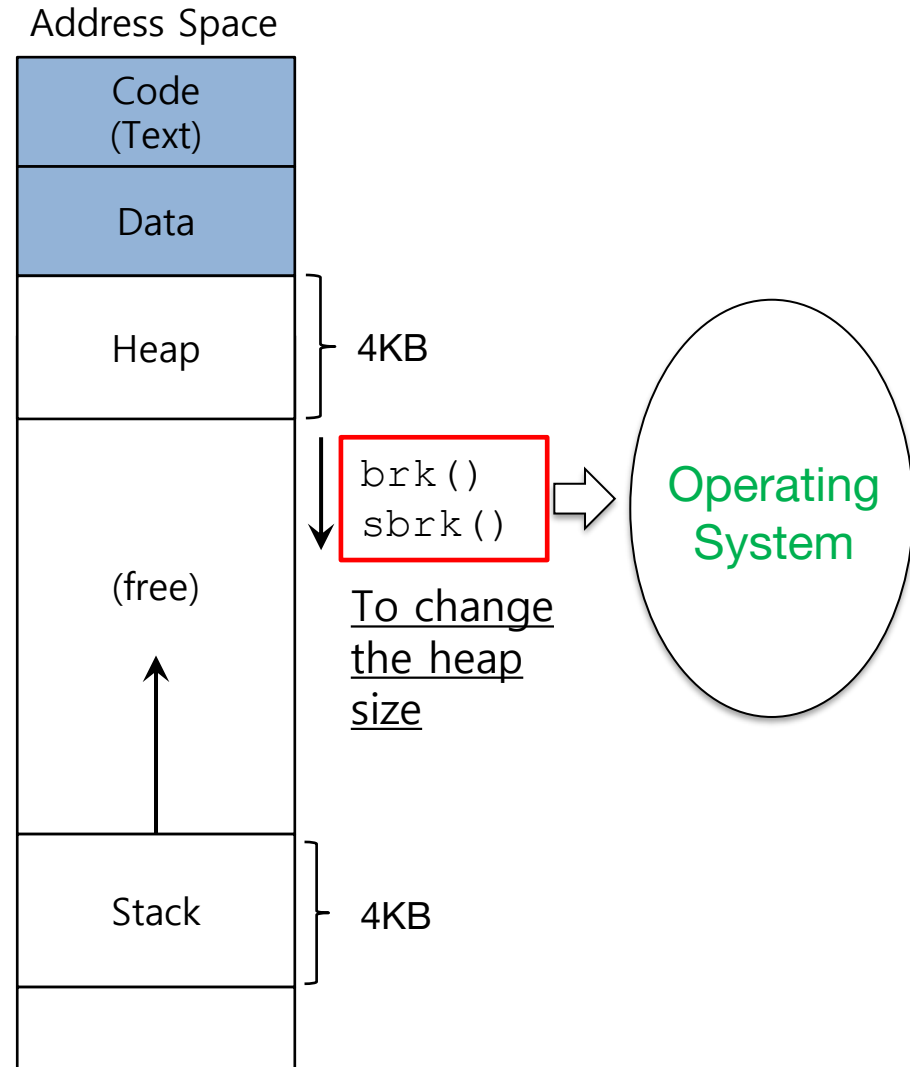
- ◆ `void *ptr`: Pointer to memory block allocated with `malloc`, `calloc` or `realloc`
- ◆ `size_t size`: New size for the memory block(in bytes)

System Calls

```
#include <unistd.h>

int brk(void *addr)
void *sbrk(intptr_t increment);
```

- There lacks of heap space. Ask OS to expand heap.
- break: The location of **the end of the heap** in address space
- malloc uses **brk** system call.
 - brk is called to expand the program's *break*.
 - sbrk is similar to brk.
 - Programmers **should never directly call** either brk or sbrk.



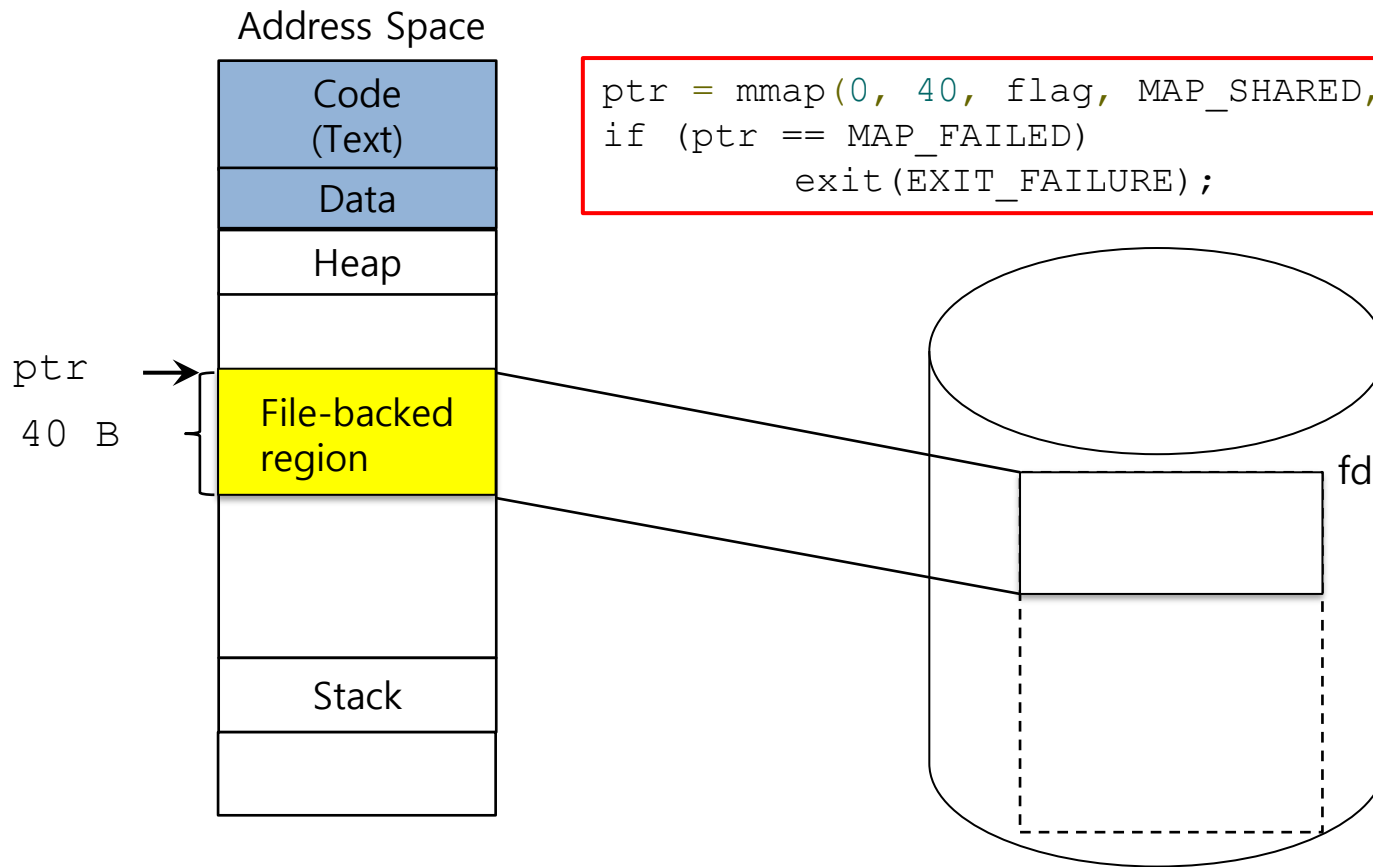
System Calls: mmap

```
#include <sys/mman.h>

void *mmap(void *ptr, size_t length, int prot, int flags,
           int fd, off_t offset)
```

- ❑ Allocate a memory region of length at ptr.
- ❑ If fd is not negative, associate the region to fd starting at offset.

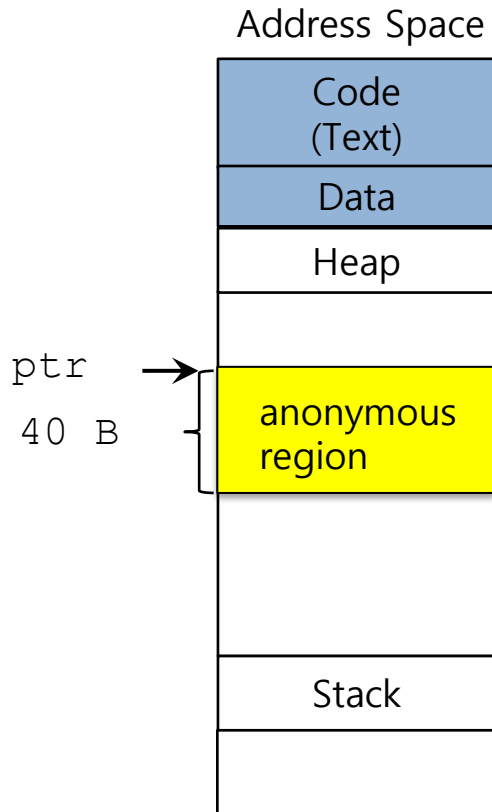
mmap: creating file-backed region



mmap: creating anonymous region

```
#include <sys/mman.h>
```

```
void *mmap(void *ptr, size_t length, int  
prot, int flags, int fd, off_t offset)
```



```
ptr= mmap(NULL, 40, PROT_READ |  
          PROT_WRITE, MAP_SHARED |  
          MAP_ANONYMOUS, -1, 0);  
if (ptr == MAP_FAILED)  
    exit(EXIT_FAILURE);
```

Summary

- ❑ malloc/free
- ❑ calloc/realloc
- ❑ mmap/munmap

Operating Systems



Youjip Won

15. Address Translation

- ▣ Memory virtualizing takes a similar strategy known as **limited direct execution(LDE)** for efficiency and control.
- ▣ In memory virtualizing, efficiency and control are attained by **hardware support**.
 - ◆ e.g., registers, TLB(Translation Look-aside Buffer)s, page-table

Address Translation

- ❑ Hardware transforms a **virtual address** to a **physical address**.
 - ◆ The desired information is actually stored in a physical address.
- ❑ The OS must get involved at key points to set up the hardware.
 - ◆ The OS must manage memory to judiciously intervene.

Example: Address Translation

▣ C - Language code

```
void func()  
    int x=3000;  
    ...  
    x = x + 3; // this is the line of code we are interested in
```

- ◆ **Load** a value from memory
- ◆ **Increment** it by three
- ◆ **Store** the value back into memory

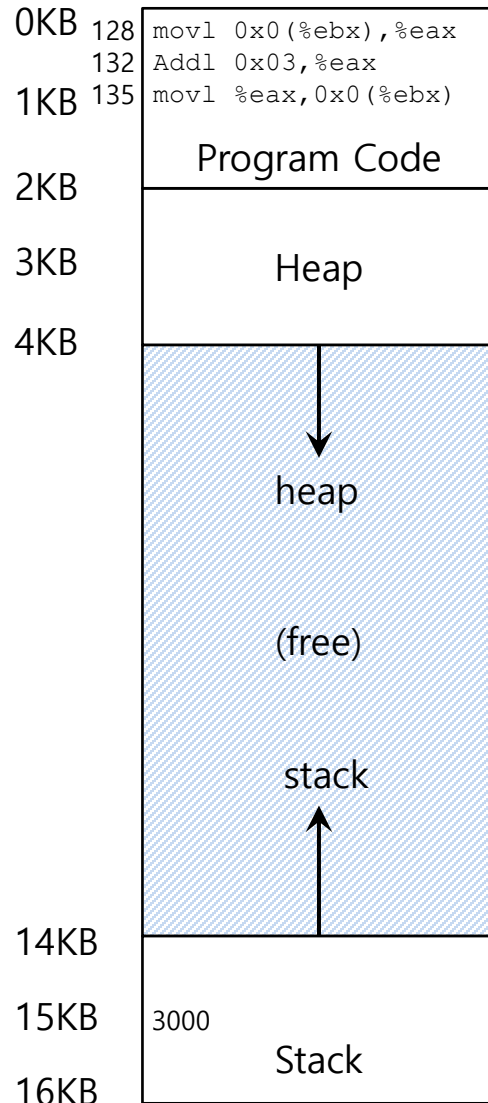
Example: Address Translation(Cont.)

▣ Assembly

```
128 : movl 0x0(%ebx), %eax      ; load 0+ebx into eax
132 : addl $0x03, %eax          ; add 3 to eax register
135 : movl %eax, 0x0(%ebx)      ; store eax back to mem
```

- ◆ **Load** the value at that address into `eax` register.
- ◆ **Add** 3 to `eax` register.
- ◆ **Store** the value in `eax` back into memory.

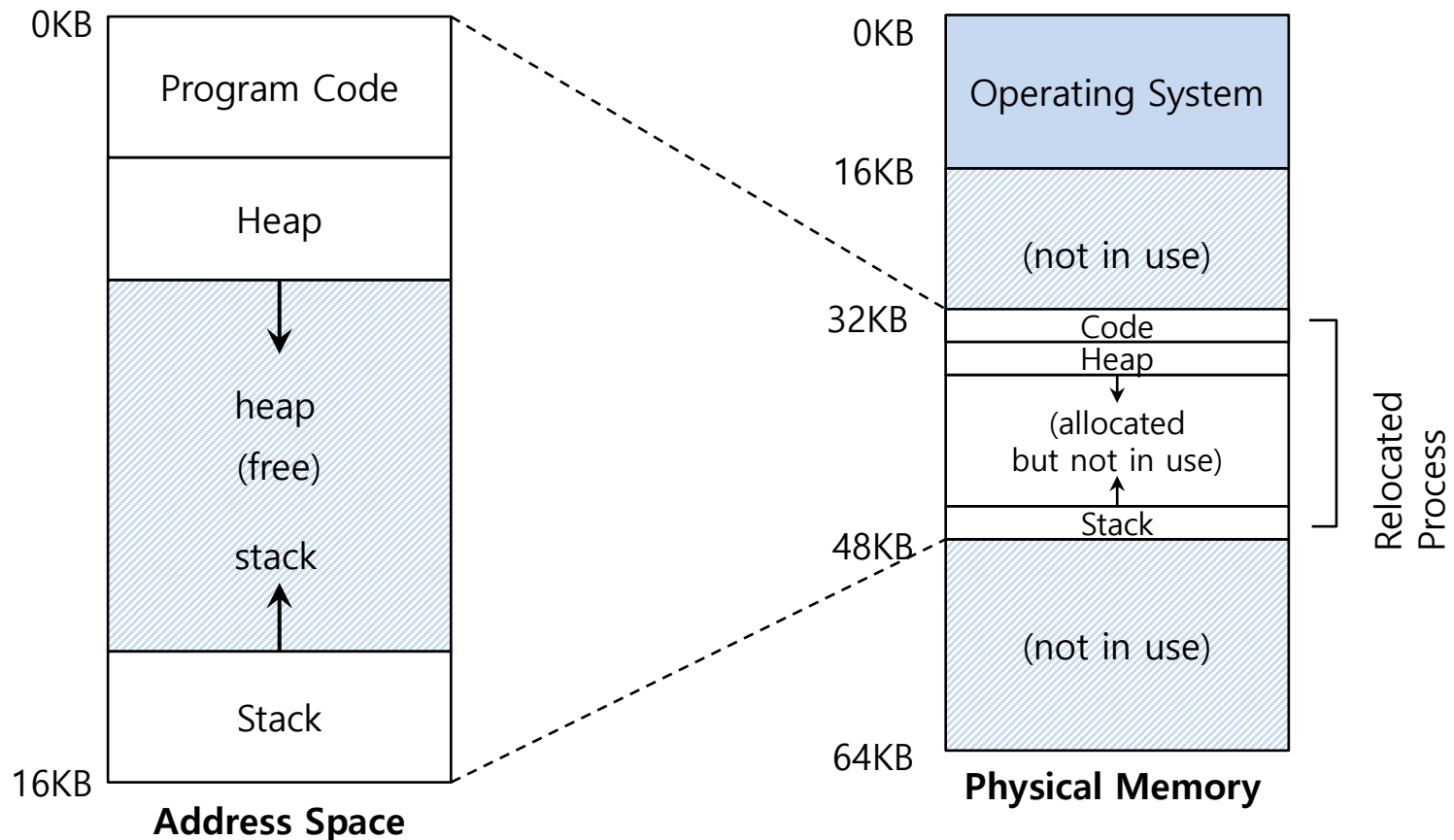
Example: Address Translation(Cont.)



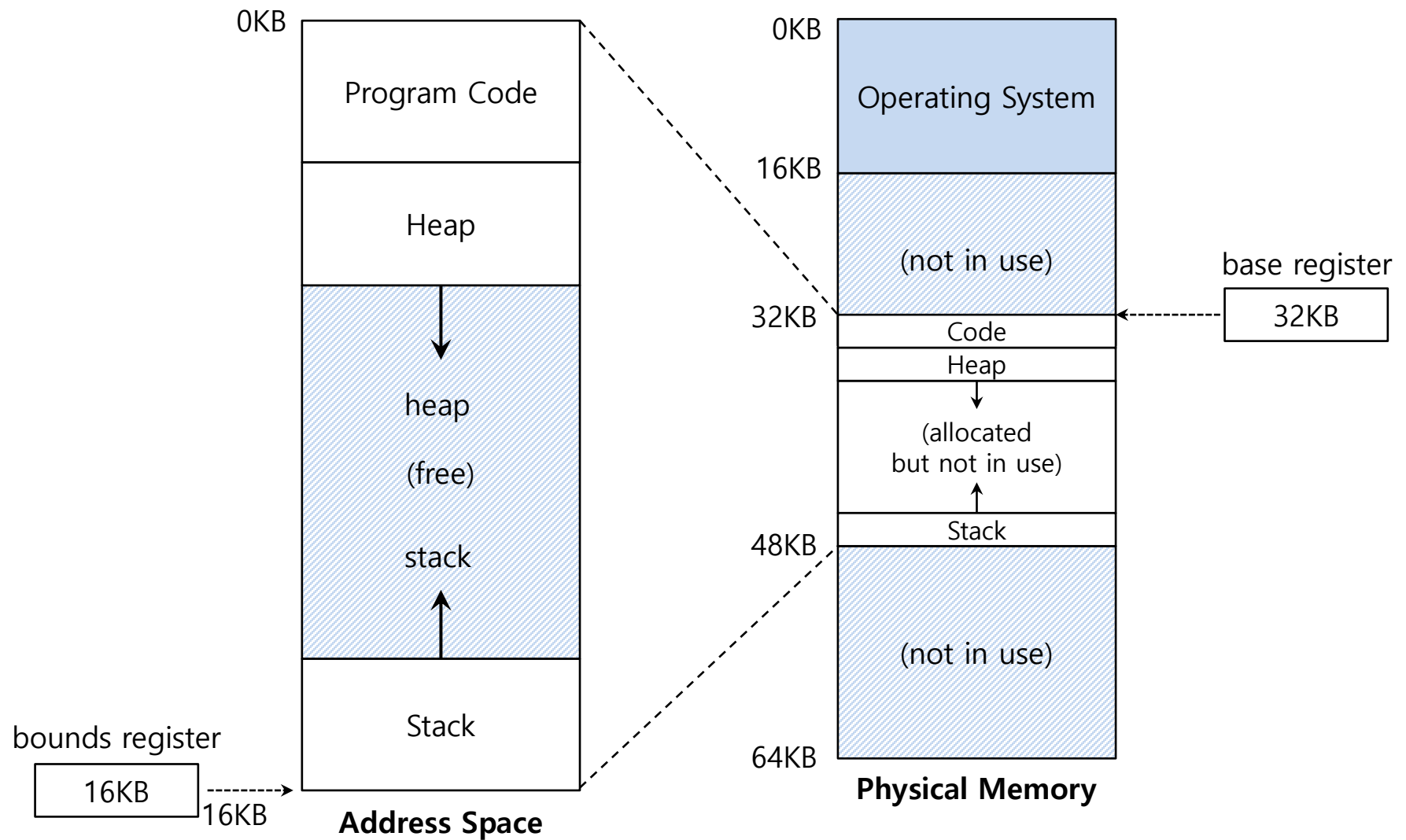
- Fetch instruction at address 128
- Execute this instruction (load from address 15KB)
- Fetch instruction at address 132
- Execute this instruction (no memory reference)
- Fetch the instruction at address 135
- Execute this instruction (store to address 15 KB)

Dynamic Relocation: Base and Bound Register

- The OS wants to place the process **somewhere else** in physical memory, not at address 0.
 - ◆ The address space start at address 0.



Base and Bounds Register



Dynamic(Hardware base) Relocation

- When a program starts running, the OS decides **where** in physical memory a process should be **loaded**.
 - ◆ Set the **base** register a value.

$$\text{physical address} = \text{virtual address} + \text{base}$$

- ◆ Every virtual address must **not be greater than bound** and **negative**.

$$0 \leq \text{virtual address} < \text{bounds}$$

Relocation and Address Translation

128 : movl 0x0(%ebx), %eax

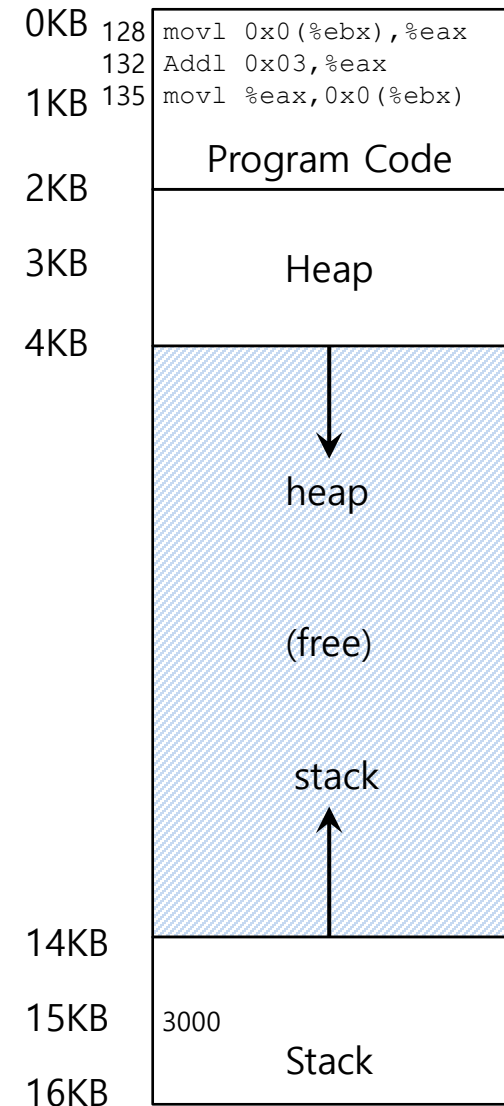
- ◆ **Fetch** instruction at address 128

$$32896 = 128 + 32KB(base)$$

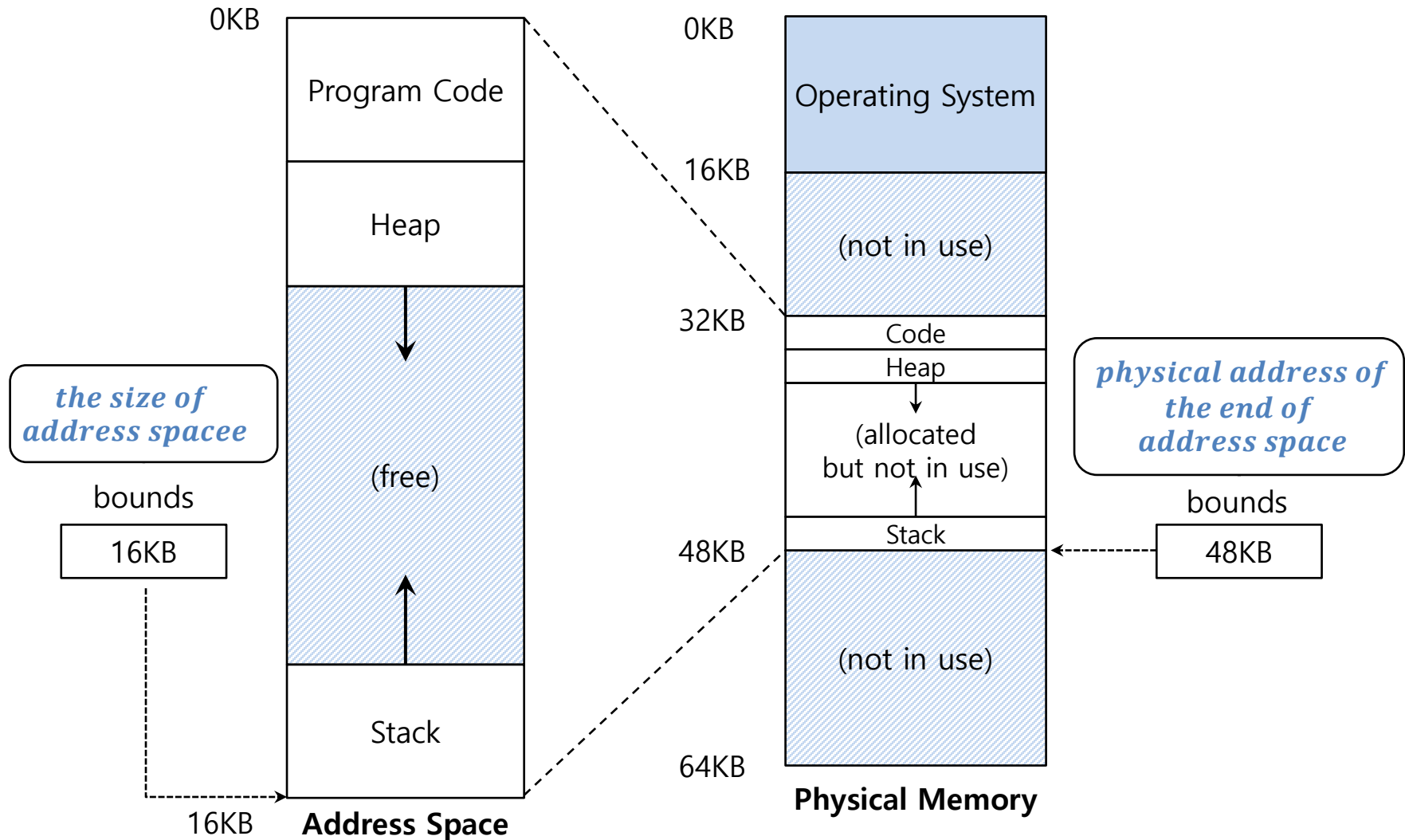
- ◆ **Execute** this instruction

- Load from address 15KB

$$47KB = 15KB + 32KB(base)$$



Two ways of Bounds Register



Hardware Requirements

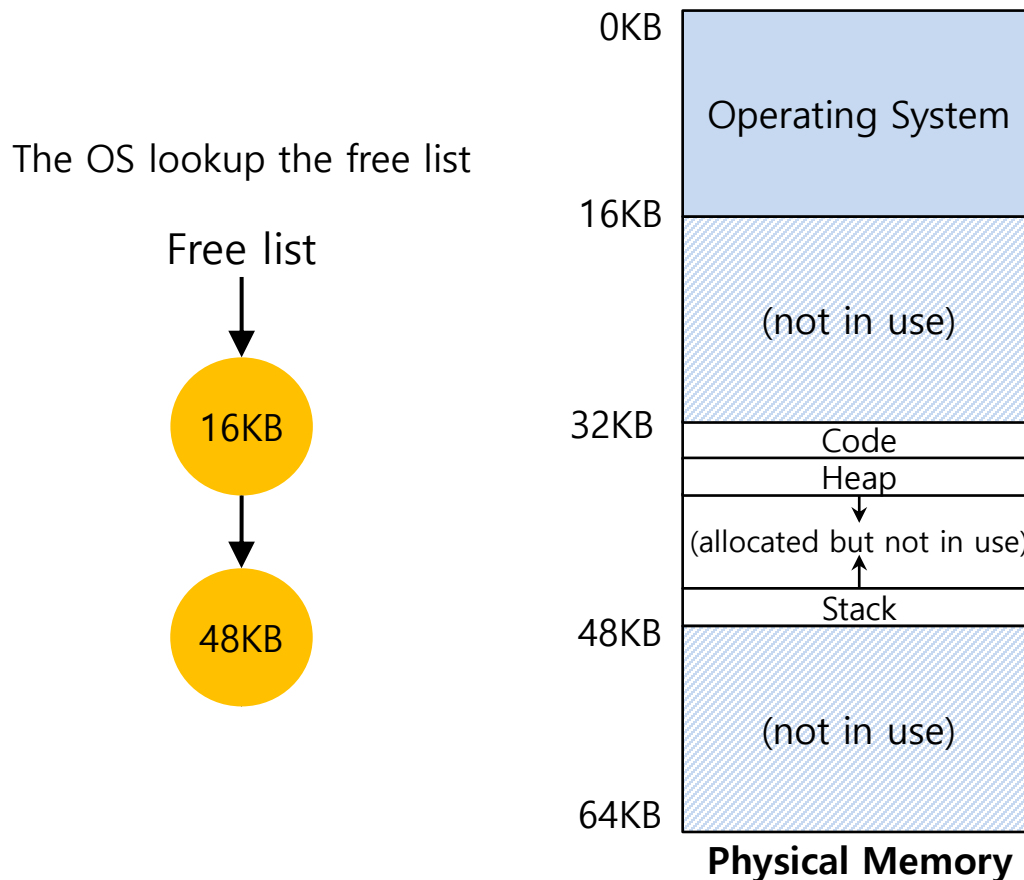
- ❑ Privileged mode: prevent user-mode processes from executing privileged operations
- ❑ Base/bounds registers: Need pair of registers per CPU to support address translation and bounds checks
- ❑ Ability to translate virtual addresses and check if within bounds limits; Circuitry to do translations.
- ❑ Privileged instruction(s) to update base/bounds: OS must be able to set these values before letting a user program run
- ❑ Privileged instruction(s) to register: OS must be able to tell hardware what exception handlers code to run if exception occurs
- ❑ Ability to raise exceptions when processes try to access privileged instructions or out-of-bounds memory

OS Issues for Memory Virtualizing

- ▣ The OS must **take action** to implement **base-and-bounds** approach.
- ▣ Three critical junctures:
 - ◆ When a process **starts running**:
 - Finding space for address space in physical memory
 - ◆ When a process is **terminated**:
 - Reclaiming the memory for use
 - ◆ When context **switch occurs**:
 - Saving and storing the base-and-bounds pair

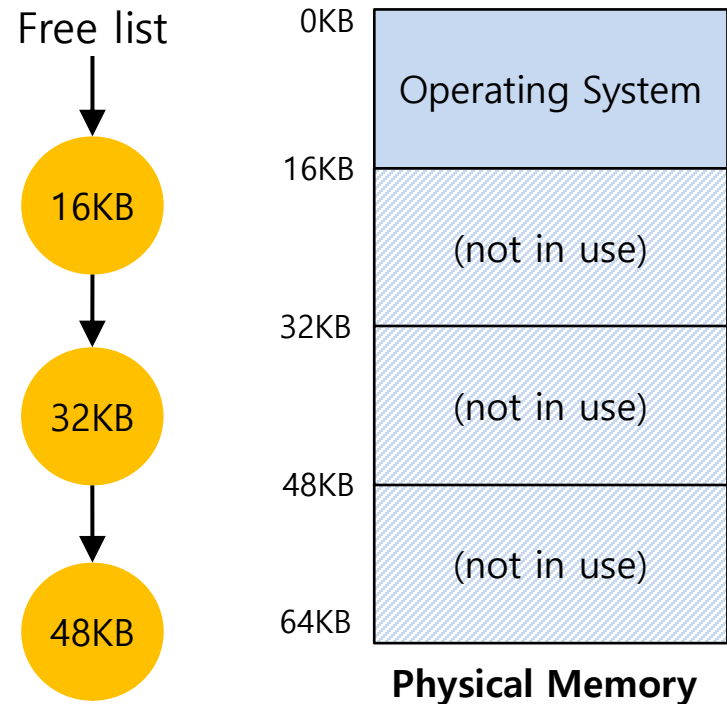
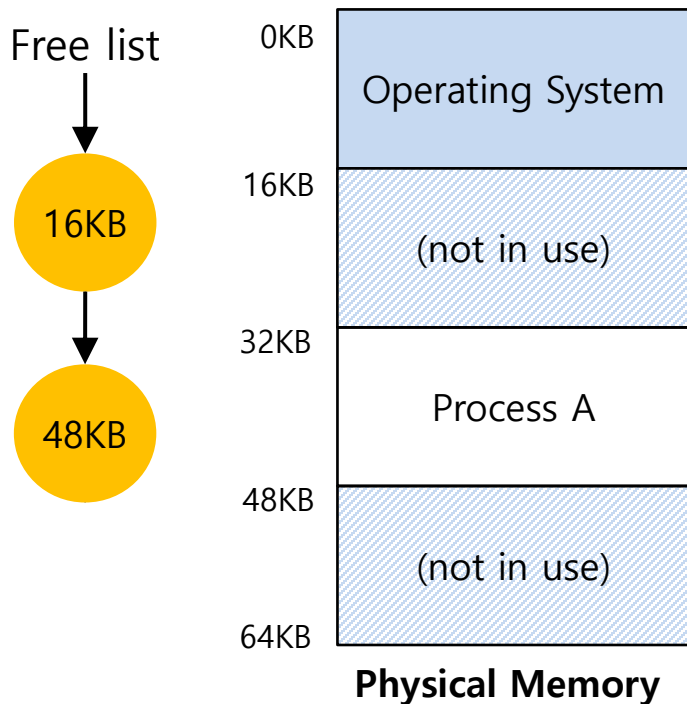
OS Issues: When a Process Starts Running

- ▣ The OS must **find a room** for a new address space.
 - ◆ free list : A list of the range of the physical memory which are not in use.



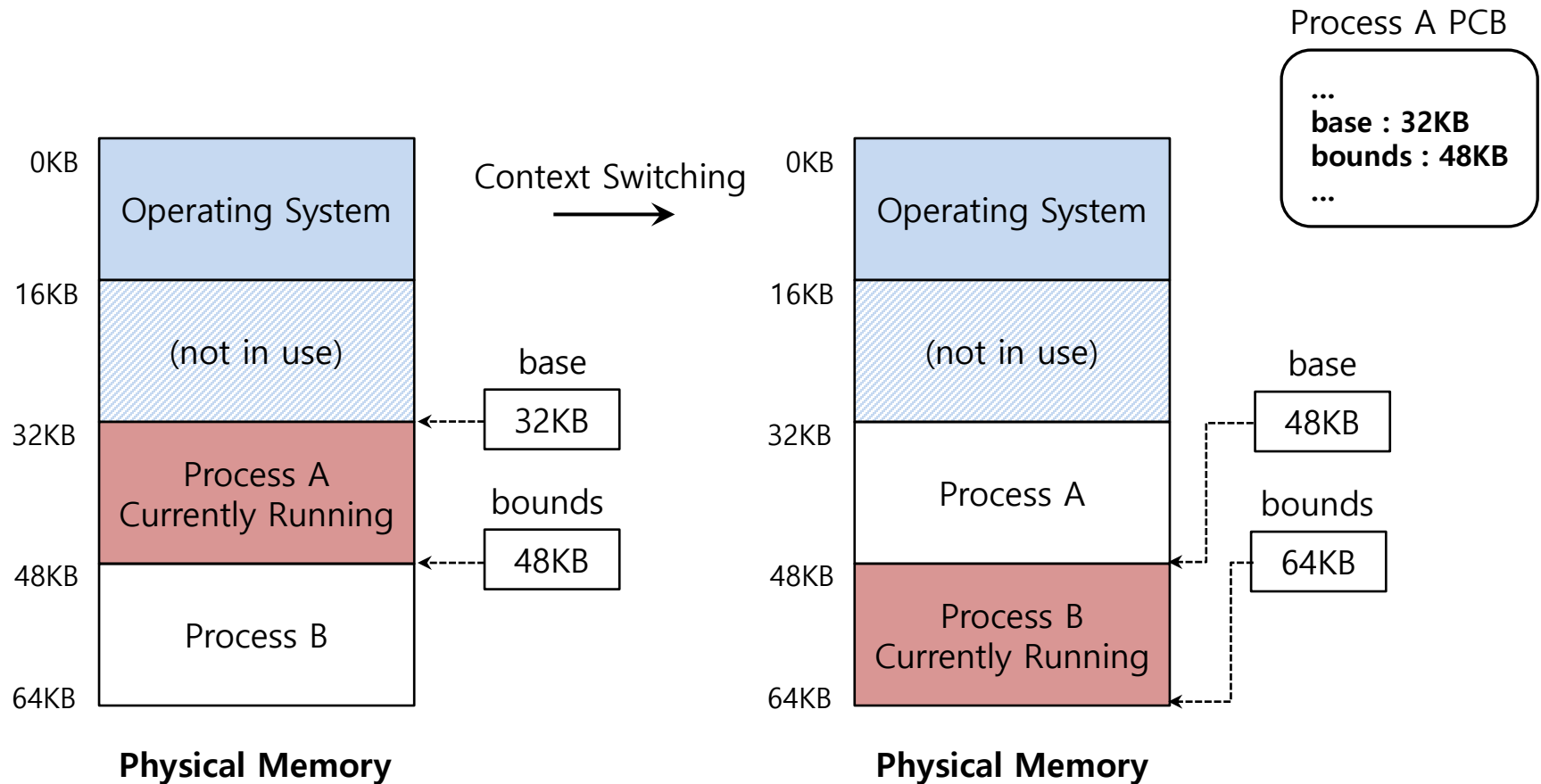
OS Issues: When a Process Is Terminated

- ▣ The OS must **put the memory back** on the free list.



OS Issues: When Context Switch Occurs

- ▣ The OS must **save and restore** the base-and-bounds pair.
 - ◆ In **process structure** or **process control block(PCB)**



OS Issues: provide exception handlers

- ▣ the OS must provide exception handlers,
- ▣ the OS installs these handlers at boot time (via privileged instructions
 - ◆ Exception handler for segmentation fault

Summary

- ▣ Address translation: hardware support and OS support
- ▣ Basic form: base and bound
- ▣ Fragmentation issue

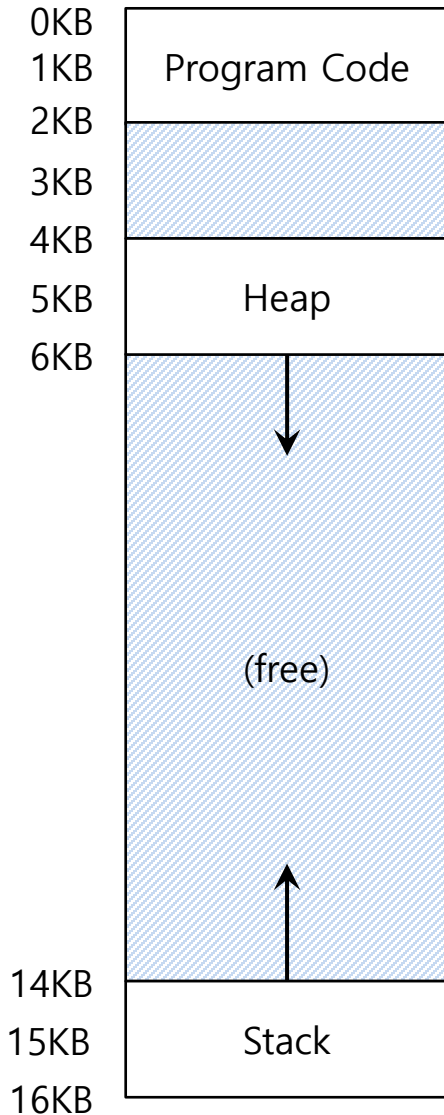
Operating Systems

Youjip Won



16. Segmentation

Inefficiency of the Base and Bound Approach

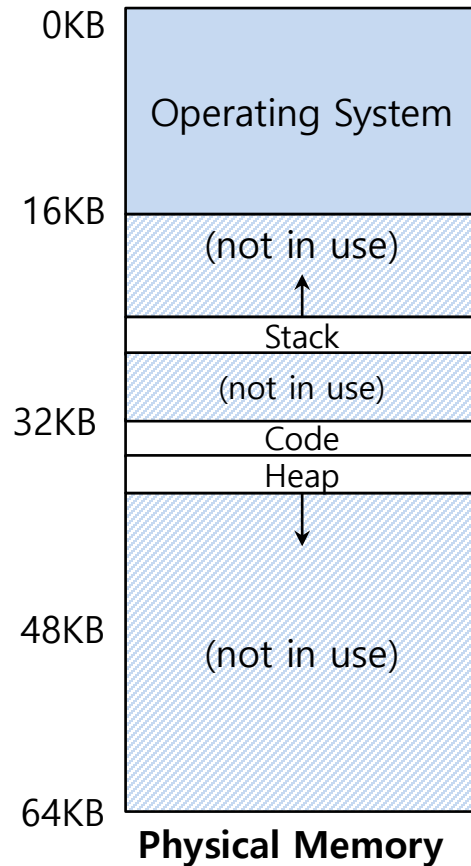


- ▣ **Big chunk of “free” space**
- ▣ “free” space **takes up** physical memory.
- ▣ Hard to run when an address space **does not fit** into physical memory

Segmentation

- ▣ Segment is just **a contiguous portion** of the address space of a particular length.
 - ◆ Logically-different segment: code, stack, heap
- ▣ Each segment can be **placed** in **different part of physical memory**.
 - ◆ **Base** and **bounds** exist **per each segment**.

Placing Segment In Physical Memory

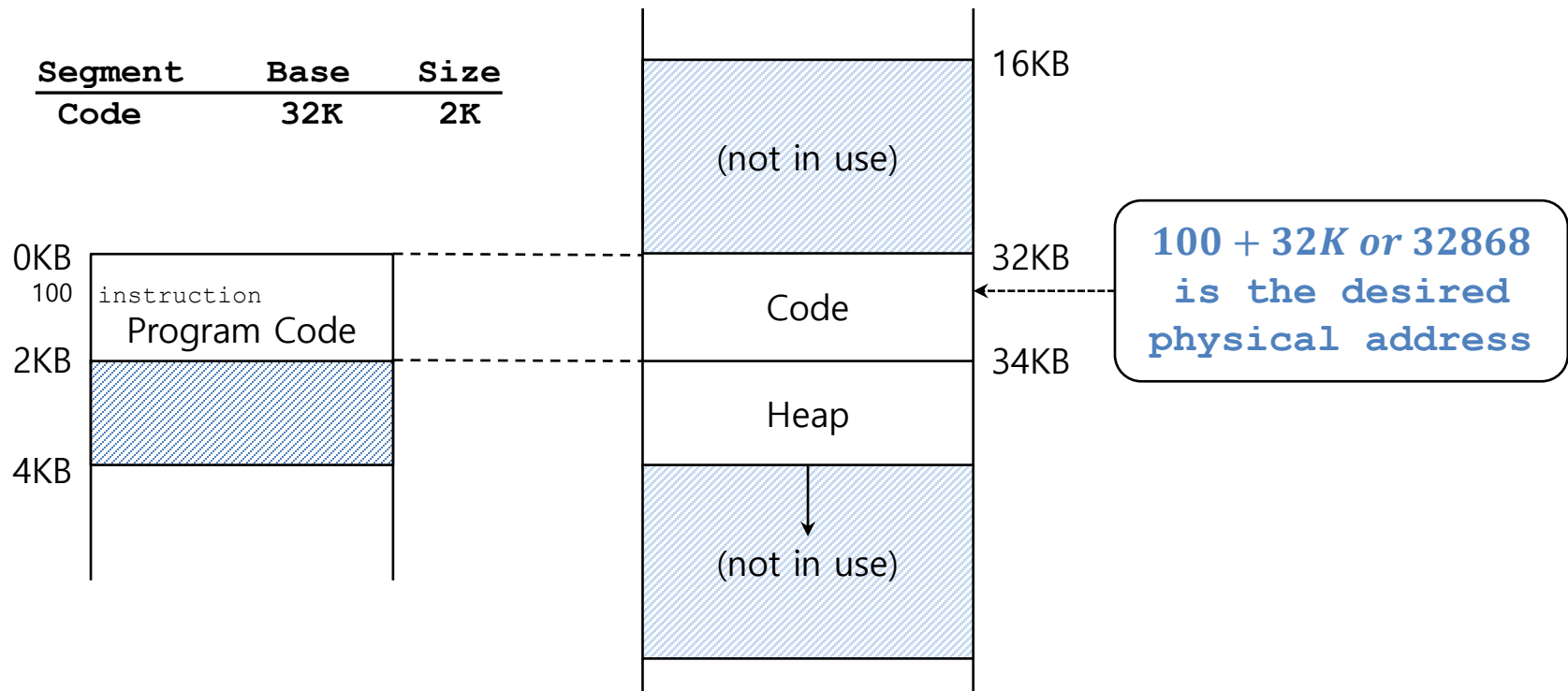


Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K

Address Translation on Segmentation: code

$$\text{physical address} = \text{offset} + \text{base}$$

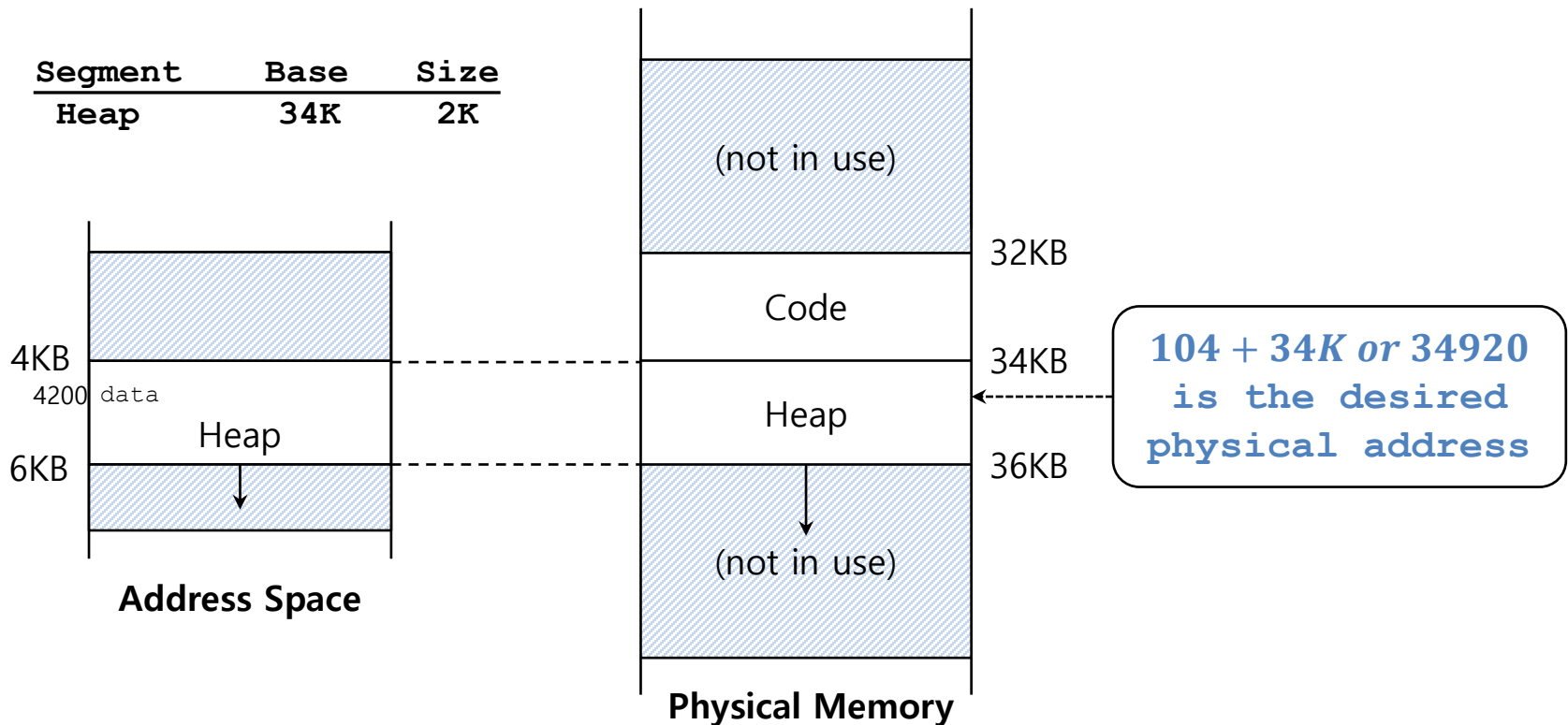
- ▣ The `offset` of virtual address 100 is 100.
 - ◆ The code segment **starts at virtual address 0** in address space.



Address Translation on Segmentation: heap

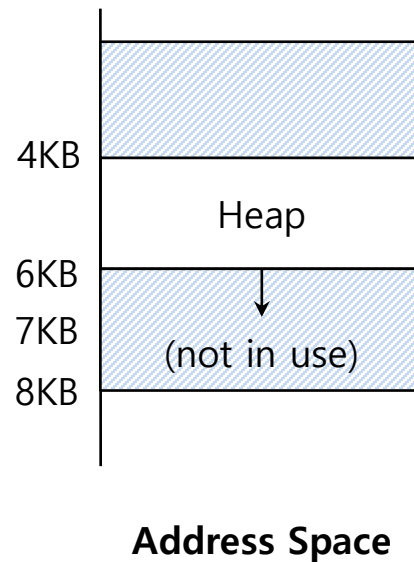
*Virtual address + base is not the correct physical address.
OFFSET of Virtual address + base is the correct physical address.*

- ▣ The offset of virtual address 4200 is 104.
 - ◆ The heap segment **starts at virtual address 4096** in address space.



Segmentation Fault or Violation

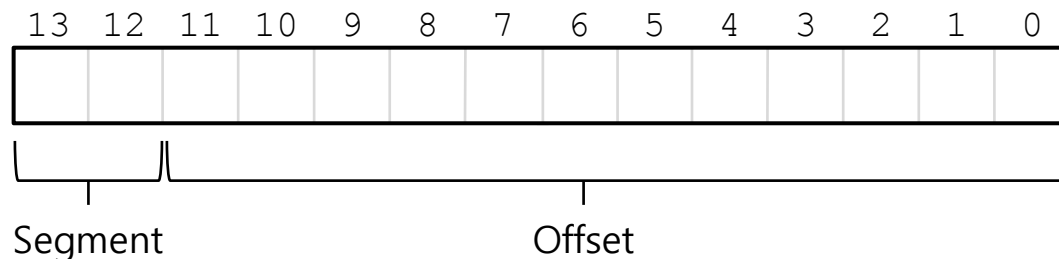
- ❑ If an **illegal address** such as 7KB which is beyond the end of heap is referenced, the OS occurs **segmentation fault**.
 - ◆ The hardware detects that address is **out of bounds**.



Referring to Segment

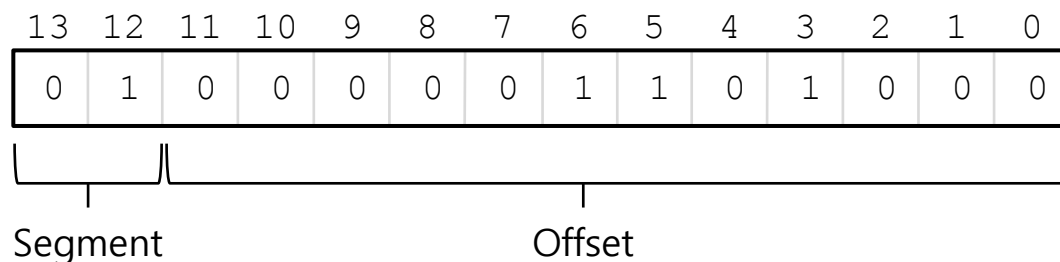
□ Explicit approach

- ◆ Chop up the address space into segments based on the **top few bits** of virtual address.



□ Example: virtual address 4200 (01000001101000)

Segment	bits
Code	00
Heap	01
Stack	10
-	11



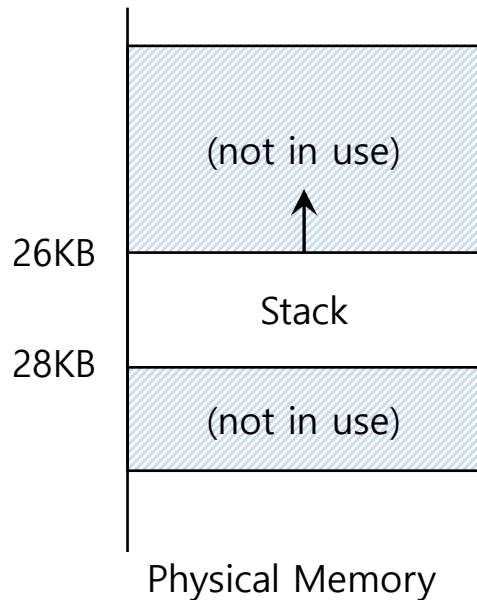
Segment selection

```
1  // get top 2 bits of 14-bit VA
2  Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3  // now get offset
4  Offset = VirtualAddress & OFFSET_MASK
5  if (Offset >= Bounds[Segment])
6      RaiseException(PROTECTION_FAULT)
7  else
8      PhysAddr = Base[Segment] + Offset
9      Register = AccessMemory(PhysAddr)
```

- ◆ `SEG_MASK = 0x3000 (11000000000000)`
- ◆ `SEG_SHIFT = 12`
- ◆ `OFFSET_MASK = 0xFFF (00111111111111)`

Referring to Stack Segment

- ❑ Stack grows **backward**.
- ❑ **Extra hardware support** is need.
 - ◆ The hardware checks which way the segment grows.
 - ◆ 1: positive direction, 0: negative direction



Segment Register(with Negative-Growth Support)

Segment	Base	Size	Grows Positive?
Code	32K	2K	1
Heap	34K	2K	1
Stack	28K	2K	0

Support for Sharing

- Segment can be **shared between address** space.
 - ◆ **Code sharing** is still in use in systems today.
 - ◆ by extra hardware support.
- Extra hardware support is need for form of **Protection bits**.
 - ◆ **A few more bits** per segment to indicate **permissions** of **read**, write and **execute**.

Segment Register Values(with Protection)

Segment	Base	Size	Grows Positive?	Protection
Code	32K	2K	1	Read-Execute
Heap	34K	2K	1	Read-Write
Stack	28K	2K	0	Read-Write

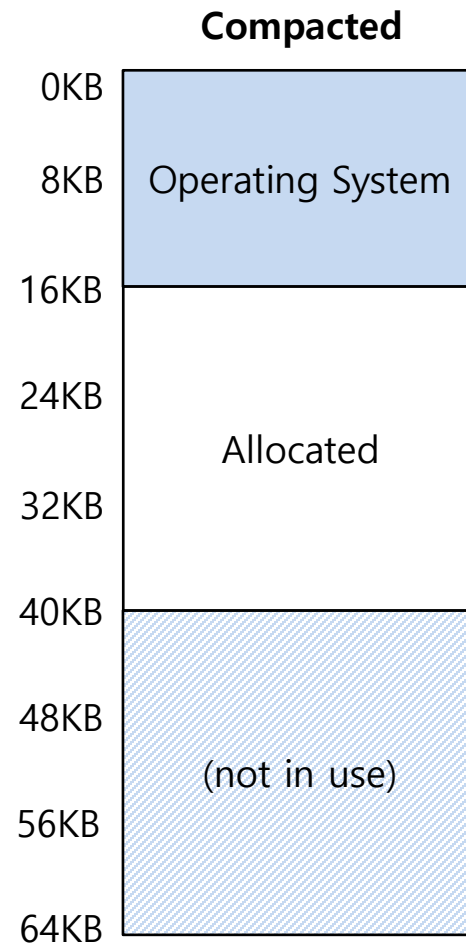
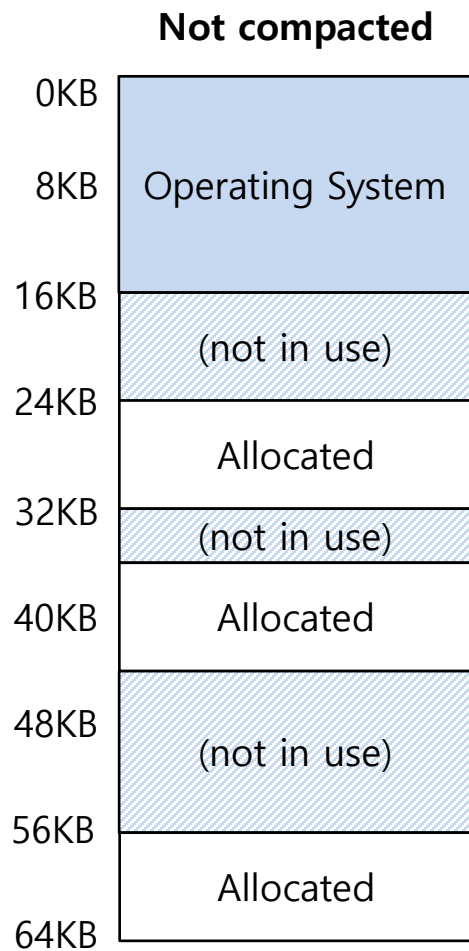
Fine-Grained and Coarse-Grained segmentation

- ▣ **Coarse-Grained** means small number of segments.
 - ◆ e.g., code, heap, stack.
- ▣ **Fine-Grained** segmentation allows **more flexibility** for address space in some early system.
 - ◆ To support many segments, Hardware support with a **segment table** is required.

- ▣ **External Fragmentation:** little holes of **free space** in physical memory that is too small for allocating segment.
 - ◆ There is **24KB free**, but **not in one contiguous** segment.
 - ◆ The OS **cannot** satisfy the **20KB request**.

- ▣ **Compaction: rearranging** the exiting segments in physical memory.
 - ◆ Compaction is **costly**.
 - **Stop** running process.
 - **Copy** data to somewhere.
 - **Change** segment register value.

Memory Compaction



History of segmentation

- ❑ In early days, OS used segmentation.
 - ◆ Burroughs B5000 (first commercial machine with virtual memory)
 - ◆ IBM AS/400
 - ◆ Intel 8086, 80286
- ❑ 80386 and later Intel CPU's support paging.
- ❑ X86-64 does not use segmentation any more in 64bit mode
 - ◆ CS,SS,DS and ES are forced to 0 and 2^{24} ..

Summary

- ▣ Segmentation can better support sparse address spaces.
- ▣ It is also fast as the overheads of translation are minimal.
- ▣ Sharing (such as code) is easy.
- ▣ Issues
 - ◆ External fragmentation issue
 - ◆ Sparse segment

Operating Systems

Youjip Won

KAIST

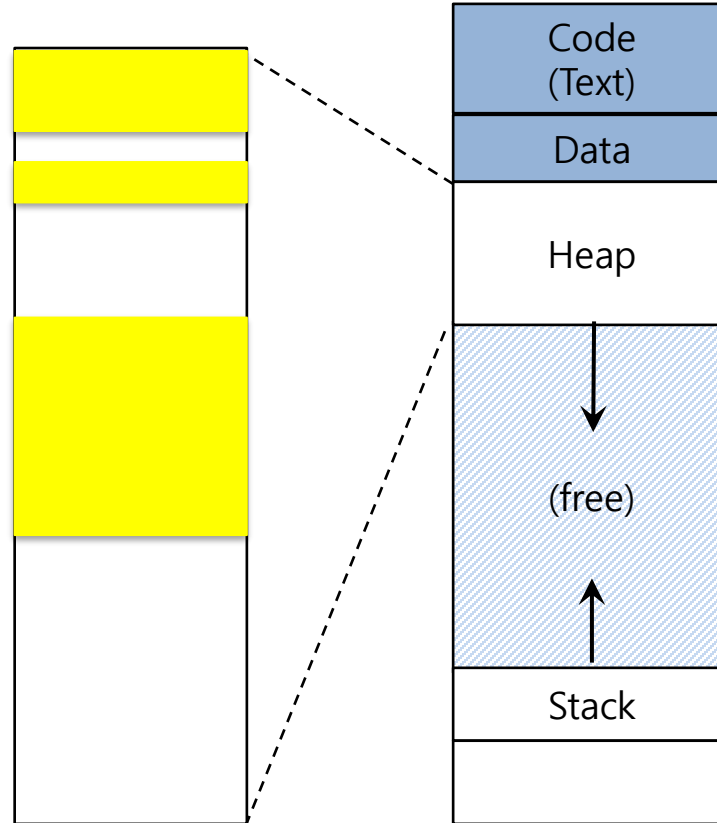


17. Free-Space Management

Managing heap

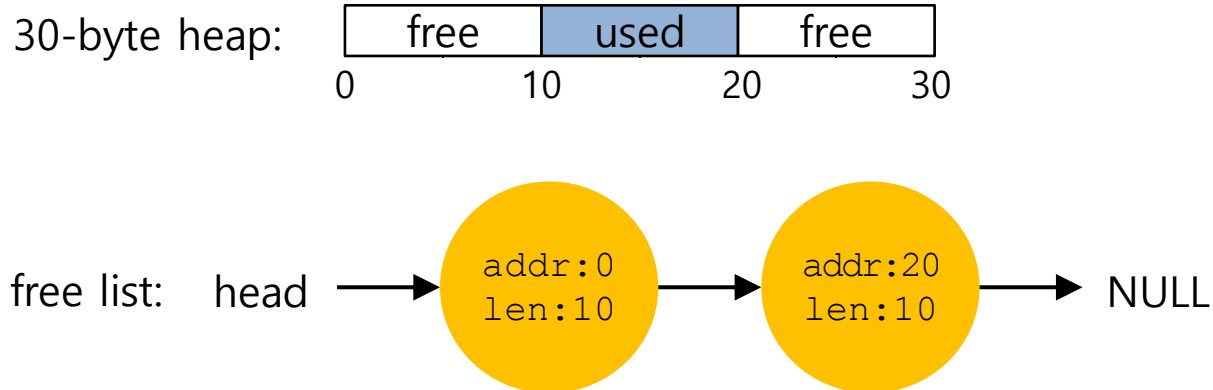
`malloc()`
`free()`
in **libc**

To manage the
memory in a
heap



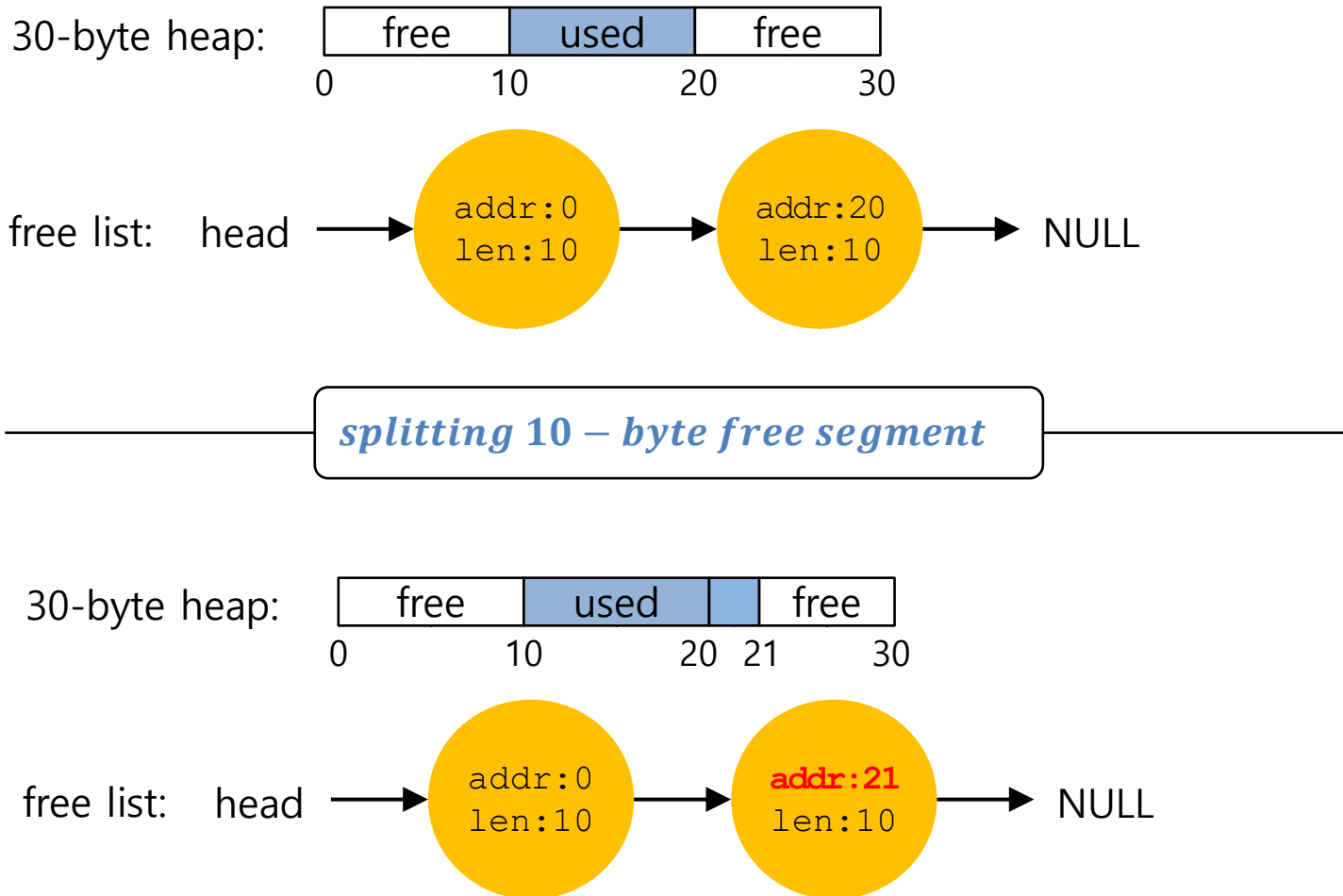
Splitting

- ▣ Finding a free chunk of memory that can satisfy the request and splitting it into two.
 - ◆ When request for memory allocation is **smaller** than the size of free chunks.



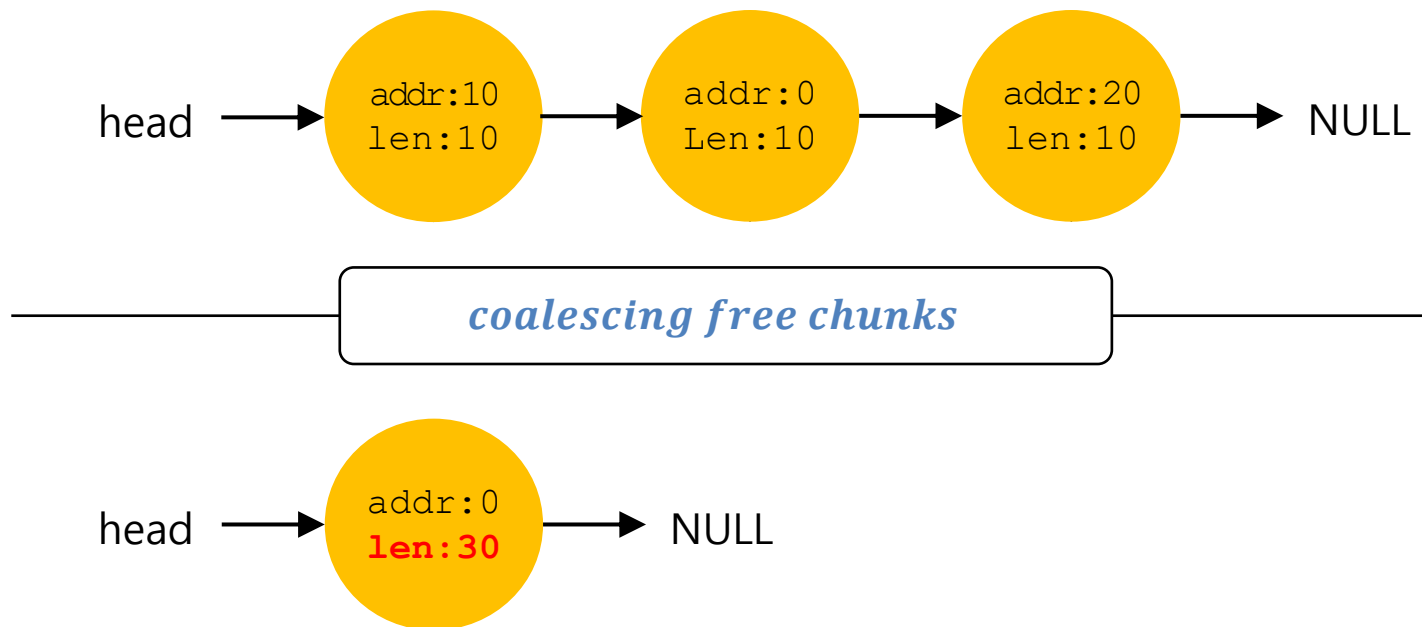
Splitting(Cont.)

- Two 10-bytes free segment with **1-byte request**



Coalescing

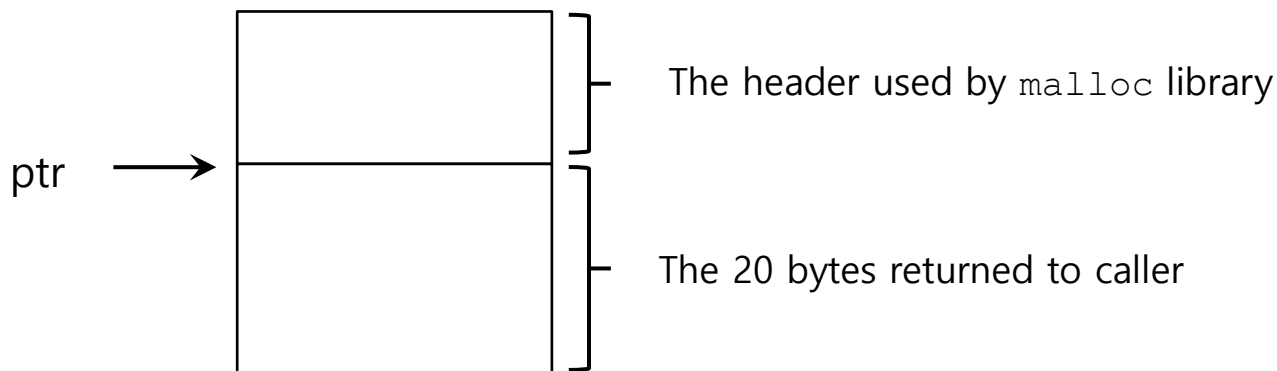
- ❑ If a user requests memory that is **bigger than free chunk size**, the list will **not find** such a free chunk.
- ❑ Coalescing: **Merge** returning a free chunk with existing chunks into a large single free chunk if **addresses** of them are **nearby**.



Tracking The Size of Allocated Regions

- ▣ The interface to `free(void *ptr)` does **not take a size parameter**.
 - ◆ How does the library **know the size** of memory region that will be back into free list?

```
ptr = malloc(20);
```

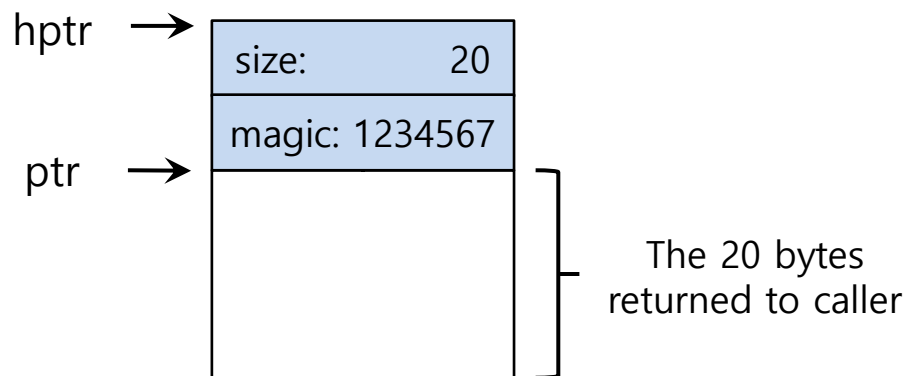


An Allocated Region Plus Header

The Header of Allocated Memory Chunk

```
typedef struct __header_t {  
    int size;  
    int magic;  
} header_t;
```

Actual chunk size of malloc(N) = N + size of header
Here, 28 Byte



The Header of Allocated Memory Chunk(Cont.)

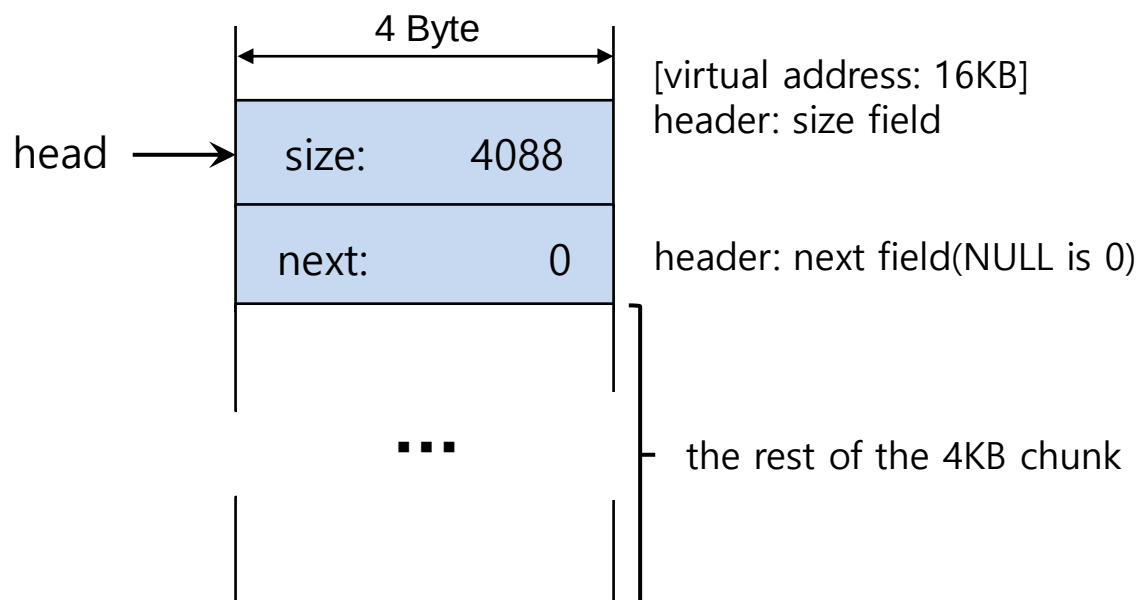
```
void free(void *ptr) {  
    header_t *hptr = (void *)ptr - sizeof(header_t) ;  
    ...  
    assert(hptr->magic==1234567) ;  
    ...  
}
```

Embedding A Free List

```
typedef struct __node_t {  
    int size;  
    struct __node_t *next;  
} nodet_t;
```

Heap Initialization

```
// mmap() returns a pointer to a chunk of free space
node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
                    MAP_ANON|MAP_PRIVATE, -1, 0);
head->size = 4096 - sizeof(node_t);
head->next = NULL;
```

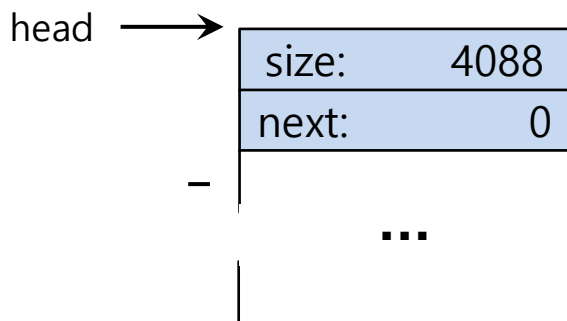


- ❑ If a chunk of memory is requested, the library **will first find** a chunk that is **large enough** to accommodate the request.
- ❑ The library will
 - ◆ **Split** the large free chunk into two.
 - **One** for the **request** and the **remaining** free chunk
 - ◆ **Shrink** the size of free chunk in the list.

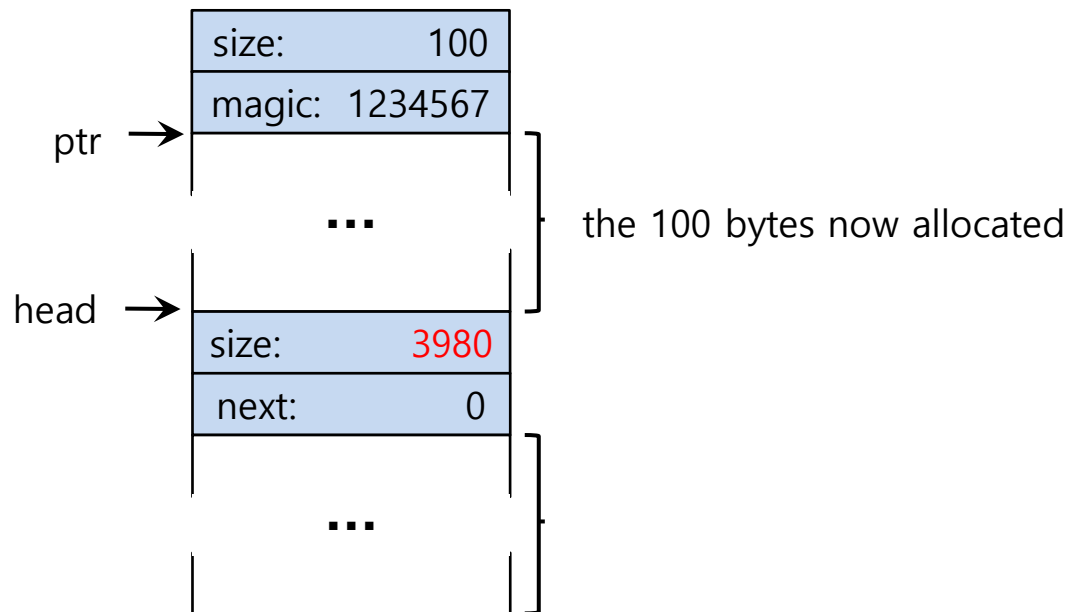
Embedding A Free List: Allocation(Cont.)

- Example: a request for 100 bytes by `ptr = malloc(100)`
→ 108 byte is returned.

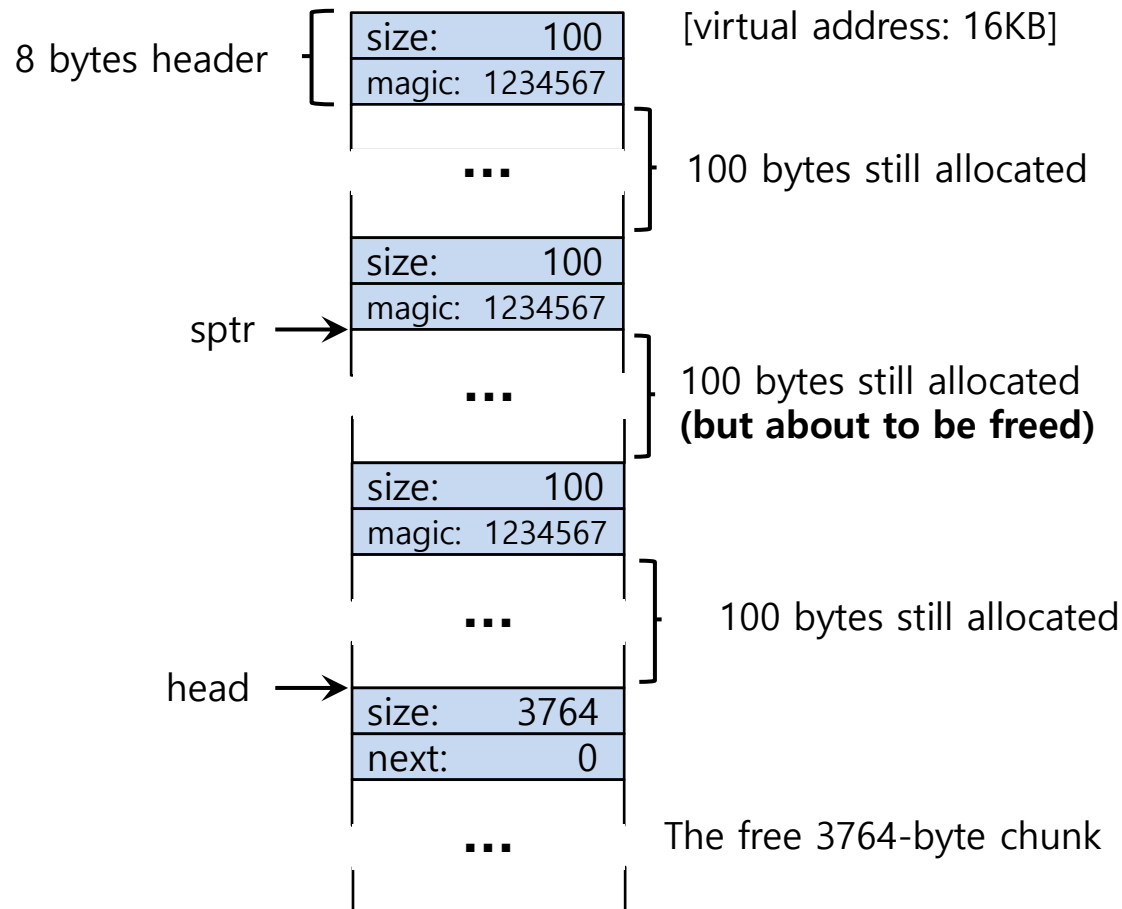
A 4KB Heap With One Free Chunk



A Heap : After One Allocation



Free Space With Chunks Allocated



Free Space With Three Chunks Allocated

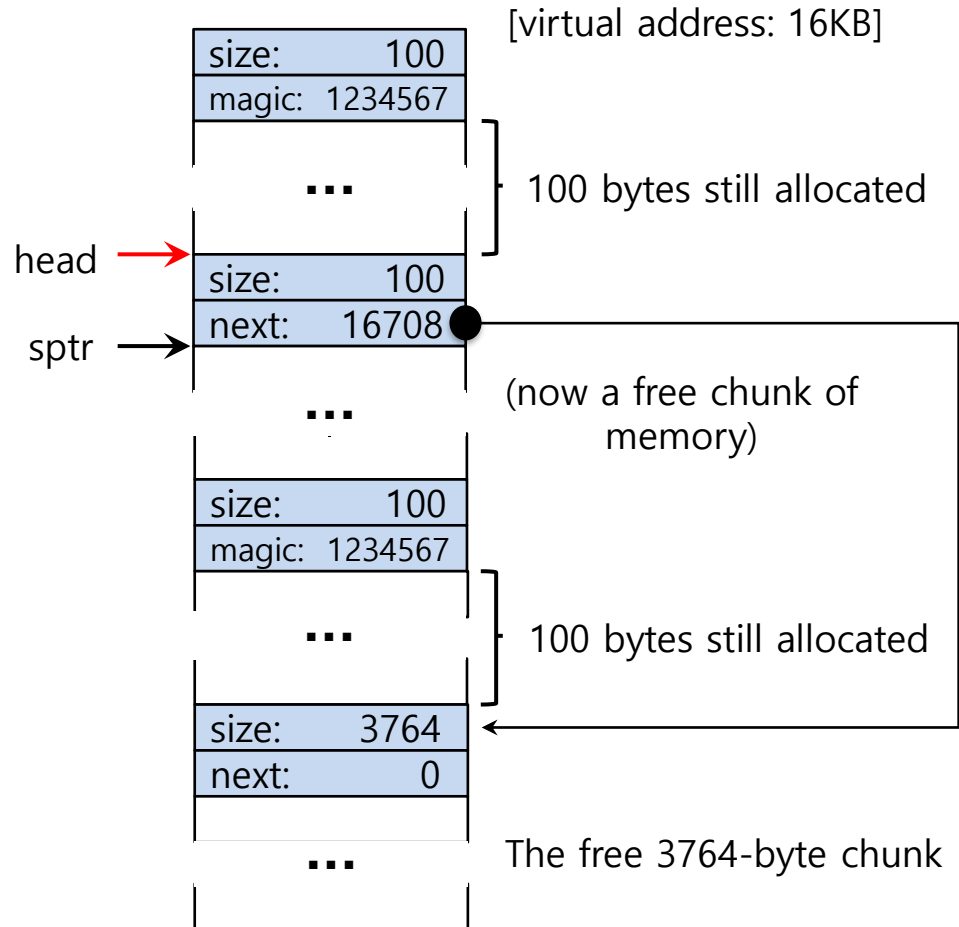
Free Space With `free()`

▣ `free(sptr)`

```
void* tmp = head ;
```

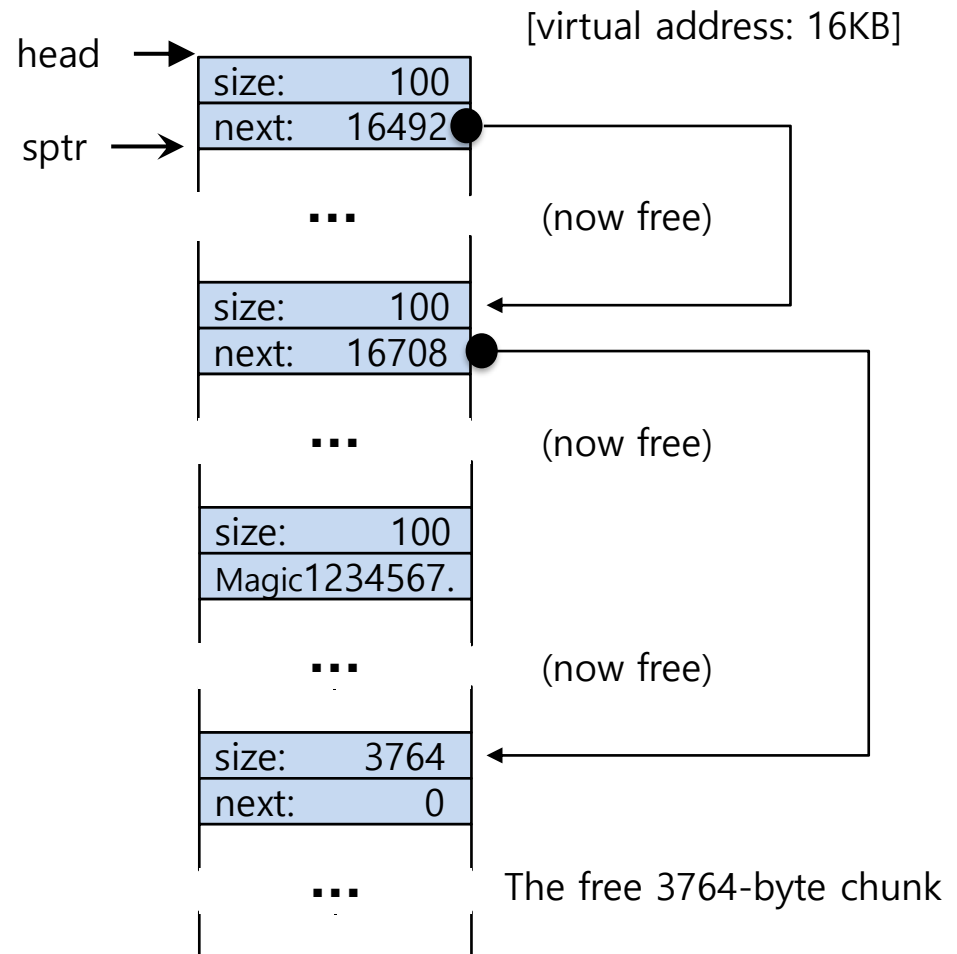
```
head = sptr ;
```

```
head->next = tmp ;
```



Free Space With Freed Chunks

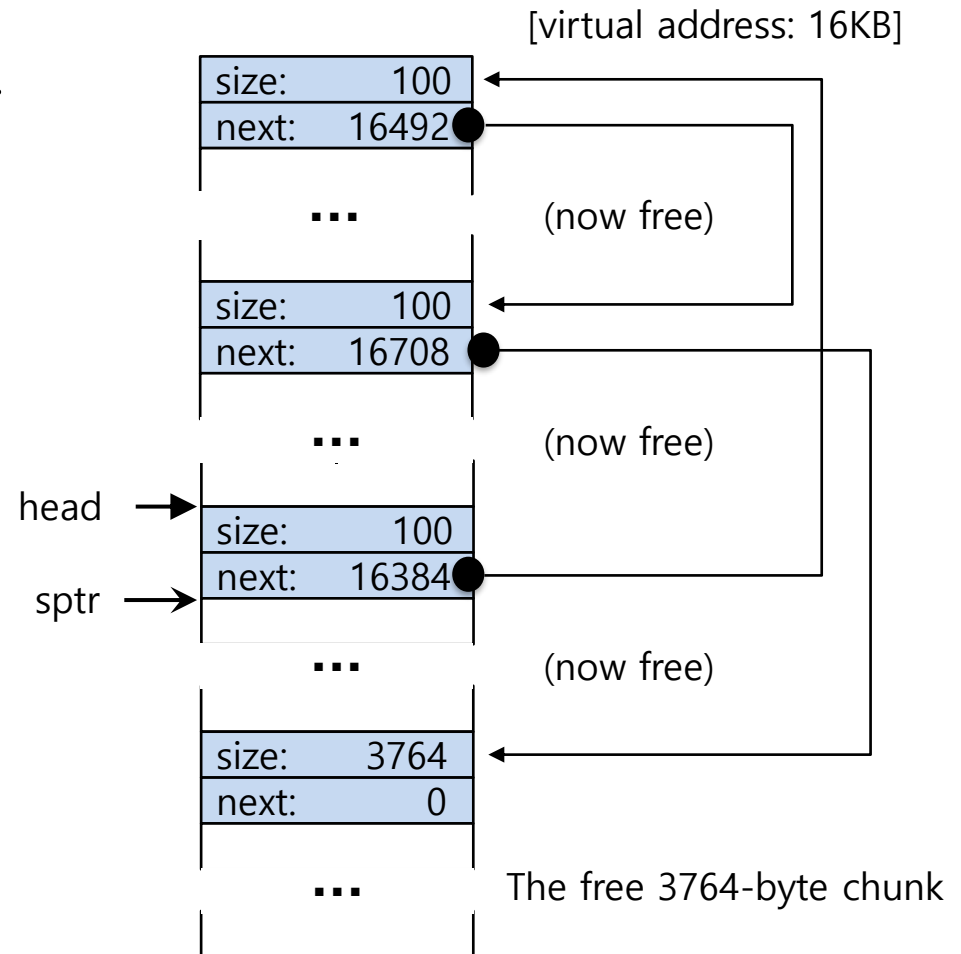
▣ `free(sptr)`



Free Space With Freed Chunks

- free(sptr)

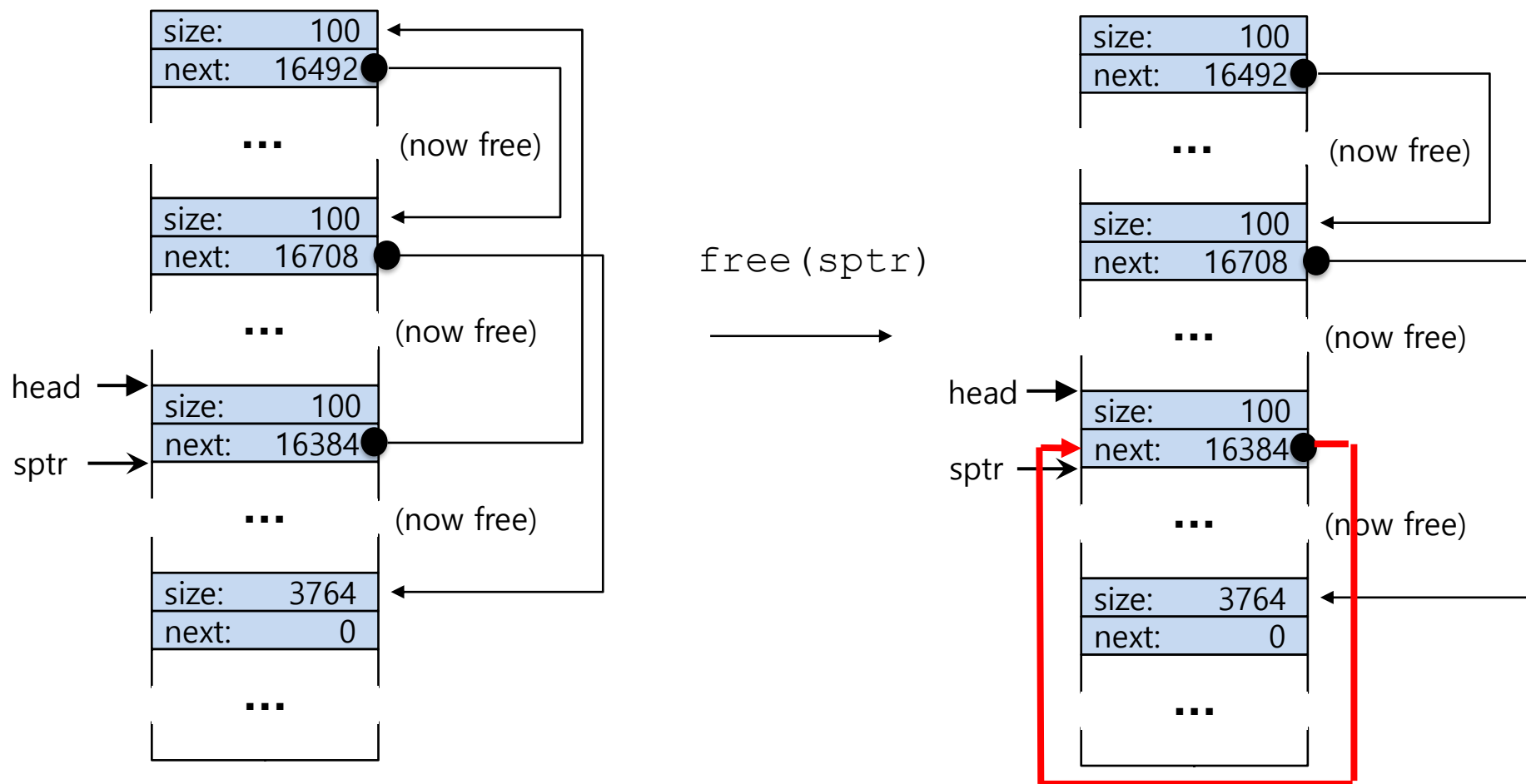
- Coalescing** is needed in the list.



Double free

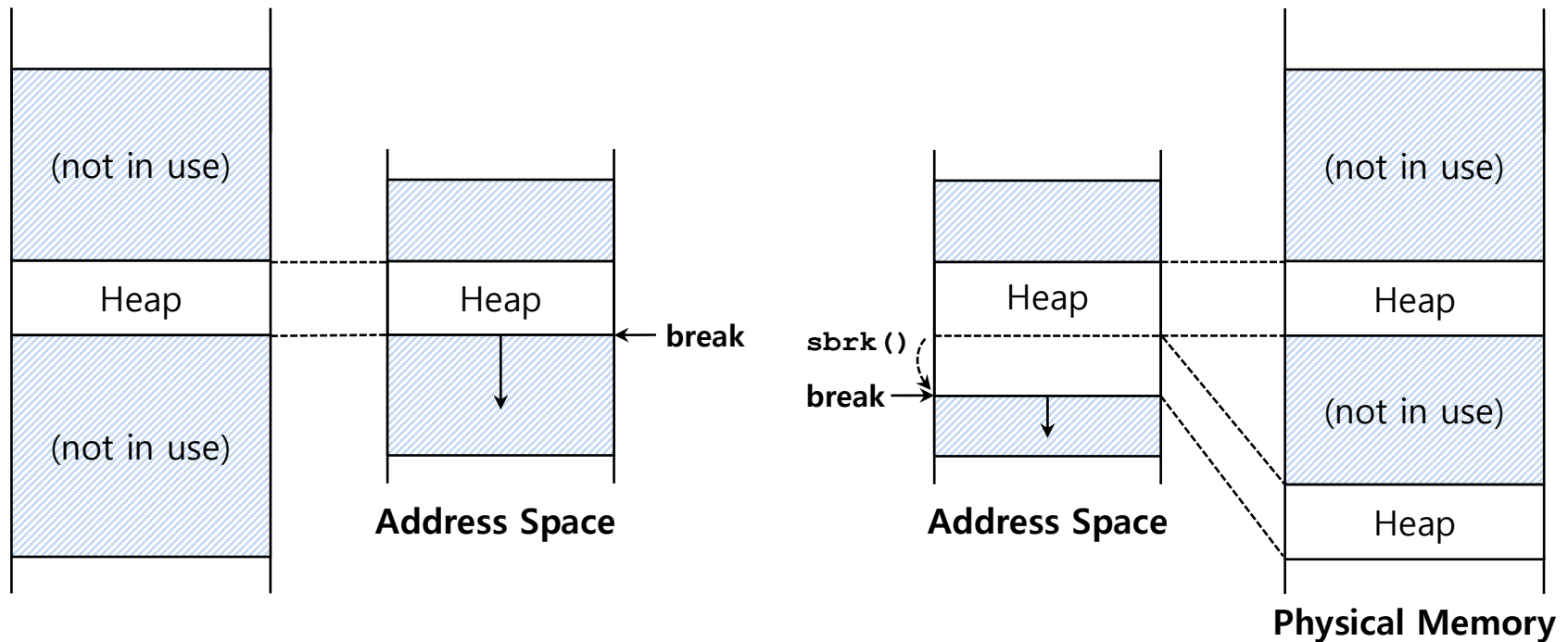
Free(sptr): dangling reference

[virtual address: 16KB]



Growing The Heap

- Most allocators **start** with a **small-sized heap** and then **request more** memory from the OS when they run out.
 - e.g., `sbrk()`, `brk()` in most UNIX systems.



Managing Free Space: Basic Strategies

□ Best Fit:

- ◆ Finding free chunks that are **big or bigger than the request**
- ◆ Returning the **one of smallest** in the chunks **in the group** of candidates

□ Worst Fit:

- ◆ Finding the **largest free chunks** and allocation the amount of the request
- ◆ **Keeping the remaining chunk** on the free list.

Managing Free Space: Basic Strategies(Cont.)

▣ First Fit:

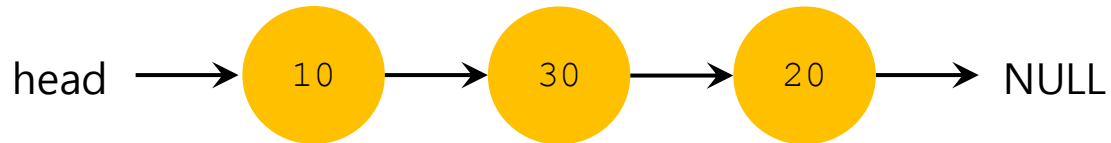
- ◆ Finding the **first chunk** that is **big enough** for the request
- ◆ Returning the requested amount and remaining the rest of the chunk.

▣ Next Fit:

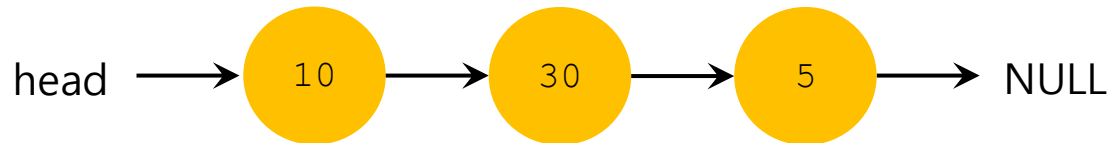
- ◆ Finding the first chunk that is big enough for the request.
- ◆ Searching at **where one was looking** at instead of the beginning of the list.

Examples of Basic Strategies

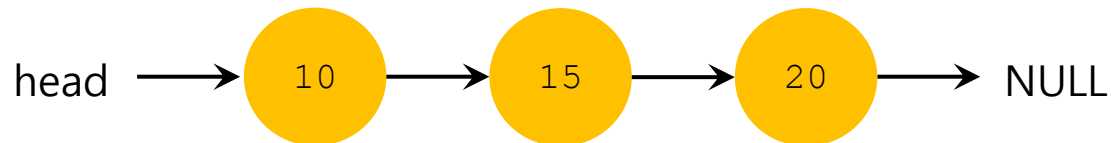
Allocation Request Size 15



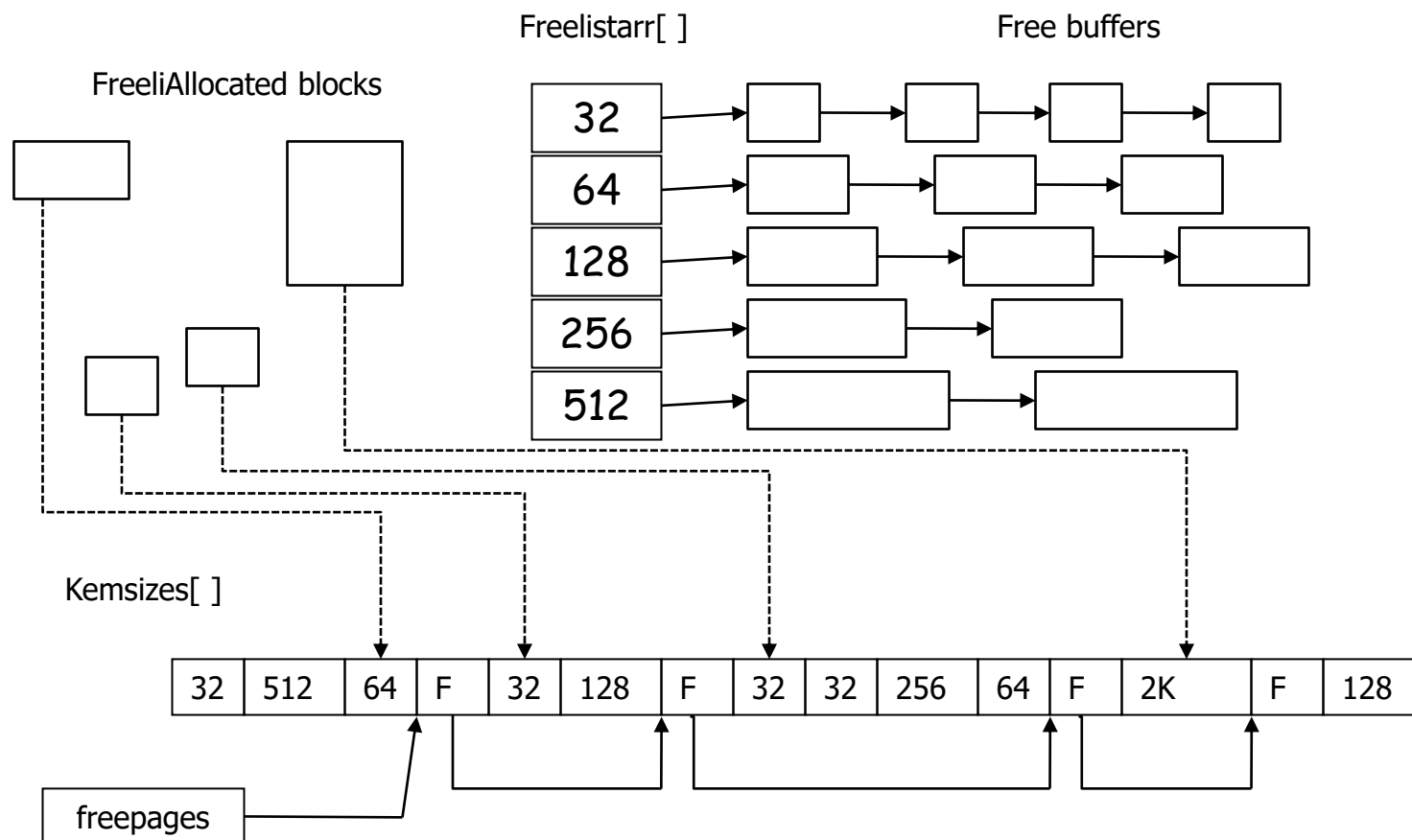
Result of Best-fit



Result of Worst-fit



Segregated List: McKusick-Karels Allocator (4.3 BSD)



McKusick-Karels Allocator

- ▣ Used in 4.3BSD and Digital UNIX.
- ▣ Management
 - ◆ Contiguous set of pages
 - ◆ Same buffer size within a page
- ▣ Manage pages by page usage array(kmemsizes[]).
 - ◆ Element of kmemsizes: free | size of buffer
- ▣ `malloc()` allocates the requested sized buffer from the free list.
- ▣ `malloc()` invoke a routine that acquires free page and splits by a requested size if free list is empty.
- ▣ How much memory should dedicate to the pool of memory that serves specialized requests of a given size?

Buddy System

- ❑ Create the small buffers by repeatedly halving a large buffer and coalesce the adjacent free buffers.
- ❑ When a buffer is split, each half is called the buddy of the other.
- ❑ Minimum size is 32byte.
- ❑ Use a bitmap to monitor each 32-byte chunk of the block.

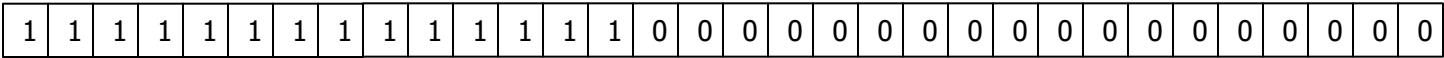
Detailed step

- ▣ Allocate 256 byte
 - ◆ Split the block into A and A'. Puts A' in the 512 byte free list.
 - ◆ Split A into B and B'. Put B' in the 256 byte free list.
 - ◆ Return B to the client
- ▣ Allocate 128 Byte
 - ◆ Split B' into C and C'. Put C' in the free list.
 - ◆ Return C to the client.
- ▣ Allocate 64 byte.
 - ◆ Split c' into D and D'. Put D' in the free list.
 - ◆ Return D to the client.
- ▣ Allocate 128
 - ◆ Split A' into E and E'. Put E' in the free list of 256 byte
 - ◆ Split E into F and F'. Put F' in the free list of 128 byte
 - ◆ Allocate F.

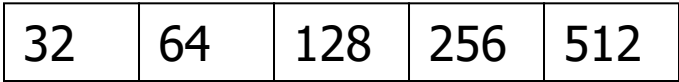
Example

- Maintain freelist for every buffer size(32-512).

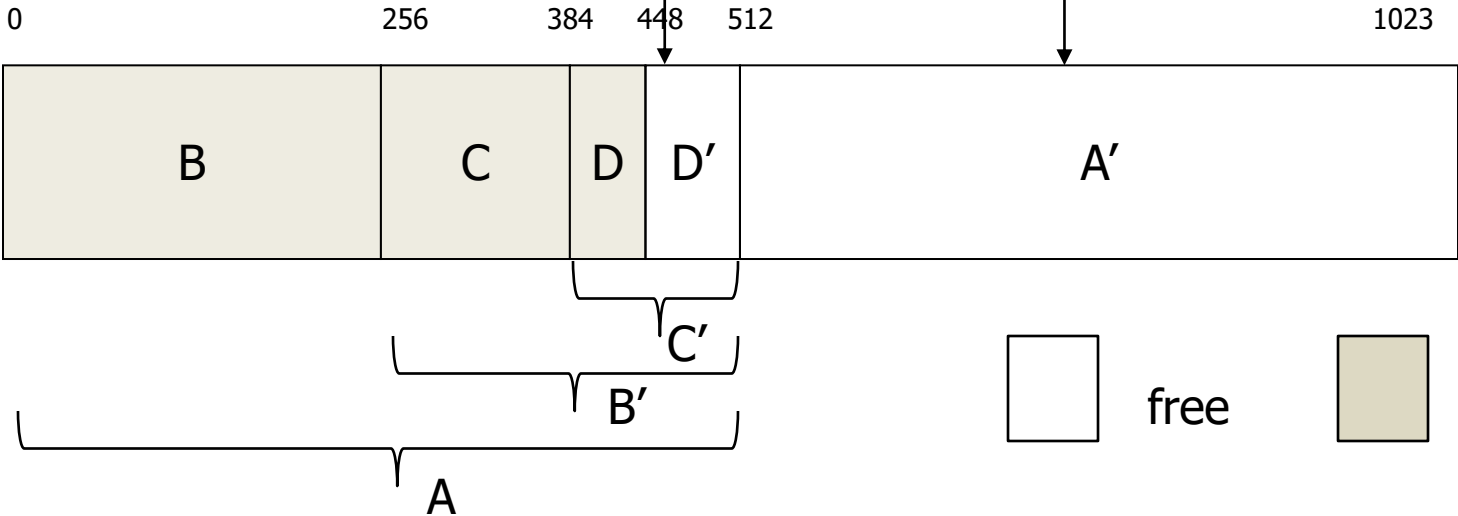
bitmap



Free list headers



Allocate(256)
Allocate(128)
Allocate(64)

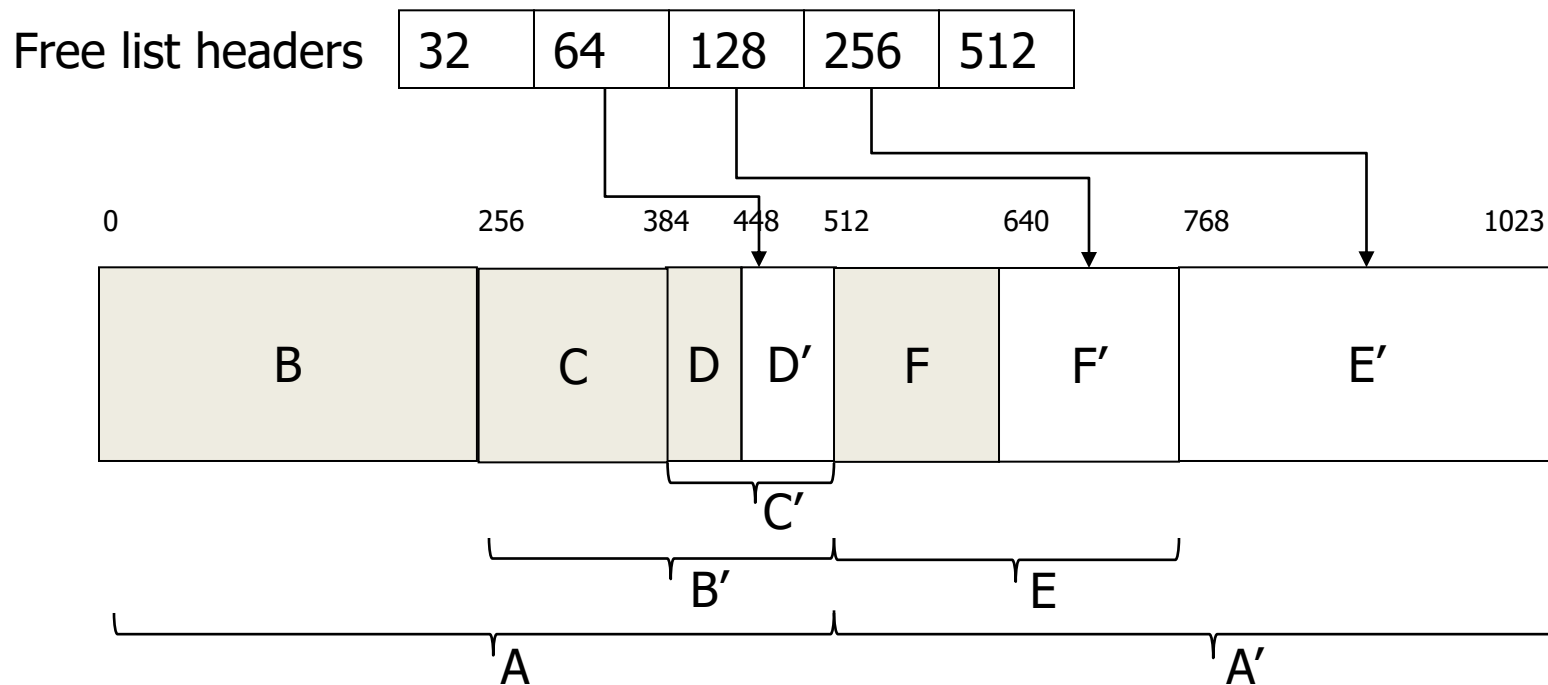


Example

bitmap

1	1	1	1	1	1	1	1	1	0	0	0	0	1	1	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Allocate(128)

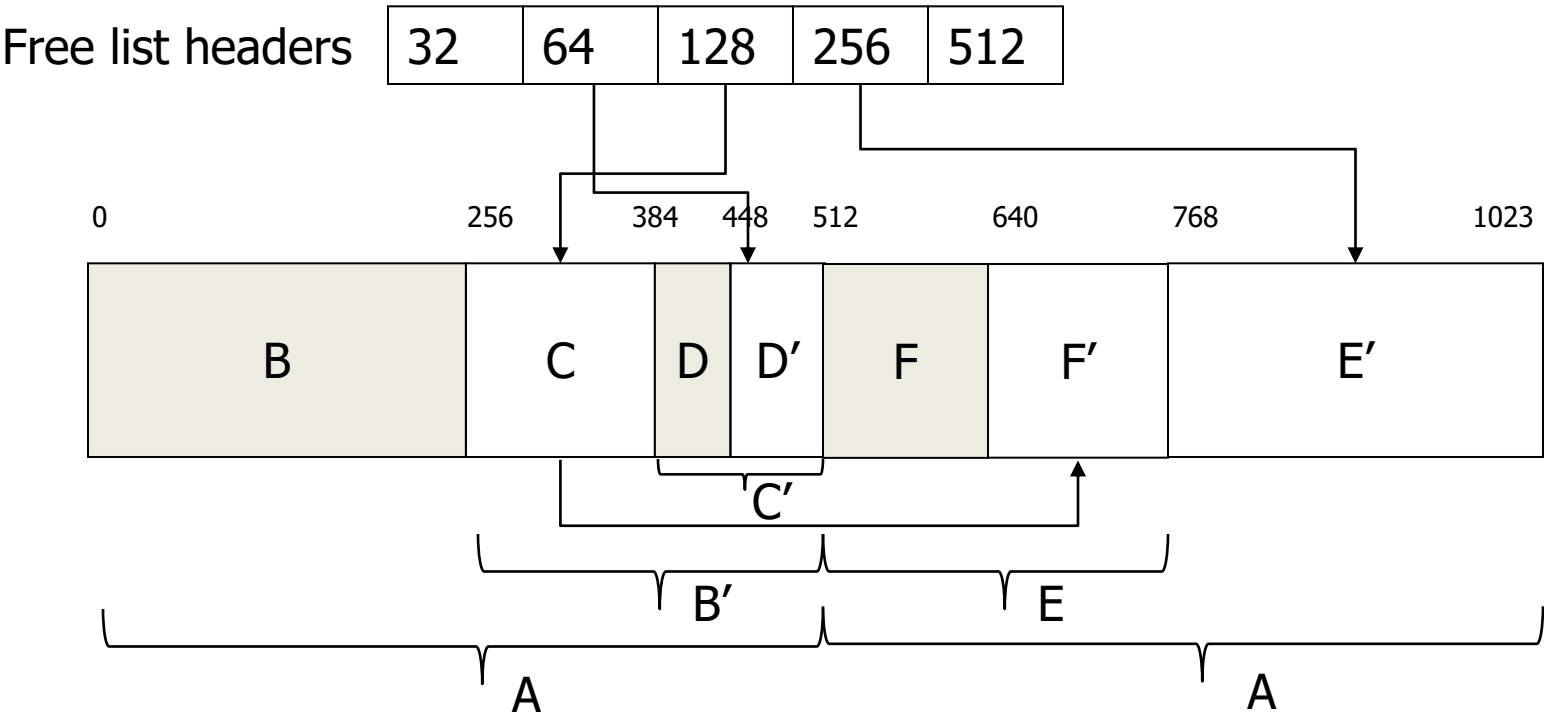


Example

bitmap

1	1	1	1	1	1	1	1	1	0	0	0	0	1	1	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Release(C, 128)

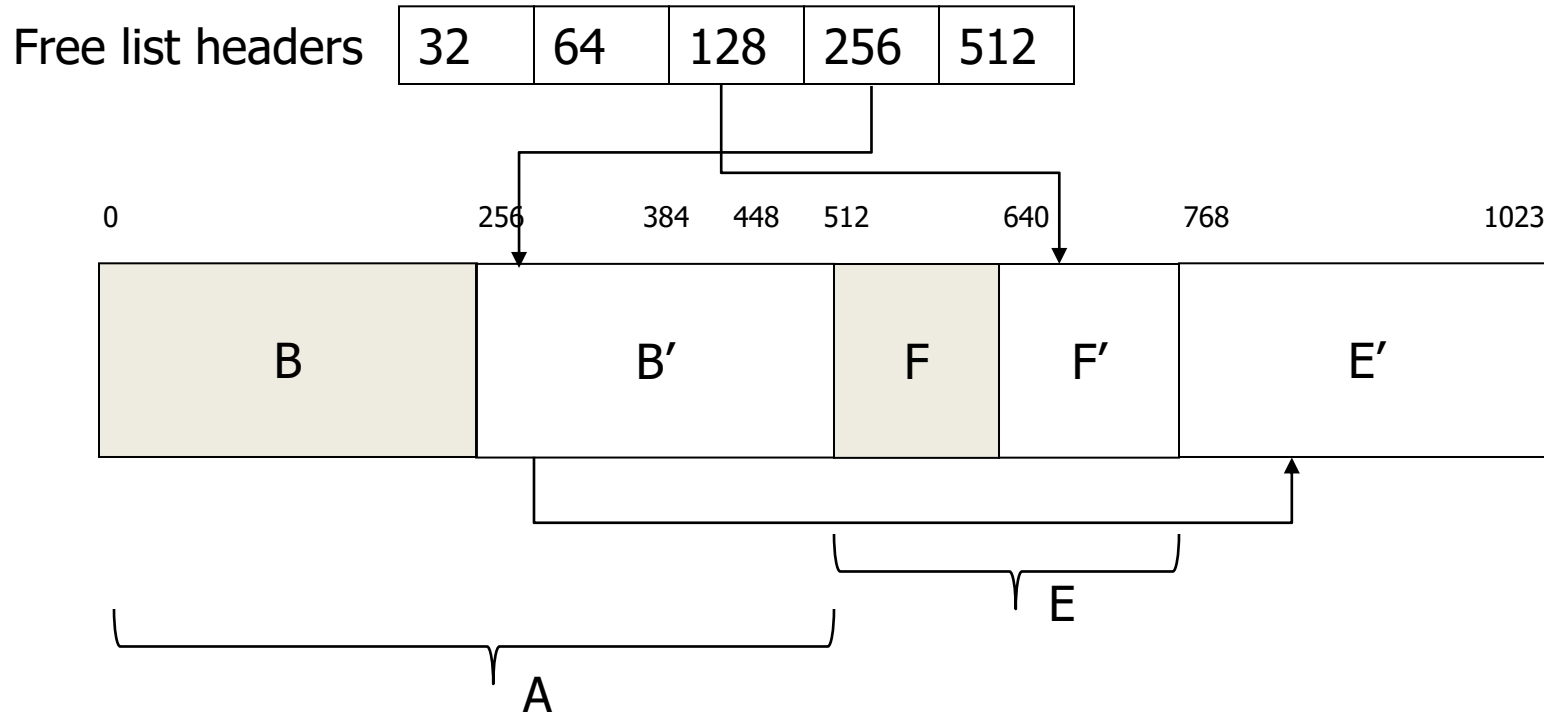


Example

bitmap

1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Release(D, 64)



- Characteristic

- Internal fragmentation by power-of-two allocation
- Easy to find a buddy of a buffer by address and size
- Use bitmap for coalescing.

- Advantage

- Does a good job of coalescing adjacent free buffers.
- Easy exchange of memory between the allocator and the paging system

- Disadvantage

- Performance degrade: every time a buffer is released, the allocator tries to coalesce as much as possible.
- Release routine needs both the address and size of the buffer.
- Partial release is insufficient.

Slab allocator

- ❑ Advanced form of segregated list
- ❑ Slab: a set of kernel pages
 - ◆ Consists of same objects, e.g. inodes, locks, sockets
 - ◆ They are all initialized before allocation.
- ❑ Using object
 - ◆ Allocate memory and Construct the object
 - ◆ Use the object.
 - ◆ Deconstruct it, Free the memory.
- ❑ Construction
 - ◆ Initialization of various fields
- ❑ Object reuse
 - ◆ It is better to reuse possible data structures rather than release

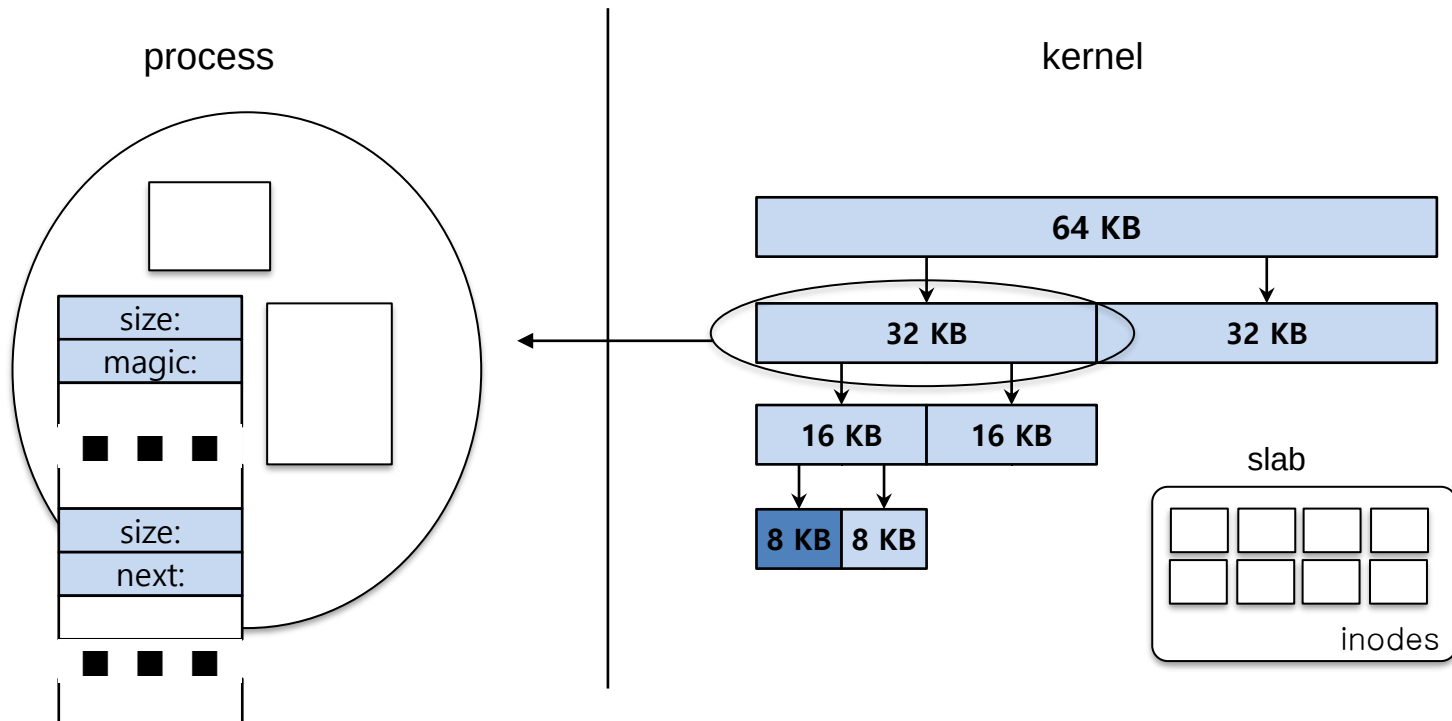
Memory allocation

- ▣ libc

- ◆ Based on linked list

- ▣ kernel

- ◆ Buddy: allocating memory to process
- ◆ Slab: allocating memory for small kernel objects (proc structure, inode, socket and etc.)



Operating Systems

Youjip Won



18. Paging: Introduction

Concept of Paging

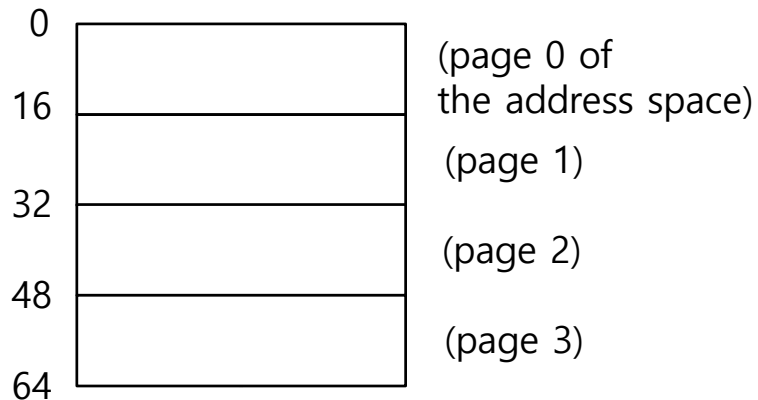
- Paging **splits up** address space into **fixed-sized** unit called a **page**.
 - ◆ Segmentation: variable size of logical segments(code, stack, heap, etc.)
- With paging, **physical memory** is also **split** into some number of pages called a **page frame**.
- **Page table** per process is needed **to translate** the virtual address to physical address.

Advantages Of Paging

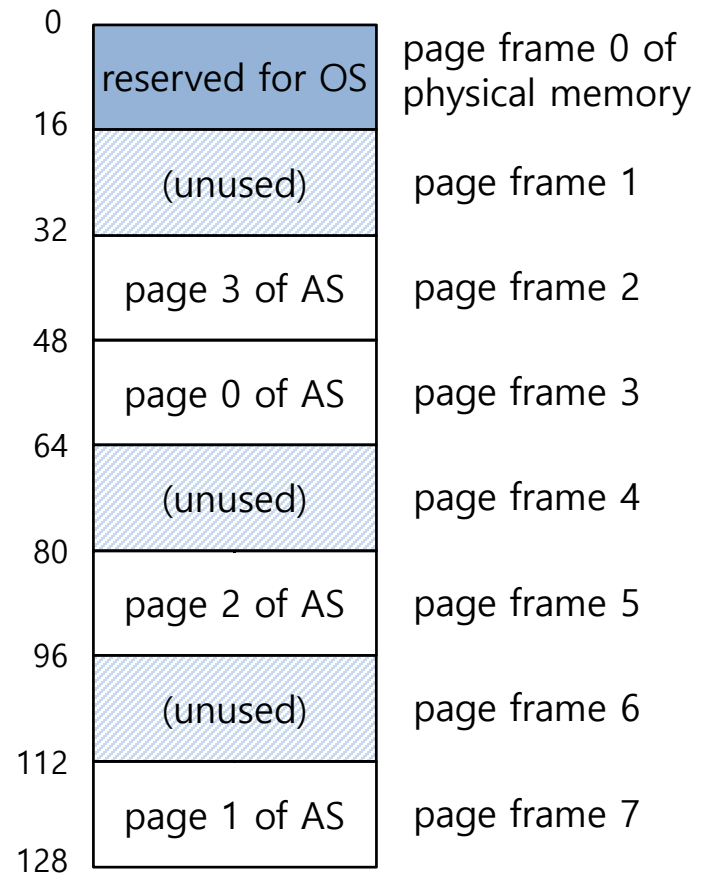
- ▣ **Flexibility:** Supporting the abstraction of address space effectively
 - ◆ Don't need assumption how heap and stack grow and are used.
- ▣ **Simplicity:** ease of free-space management
 - ◆ The page in address space and the page frame are the same size.
 - ◆ Easy to allocate and keep a free list

Example: A Simple Paging

- 128-byte physical memory with 16 bytes page frames
- 64-byte address space with 16 bytes pages



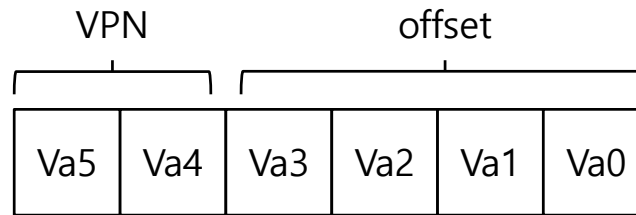
A Simple 64-byte Address Space



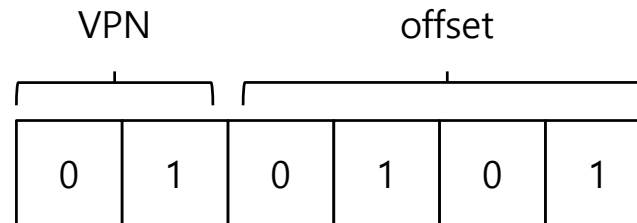
64-Byte Address Space Placed In Physical Memory

Address Translation

- Two components in the virtual address
 - VPN: virtual page number
 - Offset: offset within the page

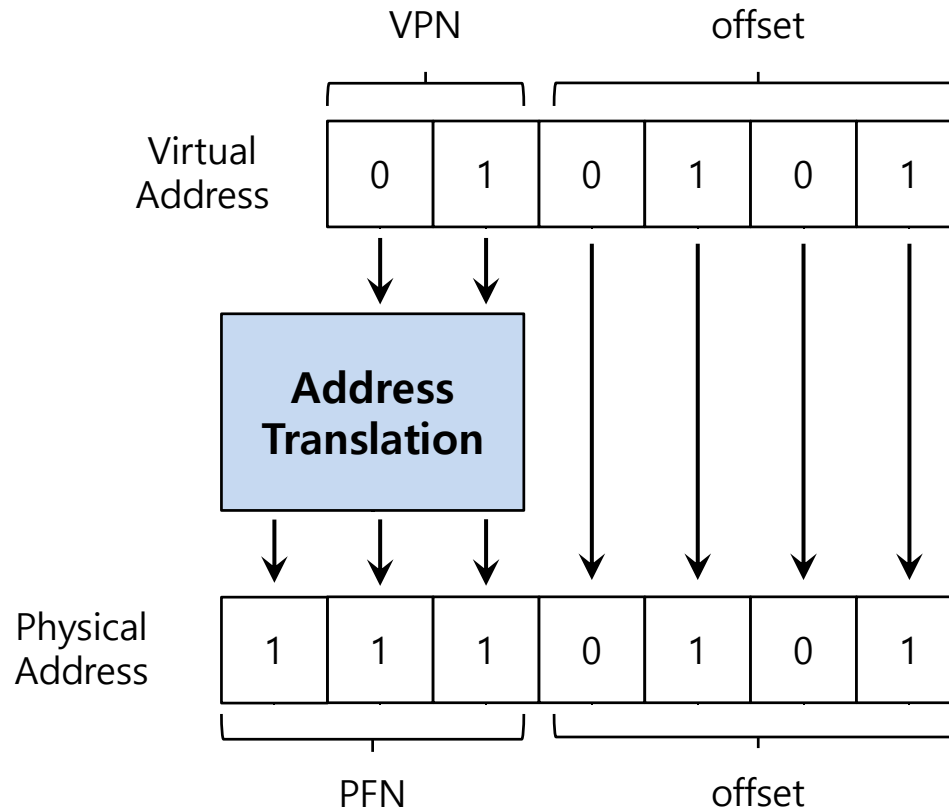


- Example: virtual address 21 in 64-byte address space



Example: Address Translation

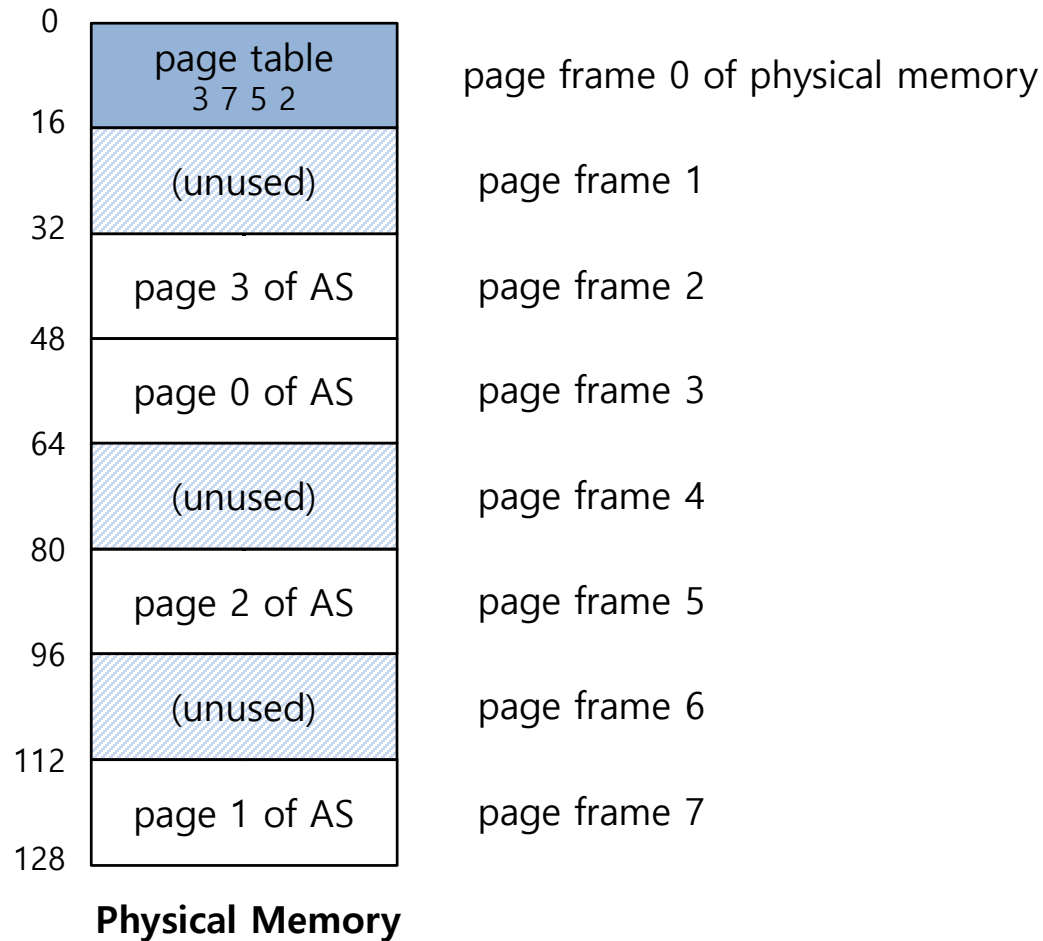
- ▣ The virtual address 21 in 64-byte address space



Where Are Page Tables Stored?

- ❑ Page tables can get awfully large.
 - ◆ 32-bit address space with 4-KB pages, 20 bits for VPN
 - Page offset for 4 Kbyte page: 20 bit
 - $4MB = 2^{20} \text{ entries} * 4 \text{ Bytes per page table entry}$
- ❑ Page tables for each process are stored in memory.

Example: Page Table in Kernel Physical Memory



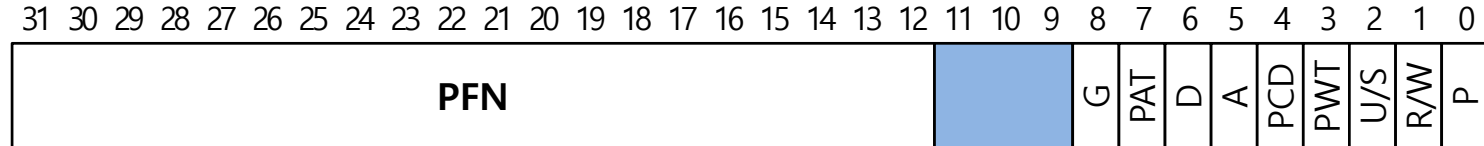
What Is In The Page Table?

- ▣ The page table is a **data structure** that is used to map the virtual address to physical address.
 - ◆ Simplest form: a linear page table, an array
- ▣ The OS **indexes** the array by VPN, and looks up the page-table entry.

Common Flags Of Page Table Entry

- ▣ **Valid Bit:** Indicating whether the particular translation is valid.
- ▣ **Protection Bit:** Indicating whether the page could be read from, written to, or executed from
- ▣ **Present Bit:** Indicating whether this page is in physical memory or on disk(swapped out)
- ▣ **Dirty Bit:** Indicating whether the page has been modified since it was brought into memory
- ▣ **Reference Bit(Accessed Bit):** Indicating that a page has been accessed

Example: x86 Page Table Entry



An x86 Page Table Entry(PTE)

- ▣ P: present
- ▣ R/W: read/write bit
- ▣ U/S: supervisor
- ▣ A: accessed bit
- ▣ D: dirty bit
- ▣ PFN: the page frame number

Paging: Too Slow

- ▣ To find a location of the desired PTE, the **starting location** of the page table is **needed**.
- ▣ For every memory reference, paging requires the OS to perform one **extra memory reference**.

Accessing Memory With Paging

```
1      // Extract the VPN from the virtual address
2      VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4      // Form the address of the page-table entry (PTE)
5      PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7      // Fetch the PTE
8      PTE = AccessMemory(PTEAddr)
9
```


Accessing Memory With Paging

```
10    // Check if process can access the page
11    if (PTE.Valid == False)
12        RaiseException(SEGMENTATION_FAULT)
13    else if (CanAccess(PTE.ProtectBits) == False)
14        RaiseException(PROTECTION_FAULT)
15    else
16        // Access is OK: form physical address and fetch it
17        offset = VirtualAddress & OFFSET_MASK
18        PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19        Register = AccessMemory(PhysAddr)
```

A Memory Trace

```
int array[1000];  
...  
for (i = 0; i < 1000; i++)  
    array[i] = 0;
```

```
prompt> gcc -o array array.c -Wall -o  
prompt> ./array
```

Memory access

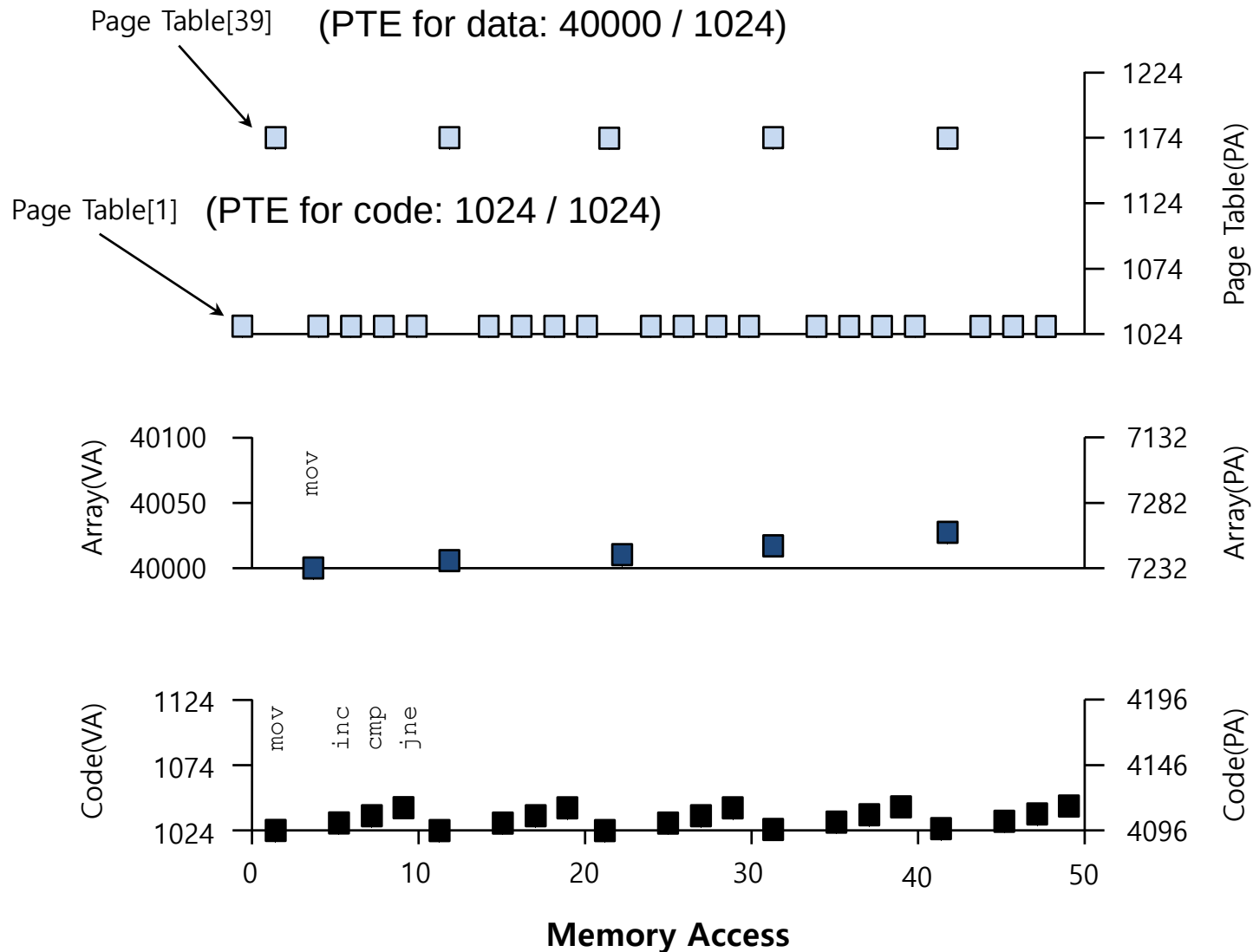
```
0x1024 movl $0x0, (%edi,%eax,4) //[edi+eax*4]= 0
```

```
0x1028 incl %eax
```

```
0x102c cmpl $0x03e8,%eax //0000 0011 1110 10002 = 100010
```

```
0x1030 jne 0x1024
```

A Virtual(And Physical) Memory Trace



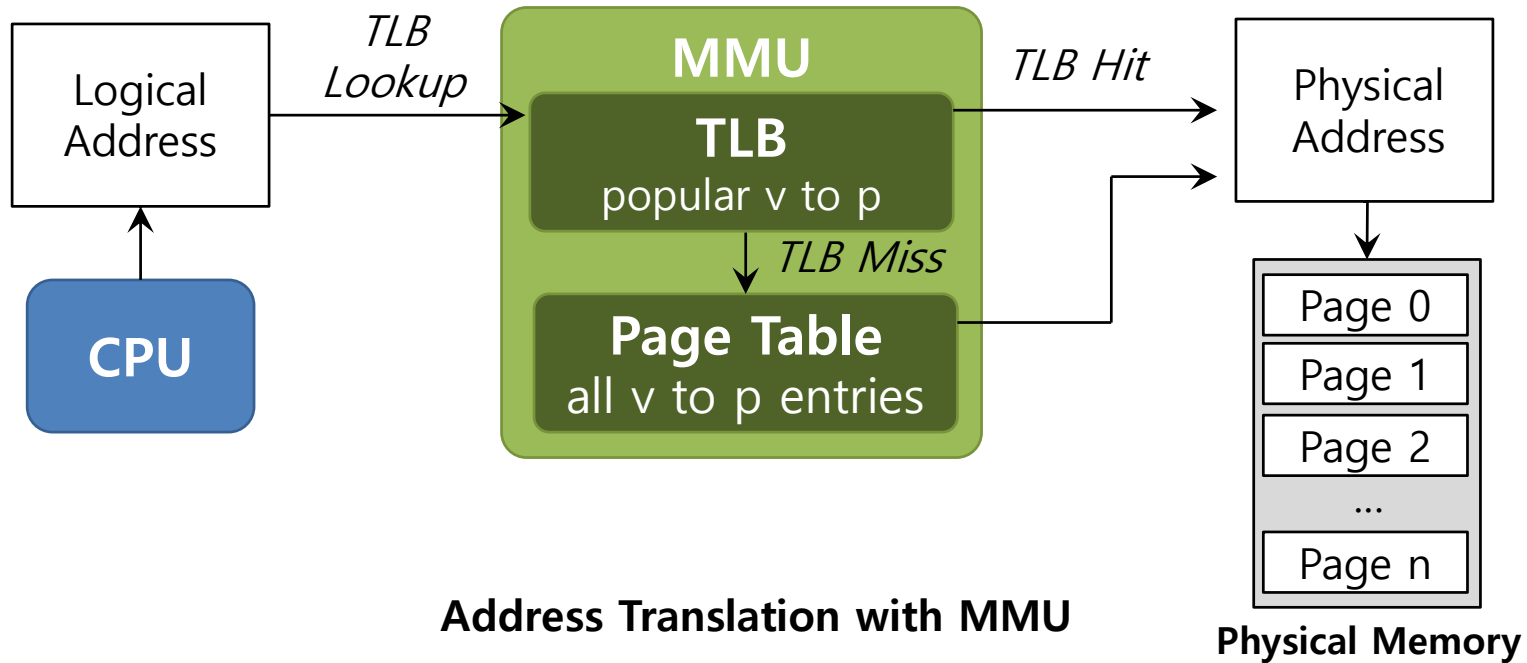
Operating Systems

Youjip Won



19. Translation Lookaside Buffer

- Part of the chip's memory-management unit(MMU).
- A hardware cache of **popular** virtual-to-physical address translation.



TLB Basic Algorithms

```
1: VPN = (VirtualAddress & VPN_MASK ) >> SHIFT
2: (Success , TlbEntry) = TLB_Lookup(VPN)
3: if (Success == TRUE) { // TLB Hit
4:     if (CanAccess(TlbEntry.ProtectBit) == True ) {
5:         offset = VirtualAddress & OFFSET_MASK
6:         PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:         AccessMemory( PhysAddr )
8:     }else RaiseException(PROTECTION_ERROR)
```

TLB Basic Algorithms (Cont.)

```
11:      }else{ //TLB Miss
12:          PTEAddr = PTBR + (VPN * sizeof(PTE))
13:          PTE = AccessMemory(PTEAddr)
14:          if(PTE.Valid == False)
15:              RaiseException(SEGFAULT) ;
16:          else{
17:              TLB_Insert( VPN , PTE.PFN , PTE.ProtectBits)
18:              RetryInstruction()
19:          }
```

- ◆ (11-12 lines) The hardware accesses the page table to find the translation.
- ◆ (16 lines) updates the TLB with the translation.

Example: Accessing An Array

- How a TLB can improve its performance.

	OFFSET				
	00	04	08	12	16
VPN = 00					
VPN = 01					
VPN = 03					
VPN = 04					
VPN = 05					
VPN = 06		a[0]	a[1]	a[2]	
VPN = 07	a[3]	a[4]	a[5]	a[6]	
VPN = 08	a[7]	a[8]	a[9]		
VPN = 09					
VPN = 10					
VPN = 11					
VPN = 12					
VPN = 13					
VPN = 14					
VPN = 15					

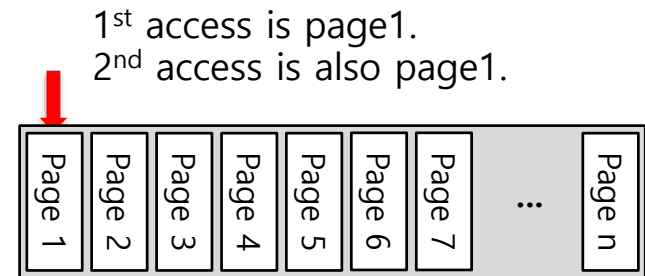
```
0:  int sum = 0 ;
1:  for( i=0; i<10; i++) {
2:      sum+=a[i] ;
3:  }
```

The TLB improves performance
due to **spatial locality**

3 misses and 7 hits.
Thus **TLB hit rate** is 70%.

▣ Temporal Locality

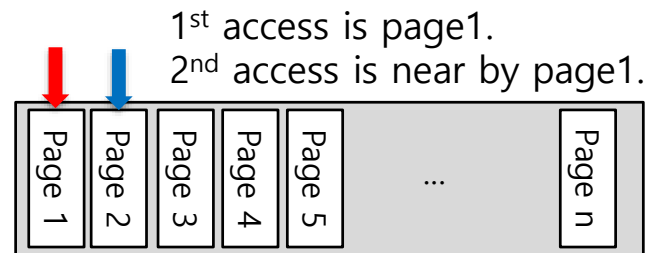
- ◆ An instruction or data item that has been recently accessed will likely be re-accessed soon in the future.



Virtual Memory

▣ Spatial Locality

- ◆ If a program accesses memory at address x , it will likely soon access memory near x .



Virtual Memory

Who Handles The TLB Miss?

- ❑ Hardware handles the TLB miss entirely on CISC.
 - ◆ The hardware has to know exactly where the page tables are located in memory.
 - ◆ The hardware would “walk” the page table, find the correct page-table entry and extract the desired translation, update and retry instruction.
 - ◆ hardware-managed TLB.
 - ◆ Intel x86
- ❑ RISC has what is known as a software-managed TLB.
 - ◆ On a TLB miss, the hardware raises exception (trap handler).
 - Trap handler is code within the OS that is written with the express purpose of handling TLB miss.

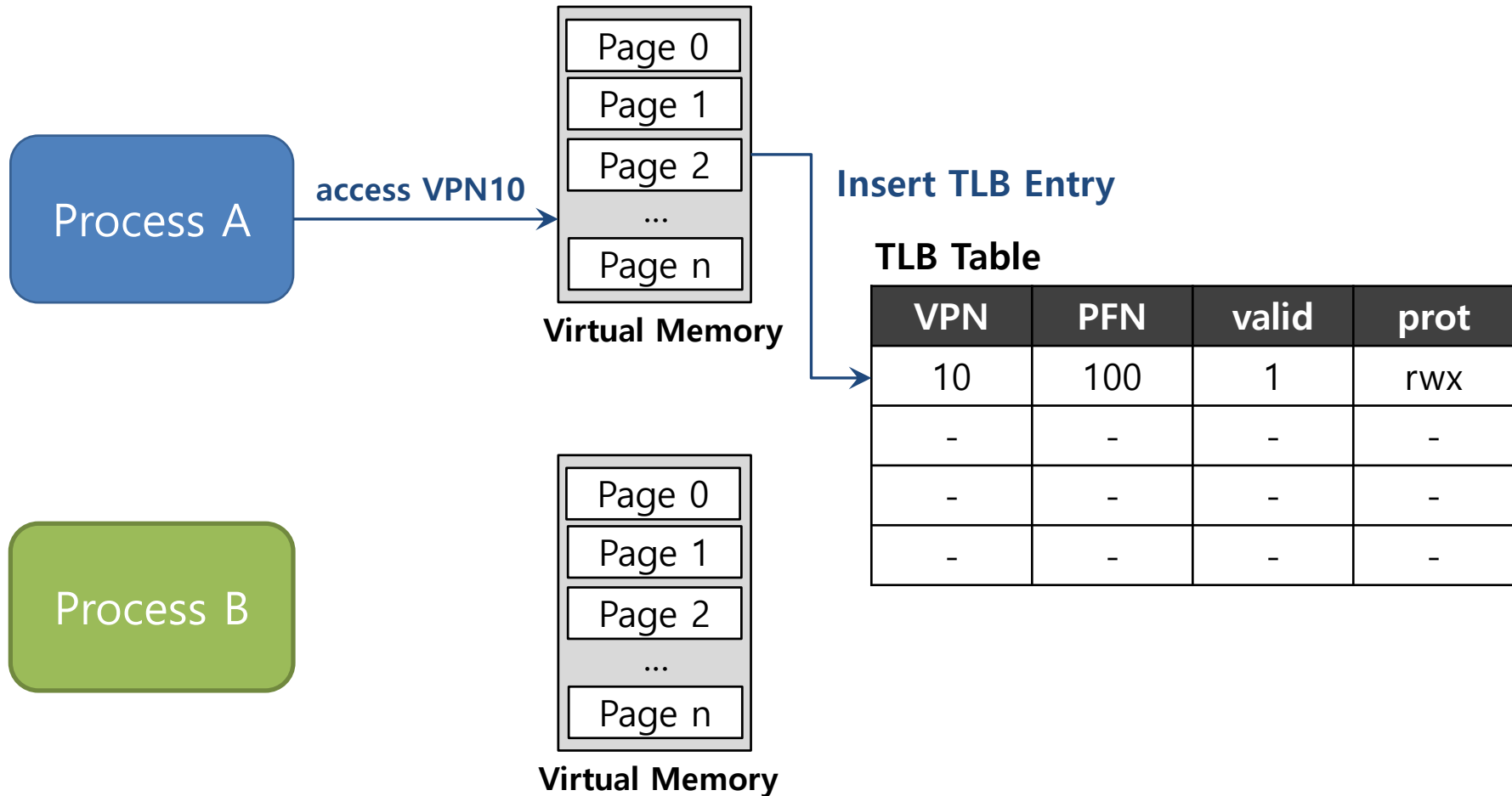
TLB Control Flow algorithm(OS Handled)

```
1:      VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2:      (Success, TlbEntry) = TLB_Lookup(VPN)
3:      if (Success == True) // TLB Hit
4:      if (CanAccess(TlbEntry.ProtectBits) == True)
5:          Offset = VirtualAddress & OFFSET_MASK
6:          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:          Register = AccessMemory(PhysAddr)
8:      else
9:          RaiseException(PROTECTION_FAULT)
10:     else // TLB Miss
11:         RaiseException(TLB_MISS)
```

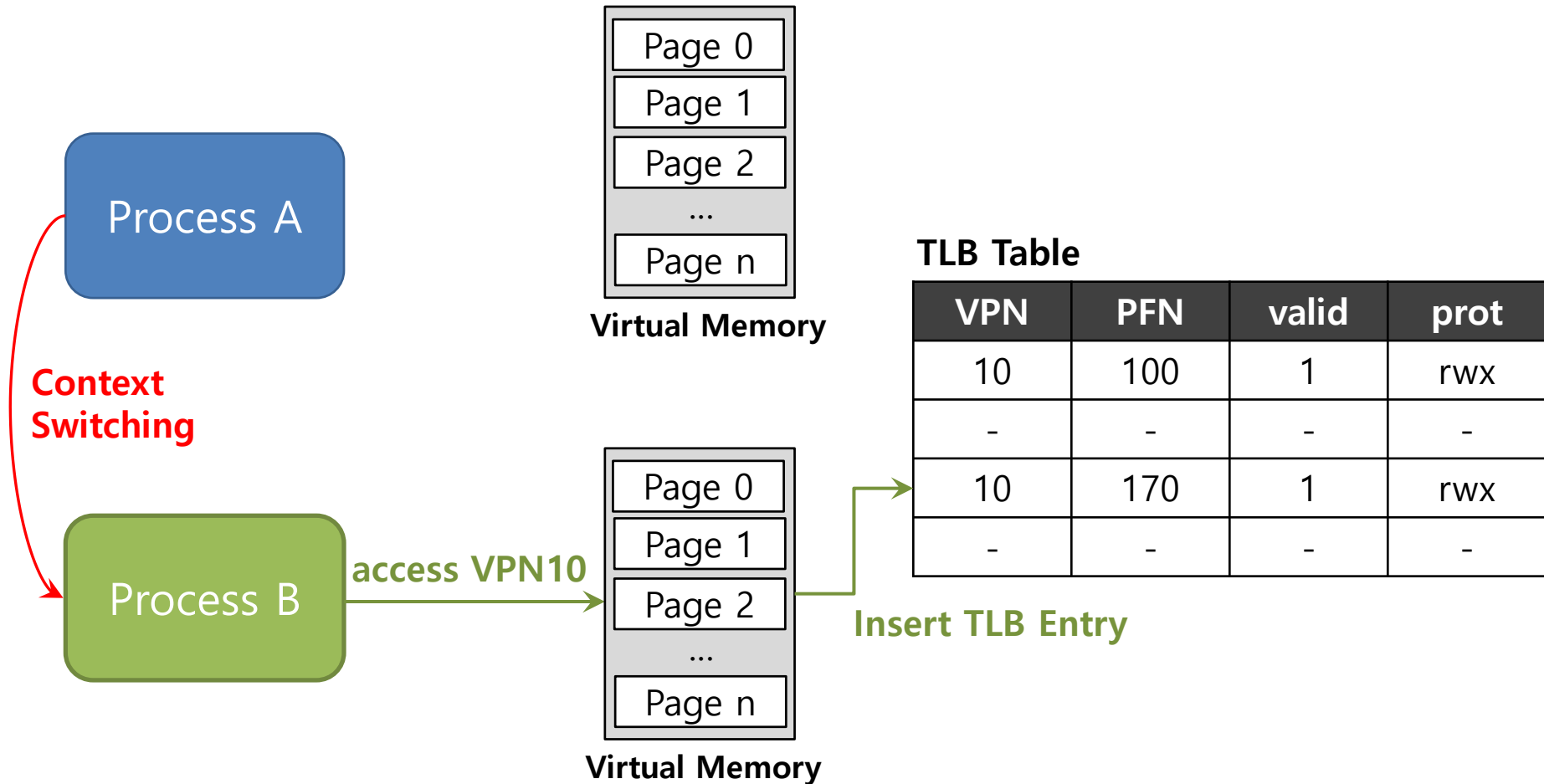
- ▣ TLB is managed by **Full Associative** method.
 - ◆ A typical TLB has 32,64, or 128 entries.
 - ◆ Hardware searches the entire TLB in parallel to find the desired translation.
 - ◆ other bits: valid bits , protection bits, address-space identifier, dirty bit



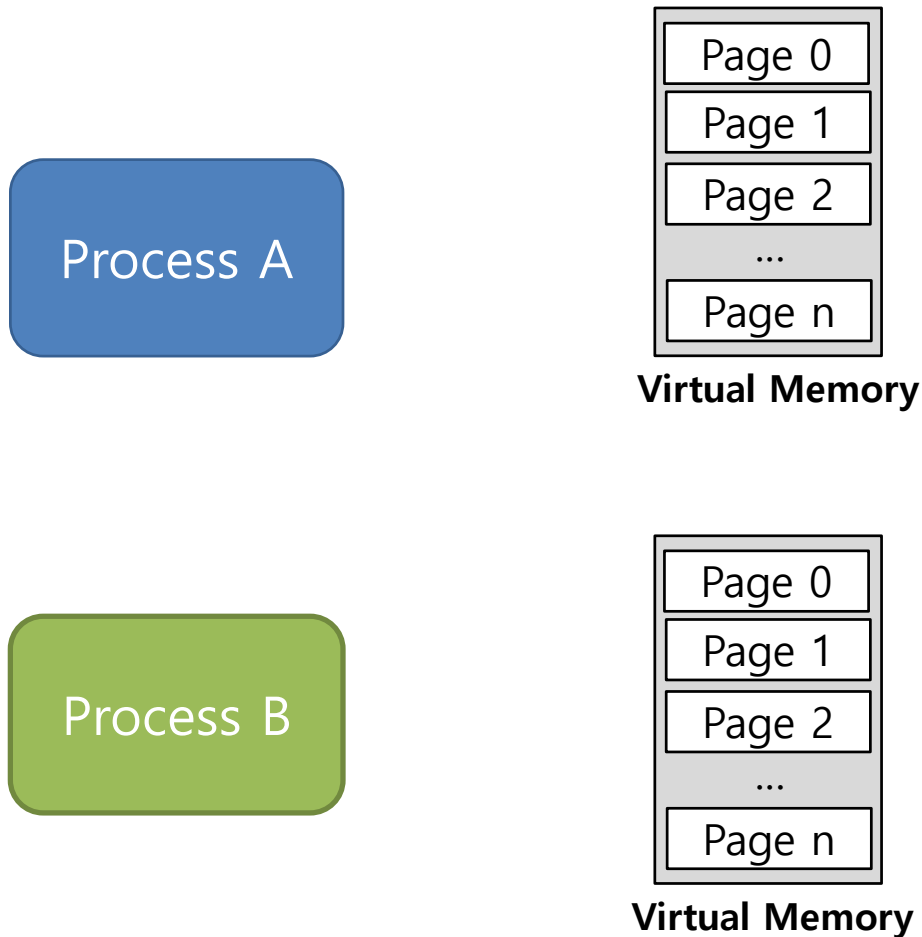
TLB Issue: Context Switching



TLB Issue: Context Switching



TLB Issue: Context Switching



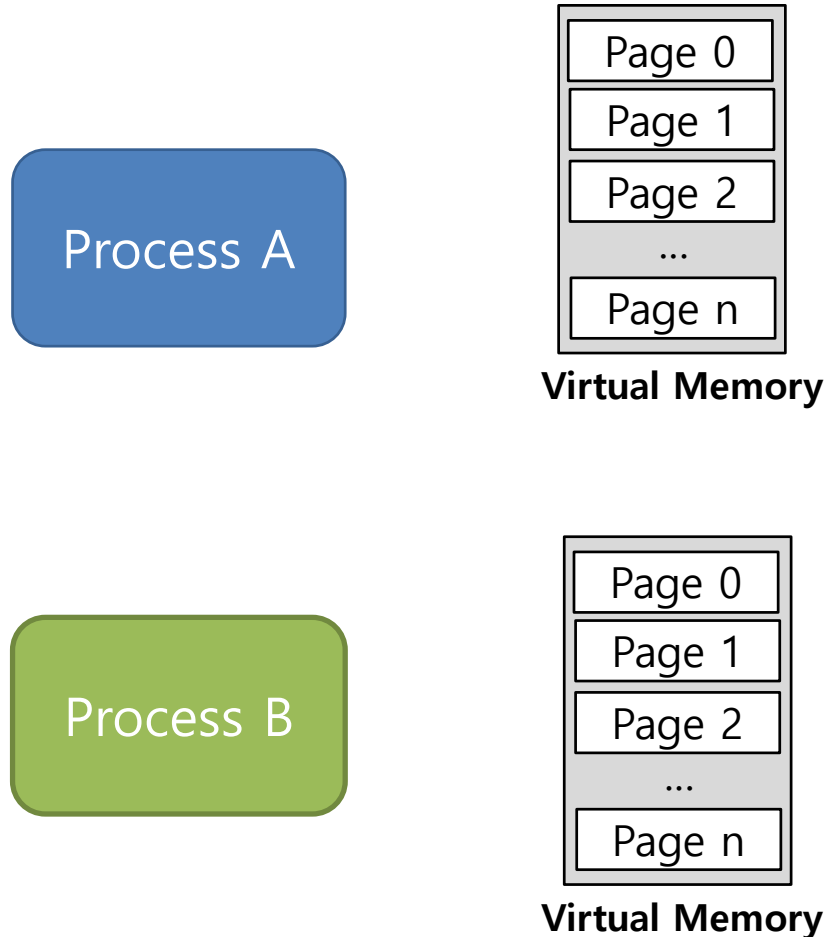
TLB Table

VPN	PFN	valid	prot
10	100	1	rwX
-	-	-	-
10	170	1	rwX
-	-	-	-

Can't Distinguish which entry is meant for which process

To Solve Problem

- Provide an address space identifier(ASID) field in the TLB.



TLB Table

VPN	PFN	valid	prot	ASID
10	100	1	rwX	1
-	-	-	-	-
10	170	1	rwX	2
-	-	-	-	-

Another Case

- Two processes **share a page**.
 - Process 1 is sharing physical page 101 with Process2.
 - P1 maps this page into the 10th page of its address space.
 - P2 maps this page to the 50th page of its address space.

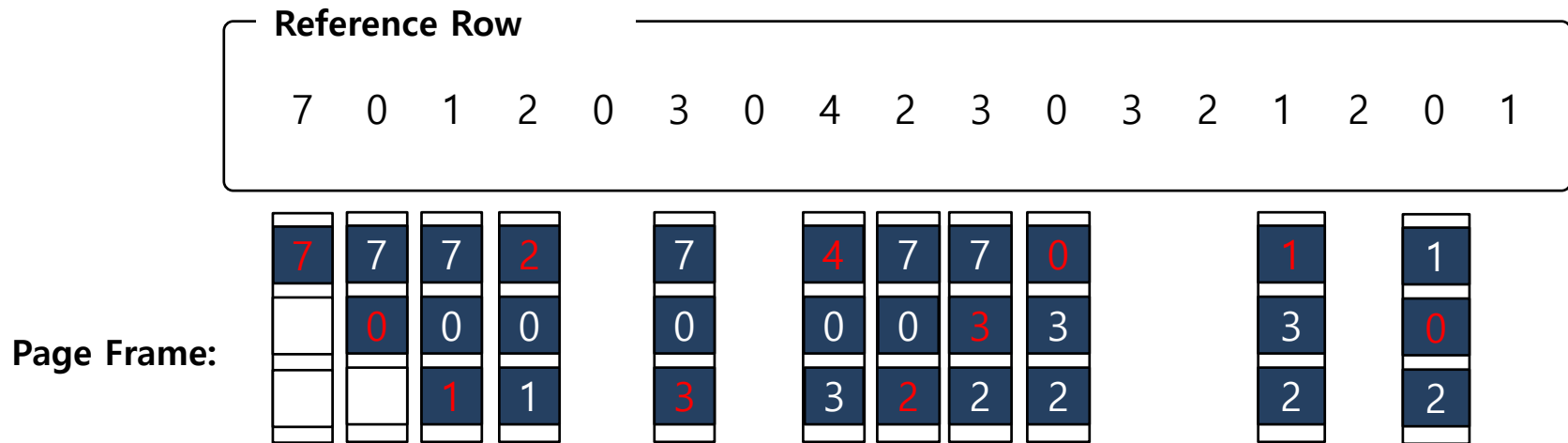
VPN	PFN	valid	prot	ASID
10	101	1	rwX	1
-	-	-	-	-
50	101	1	rwX	2
-	-	-	-	-

Sharing of pages is **useful** as it reduces the number of physical pages in use.

TLB Replacement Policy

□ LRU(Least Recently Used)

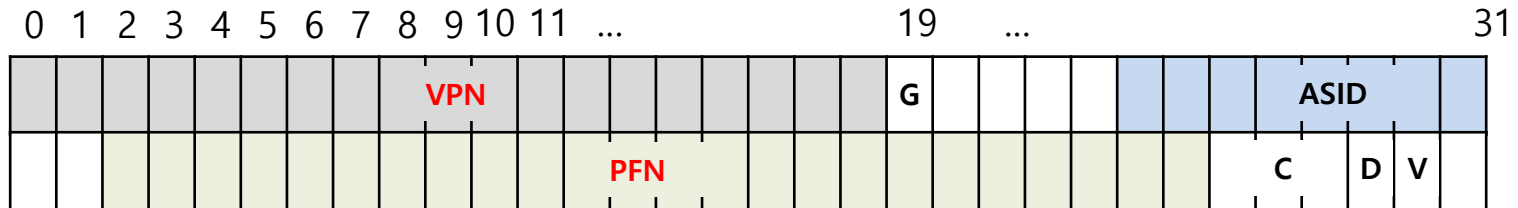
- ◆ Evict an entry that has not recently been used.
- ◆ Take advantage of *locality* in the memory-reference stream.



Total 11 TLB miss

A Real TLB Entry

All 64 bits of this TLB entry(example of MIPS R4000)



Flag	Content
19-bit VPN	The rest reserved for the kernel.
24-bit PFN	Systems can support with up to 64GB of main memory(pages).
Global bit(G)	Used for pages that are globally-shared among processes.
ASID	OS can use to distinguish between address spaces.
Coherence bit(C)	determine how a page is cached by the hardware.
Dirty bit(D)	marking when the page has been written.
Valid bit(V)	tells the hardware if there is a valid translation present in the entry.

Operating Systems

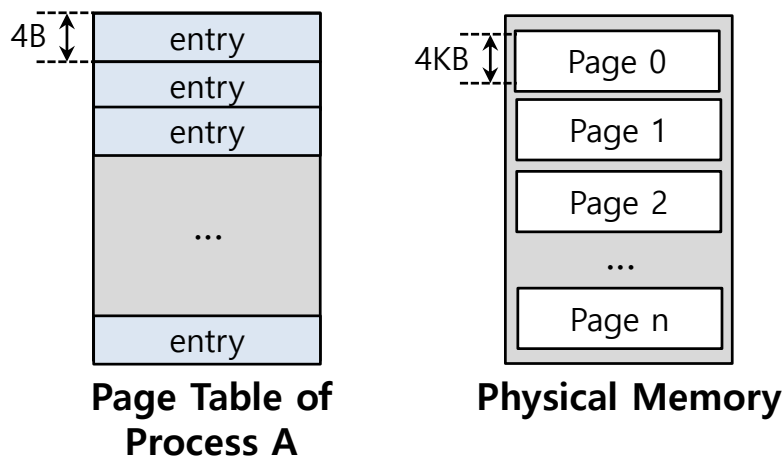
Youjip Won



20. Advanced Page Tables

Paging: Linear Tables

- We usually have one page table for every process in the system.
 - ◆ Assume that 32-bit address space with 4KB pages and 4-byte page-table entry.

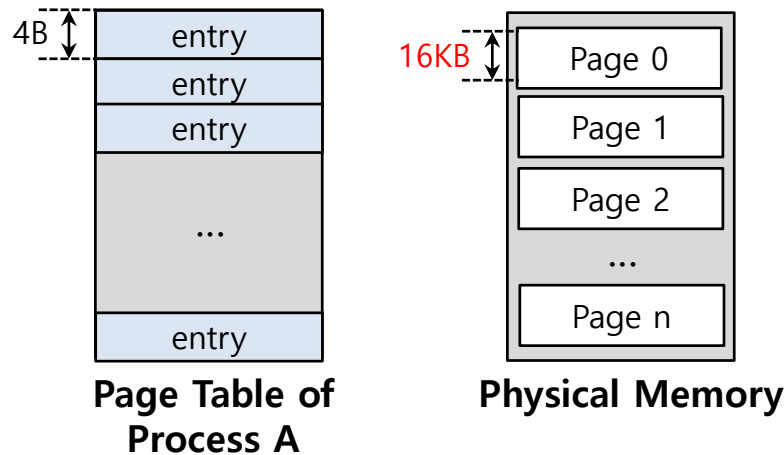


$$\text{Page table size} = \frac{2^{32}}{2^{12}} * 4\text{Byte} = 4\text{MByte}$$

Page table are **too big** and thus consume too much memory.

Paging: Smaller Tables

- ❑ Page tables are too big and thus consume too much memory.
 - ◆ Assume that 32-bit address space with **16KB** pages and 4-byte page-table entry.

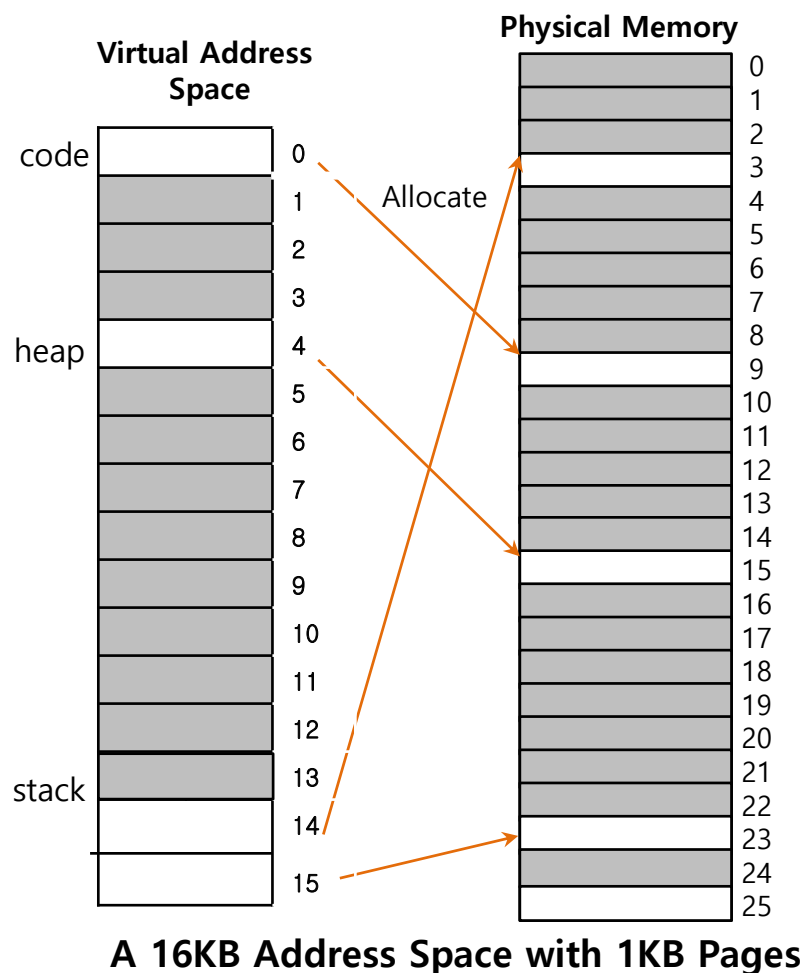


$$\frac{2^{32}}{2^{16}} * 4 = 1MB \text{ per page table}$$

Big pages lead to internal fragmentation.

Problem

- Single page table for the address space of process.

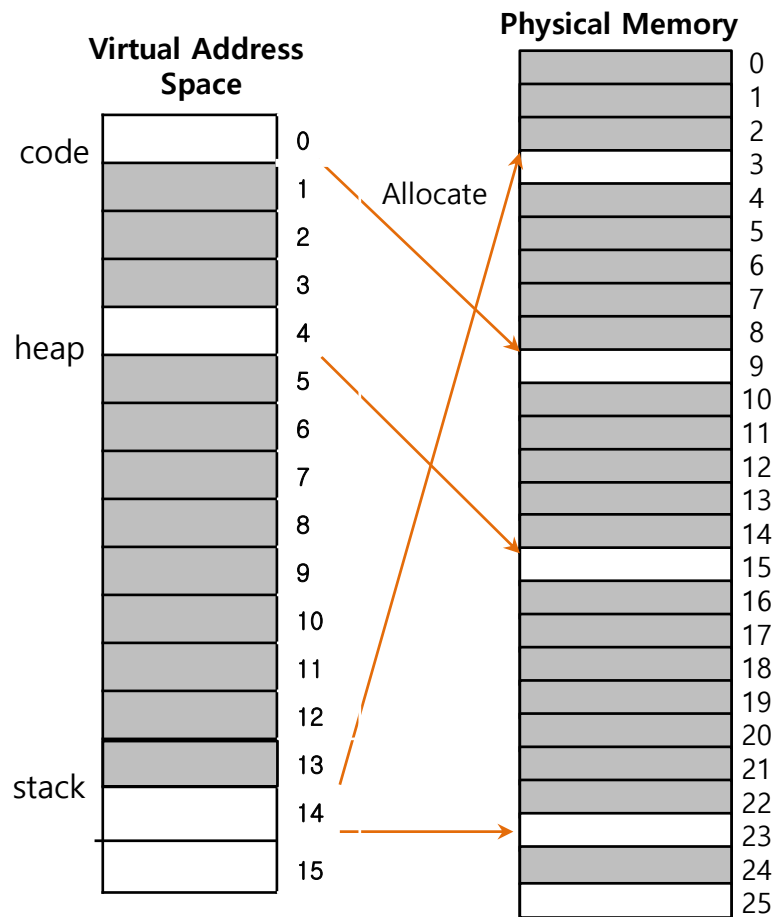


PFN	valid	prot	present	dirty
10	1	r-x	1	0
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
15	1	rw-	1	1
...	...	...	...	...
-	0	-	-	-
3	1	rw-	1	1
23	1	rw-	1	1

A Page Table For 16KB Address Space

Problem

- Most of the page table is **unused**, full of invalid entries.



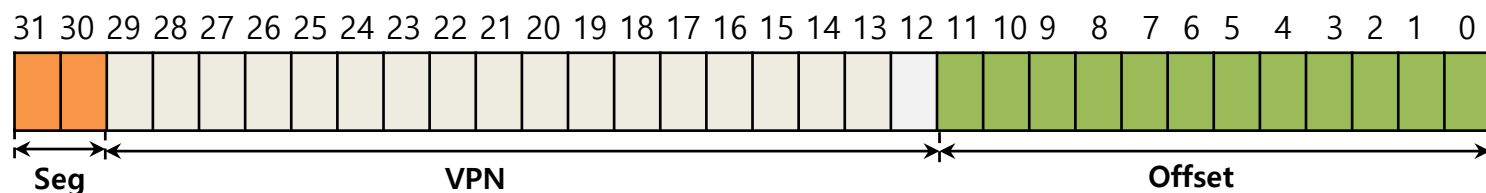
A 16KB Address Space with 1KB Pages

PFN	valid	prot	present	dirty
9	1	r-x	1	0
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
15	1	rw-	1	1
...	...	...	...	...
-	0	-	-	-
3	1	rw-	1	1
23	1	rw-	1	1

A Page Table For 16KB Address Space

Hybrid Approach: Paging and Segments

- Page table for each segment
 - ◆ the base register for each of these segments contains the physical address of a linear page table for that segment.
 - ◆ The bound register: indicate the end of the page table.
- Example: Each process has **three** page tables associated with it.



32-bit Virtual address space with 4KB pages

Seg value	Content
00	unused segment
01	code
10	heap
11	stack

TLB miss on Hybrid Approach

- ▣ The hardware gets **physical address** from **page table**.
 - ◆ The hardware uses the segment bits(SN) to determine which base and bounds pair to use.
 - ◆ The hardware then takes the **physical address** therein and **combines** it with the VPN as follows to form the address of the page table entry(PTE) .

```
01:    SN = (VirtualAddress & SEG_MASK) >> SN_SHIFT
```

```
02:    VPN = (VirtualAddress & VPN_MASK) >> VPN_SHIFT
```

```
03:    AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))
```

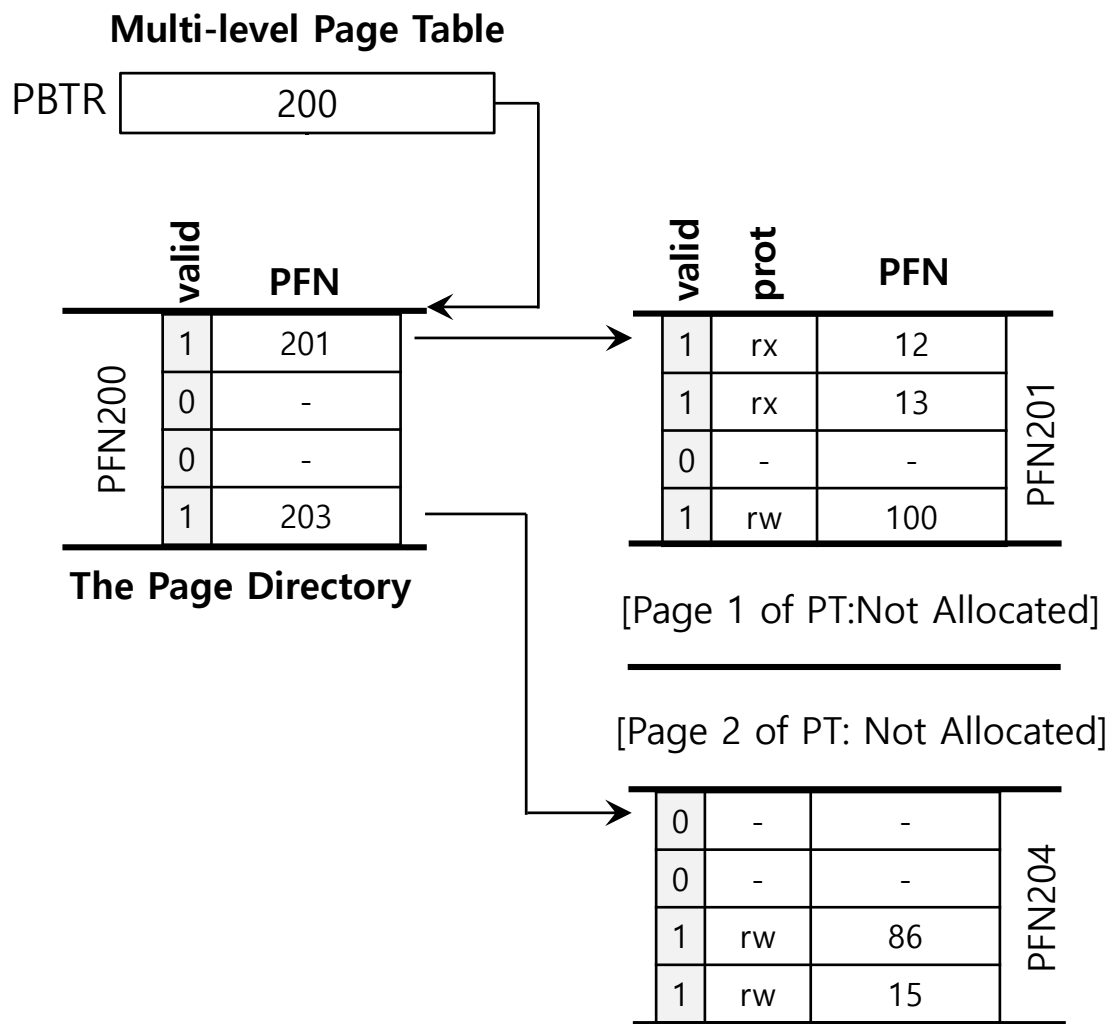
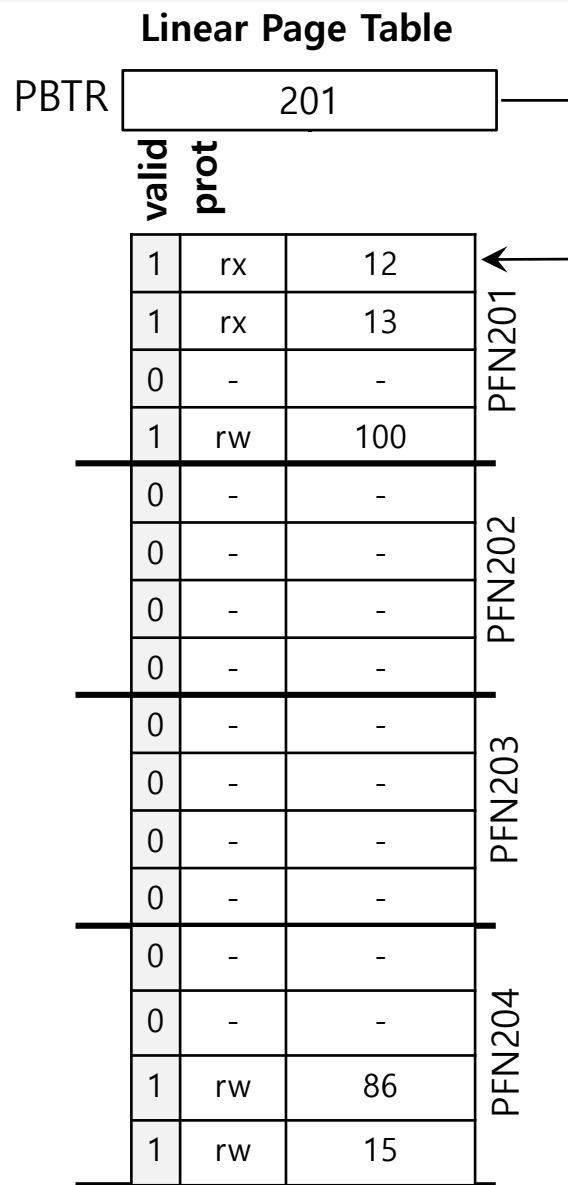
Problem of Hybrid Approach

- ▣ Hybrid Approach is not without problems.
 - ◆ If we have a large but sparsely-used heap, we can still end up with a lot of page table waste.
 - ◆ Causing external fragmentation to arise again.

Multi-level Page Tables

- ▣ Turn the linear page table into something like a tree.
 - ◆ Chop up the page table into page-sized units.
 - ◆ If an entire page of page-table entries is invalid, don't allocate that page of the page table at all.
 - ◆ To track whether a page of the page table is valid, use a new structure, called **page directory**.

Multi-level Page Tables: Page directory



Multi-level Page Tables

▣ Page Directory

- ◆ The page directory contains one entry per page of the page table.
- ◆ It consists of a number of **page directory entries(PDE)**.
- ◆ PDE has a valid bit and page frame number(PFN).

▣ Advantage

- ◆ Only allocates page-table space in proportion to the amount of address space you are using.
- ◆ The OS can grab the next free page when it needs to allocate or grow a page table.

▣ Disadvantage

- ◆ Multi-level table is a small example of a **time-space trade-off**.
- ◆ **Complexity**.

Example

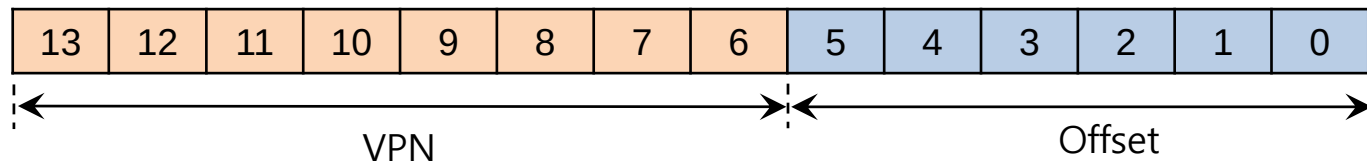
0000 0000	code
0000 0001	code
...	(free)
	(free)
0000 0100	heap
0000 0101	heap
.....	(free)
	(free)
1111 1110	stack
1111 1111	stack

- ▣ Page 0,1: code
- ▣ Page 4,5: heap
- ▣ Page 254, 255: stack

Example

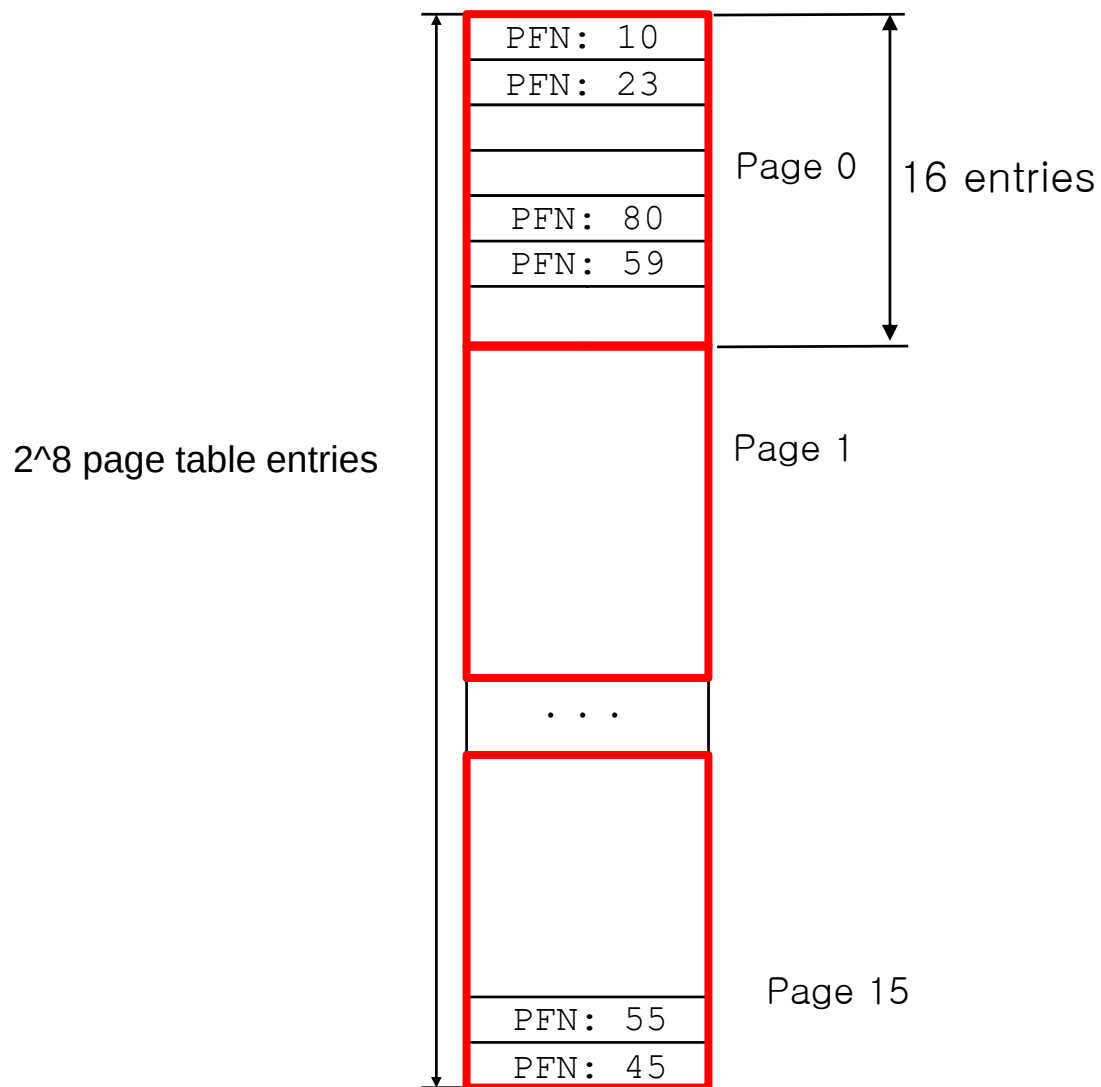
Flag	Detail
Address space	16 KB (2^{14} Byte)
Page size	64 byte
Virtual address	14 bit
VPN	8 bit
Offset	6 bit
Page table entry	4 Byte

A 16-KB Address Space With 64-byte Pages



- Single level paging
 - 256 page table entries: 2^8 entries
 - Page table size: $256 * 4 \text{ Byte} = 1 \text{ Kbyte}$
 - Sixteen pages (64 byte each): $1024/64 = 16$

Example: single level page table

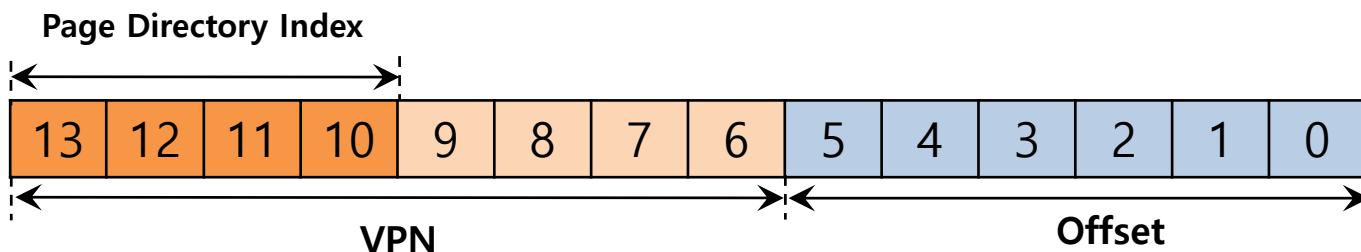


Example: two level paging

▣ Page directory index

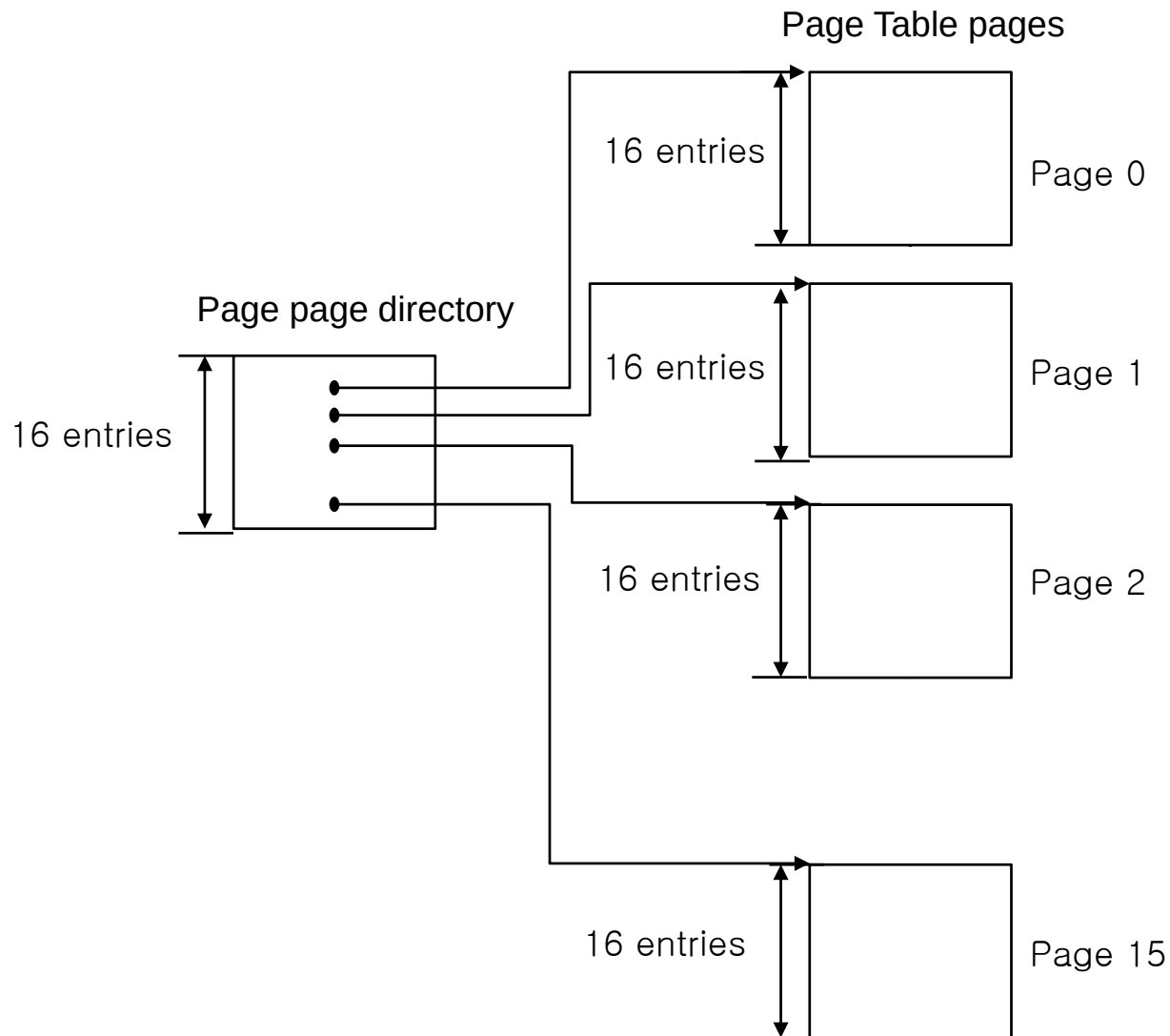
- ◆ A page table consists of 16 pages.
- ◆ 16 entries for page directory: one entry per page of the page table.
- ◆ $16 * 4 \text{ byte} = 64 \text{ byte}$ is required for page directory. → it can fit in a page.
- ◆ 4 bits for page directory index.

$$\text{PDEAddr} = \text{PageDirBase} + (\text{PDIndex} * \text{sizeof(PDE)})$$



- ▣ If the page-directory entry is **invalid**, raise an exception (The access is invalid).

Example: two level paging

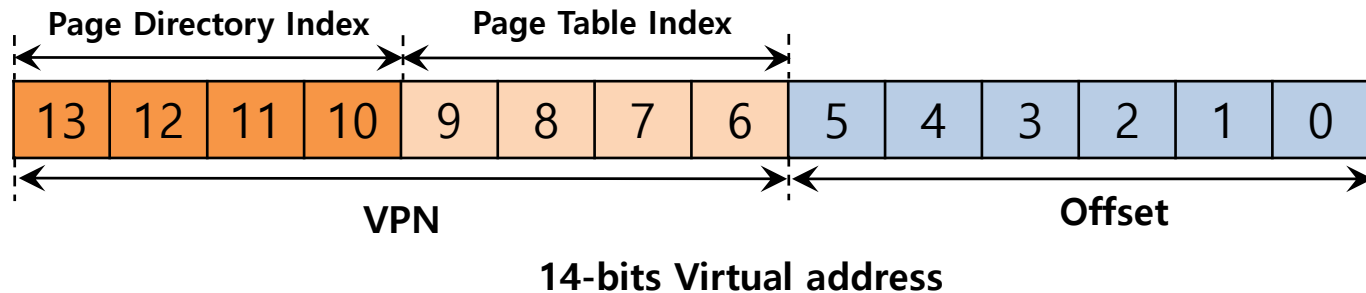


Example: Page Table index

□ Page table index

- ◆ It is used to find the address of the page table entry.

$$\text{PTEAddr} = (\text{PDE.PFN} \ll \text{SHIFT}) + (\text{PTIndex} * \text{sizeof}(\text{PTE}))$$

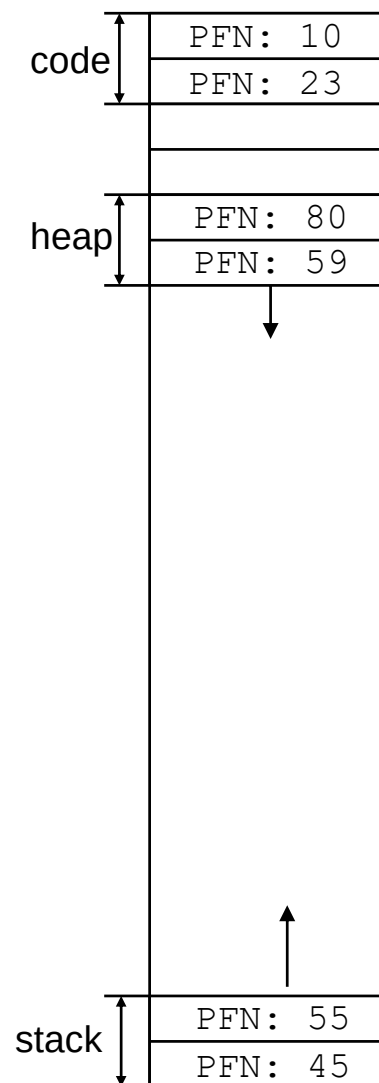


Examples

Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

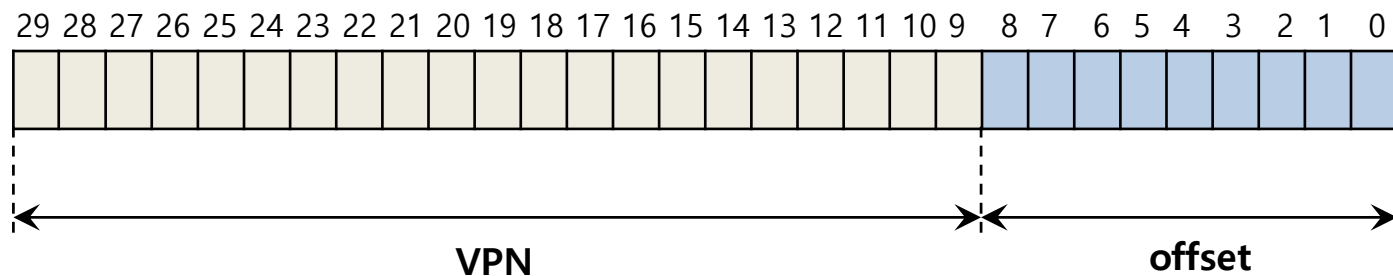
Single level paging: 16 pages

→ Two level paging: 3 pages



More than Two Levels

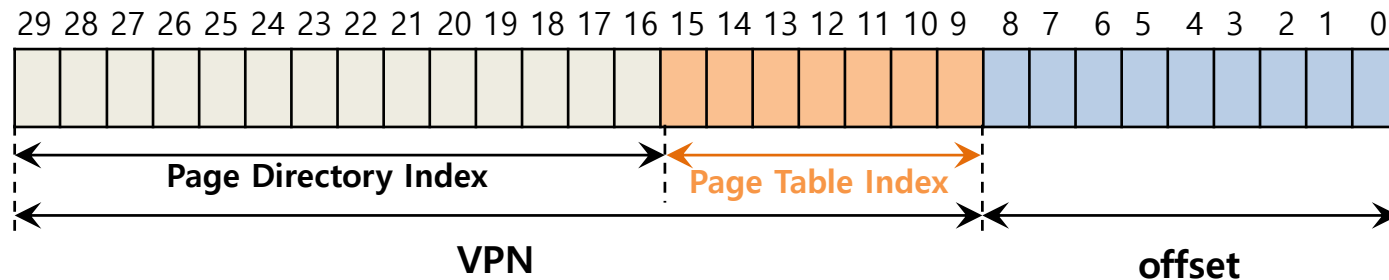
- In some cases, a deeper tree is required.
- Consider the following address space.



Flag	Detail
Virtual address	30 bit
Page size	512 byte
VPN	21 bit
Offset	9 bit

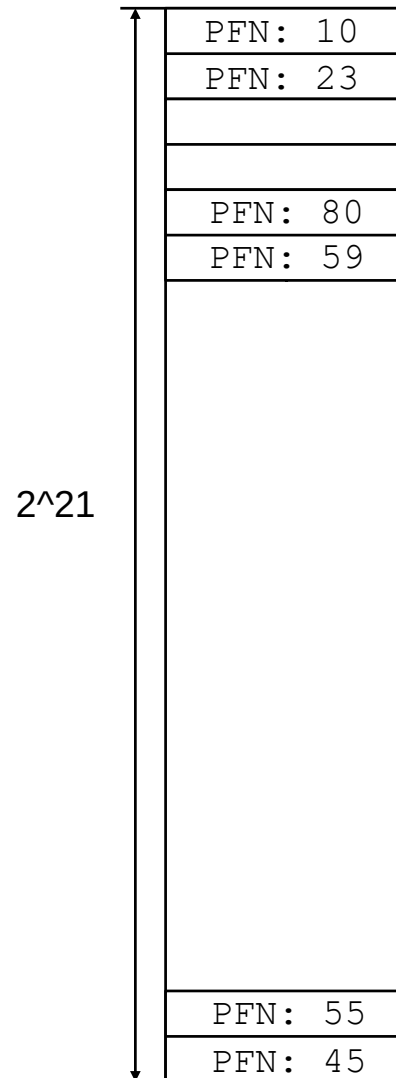
More than Two Level : Page Table Index

- Number of pages in a page table
 - 2^{21} page table entries
 - 2^7 (128) PTE's in a single page (512/4, 512 page size, 4 byte pte size)
 - 2^{14} pages in a page table
- Number of page directory entries: 2^{14}

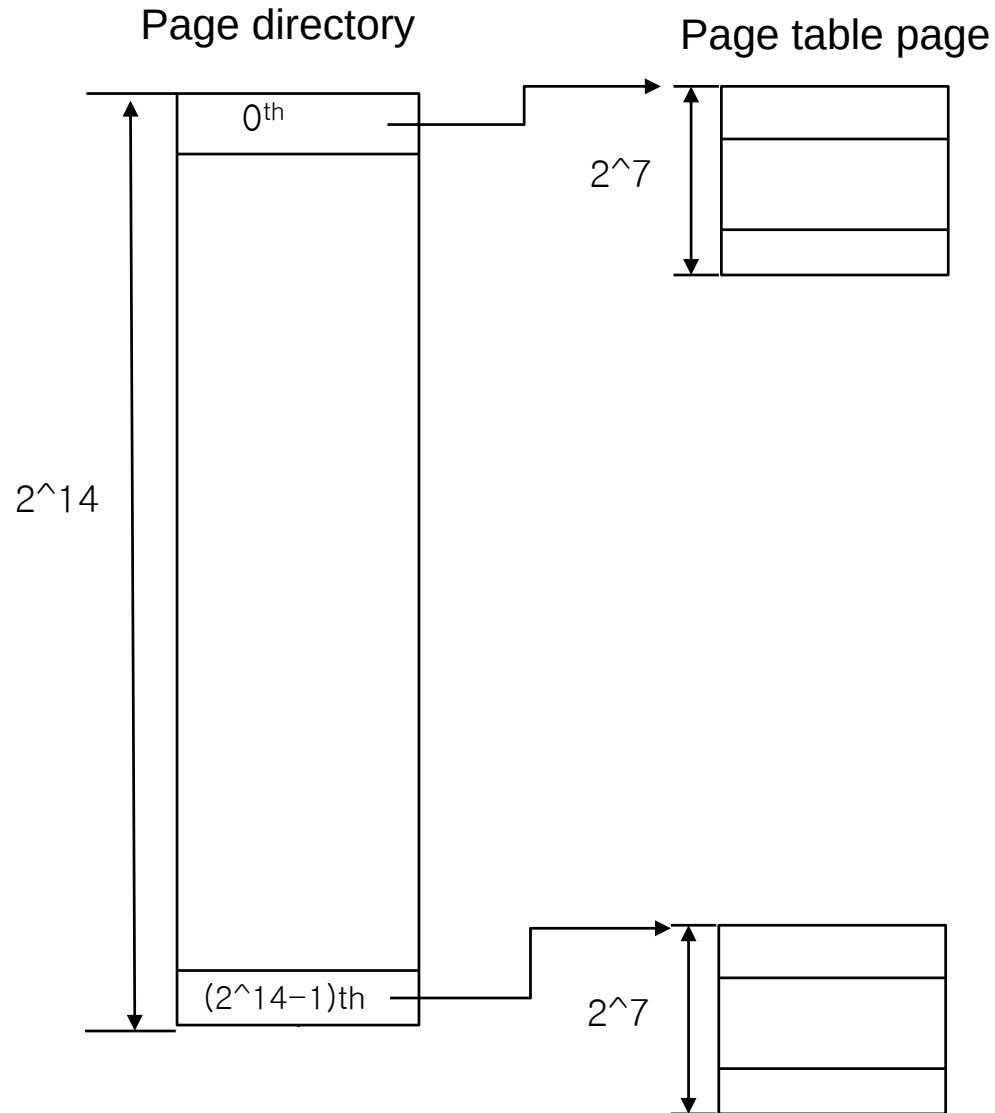


Flag	Detail
Virtual address	30 bit
Page size	512 byte
VPN	21 bit
Offset	9 bit
Page entry per page	128 PTEs (512/4)

Single level page table

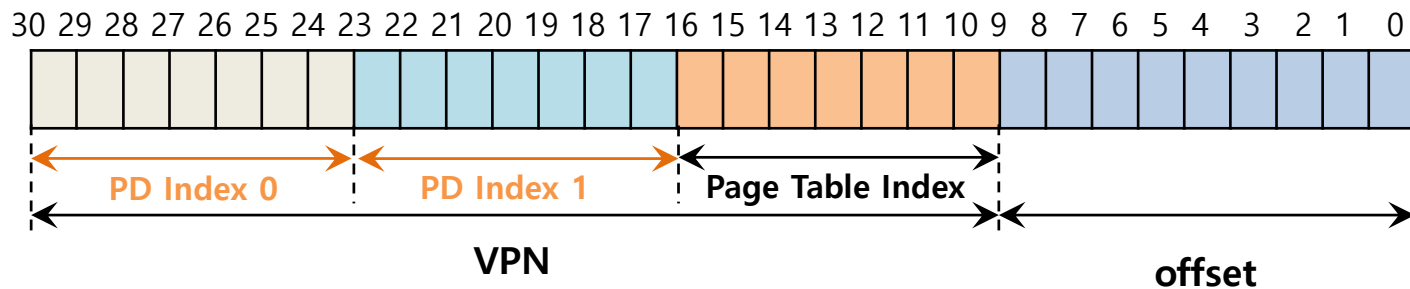


two level page table

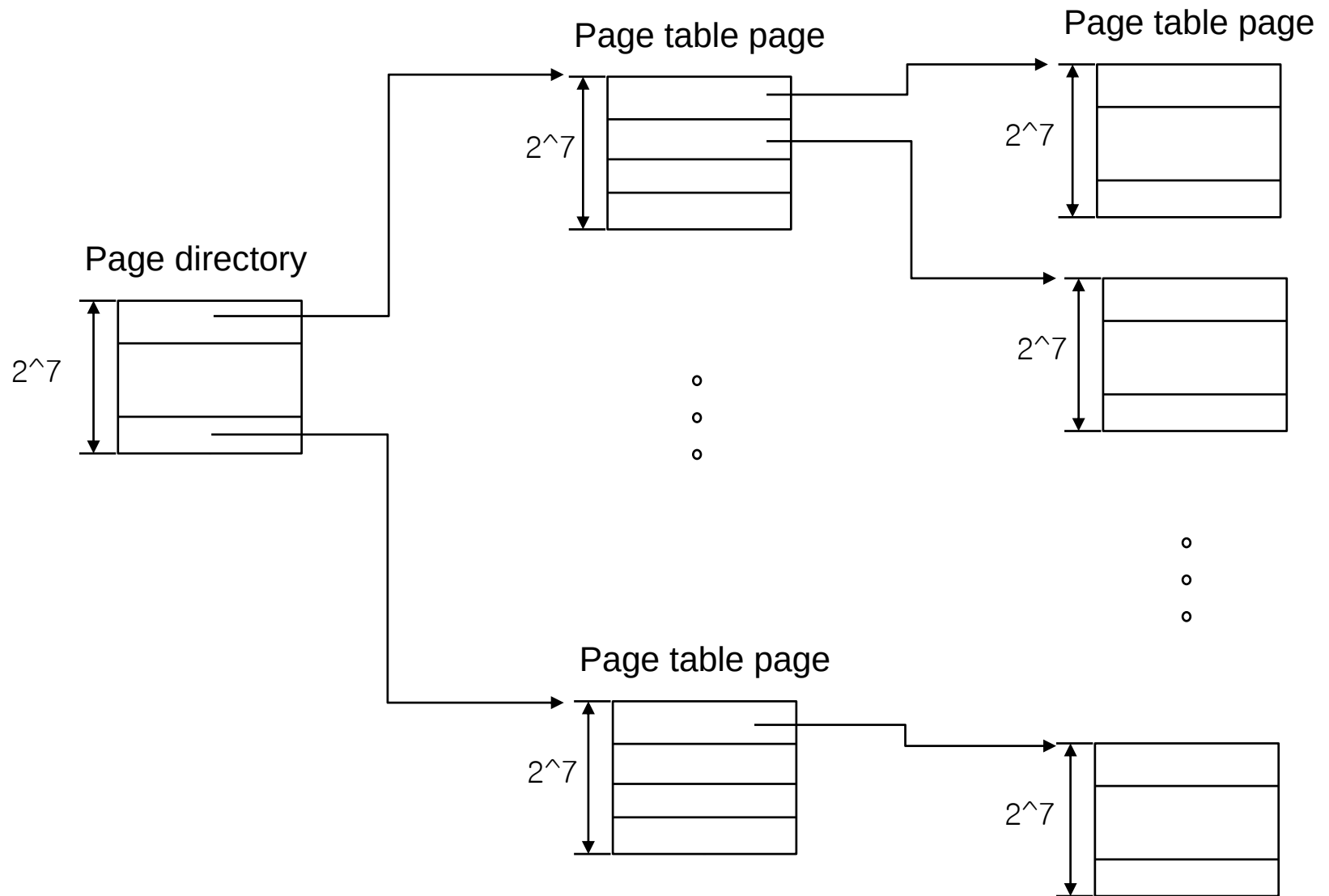


More than Two Level : Page Directory

- ▣ The number of pages in a page directory: $2^{14}/2^7 = 2^7$ (128)
- ▣ Page the page directory



three level page table



Multi-level Page Table Control Flow

```
01:     VPN = (VirtualAddress & VPN_MASK) >> SHIFT
02:     (Success, TlbEntry) = TLB_Lookup(VPN)
03:     if (Success == True)           //TLB Hit
04:         if (CanAccess(TlbEntry.ProtectBits) == True)
05:             Offset = VirtualAddress & OFFSET_MASK
06:             PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
07:             Register = AccessMemory(PhysAddr)
08:         else RaiseException(PROTECTION_FAULT);
09:     else // perform the full multi-level lookup
```

- ◆ (1 lines) extract the virtual page number(VPN)
- ◆ (2 lines) check if the TLB holds the translation for this VPN
- ◆ (5-8 lines) extract the page frame number from the relevant TLB entry, and form the desired physical address and access memory

Multi-level Page Table Control Flow

```
11:         else
12:             PDIndex = (VPN & PD_MASK) >> PD_SHIFT
13:             PDEAddr = PDBR + (PDIndex * sizeof(PDE))
14:             PDE = AccessMemory(PDEAddr)
15:             if(PDE.Valid == False)
16:                 RaiseException(SEGMENTATION_FAULT)
17:             else // PDE is Valid: now fetch PTE from PT
```

- ◆ (11 lines) extract the Page Directory Index(PDIndex)
- ◆ (13 lines) get Page Directory Entry(PDE)
- ◆ (15-17 lines) Check PDE valid flag. If valid flag is true, fetch Page Table entry from Page Table

The Translation Process: Remember the TLB

```
18:     PTIndex = (VPN & PT_MASK) >> PT_SHIFT
19:     PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
20:     PTE = AccessMemory(PTEAddr)
21:     if(PTE.Valid == False)
22:         RaiseException(SEGMENTATION_FAULT)
23:     else if(CanAccess(PTE.ProtectBits) == False)
24:         RaiseException(PROTECTION_FAULT);
25:     else
26:         TLB_Insert(VPN, PTE.PFN , PTE.ProtectBits)
27:         RetryInstruction()
```


Inverted Page Tables

- ▣ Keeping a single page table that has an entry for each physical page of the system.
- ▣ The entry tells us which process is using this page, and which virtual page of that process maps to this physical page.

Summary

- ▣ Reducing the page table size
- ▣ Per segment page table
- ▣ Multi-level paging
 - ◆ Efficient use of memory
 - ◆ Severe TLB miss

Operating Systems

Youjip Won

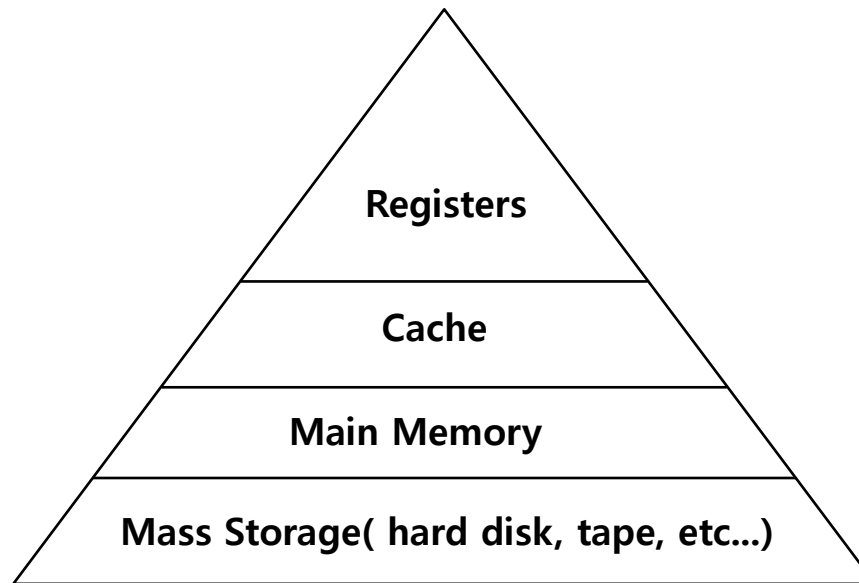
KAIST



21. Swapping: Mechanisms

Beyond Physical Memory: Mechanisms

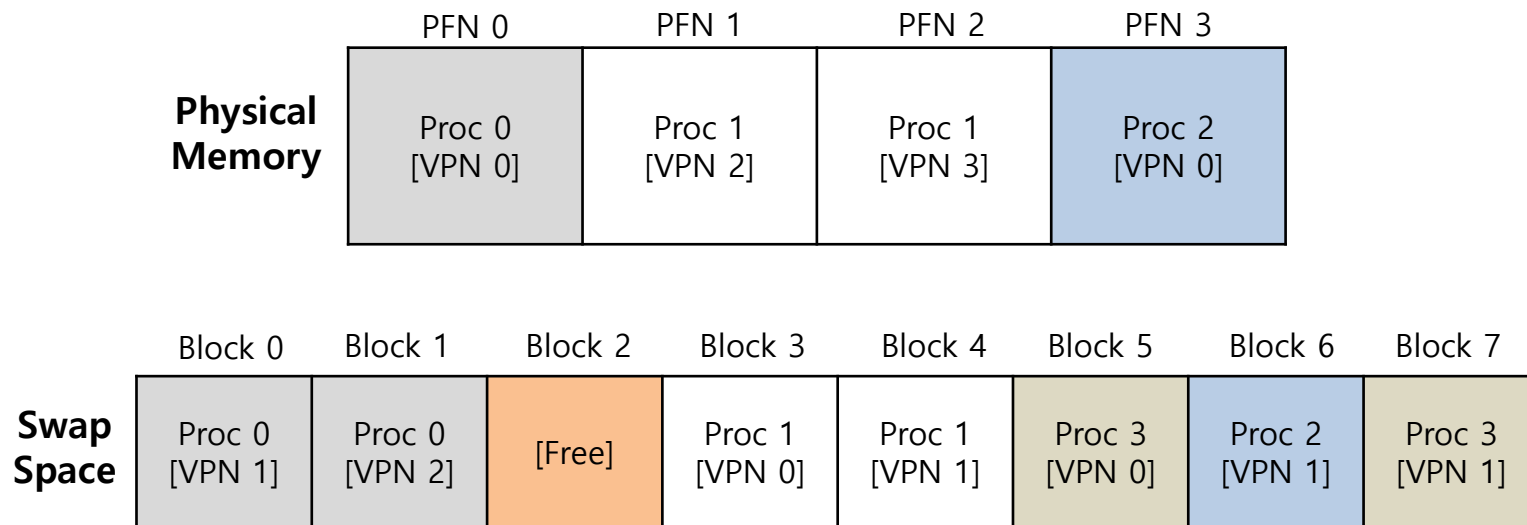
- ▣ Use part of disk as memory.
 - ◆ OS need a place to stash away portions of address space that currently aren't in great demand.
 - ◆ In modern systems, this role is usually served by a **hard disk drive**.



Memory Hierarchy in modern system

Swap Space

- ❑ Reserve some space on the disk for moving pages back and forth.
- ❑ OS needs to remember the swap space, in **page-sized unit**.



Physical Memory and Swap Space

Present Bit

- ▣ Add some machinery higher up in the system in order to support swapping the pages to and from the disk.
 - ◆ When the hardware looks in the PTE, it may find that the page is not present in physical memory.

Value	Meaning
1	page is present in physical memory
0	The page is not in memory but rather on disk.

❑ Page fault

- ◆ Accessing page that is **not in physical memory**.
- ◆ If a page is not present and has been swapped disk, the OS needs to swap the page back into memory in order to service the page fault.

❑ Page replacement

- ◆ The OS likes to page out pages to make room for the new pages the OS is about to bring in.
- ◆ The process of picking a page to kick out, or replace is known as **page-replacement** policy.

When to perform replacement

▣ Lazy approach

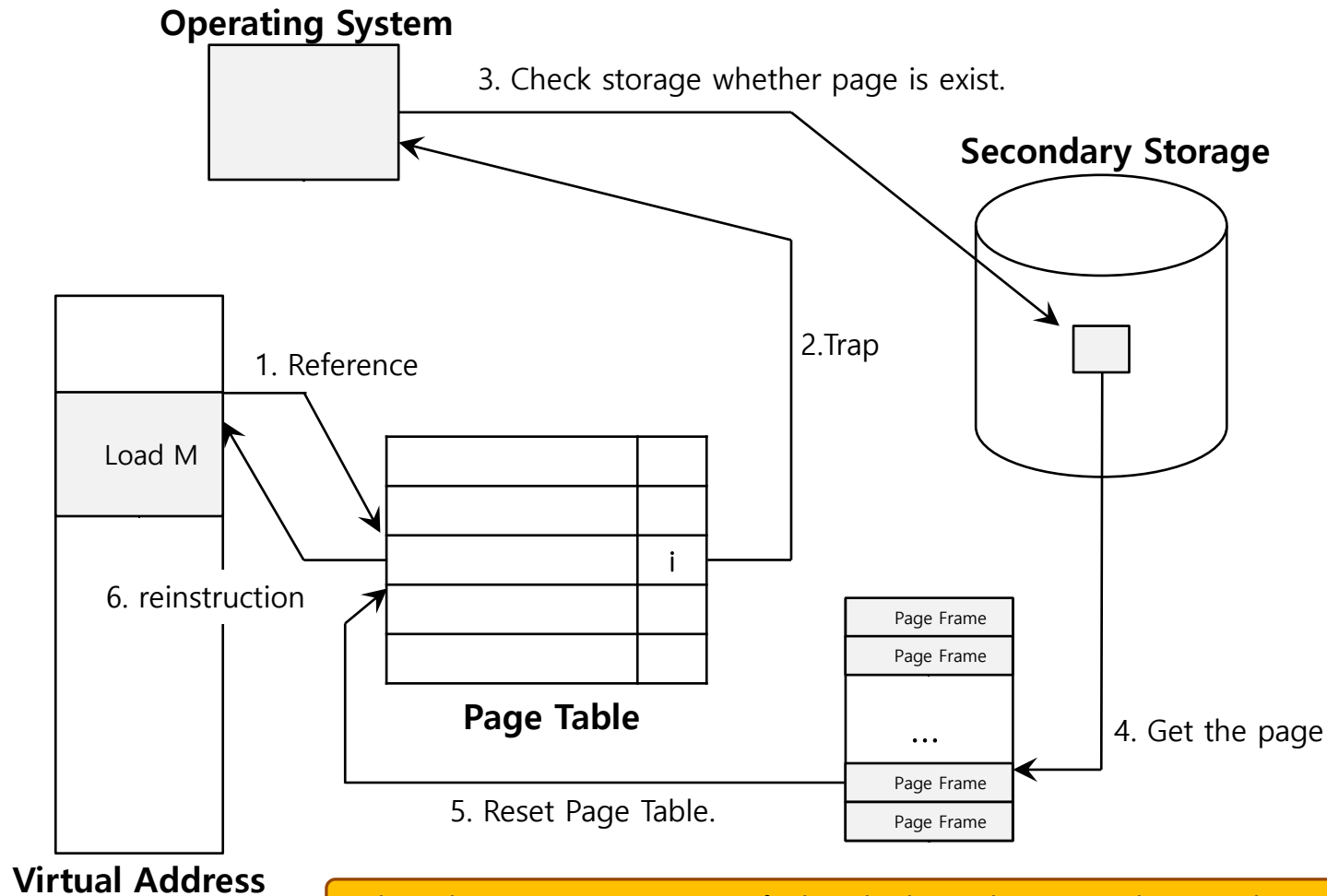
- ◆ OS waits until memory is entirely full, and only then replaces a page to make room for some other page.
- ◆ This is unrealistic.

▣ Swap Daemon, Page Daemon

- ◆ There are fewer than LW (low watermark) pages available, a background thread that is responsible for freeing memory runs.
- ◆ The thread evicts pages until there are HW(high watermark) pages available.

Page Fault Control Flow

- PTE used for data such as the PFN of the page for a disk address.



When the OS receives a page fault, it looks in the PTE and issues the request to disk.

Page Fault Control Flow – Hardware

```
1:     VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2:     (Success, TlbEntry) = TLB_Lookup(VPN)
3:     if (Success == True) // TLB Hit
4:         if (CanAccess(TlbEntry.ProtectBits) == True)
5:             Offset = VirtualAddress & OFFSET_MASK
6:             PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:             Register = AccessMemory(PhysAddr)
8:         else RaiseException(PROTECTION_FAULT)
9:     else // TLB Miss
10:         PTEAddr = PTBR + (VPN * sizeof(PTE))
11:         PTE = AccessMemory(PTEAddr)
12:         if (PTE.Valid == False)
13:             RaiseException(SEGMENTATION_FAULT)
14:         else
15:             if (CanAccess(PTE.ProtectBits) == False)
16:                 RaiseException(PROTECTION_FAULT)
17:             else if (PTE.Present == True)
18:                 // assuming hardware-managed TLB
19:                 TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
20:                 RetryInstruction()
21:             else if (PTE.Present == False)
22:                 RaiseException(PAGE_FAULT)
```

Page Fault Control Flow – Software

```
1:      PFN = FindFreePhysicalPage()
2:      if (PFN == -1) // no free page found
3:          PFN = EvictPage() // run replacement algorithm
4:      DiskRead(PTE.DiskAddr, pfn) // sleep (waiting for I/O)
5:      PTE.present = True // update page table with present
6:      PTE.PFN = PFN // bit and translation (PFN)
7:      RetryInstruction() // retry instruction
```

- ◆ The OS must find a physical frame for the soon-be-faulted-in page to reside within.
- ◆ If there is no such page, waiting for the replacement algorithm to run and kick some pages out of memory.

Summary

- ▣ Swapping: making the part of disk as memory
- ▣ Present bit required

Operating Systems

Youjip Won

KAIST



22. Swapping: Policies

Goal of Cache Management

- ▣ to minimize the number of cache misses.
- ▣ the *average memory access time(AMAT)*.

$$AMAT = (P_{Hit} * T_M) + (P_{Miss} * T_D)$$

Argument	Meaning
T_M	The cost of accessing memory
T_D	The cost of accessing disk
P_{Hit}	The probability of finding the data item in the cache(a hit)
P_{Miss}	The probability of not finding the data in the cache(a miss)

The Optimal Replacement Policy

- ❑ Lead to the fewest number of misses overall.
 - ◆ Replace the page that will be accessed furthest in the future.
 - ◆ Result in the **fewest-possible** cache misses.
- ❑ Serve only as a comparison point, to know how close we are to **perfect**.

Tracing the Optimal Policy

Reference Row

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	2	0,1,3
0	Hit		0,1,3
3	Hit		0,1,3
1	Hit		0,1,3
2	Miss	3	0,1,2
1	Hit		0,1,2

Hit rate is $\frac{Hits}{Hits+Misses} = 54.6\%$

Future is not known.

A Simple Policy: FIFO

- ▣ Pages were placed in a queue when they enter the system.
- ▣ When a replacement occurs, the page on the tail of the queue(the "**First-in**" pages) is evicted.
 - ◆ It is simple to implement, but can't determine the importance of blocks.

Tracing the FIFO Policy

Reference Row

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	1,2,3
0	Miss	1	2,3,0
3	Hit		2,3,0
1	Miss		3,0,1
2	Miss	3	0,1,2
1	Hit		0,1,2

Hit rate is $\frac{\text{Hits}}{\text{Hits} + \text{Misses}} = 36.4\%$

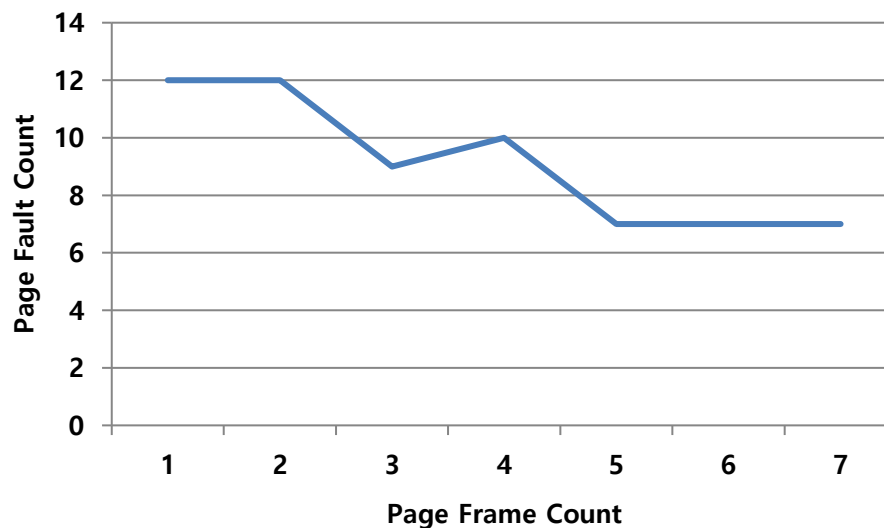
Even though page 0 had been accessed a number of times, **FIFO still kicks it out.**

BELADY'S ANOMALY

- We would expect the cache hit rate to **increase** when the cache gets larger. But in this case, with FIFO, it gets worse.

Reference Row

1 2 3 4 1 2 5 1 2 3 4 5



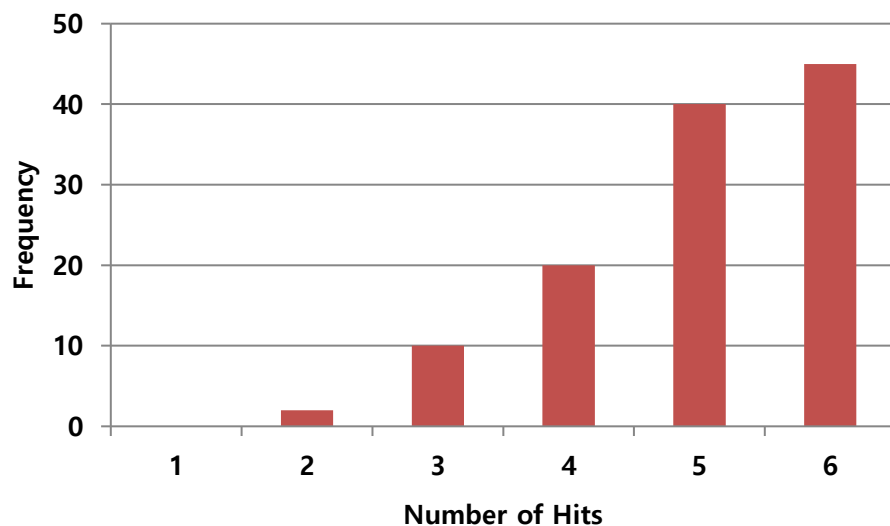
Another Simple Policy: Random

- ▣ Picks a random page to replace under memory pressure.
 - ◆ It doesn't really try to be too intelligent in picking which blocks to evict.
 - ◆ Random does depends entirely upon how lucky Random gets in its choice.

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	1,2,3
0	Miss	1	2,3,0
3	Hit		2,3,0
1	Miss	3	2,0,1
2	Hit		2,0,1
1	Hit		2,0,1

Random Performance

- Sometimes, **Random is as good as optimal**, achieving 6 hits on the example trace.



Random Performance over 10,000 Trials

Using History

- ▣ Learn on the past and use history.
 - ◆ Two type of historical information.

Historical Information	Meaning	Algorithms
recency	The more recently a page has been accessed, the more likely it will be accessed again	LRU
frequency	If a page has been accessed many times, It should not be replcaed as it clearly has some value	LFU

Using History : LRU

- ▣ Replace the least-recently-used page.

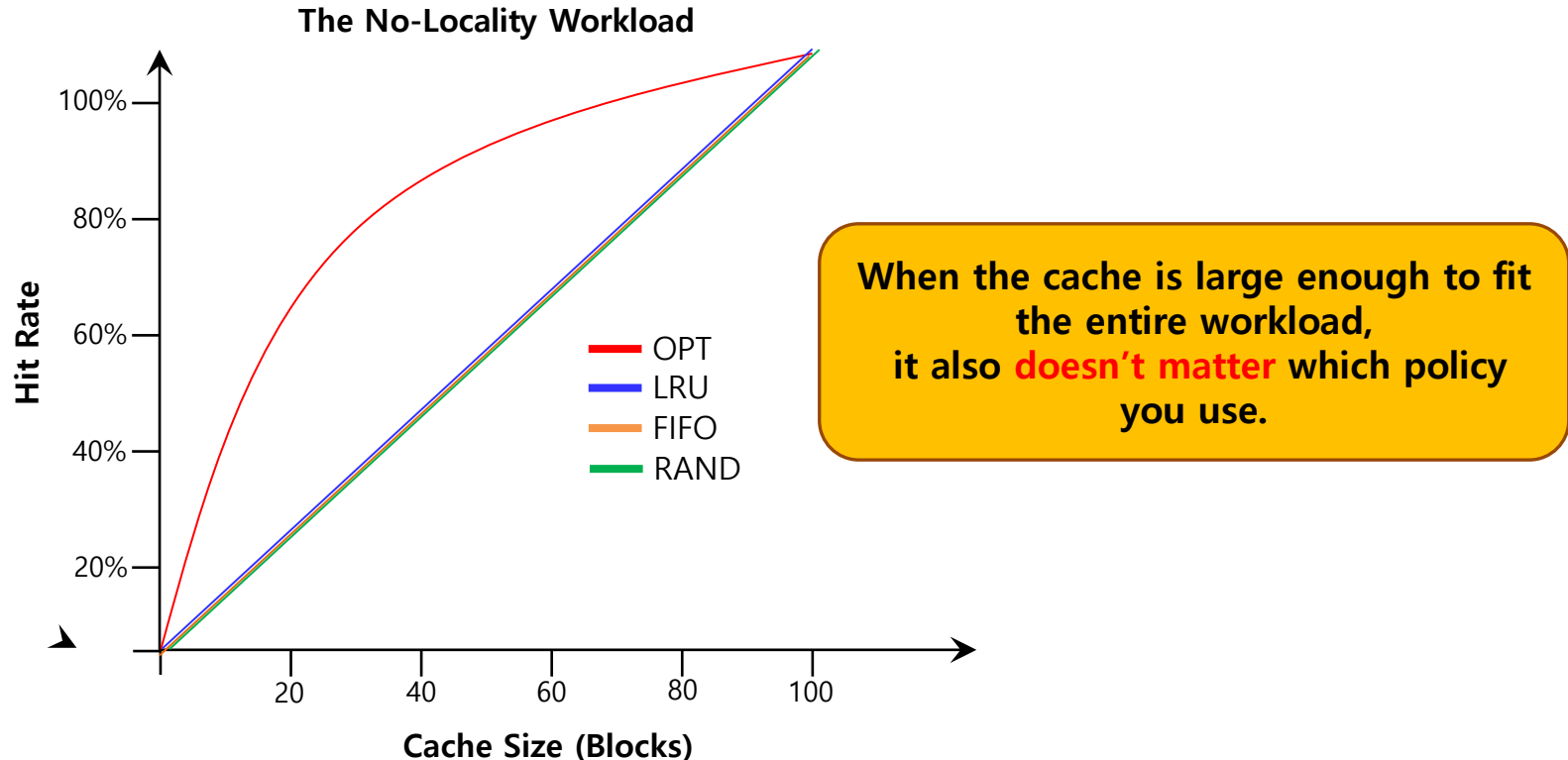
Reference Row

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		3,2,1

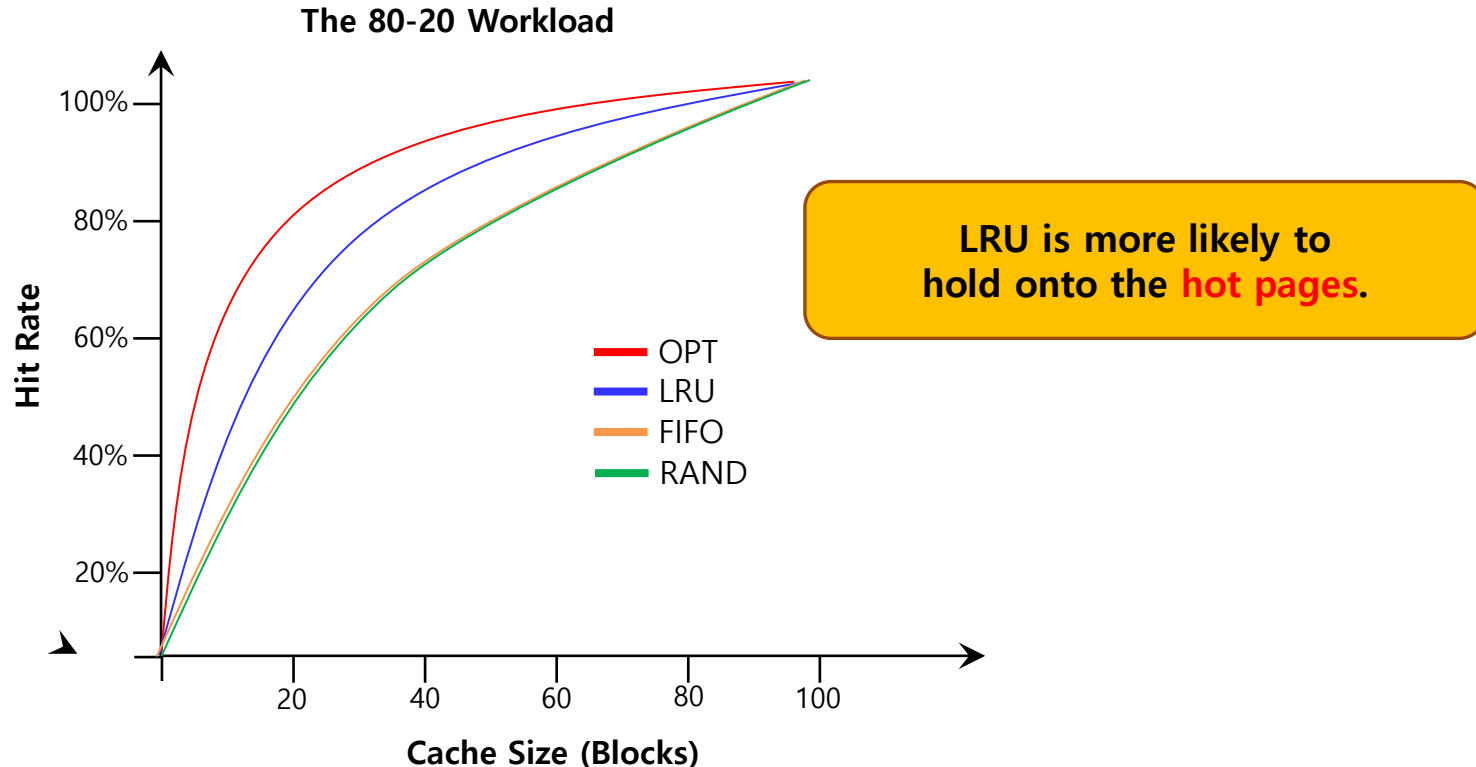
Workload Example : The No-Locality Workload

- Each reference is to a random page within the set of accessed pages.
 - Workload accesses 100 unique pages over time.
 - Choosing the next page to refer to at random



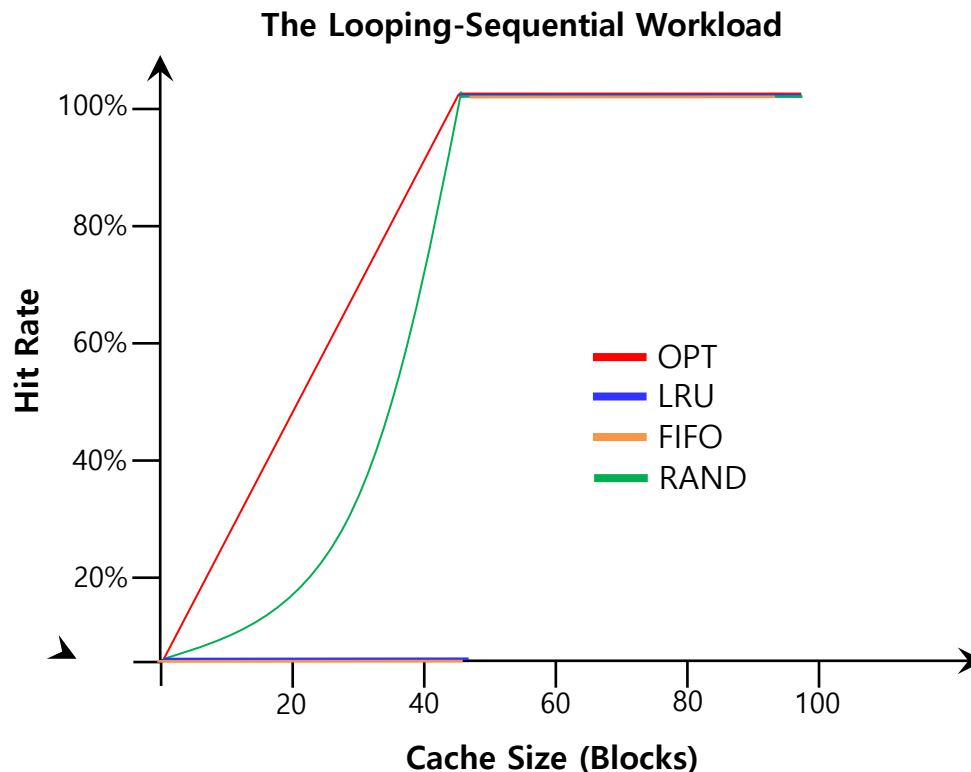
Workload Example : The 80-20 Workload

- ▣ Exhibits locality: 80% of the **reference** are made to 20% of the page
- ▣ The remaining 20% of the **reference** are made to the remaining 80% of the pages.



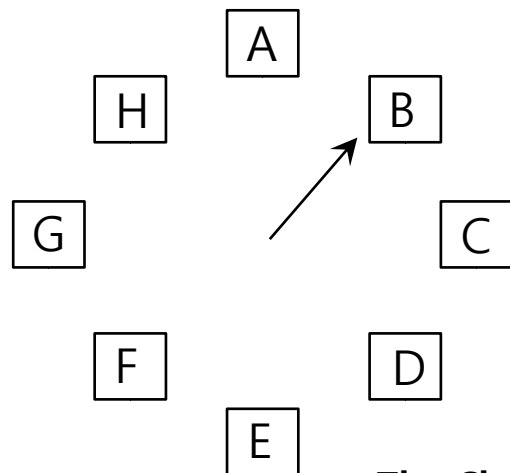
Workload Example : The Looping Sequential

- Refer to 50 pages in sequence.
 - Starting at 0, then 1, ... up to page 49, and then we Loop, repeating those accesses, for total of 10,000 accesses to 50 unique pages.



Approximating LRU: Clock Algorithm

- Require hardware support: a **use bit**
 - Whenever a **page is referenced**, the use bit is set by hardware to 1.
 - Hardware **never** clears the bit, though; that is the responsibility of the OS
- Clock Algorithm
 - All pages of the system arranges in a circular list.
 - A clock hand points to some particular page to begin with.
 - The algorithm continues until it finds a use bit that is set to 0.

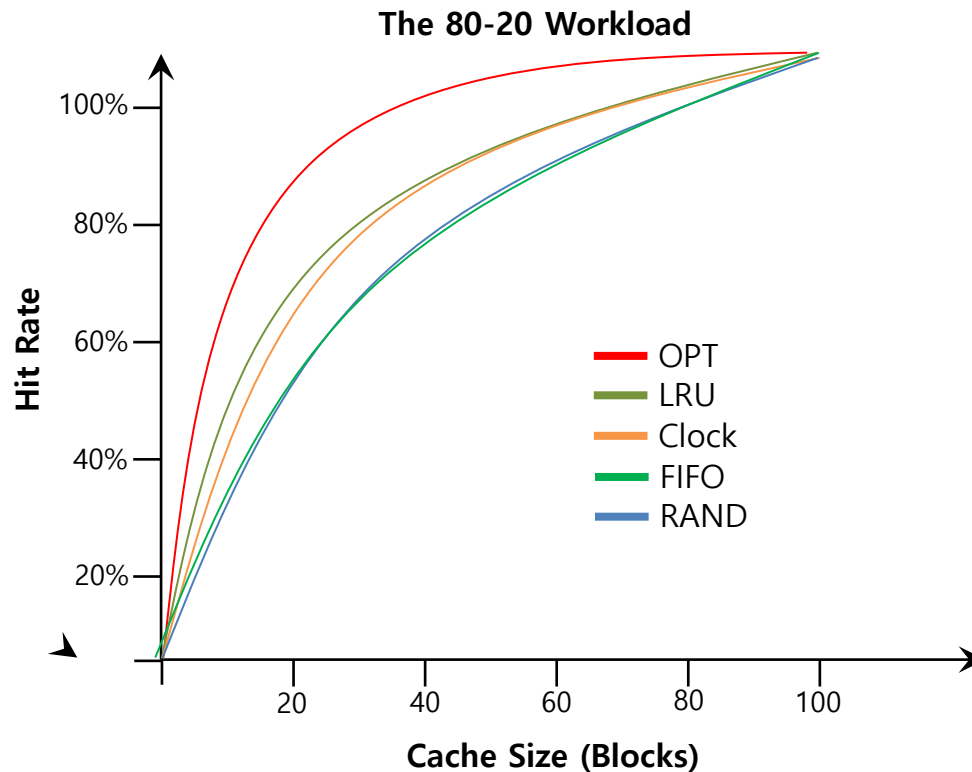


Use bit	Meaning
0	Evict the page
1	Clear <u>Use bit</u> and advance hand

The Clock page replacement algorithm

Workload with Clock Algorithm

- Clock algorithm doesn't do as well as perfect LRU, it does better than approach that don't consider history at all.

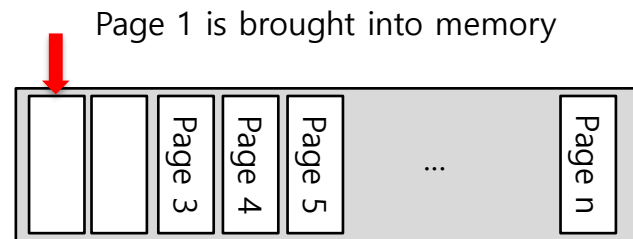


Considering Dirty Pages

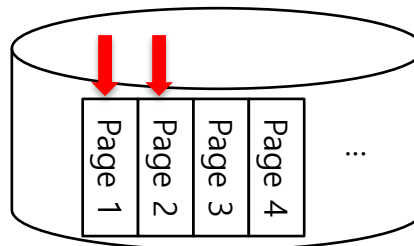
- The hardware includes a **modified bit** (a.k.a **dirty bit**)
 - ◆ Page has been **modified** and is thus **dirty**, it must be written back to disk to evict it.
 - ◆ Page has not been modified, the eviction is free.

Prefetching

- The OS guesses that a page is about to be used, and thus bring it in ahead of time.



Physical Memory

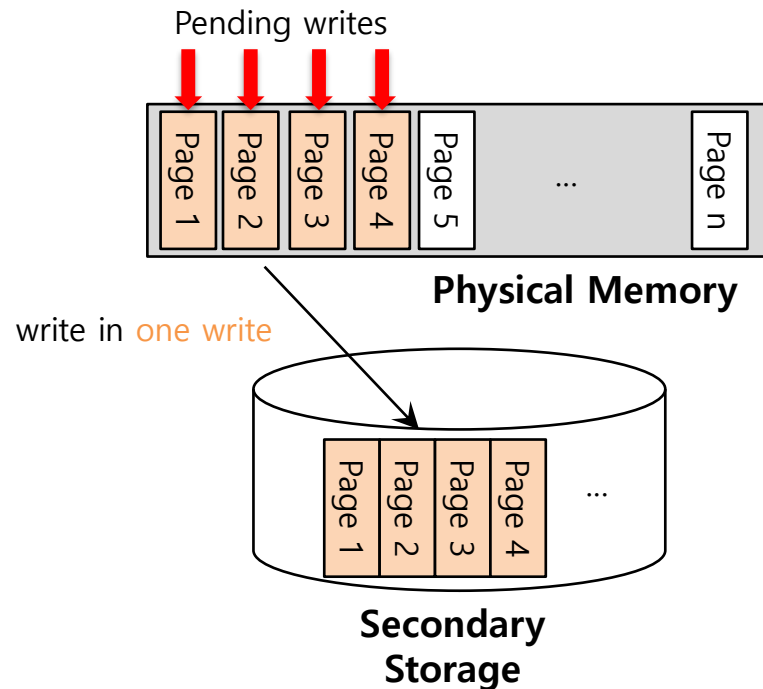


Secondary Storage

Page 2 likely soon be accessed and thus should be brought into memory too

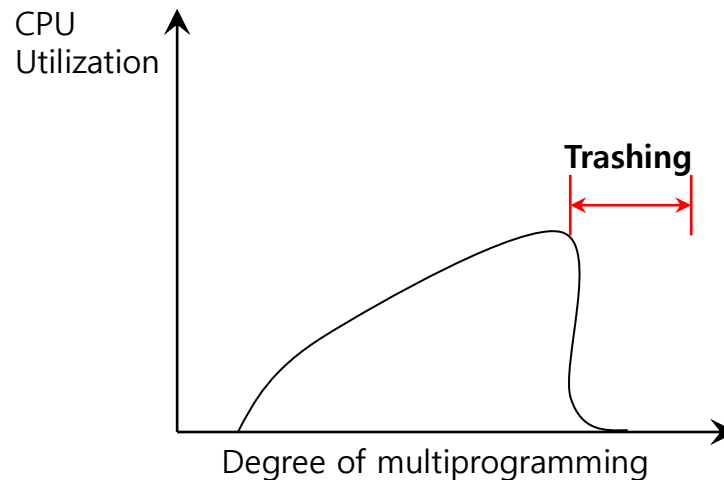
Clustering, Grouping

- ❑ Collect a number of **pending writes** together in memory and write them to disk in **one write**.
 - ◆ Perform a **single large write** more efficiently than **many small ones**.



Thrashing

- ❑ Memory is **oversubscribed** and the memory demands of the set of running processes **exceeds** the available physical memory.
 - ◆ Decide not to run a subset of processes.
 - ◆ Reduced set of processes working sets fit in memory.



Summary

- ▣ Swapping: use part of disk as memory
- ▣ LRU, LFU, RANDOM, FIFO
- ▣ Approximation to LRU: Clock
- ▣ Making the disk IO in larger unit
 - ◆ Clustering
 - ◆ Grouping
 - ◆ prefetching

Operating Systems

Youjip Won

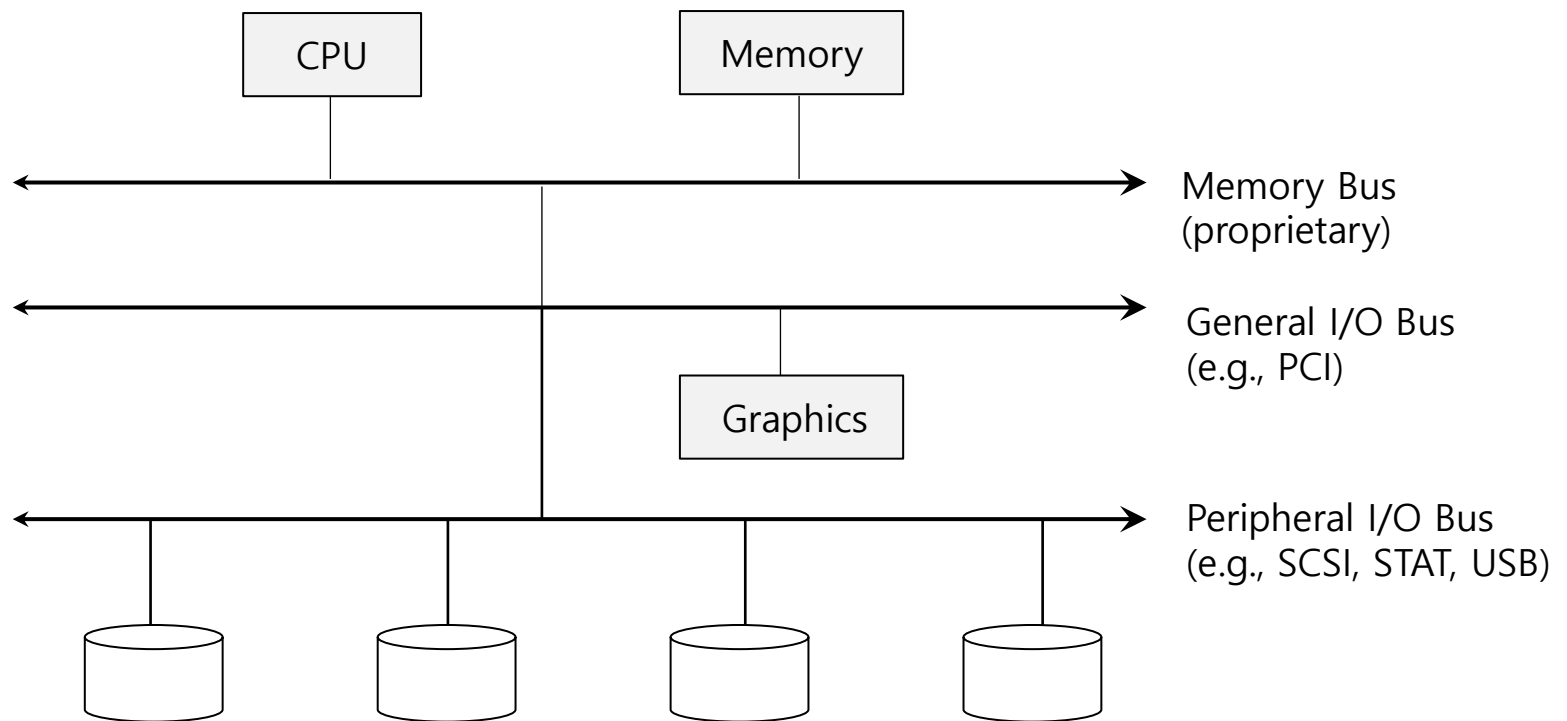
KAIST



36. I/O Devices

- ▣ I/O is **critical** to computer system to **interact with systems**.
- ▣ Issue :
 - ◆ How should I/O be integrated into systems?
 - ◆ What are the general mechanisms?
 - ◆ How can we make the efficiently?

Structure of input/output (I/O) device



Prototypical System Architecture

CPU is attached to the main memory of the system via some kind of memory **bus.**
Some devices are connected to the system via a general **I/O bus.**

- ▣ Buses

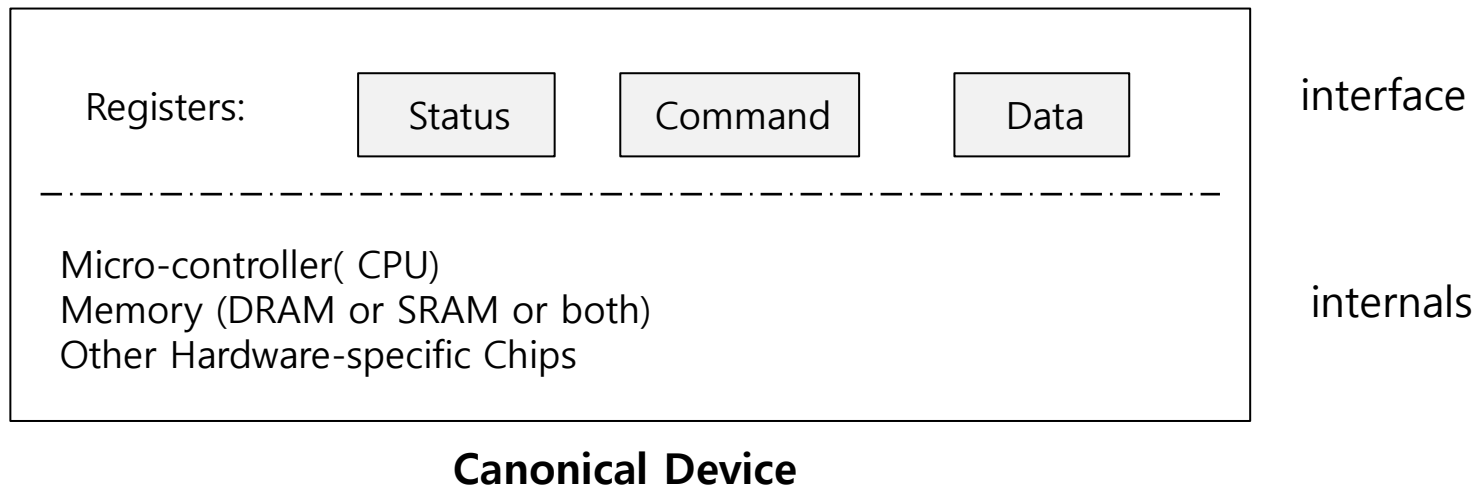
- ◆ Data paths that provided to enable information between CPU(s), RAM, and I/O devices.

- ▣ I/O bus

- ◆ Data path that connects a CPU to an I/O device.
- ◆ I/O bus is connected to I/O device by three hardware components: I/O ports, interfaces and device controllers.

Canonical Device

- ▣ Canonical Devices has two important components.
 - ◆ **Hardware interface** allows the system software to control its operation.
 - ◆ **Internals** which is implementation specific.



Hardware interface of Canonical Device

- ▣ **status register**

- ◆ See the current status of the device

- ▣ **command register**

- ◆ Tell the device to perform a certain task

- ▣ **data register**

- ◆ Pass data to the device, or get data from the device

**By reading and writing above three registers,
the operating system can control device behavior.**

Hardware interface of Canonical Device (Cont.)

▣ Typical interaction example

```
while ( STATUS == BUSY)
    ; //wait until device is not busy
write data to data register
write command to command register
    Doing so starts the device and executes the command
while ( STATUS == BUSY)
    ; //wait until device is done with your request
```

Polling

- ❑ Operating system waits until the device is ready by **repeatedly** reading the status register.
 - ◆ Positive aspect is simple and working.
 - ◆ **However, it wastes CPU time just waiting for the device.**
 - Switching to another ready process is better utilizing the CPU.

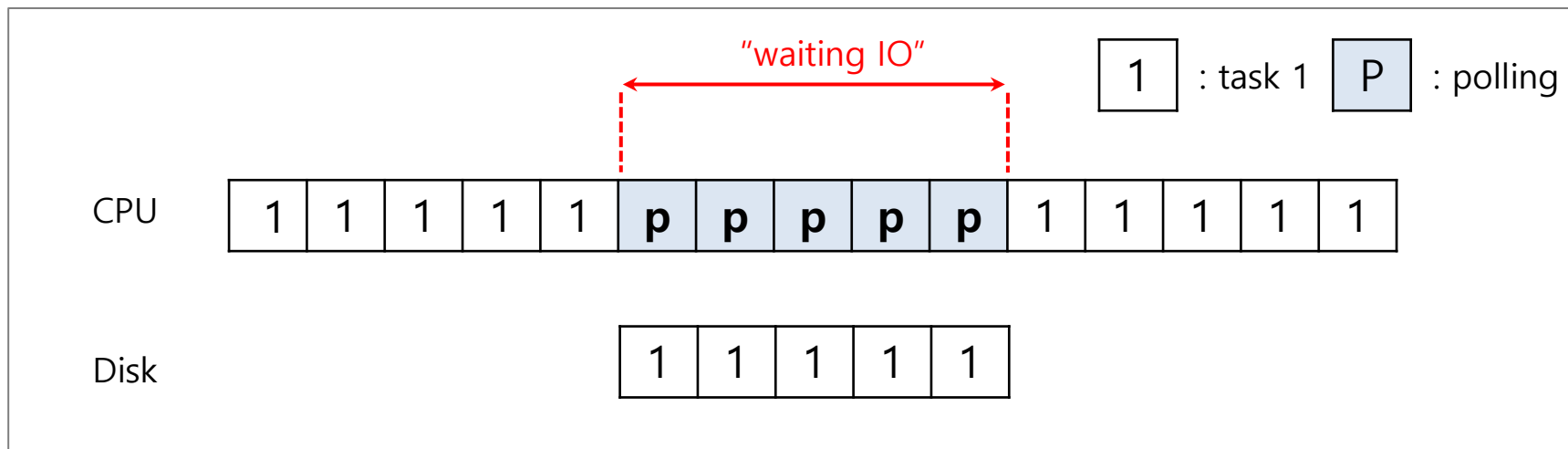


Diagram of CPU utilization by polling

interrupts

- ❑ Put the I/O request process to sleep and context switch to another.
- ❑ When the device is finished, wake the process waiting for the I/O by **interrupt**.
 - ◆ Positive aspect is allow to **CPU and the disk are properly utilized.**

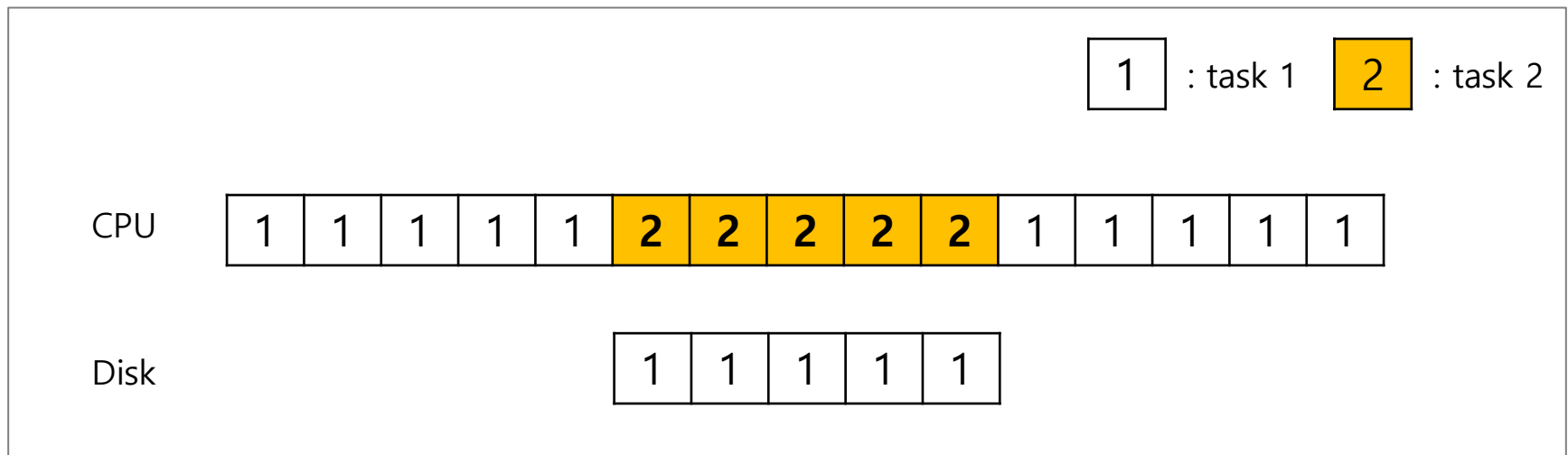


Diagram of CPU utilization by interrupt

Polling vs interrupts

- *However, “interrupts is not always the best solution”*
 - ◆ If, device performs very quickly, interrupt will “slow down” the system.
 - ◆ Because **context switch is expensive (switching to another process)**

If a device is fast → **poll** is best.
If it is slow → **interrupts** is better.

CPU is once again over-burdened

- ❑ CPU **wastes a lot of time** to copy the *a large chunk of data* from memory to the device.

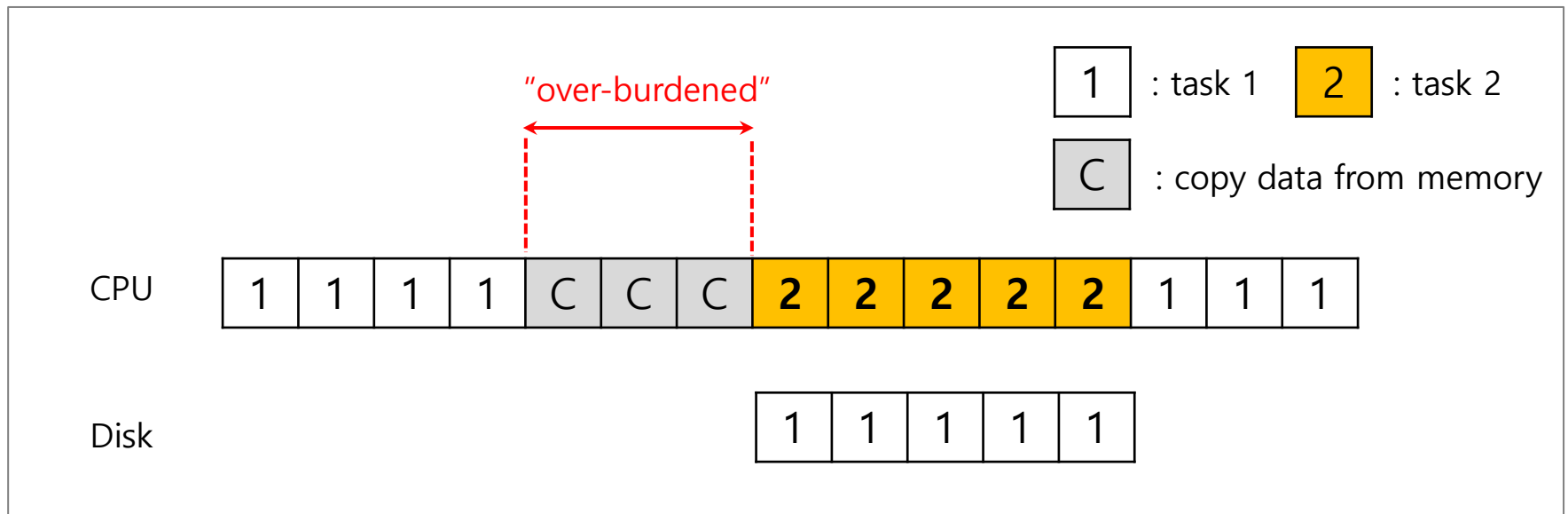


Diagram of CPU utilization

DMA (Direct Memory Access)

- **Copy data** in memory by knowing “where the data lives in memory, how much data to copy”
- When completed, DMA raises an interrupt, I/O begins on Disk.

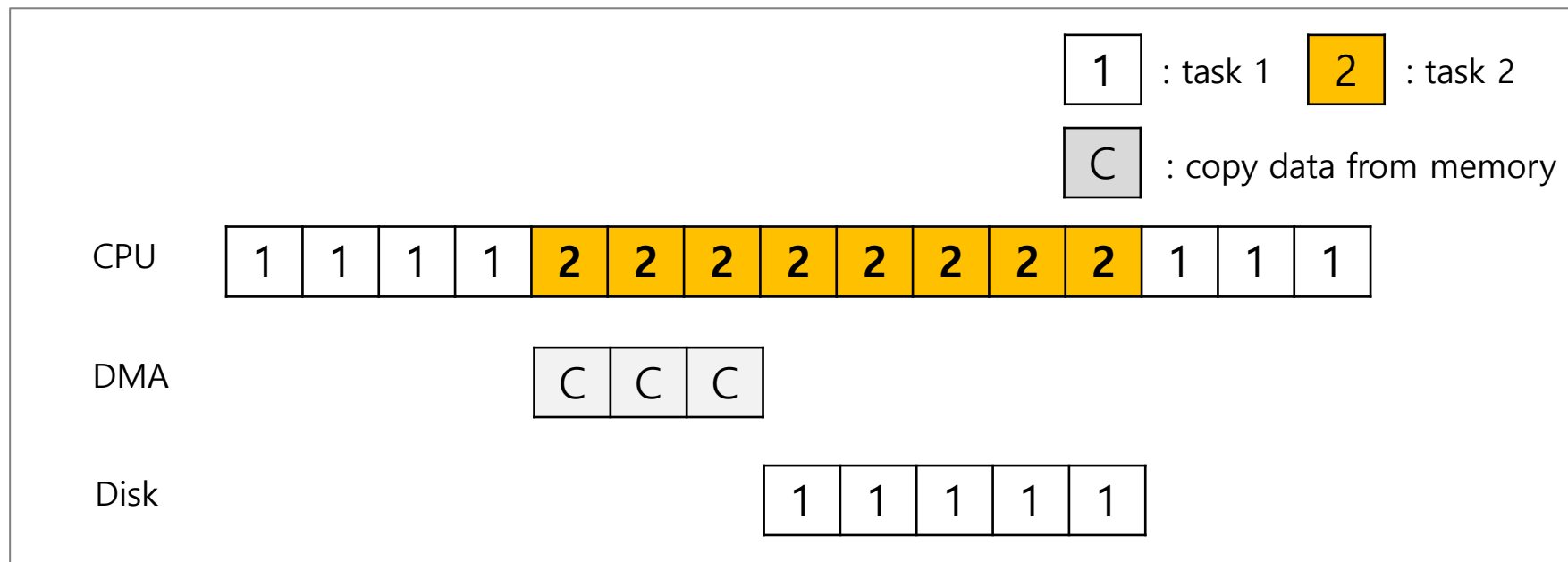


Diagram of CPU utilization by DMA

- ▣ How the OS communicates with the **device**?
- ▣ Solutions
 - ◆ I/O instructions: a way for the OS to send data to specific device registers.
 - Ex) `in` and `out` instructions on x86
 - ◆ memory-mapped I/O
 - Device registers available as if they were memory locations.
 - The OS `load` (to read) or `store` (to write) to the device instead of main memory.

- ▣ For registers
 - ◆ Control register (8 bit)
 - ◆ Command block registers
 - ◆ Status register (8 bit)
 - ◆ Error register (8 bit)

The IDE Interface

- ▣ Control Register (1 Byte):
Address 0x3F6 = 0x08 (0000 1RE0): R=reset, E="Enable Interrupt", E=0 means "enable interrupt"
- ▣ Command Block Registers:
Address 0x1F0 = Data Port (128 Byte)
Address 0x1F1 = Error (1 Byte)
Address 0x1F2 = Sector Count (1 Byte)
Address 0x1F3 = LBA low byte (1 Byte)
Address 0x1F4 = LBA mid byte (1 Byte)
Address 0x1F5 = LBA hi byte (1 Byte)
Address 0x1F6 = 1B1D TOP4LBA: B=LBA, D=drive
Address 0x1F7 = Command/status(1 Byte)
- ▣ Status Register (Address 0x1F7): (1 Byte)

7	6	5	4	3	2	1	0
BUSY	READY	FAULT	SEEK	DRQ	CORR	IDDEX	ERROR

The IDE Interface (Cont.)

- Error Register (Address 0x1F1): (check when Status ERROR==1)

7	6	5	4	3	2	1	0
BBK	UNC	MC	IDNF	MCR	ABRT	T0NF	AMNF

BBK = Bad Block,

UNC = Uncorrectable data error,

MC = Media Changed,

IDNF = ID mark Not Found,

MCR = Media Change Requested,

ABRT = Command aborted,

T0NF = Track 0 Not Found,

AMNF = Address Mark Not Found

The basic protocol of IDE device driver

- ❑ Wait for the drive to be ready.
- ❑ Write parameters to command registers.
 - ◆ Sector count
 - ◆ logical block address (LBA) of sector
 - ◆ Driver number
- ❑ Issue read/write to command register.
 - ◆ Read or Write
 - ◆ For write command, transfer data
- ❑ Handle interrupt.
- ❑ Handle Error.

IDE device driver in xv6

```
void ide_rw(struct buf *b) {  
    acquire(&ide_lock);  
  
    1 for (struct buf **pp = &ide_queue; *pp; pp=&(*pp)->qnext) ;  
    *pp = b;  
  
    2 if (ide_queue == b)                // if q is empty,  
        ide_start_request(b);          // send request to disk.  
  
    3 while ((b->flags & (B_VALID|B_DIRTY)) != B_VALID)  
        sleep(b, &ide_lock);          // wait for completion  
  
    release(&ide_lock);  
}
```

Mechanism: add the buffer to the `ide_queue` and perform IO.

1. Enqueue the buffer to the `ide_queue`.
2. If the buffer is the only entry in the `ide_queue`, start the IO right away.
3. Wait for the completion of the IO.
 1. If the IO is READ, interrupt handler resets the `B_VALID` flag.
 2. If the IO is write, interrupt handler resets the `B_DIRTY` flag.

Device driver

```
static void ide_start_request(struct buf *b) {  
    ide_wait_ready();  
    outb(0x3f6, 0); // generate interrupt  
    outb(0x1f2, 1); // the count of sectors  
    outb(0x1f3, b->sector & 0xff);  
    outb(0x1f4, (b->sector >> 8) & 0xff);  
    outb(0x1f5, (b->sector >> 16) & 0xff);  
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f));  
    if (b->flags & B_DIRTY){  
        outb(0x1f7, IDE_CMD_WRITE); // WRITE  
        outsl(0x1f0, b->data, 512/4); // transfer data  
    } else {  
        outb(0x1f7, IDE_CMD_READ); // READ  
    }  
}
```

Device driver

```
static void ide_start_request(struct buf *b) {  
    ide_wait_ready();  
    ...  
}
```

Call



```
static int ide_wait_ready() {  
    int r;  
  
    while ((r = inb(0x1f7)) & IDE_BSY) || !(r & IDE_DRDY))  
        ;  
}
```

1. Read Status Register(0x1F7) and save it to r.
2. Check if the drive is the busy. (IDE_BSY).
3. Check if the drive is ready .
4. If the device is not busy and ready, get out of the while loop.

Status Register is also used as a command register. The device uses it to store the status of the register. The host uses it to store the command.

Enable interrupt and set the sector count.

```
static void ide_start_request(struct buf *b) {  
    ide_wait_ready();  
    outb(0x3f6, 0); // generate interrupt  
    outb(0x1f2, 1); // the number of sectors  
    ...  
}
```

1. Enable interrupt for the device: set the Control Register(0x3F6).
2. Set the number of sectors for an IO: set the sector count register(0x1F2).

Specifying the LBA

```
static void ide_start_request(struct buf *b) {  
    ide_wait_ready();  
    outb(0x3f6, 0); // generate interrupt  
    outb(0x1f2, 1); // the count of sectors  
    outb(0x1f3, b->sector & 0xff);          // 1  
    outb(0x1f4, (b->sector >> 8) & 0xff);    // 2  
    outb(0x1f5, (b->sector >> 16) & 0xff);    // 3  
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f)); // 4
```

Write Command and the data transfer

```
static void ide_start_request(struct buf *b) {  
    ...  
    outb(0x1f3, b->sector & 0xff);  
    outb(0x1f4, (b->sector >> 8) & 0xff);  
    outb(0x1f5, (b->sector >> 16) & 0xff);  
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f));  
    if (b->flags & B_DIRTY) {  
        outb(0x1f7, IDE_CMD_WRITE); // WRITE  
        outsl(0x1f0, b->data, 512/4); // transfer data  
    }  
}
```

1. If the DIRTY bit is set, perform write. (In xv6, device driver performs write only when the buffer is dirty.)
2. Set the write command at the Command Register (0x1F7).
3. Write the data to the Data Register (0x1F0).

1. The amount of data to write: 512/4 byte

```
2. static inline void outsl (int port, const void *addr, int cnt)
```

Read command

```
static void ide_start_request(struct buf *b) {  
    ...  
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f));  
    if (b->flags & B_DIRTY){  
        outb(0x1f7, IDE_CMD_WRITE); // WRITE  
        outsl(0x1f0, b->data, 512/4); // transfer data  
    } else {  
        outb(0x1f7, IDE_CMD_READ); // READ  
    }  
}
```

1. If the buffer is not dirty, perform read.
2. Set the command READ at Command Register (0x1F7).
3. Perform read.

Interrupt handler (IO Completion)

```
void ide_intr() {  
    struct buf *b;  
    acquire(&ide_lock);  
  
    if (!(b->flags & B_DIRTY) && ide_wait_ready() >= 0) Read  
        insl(0x1f0, b->data, 512/4); // if READ: get data  
  
    b->flags |= B_VALID;  
    b->flags &= B_DIRTY;  
    wakeup(b); // wake waiting process  
  
    if ((ide_queue = b->qnext) != 0) // start next request  
        ide_start_request(ide_queue); // (if one exists)  
  
    release(&ide_lock);  
  
}
```

Set the flag properly.
→wake up from ide_rw()

Start the next request in the ide_queue.

Device interaction (Cont.)

- ▣ How the OS interact with **different types of interfaces**?
 - ◆ Ex) We'd like to build a file system that worked on top of SCSI disks, IDE disks, USB keychain drivers, and so on.
- ▣ Solutions: **Abstraction**
 - ◆ Abstraction encapsulate **any specifics of device interaction**.

Summary

- ▣ To save the CPU cycles for IO
 - ◆ Use Interrupt
 - ◆ Use DMA
- ▣ To access the device registers
 - ◆ Memory mapped IO
 - ◆ Explicit IO instruction

I/O Devices

Before delving into the main content of this part of the book (on persistence), we first introduce the concept of an **input/output (I/O) device** and show how the operating system might interact with such an entity. I/O is quite critical to computer systems, of course; imagine a program without any input (it produces the same result each time); now imagine a program with no output (what was the purpose of it running?). Clearly, for computer systems to be interesting, both input and output are required. And thus, our general problem:

CRUX: HOW TO INTEGRATE I/O INTO SYSTEMS

How should I/O be integrated into systems? What are the general mechanisms? How can we make them efficient?

36.1 System Architecture

To begin our discussion, let's look at a "classical" diagram of a typical system (Figure 36.1, page 2). The picture shows a single CPU attached to the main memory of the system via some kind of **memory bus** or interconnect. Some devices are connected to the system via a general **I/O bus**, which in many modern systems would be **PCI** (or one of its many derivatives); graphics and some other higher-performance I/O devices might be found here. Finally, even lower down are one or more of what we call a **peripheral bus**, such as **SCSI**, **SATA**, or **USB**. These connect slow devices to the system, including **disks**, **mice**, and **keyboards**.

One question you might ask is: why do we need a hierarchical structure like this? Put simply: physics, and cost. The faster a bus is, the shorter it must be; thus, a high-performance memory bus does not have much room to plug devices and such into it. In addition, engineering a bus for high performance is quite costly. Thus, system designers have adopted this hierarchical approach, where components that demand high performance (such as the graphics card) are nearer the CPU. Lower performance components are further away. The benefits of placing disks and other slow devices on a peripheral bus are manifold; in particular, you can place a large number of devices on it.

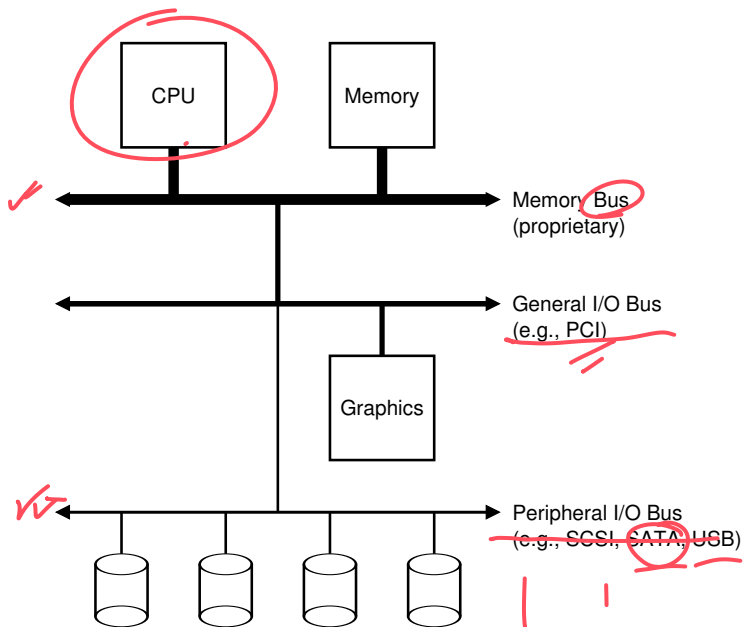


Figure 36.1: Prototypical System Architecture

Of course, modern systems increasingly use specialized chipsets and faster point-to-point interconnects to improve performance. Figure 36.2 (page 3) shows an approximate diagram of Intel's Z270 Chipset [H17]. Along the top, the CPU connects most closely to the memory system, but also has a high-performance connection to the graphics card (and thus, the display) to enable gaming (oh, the horror!) and other graphics-intensive applications.

The CPU connects to an I/O chip via Intel's proprietary **DMI (Direct Media Interface)**, and the rest of the devices connect to this chip via a number of different interconnects. On the right, one or more hard drives connect to the system via the **eSATA** interface; **ATA** (the **AT Attachment**, in reference to providing connection to the IBM PC AT), then **SATA** (for **Serial ATA**), and now eSATA (for **external SATA**) represent an evolution of storage interfaces over the past decades, with each step forward increasing performance to keep pace with modern storage devices.

Below the I/O chip are a number of **USB (Universal Serial Bus)** connections, which in this depiction enable a keyboard and mouse to be attached to the computer. On many modern systems, USB is used for low performance devices such as these.

Finally, on the left, other higher performance devices can be connected

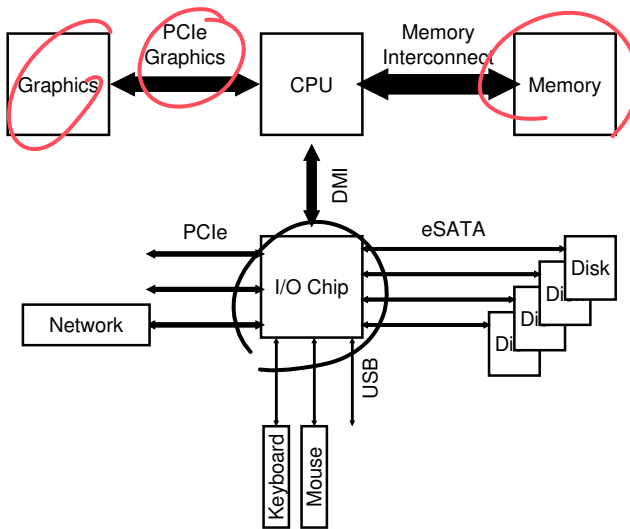


Figure 36.2: Modern System Architecture

to the system via **PCIe (Peripheral Component Interconnect Express)**. In this diagram, a network interface is attached to the system here; higher performance storage devices (such as **NVMe** persistent storage devices) are often connected here.

36.2 A Canonical Device

Let us now look at a canonical device (not a real one), and use this device to drive our understanding of some of the machinery required to make device interaction efficient. From Figure 36.3 (page 4), we can see that a device has two important components. The first is the hardware **interface** it presents to the rest of the system. Just like a piece of software, hardware must also present some kind of interface that allows the system software to control its operation. Thus, all devices have some specified interface and protocol for typical interaction.

The second part of any device is its **internal structure**. This part of the device is implementation specific and is responsible for implementing the abstraction the device presents to the system. Very simple devices will have one or a few hardware chips to implement their functionality; more complex devices will include a simple CPU, some general purpose memory, and other device-specific chips to get their job done. For example, modern RAID controllers might consist of hundreds of thousands of lines of **firmware** (i.e., software within a hardware device) to implement its functionality.

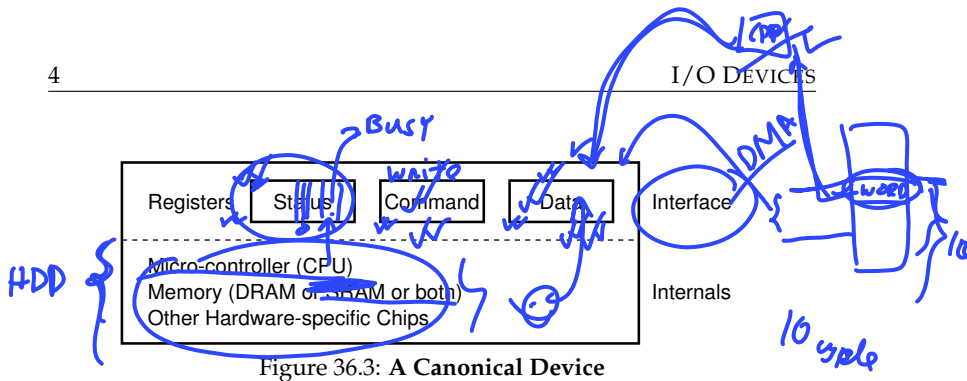


Figure 36.3: A Canonical Device

36.3 The Canonical Protocol

In the picture above, the (simplified) device interface is comprised of three registers: a **status** register, which can be read to see the current status of the device; a **command** register, to tell the device to perform a certain task; and a **data** register to pass data to the device, or get data from the device. By reading and writing these registers, the operating system can control device behavior.

Let us now describe a typical interaction that the OS might have with the device in order to get the device to do something on its behalf. The protocol is as follows:

```

HDD {
  While (STATUS == BUSY)
    ; // wait until device is not busy
  Write data to DATA register
  Write command to COMMAND register
    (starts the device and executes the command)
  While (STATUS == BUSY)
    ; // wait until device is done with your request

```

The protocol has four steps. In the first, the OS waits until the device is ready to receive a command by repeatedly reading the status register; we call this **polling** the device (basically, just asking it what is going on). Second, the OS sends some data down to the data register; one can imagine that if this were a disk, for example, that multiple writes would need to take place to transfer a disk block (say 4KB) to the device. When the main CPU is involved with the data movement (as in this example protocol), we refer to it as **programmed I/O (PIO)**. Third, the OS writes a command to the command register; doing so implicitly lets the device know that both the data is present and that it should begin working on the command. Finally, the OS waits for the device to finish by again polling it in a loop, waiting to see if it is finished (it may then get an error code to indicate success or failure).

This basic protocol has the positive aspect of being simple and working. However, there are some inefficiencies and inconveniences involved. The first problem you might notice in the protocol is that polling seems inefficient; specifically, it wastes a great deal of CPU time just waiting for the (potentially slow) device to complete its activity, instead of switching to another ready process and thus better utilizing the CPU.

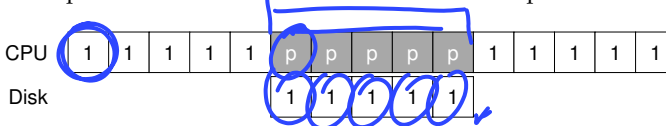
THE CRUX: HOW TO AVOID THE COSTS OF POLLING

How can the OS check device status without frequent polling, and thus lower the CPU overhead required to manage the device?

36.4 Lowering CPU Overhead With Interrupts

The invention that many engineers came upon years ago to improve this interaction is something we've seen already: the **interrupt**. Instead of polling the device repeatedly, the OS can issue a request, put the calling process to sleep, and context switch to another task. When the device is finally finished with the operation, it will raise a hardware interrupt, causing the CPU to jump into the OS at a predetermined **interrupt service routine (ISR)** or more simply an **interrupt handler**. The handler is just a piece of operating system code that will finish the request (for example, by reading data and perhaps an error code from the device) and wake the process waiting for the I/O, which can then proceed as desired.

Interrupts thus allow for **overlap** of computation and I/O, which is key for improved utilization. This timeline shows the problem:



In the diagram, Process 1 runs on the CPU for some time (indicated by a repeated 1 on the CPU line), and then issues an I/O request to the disk to read some data. Without interrupts, the system simply spins, polling the status of the device repeatedly until the I/O is complete (indicated by a p). The disk services the request and finally Process 1 can run again.

If instead we utilize interrupts and allow for overlap, the OS can do something else while waiting for the disk:



In this example, the OS runs Process 2 on the CPU while the disk services Process 1's request. When the disk request is finished, an interrupt occurs, and the OS wakes up Process 1 and runs it again. Thus, *both* the CPU and the disk are properly utilized during the middle stretch of time.

Note that using interrupts is not *always* the best solution. For example, imagine a device that performs its tasks very quickly: the first poll usually finds the device to be done with task. Using an interrupt in this case will actually *slow down* the system: switching to another process, handling the interrupt, and switching back to the issuing process is expensive. Thus, if a device is fast, it may be best to poll; if it is slow, interrupts, which allow

TIP: INTERRUPTS NOT ALWAYS BETTER THAN POLLING

Although interrupts allow for overlap of computation and I/O, they only really make sense for slow devices. Otherwise, the cost of interrupt handling and context switching may outweigh the benefits interrupts provide. There are also cases where a flood of interrupts may overload a system and lead it to livelock [MR96]; in such cases, polling provides more control to the OS in its scheduling and thus is again useful.

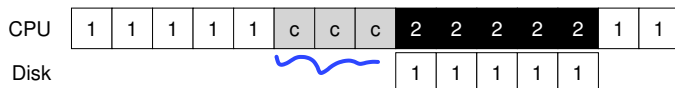
overlap, are best. If the speed of the device is not known, or sometimes fast and sometimes slow, it may be best to use a **hybrid** that polls for a little while and then, if the device is not yet finished, uses interrupts. This **two-phased** approach may achieve the best of both worlds.

Another reason not to use interrupts arises in networks [MR96]. When a huge stream of incoming packets each generate an interrupt, it is possible for the OS to **livelock**, that is, find itself only processing interrupts and never allowing a user-level process to run and actually service the requests. For example, imagine a **web server** that experiences a load burst because it became the top-ranked entry on hacker news [H18]. In this case, it is better to occasionally use polling to better control what is happening in the system and allow the web server to service some requests before going back to the device to check for more packet arrivals.

Another interrupt-based optimization is **coalescing**. In such a setup, a device which needs to raise an interrupt first waits for a bit before delivering the interrupt to the CPU. While waiting, other requests may soon complete, and thus multiple interrupts can be coalesced into a single interrupt delivery, thus lowering the overhead of interrupt processing. Of course, waiting too long will increase the latency of a request, a common trade-off in systems. See Ahmad et al. [A+11] for an excellent summary.

36.5 More Efficient Data Movement With DMA

Unfortunately, there is one other aspect of our canonical protocol that requires our attention. In particular, when using programmed I/O (PIO) to transfer a large chunk of data to a device, the CPU is once again overburdened with a rather trivial task, and thus wastes a lot of time and effort that could better be spent running other processes. This timeline illustrates the problem:



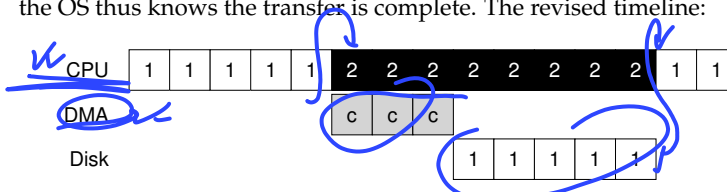
In the timeline, Process 1 is running and then wishes to write some data to the disk. It then initiates the I/O, which must copy the data from memory to the device explicitly, one word at a time (marked **c** in the diagram). When the copy is complete, the I/O begins on the disk and the CPU can finally be used for something else.

THE CRUX: HOW TO LOWER PIO OVERHEADS

With PIO, the CPU spends too much time moving data to and from devices by hand. How can we offload this work and thus allow the CPU to be more effectively utilized?

The solution to this problem is something we refer to as **Direct Memory Access (DMA)**. A DMA engine is essentially a very specific device within a system that can orchestrate transfers between devices and main memory without much CPU intervention.

DMA works as follows. To transfer data to the device, for example, the OS would program the DMA engine by telling it where the data lives in memory, how much data to copy, and which device to send it to. At that point, the OS is done with the transfer and can proceed with other work. When the DMA is complete, the DMA controller raises an interrupt, and the OS thus knows the transfer is complete. The revised timeline:



From the timeline, you can see that the copying of data is now handled by the DMA controller. Because the CPU is free during that time, the OS can do something else, here choosing to run Process 2. Process 2 thus gets to use more CPU before Process 1 runs again.

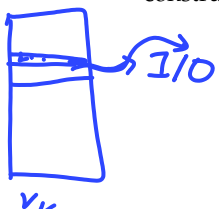
36.6 Methods Of Device Interaction

Now that we have some sense of the efficiency issues involved with performing I/O, there are a few other problems we need to handle to incorporate devices into modern systems. One problem you may have noticed thus far: we have not really said anything about how the OS actually communicates with the device! Thus, the problem:

THE CRUX: HOW TO COMMUNICATE WITH DEVICES

How should the hardware communicate with a device? Should there be explicit instructions? Or are there other ways to do it?

Over time, two primary methods of device communication have developed. The first, oldest method (used by IBM mainframes for many years) is to have explicit **I/O instructions**. These instructions specify a way for the OS to send data to specific device registers and thus allow the construction of the protocols described above.



HDD-1
mouse-2

in 1.2.3.4, (2)

For example, on x86, the in and out instructions can be used to communicate with devices. For example, to send data to a device, the caller specifies a register with the data in it, and a specific *port* which names the device. Executing the instruction leads to the desired behavior.

Such instructions are usually **privileged**. The OS controls devices, and the OS thus is the only entity allowed to directly communicate with them. Imagine if any program could read or write the disk, for example: total chaos (as always), as any user program could use such a loophole to gain complete control over the machine.

The second method to interact with devices is known as **memory-mapped I/O**. With this approach, the hardware makes device registers available as if they were memory locations. To access a particular register, the OS issues a load (to read) or store (to write) the address; the hardware then routes the load/store to the device instead of main memory.

There is not some great advantage to one approach or the other. The memory-mapped approach is nice in that no new instructions are needed to support it, but both approaches are still in use today.

36.7 Fitting Into The OS: The Device Driver

One final problem we will discuss: how to fit devices, each of which have very specific interfaces, into the OS, which we would like to keep as general as possible. For example, consider a file system. We'd like to build a file system that worked on top of SCSI disks, IDE disks, USB keychain drives, and so forth, and we'd like the file system to be relatively oblivious to all of the details of how to issue a read or write request to these different types of drives. Thus, our problem:

THE CRUX: HOW TO BUILD A DEVICE-NEUTRAL OS

How can we keep most of the OS device-neutral, thus hiding the details of device interactions from major OS subsystems?

The problem is solved through the age-old technique of **abstraction**. At the lowest level, a piece of software in the OS must know in detail how a device works. We call this piece of software a **device driver**, and any specifics of device interaction are encapsulated within.

Let us see how this abstraction might help OS design and implementation by examining the Linux file system software stack. Figure 36.4 is a rough and approximate depiction of the Linux software organization. As you can see from the diagram, a file system (and certainly, an application above) is completely oblivious to the specifics of which disk class it is using; it simply issues block read and write requests to the generic block layer, which routes them to the appropriate device driver, which handles the details of issuing the specific request. Although simplified, the diagram shows how such detail can be hidden from most of the OS.

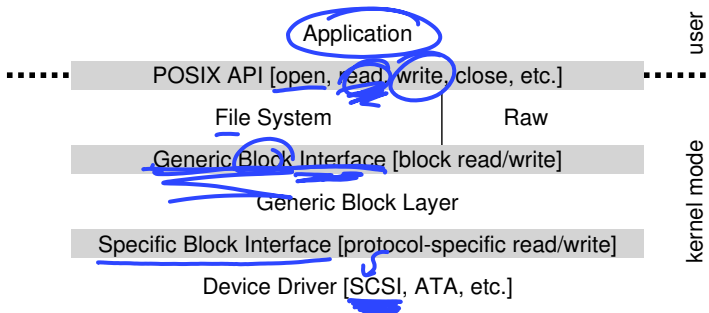


Figure 36.4: The File System Stack

The diagram also shows a **raw interface** to devices, which enables special applications (such as a **file-system checker**, described later [AD14], or a **disk defragmentation** tool) to directly read and write blocks without using the file abstraction. Most systems provide this type of interface to support these low-level storage management applications.

Note that the encapsulation seen above can have its downside as well. For example, if there is a device that has many special capabilities, but has to present a generic interface to the rest of the kernel, those special capabilities will go unused. This situation arises, for example, in Linux with SCSI devices, which have very rich error reporting; because other block devices (e.g., ATA/IDE) have much simpler error handling, all that higher levels of software ever receive is a generic EIO (generic IO error) error code; any extra detail that SCSI may have provided is thus lost to the file system [G08].

Interestingly, because device drivers are needed for any device you might plug into your system, over time they have come to represent a huge percentage of kernel code. Studies of the Linux kernel reveal that over 70% of OS code is found in device drivers [C01]; for Windows-based systems, it is likely quite high as well. Thus, when people tell you that the OS has millions of lines of code, what they are really saying is that the OS has millions of lines of device-driver code. Of course, for any given installation, most of that code may not be active (i.e., only a few devices are connected to the system at a time). Perhaps more depressingly, as drivers are often written by “amateurs” (instead of full-time kernel developers), they tend to have many more bugs and thus are a primary contributor to kernel crashes [S03].

36.8 Case Study: A Simple IDE Disk Driver

To dig a little deeper here, let’s take a quick look at an actual device: an IDE disk drive [L94]. We summarize the protocol as described in this reference [W10]; we’ll also peek at the xv6 source code for a simple example of a working IDE driver [CK+08].

Control Register:

Address 0x3F6 = 0x08 (0000 1RE0): R=reset,
E=0 means "enable interrupt"

Command Block Registers:

Address 0x1F0 = Data Port
Address 0x1F1 = Error
Address 0x1F2 = Sector Count
Address 0x1F3 = LBA low byte
Address 0x1F4 = LBA mid byte
Address 0x1F5 = LBA hi byte
Address 0x1F6 = 1B1D TOP4LBA: B=LBA, D=drive
Address 0x1F7 = Command/status

Status Register (Address 0x1F7):

7	6	5	4	3	2	1	0
BUSY	READY	FAULT	SEEK	DRQ	CORR	IDDEX	ERROR

Error Register (Address 0x1F1): (check when ERROR==1)

7	6	5	4	3	2	1	0
BBK	UNC	MC	IDNF	MCR	ABRT	T0NF	AMNF

BBK = Bad Block
UNC = Uncorrectable data error
MC = Media Changed
IDNF = ID mark Not Found
MCR = Media Change Requested
ABRT = Command aborted
T0NF = Track 0 Not Found
AMNF = Address Mark Not Found

Figure 36.5: The IDE Interface

An IDE disk presents a simple interface to the system, consisting of four types of register: control, command block, status, and error. These registers are available by reading or writing to specific "I/O addresses" (such as 0x3F6 below) using (on x86) the `in` and `out` I/O instructions.

The basic protocol to interact with the device is as follows, assuming it has already been initialized.

- **Wait for drive to be ready.** Read Status Register (0x1F7) until drive is READY and not BUSY.
- **Write parameters to command registers.** Write the sector count, logical block address (LBA) of the sectors to be accessed, and drive number (master=0x00 or slave=0x10, as IDE permits just two drives) to command registers (0x1F2-0x1F6).
- **Start the I/O.** Write READ | WRITE command to command register (0x1F7).

- **Data transfer (for writes):** Wait until drive status is READY and DRQ (drive request for data); write data to data port.
- **Handle interrupts.** In the simplest case, handle an interrupt for each sector transferred; more complex approaches allow batching and thus one final interrupt when the entire transfer is complete.
- **Error handling.** After each operation, read the status register. If the ERROR bit is on, read the error register for details.

Most of this protocol is found in the xv6 IDE driver (Figure 36.6), which (after initialization) works through four primary functions. The first is `ide_rw()`, which queues a request (if there are others pending), or issues it directly to the disk (via `ide_start_request()`); in either case, the routine waits for the request to complete and the calling process is put to sleep. The second is `ide_start_request()`, which is used to send a request (and perhaps data, in the case of a write) to the disk; the `in` and `out` x86 instructions are called to read and write device registers, respectively. The start request routine uses the third function, `ide_wait_ready()`, to ensure the drive is ready before issuing a request to it. Finally, `ide_intr()` is invoked when an interrupt takes place; it reads data from the device (if the request is a read, not a write), wakes the process waiting for the I/O to complete, and (if there are more requests in the I/O queue), launches the next I/O via `ide_start_request()`.

36.9 Historical Notes

Before ending, we include a brief historical note on the origin of some of these fundamental ideas. If you are interested in learning more, read Smotherman's excellent summary [S08].

Interrupts are an ancient idea, existing on the earliest of machines. For example, the UNIVAC in the early 1950's had some form of interrupt vectoring, although it is unclear in exactly which year this feature was available [S08]. Sadly, even in its infancy, we are beginning to lose the origins of computing history.

There is also some debate as to which machine first introduced the idea of DMA. For example, Knuth and others point to the DYSEAC (a "mobile" machine, which at the time meant it could be hauled in a trailer), whereas others think the IBM SAGE may have been the first [S08]. Either way, by the mid 50's, systems with I/O devices that communicated directly with memory and interrupted the CPU when finished existed.

The history here is difficult to trace because the inventions are tied to real, and sometimes obscure, machines. For example, some think that the Lincoln Labs TX-2 machine was first with vectored interrupts [S08], but this is hardly clear.

```

static int ide_wait_ready() {
    while (((int r = inb(0x1f7)) & IDE_BSY) || !(r & IDE_DRDY))
        ; // loop until drive isn't busy
    // return -1 on error, or 0 otherwise
}

static void ide_start_request(struct buf *b) {
    ide_wait_ready();
    outb(0x3f6, 0); // generate interrupt
    outb(0x1f2, 1); // how many sectors?
    outb(0x1f3, b->sector & 0xff); // LBA goes here ...
    outb(0x1f4, (b->sector >> 8) & 0xff); // ... and here
    outb(0x1f5, (b->sector >> 16) & 0xff); // ... and here!
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f));
    if(b->flags & B_DIRTY){
        outb(0x1f7, IDE_CMD_WRITE); // this is a WRITE
        outsl(0x1f0, b->data, 512/4); // transfer data too!
    } else {
        outb(0x1f7, IDE_CMD_READ); // this is a READ (no data)
    }
}

void ide_rw(struct buf *b) {
    acquire(&ide_lock);
    for (struct buf **pp = &ide_queue; *pp; pp=&(*pp)->qnext)
        ; // walk queue
    *pp = b; // add request to end
    if (ide_queue == b) // if q is empty
        ide_start_request(b); // send req to disk
    while ((b->flags & (B_VALID|B_DIRTY)) != B_VALID)
        sleep(b, &ide_lock); // wait for completion
    release(&ide_lock);
}

void ide_intr() {
    struct buf *b;
    acquire(&ide_lock);
    if (!(b->flags & B_DIRTY) && ide_wait_ready() >= 0)
        insl(0x1f0, b->data, 512/4); // if READ: get data
    b->flags |= B_VALID;
    b->flags &= ~B_DIRTY;
    wakeup(b); // wake waiting process
    if ((ide_queue = b->qnext) != 0) // start next request
        ide_start_request(ide_queue); // (if one exists)
    release(&ide_lock);
}

```

Figure 36.6: The xv6 IDE Disk Driver (Simplified)

Because the ideas are relatively obvious — no Einsteinian leap is required to come up with the idea of letting the CPU do something else while a slow I/O is pending — perhaps our focus on “who first?” is misguided. What is certainly clear: as people built these early machines, it became obvious that I/O support was needed. Interrupts, DMA, and related ideas are all direct outcomes of the nature of fast CPUs and slow devices; if you were there at the time, you might have had similar ideas.

36.10 Summary

You should now have a very basic understanding of how an OS interacts with a device. Two techniques, the interrupt and DMA, have been introduced to help with device efficiency, and two approaches to accessing device registers, explicit I/O instructions and memory-mapped I/O, have been described. Finally, the notion of a device driver has been presented, showing how the OS itself can encapsulate low-level details and thus make it easier to build the rest of the OS in a device-neutral fashion.

References

- [A+11] “vIC: Interrupt Coalescing for Virtual Machine Storage Device IO” by Irfan Ahmad, Ajay Gulati, Ali Mashtizadeh. USENIX ’11. *A terrific survey of interrupt coalescing in traditional and virtualized environments.*
- [AD14] “Operating Systems: Three Easy Pieces” (Chapters: Crash Consistency: FSCCK and Journaling and Log-Structured File Systems) by Remzi Arpaci-Dusseau and Andrea Arpaci-Dusseau. Arpaci-Dusseau Books, 2014. *A description of a file-system checker and how it works, which requires low-level access to disk devices not normally provided by the file system directly.*
- [C01] “An Empirical Study of Operating System Errors” by Andy Chou, Junfeng Yang, Benjamin Chelf, Seth Hallem, Dawson Engler. SOSP ’01. *One of the first papers to systematically explore how many bugs are in modern operating systems. Among other neat findings, the authors show that device drivers have something like seven times more bugs than mainline kernel code.*
- [CK+08] “The xv6 Operating System” by Russ Cox, Frans Kaashoek, Robert Morris, Nickolai Zeldovich. From: <http://pdos.csail.mit.edu/6.828/2008/index.html>. See `ide.c` for the IDE device driver, with a few more details therein.
- [D07] “What Every Programmer Should Know About Memory” by Ulrich Drepper. November, 2007. Available: <http://www.akkadia.org/drepper/cpumemory.pdf>. *A fantastic read about modern memory systems, starting at DRAM and going all the way up to virtualization and cache-optimized algorithms.*
- [G08] “EIO: Error-handling is Occasionally Correct” by Haryadi Gunawi, Cindy Rubio-Gonzalez, Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau, Ben Liblit. FAST ’08, San Jose, CA, February 2008. *Our own work on building a tool to find code in Linux file systems that does not handle error return properly. We found hundreds and hundreds of bugs, many of which have now been fixed.*
- [H17] “Intel Core i7-7700K review: Kaby Lake Debuts for Desktop” by Joel Hruska. January 3, 2017. www.extremetech.com/extreme/241950-intels-core-i7-7700k-reviewed-kaby-lake-debuts-desktop. *An in-depth review of a recent Intel chipset, including CPUs and the I/O subsystem.*
- [H18] “Hacker News” by Many contributors. Available: <https://news.ycombinator.com>. *One of the better aggregators for tech-related stuff. Once back in 2014, this book became a highly-ranked entry, leading to 1 million chapter downloads in just one day! Sadly, we have yet to re-experience such a high.*
- [L94] “AT Attachment Interface for Disk Drives” by Lawrence J. Lamers. Reference number: ANSI X3.221, 1994. Available: <ftp://ftp.t10.org/t13/project/d0791r4c-ATA-1.pdf>. *A rather dry document about device interfaces. Read it at your own peril.*
- [MR96] “Eliminating Receive Livelock in an Interrupt-driven Kernel” by Jeffrey Mogul, K. K. Ramakrishnan. USENIX ’96, San Diego, CA, January 1996. *Mogul and colleagues did a great deal of pioneering work on web server network performance. This paper is but one example.*
- [S08] “Interrupts” by Mark Smotherman. July ’08. Available: <http://people.cs.clemson.edu/~mark/interrupts.html>. *A treasure trove of information on the history of interrupts, DMA, and related early ideas in computing.*
- [S03] “Improving the Reliability of Commodity Operating Systems” by Michael M. Swift, Brian N. Bershad, Henry M. Levy. SOSP ’03. *Swift’s work revived interest in a more microkernel-like approach to operating systems; minimally, it finally gave some good reasons why address-space based protection could be useful in a modern OS.*
- [W10] “Hard Disk Driver” by Washington State Course Homepage. Available online at this site: <http://eeecs.wsu.edu/~cs460/cs560/HDDriver.html>. *A nice summary of a simple IDE disk drive’s interface and how to build a device driver for it.*

Operating Systems

Youjip Won

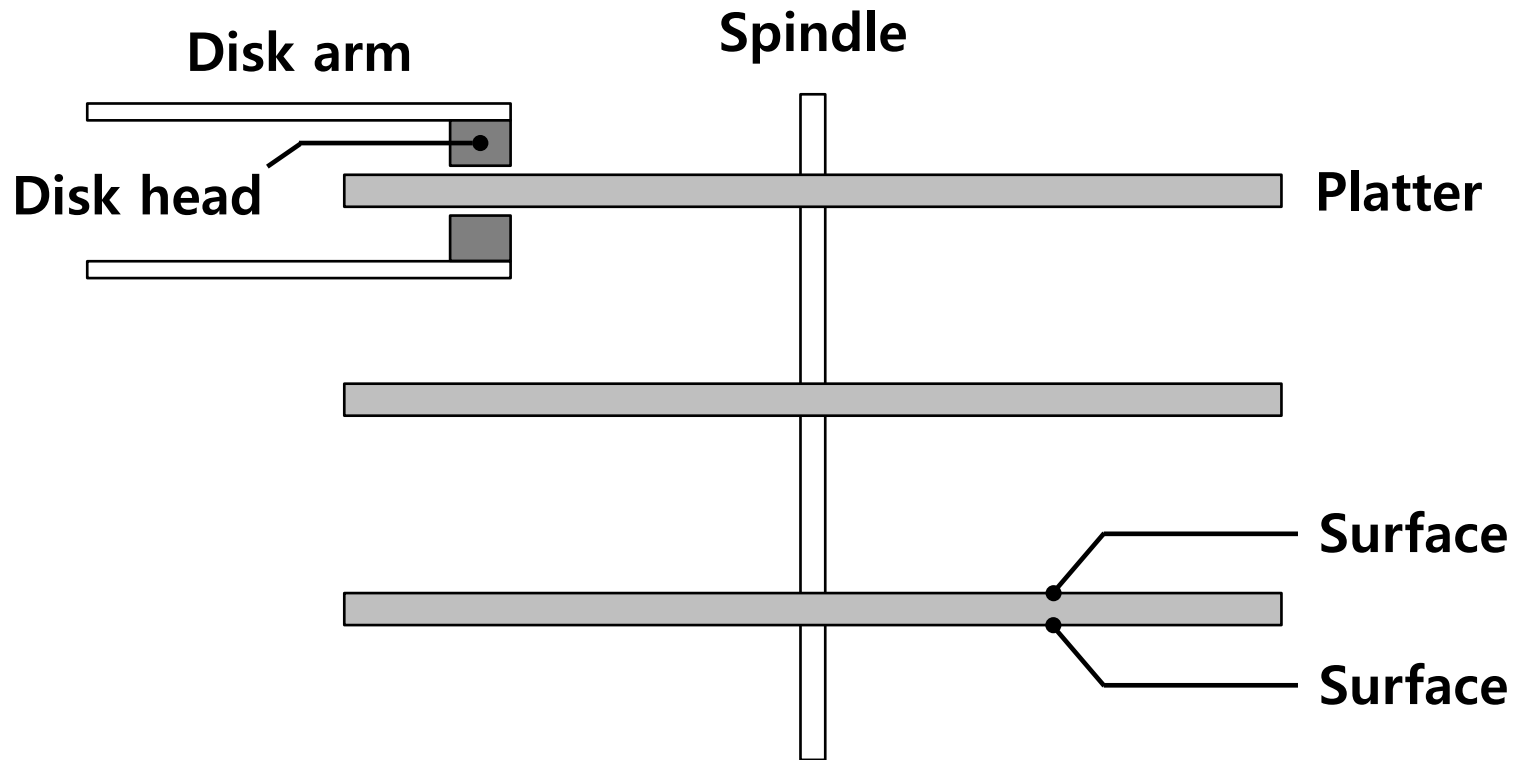
KAIST



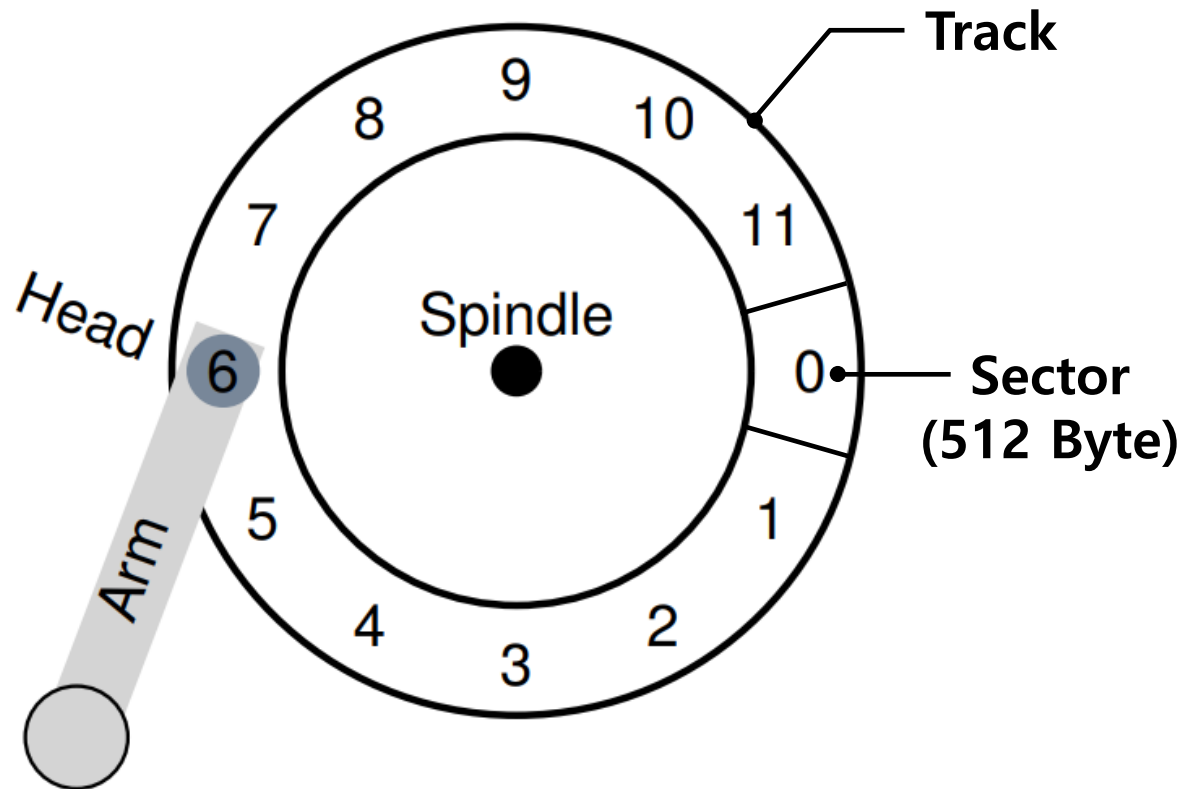
37. Hard Disk Drives

Operating System: Three Easy Pieces

Base Geometry

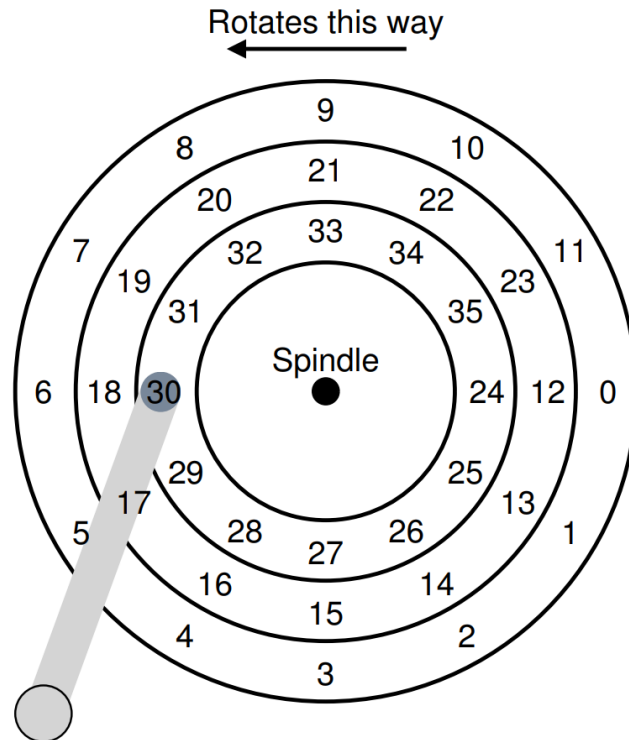


Base Geometry (Cont.)



A Simple Disk Drive

▣ Rotation Delay

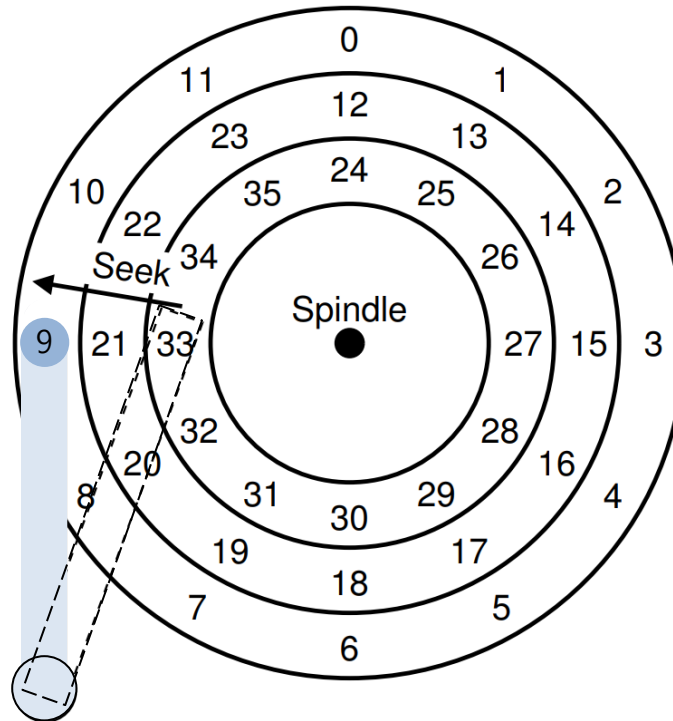


If the full rotation delay is R ,

the rotation delay of $(30 \rightarrow 24)$ is $\frac{R}{2}$

A Simple Disk Drive (Cont.)

▣ Seek Time



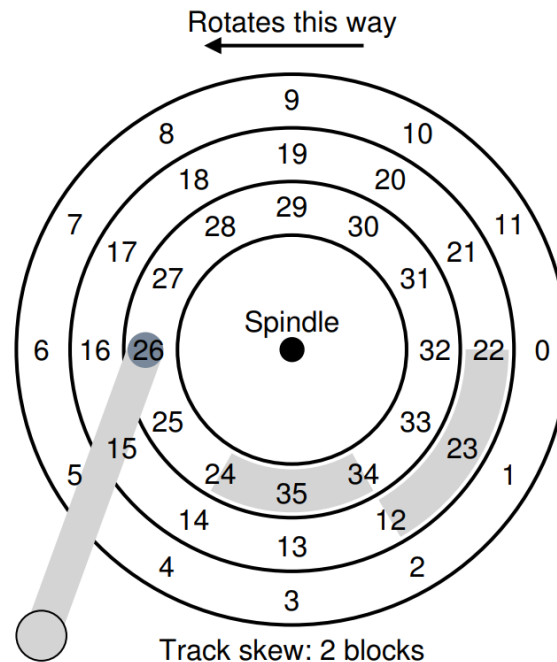
Phases of seek

Acceleration → *Coasting* → *Deceleration* → *Setting time*
(about 0.5~2 ms)

A Simple Disk Drive (Cont.)

▣ Track skew

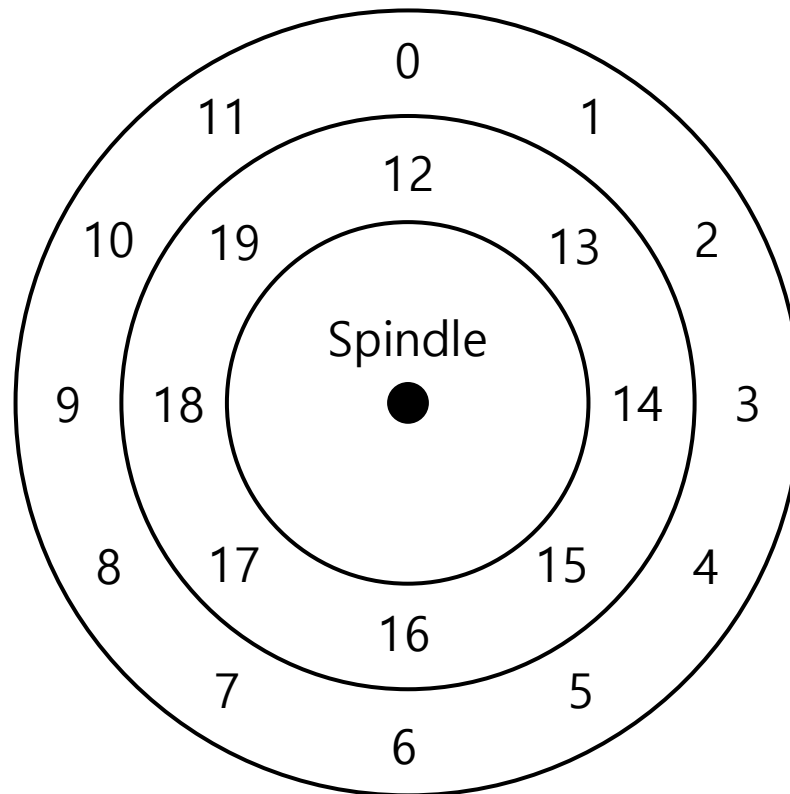
- ◆ Make sure that sequential reads can be properly serviced even when crossing track boundaries
- ◆ Without such skew, the head would be moved to the next track but the desired next block would have already rotated under the head



A Simple Disk Drive (Cont.)

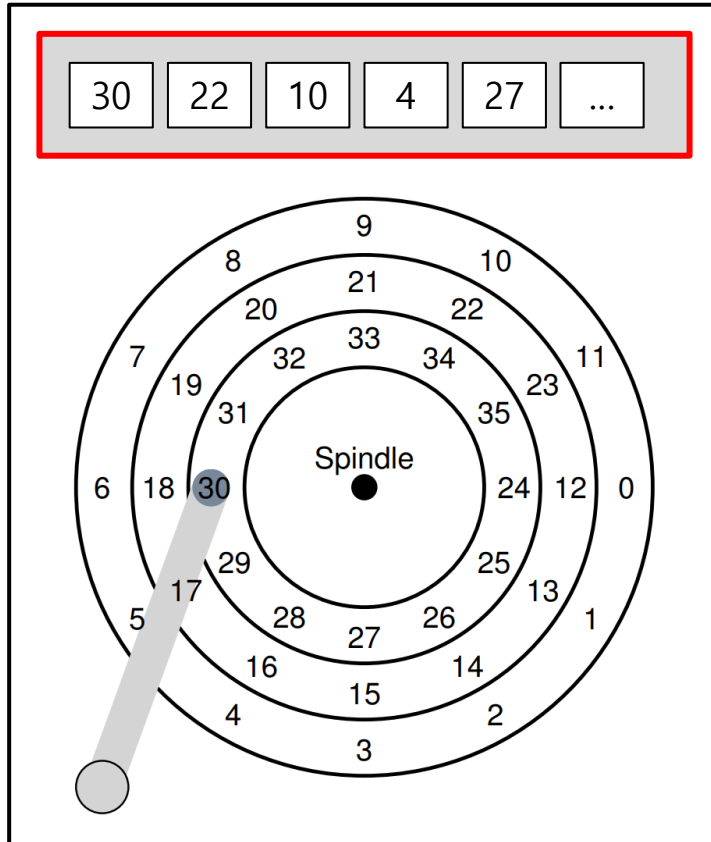
▣ Multi-zoned

- ◆ Outer tracks tend to have more sectors than inner tracks, because of result of geometry



A Simple Disk Drive (Cont.)

Cache (Track buffer)



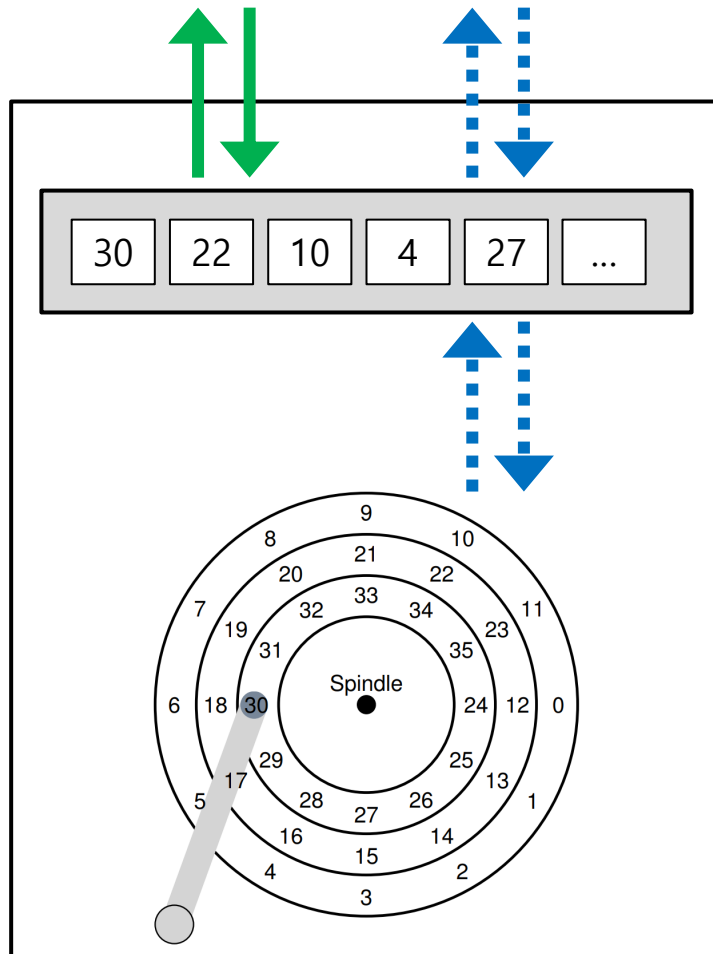
Small amount of memory
(usually around 8 or 16MB)

Hold data read from or written to the disk

Allow the drive to quickly respond to requests

A Simple Disk Drive (Cont.)

Cache (Track buffer)



→ Write-Back

Acknowledge the write has completed when it has put the data in its memory

→ Write-Through

Acknowledge after the write has actually been written to disk

I/O Time: Doing The Math

- ▣ I/O Time

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

- ▣ I/O Rate

$$R_{I/O} = \frac{Size_{Transfer}}{T_{I/O}}$$

I/O Time: Doing The Math (Cont.)

▣ 4KB Random Write Example

	Cheetah 15K.5	Barracuda
Capacity	300 GB	1 TB
RPM	15,000	7,200
Average Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s
Platters	4	4
Cache	16 MB	16/32 MB
Connects via	SCSI	SATA

$$T_{seek} = 4\text{ms}$$

$$T_{rotation} = 15,000 \text{ RPM}(= 250\text{RPS} = 4\text{ms} / 1 \text{ rotation}) / 2 \\ = 2\text{ms}$$

$$T_{transfer} = 4\text{KB} / 125(\text{MB/s}) \\ = 30\mu\text{s}$$

$$T_{I/O} = 4\text{ms} + 2\text{ms} + 30\mu\text{s} \approx 6\text{ms}$$

$$R_{I/O} = 4\text{KB} / 6\text{ms} = 0.66\text{MB/s}$$

I/O Time: Doing The Math (Cont.)

▣ 4KB Random Write Example (Cont.)

	Cheetah 15K.5	Barracuda
Capacity	300 GB	1 TB
RPM	15,000	7,200
Average Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s
Platters	4	4
Cache	16 MB	16/32 MB
Connects via	SCSI	SATA

$$T_{seek} = 9\text{ms}$$

$$T_{rotation} = 7,200 \text{ RPM}(= 120\text{RPS} = 8\text{ms} / 1 \text{ rotation}) / 2 \\ = 4\text{ms}$$

$$T_{transfer} = 4\text{KB} / 105(\text{MB/s}) \\ = 38\mu\text{s}$$

$$T_{I/O} = 9\text{ms} + 4\text{ms} + 38\mu\text{s} \approx 13\text{ms}$$

$$R_{I/O} = 4\text{KB} / 13\text{ms} = 0.31\text{MB/s}$$

I/O Time: Doing The Math (Cont.)

▣ Sequential Write Example

	Cheetah 15K.5	Barracuda
Capacity	300 GB	1 TB
RPM	15,000	7,200
Average Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s
Platters	4	4
Cache	16 MB	16/32 MB
Connects via	SCSI	SATA

$$T_{seek} = 4\text{ms}$$

$$\begin{aligned} T_{rotation} &= 15,000 \text{ RPM} (= 250\text{RPS} = 4\text{ms} / 1 \text{ rotation}) / 2 \\ &= 2\text{ms} \end{aligned}$$

$$\begin{aligned} T_{transfer} &= 100\text{MB} / 125(\text{MB/s}) \\ &= 800\text{ms} \end{aligned}$$

$$T_{I/O} = 4\text{ms} + 2\text{ms} + 800\text{ms} = 806\text{ms} \approx 800\text{ms}$$

$$R_{I/O} = 100\text{MB} / 800\text{ms} = 125\text{MB/s}$$

I/O Time: Doing The Math (Cont.)

▣ Sequential Write Example (Cont.)

	Cheetah 15K.5	Barracuda
Capacity	300 GB	1 TB
RPM	15,000	7,200
Average Seek	4 ms	9 ms
Max Transfer	125 MB/s	105 MB/s
Platters	4	4
Cache	16 MB	16/32 MB
Connects via	SCSI	SATA

$$T_{seek} = 9\text{ms}$$

$$T_{rotation} = 7,200 \text{ RPM}(= 120\text{RPS} = 8\text{ms} / 1 \text{ rotation}) / 2 \\ = 4\text{ms}$$

$$T_{transfer} = 100\text{MB} / 105(\text{MB/s}) \\ = 950\text{ms}$$

$$T_{I/O} = 9\text{ms} + 4\text{ms} + 950\text{ms} = 963\text{ms} \approx 950\text{ms}$$

$$R_{I/O} = 100\text{MB} / 950\text{ms} = 105\text{MB/s}$$

I/O Time: Doing The Math (Cont.)

	Cheetah	Barracuda
$R_{I/O}$ Random	0.66 MB/s	0.31 MB/s
$R_{I/O}$ Sequential	125 MB/s	105 MB/s

Performance vs Capacity

	Cheetah	Barracuda
$R_{I/O}$ Random	0.66 MB/s	0.31 MB/s
$R_{I/O}$ Sequential	125 MB/s	105 MB/s

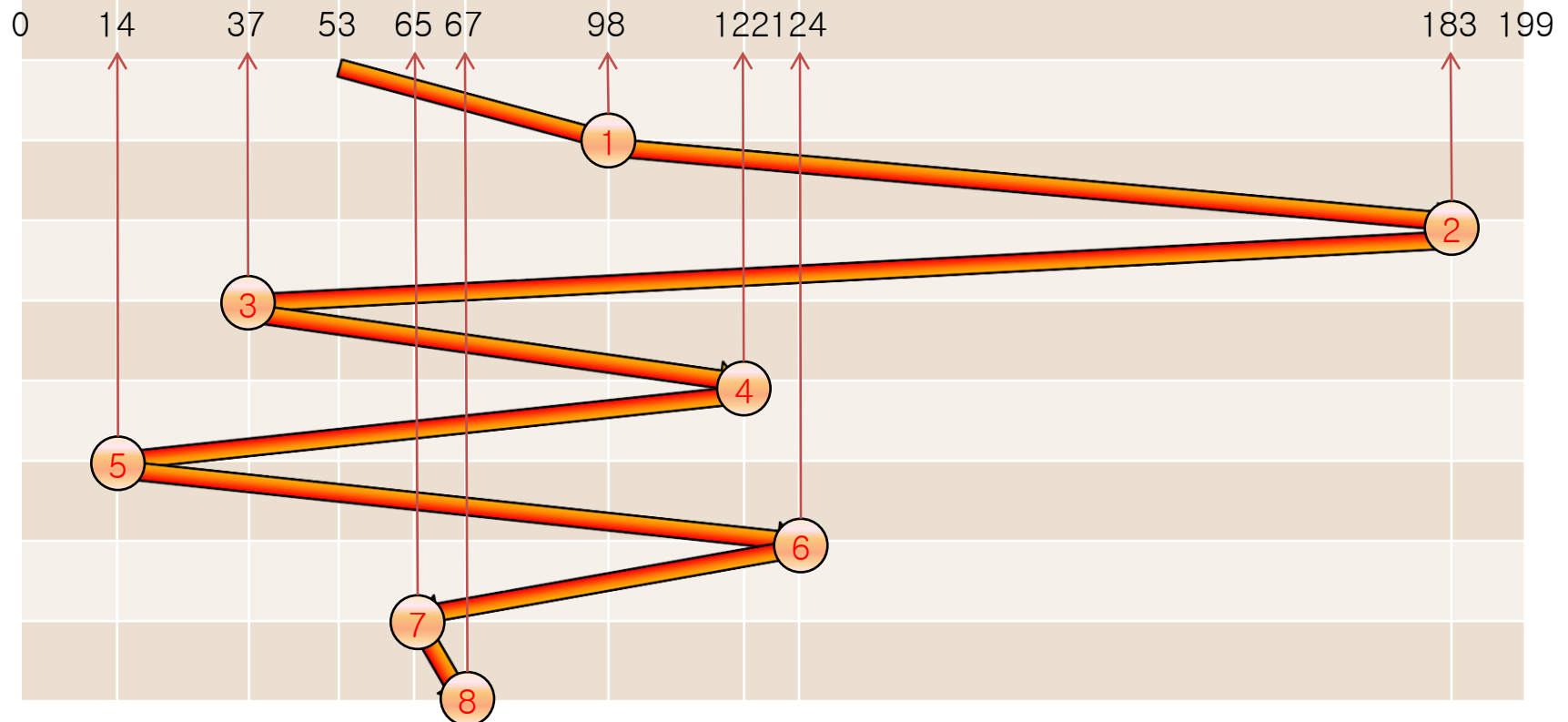
Random Write vs Sequential Write

Disk Scheduling: FCFS

How to order the services for the requests in the queue?

Illustration shows total head movement of 640 cylinders.

Queue = 98, 183, 37, 122, 14, 124, 65, 67
Head starts at 53

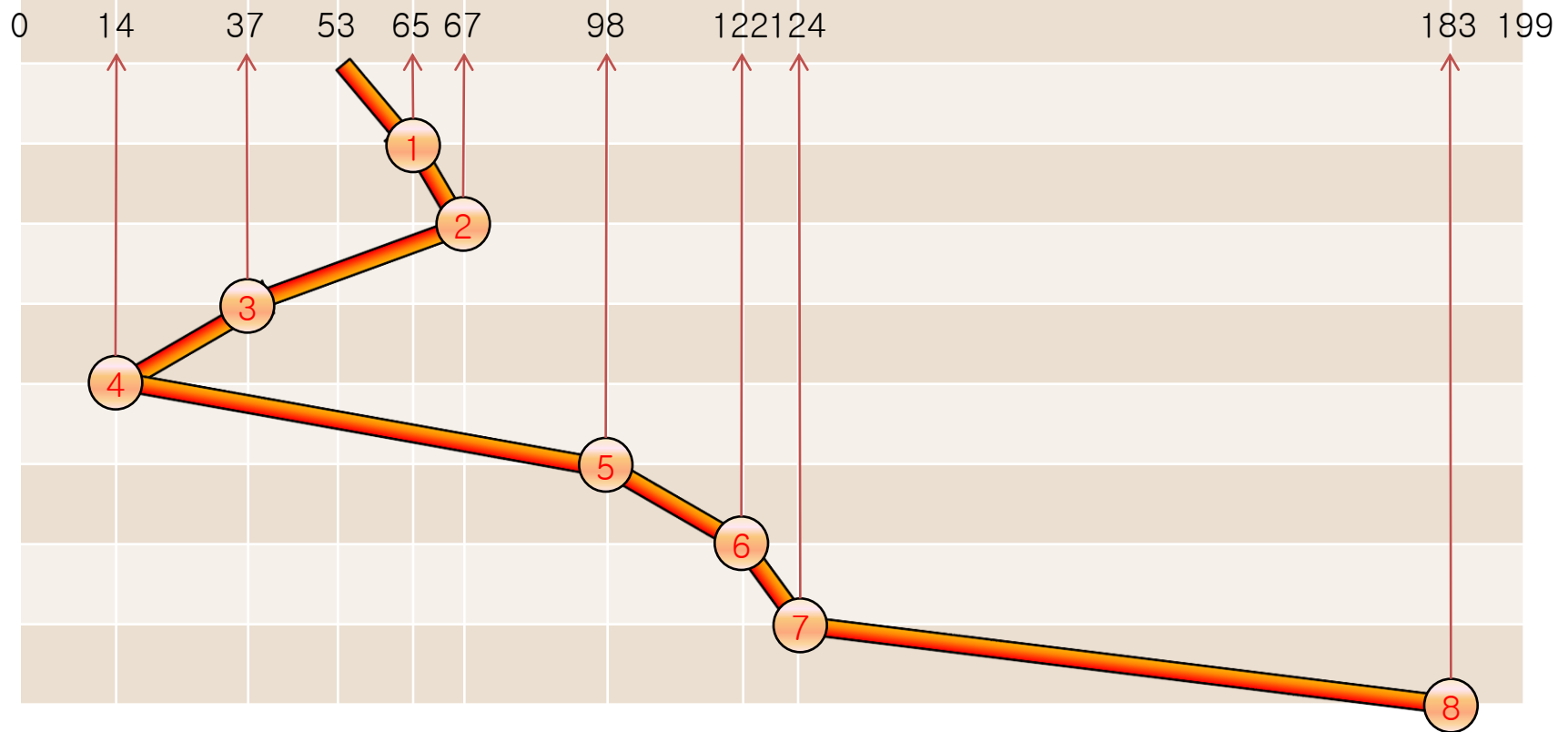


SSTF

- ❑ Selects the request with the minimum seek time from the current head position.
- ❑ SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests.
- ❑ Illustration shows total head movement of 236 cylinders.

SSTF (Cont.)

Queue = 98, 183, 37, 122, 14, 124, 65, 67
Head starts at 53

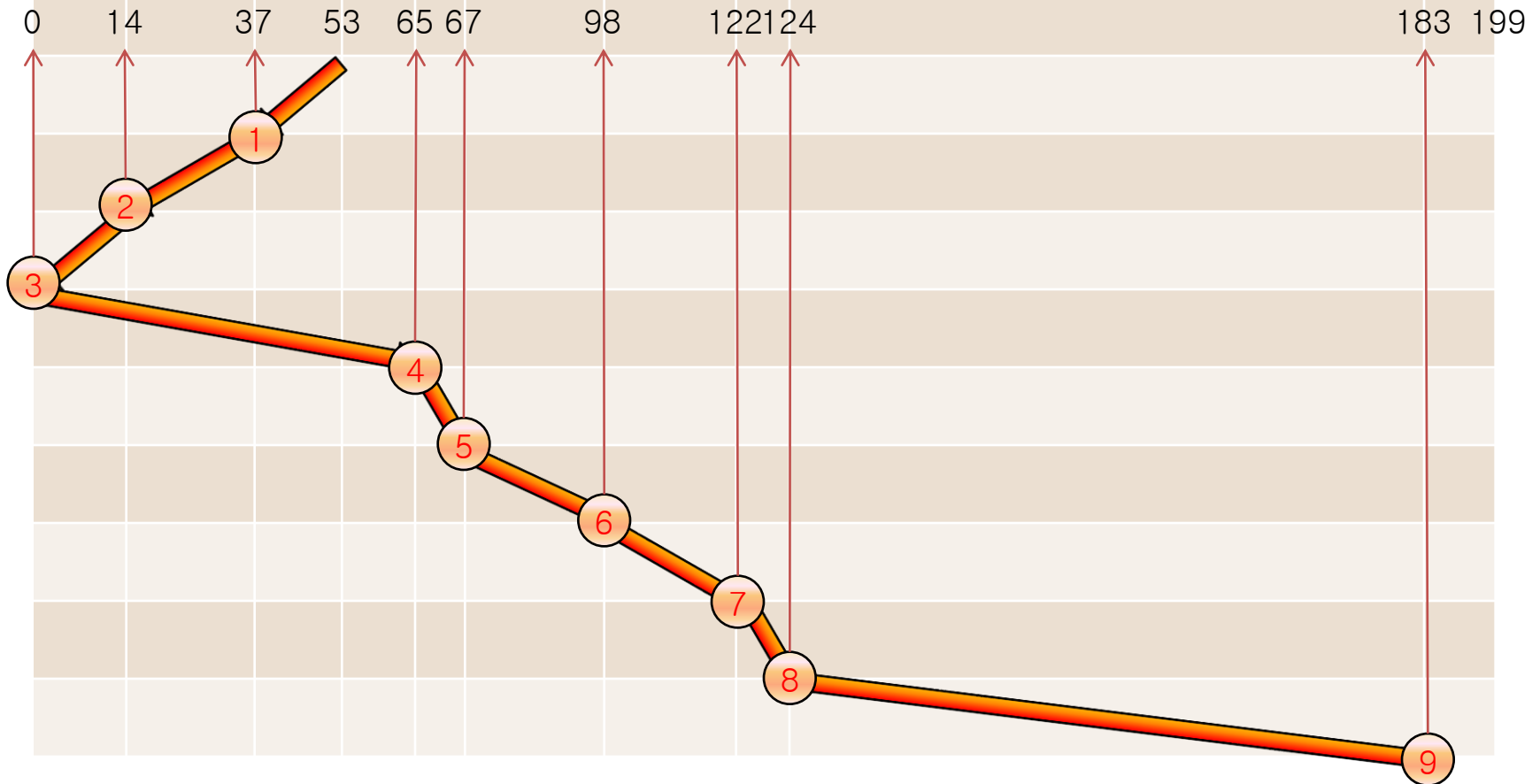


SCAN

- ▣ The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- ▣ Sometimes called the *elevator algorithm*.
- ▣ Illustration shows total head movement of 208 cylinders.

SCAN (Cont.)

Queue = 98, 183, 37, 122, 14, 124, 65, 67
Head starts at 53

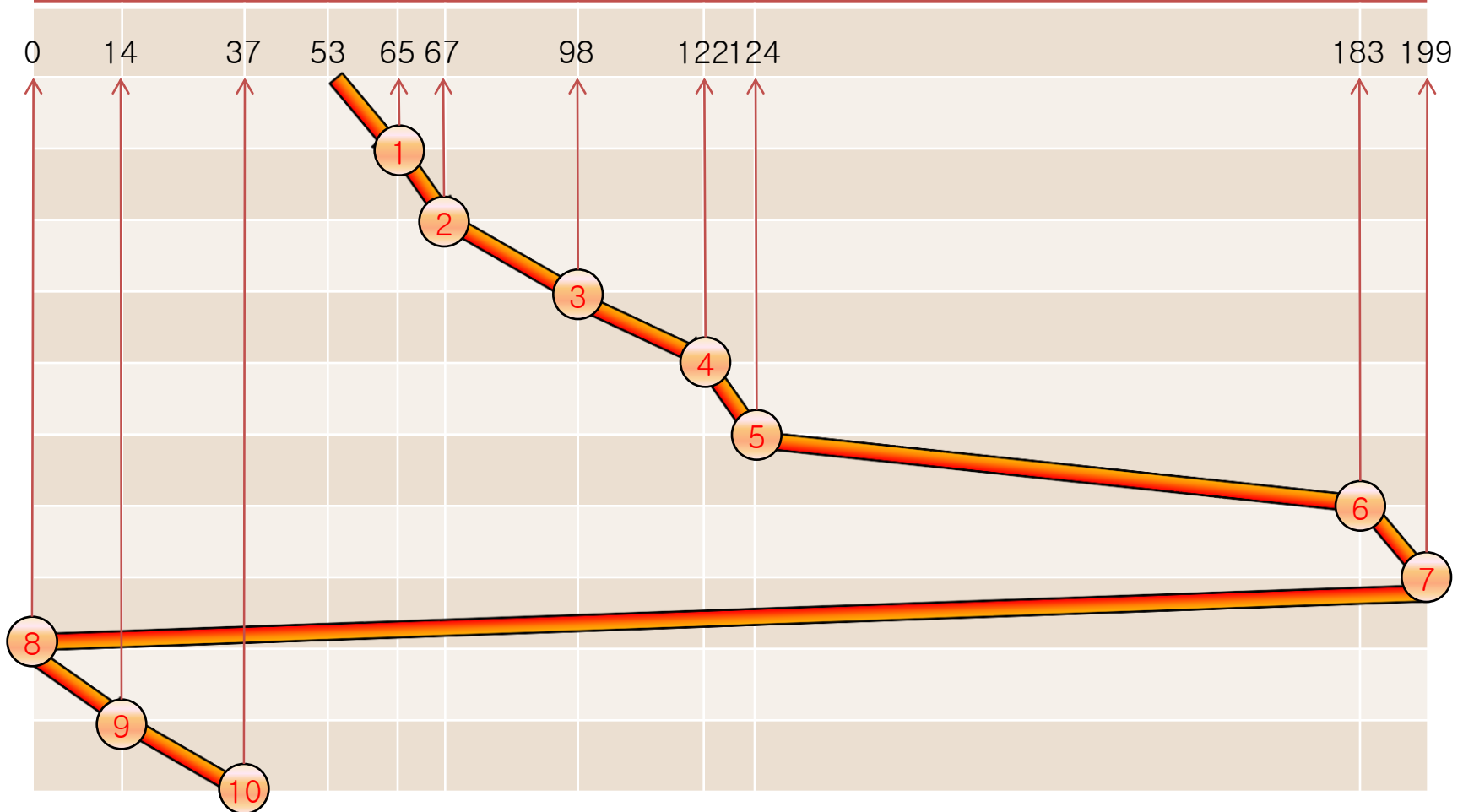


C-SCAN

- ▣ Provides a more uniform wait time than SCAN.
- ▣ The head moves from one end of the disk to the other, servicing requests as it goes. When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip.
- ▣ Treats the cylinders as a circular list that wraps around from the last cylinder to the first one.

C-SCAN (Cont.)

Queue = 98, 183, 37, 122, 14, 124, 65, 67
Head starts at 53

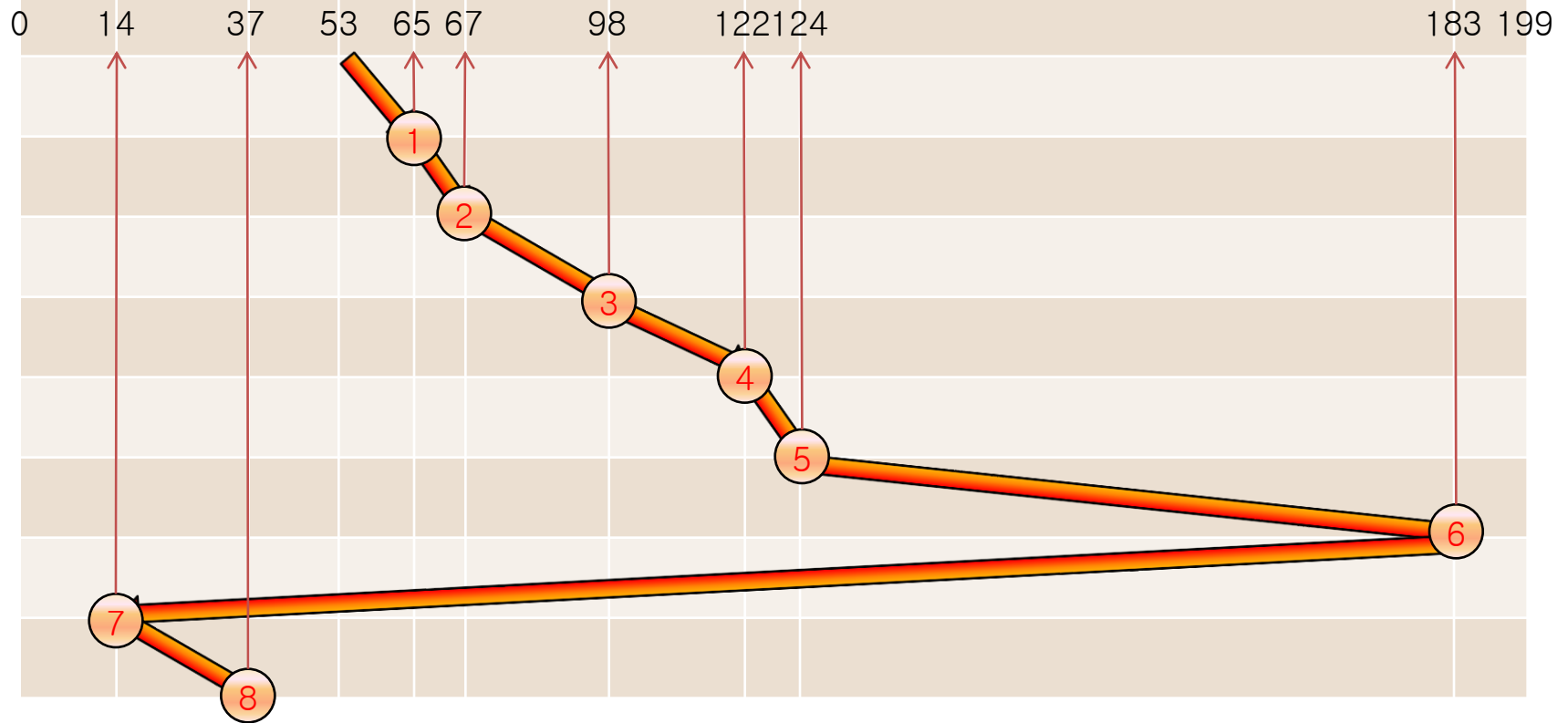


C-LOOK

- ▣ Version of C-SCAN
- ▣ Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk.

C-LOOK (Cont.)

Queue = 98, 183, 37, 122, 14, 124, 65, 67
Head starts at 53

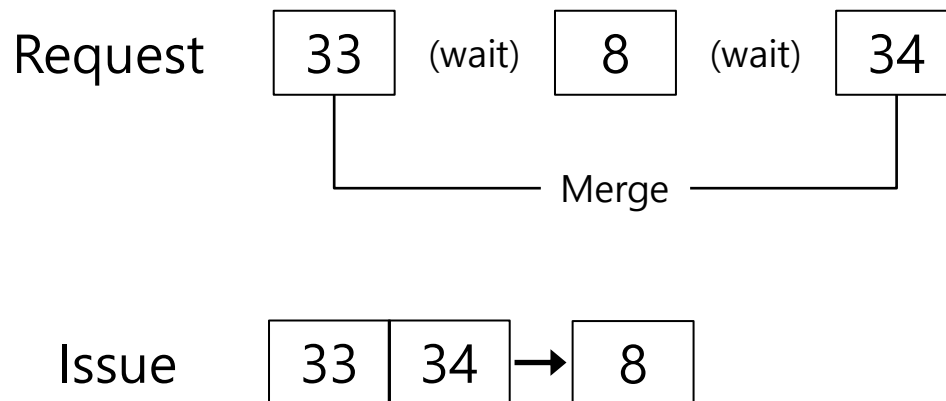


Selecting a Disk-Scheduling Algorithm

- ❑ SSTF is common and has a natural appeal
- ❑ SCAN and C-SCAN perform better for systems that place a heavy load on the disk.
- ❑ Performance depends on the number and types of requests.
- ❑ Requests for disk service can be influenced by the file-allocation method.
- ❑ The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary.
- ❑ Either SSTF or LOOK is a reasonable choice for the default algorithm.

Disk Scheduling (Cont.)

- ▣ I/O merging
 - ◆ Merge the continuous requests into a single two-block request
 - ◆ By waiting, a new and "better" request may arrive at the disk



Operating Systems

Youjip Won

KAIST



38. RAID

RAID (Redundant Array of Inexpensive Disks)

- ▣ **RAID** is to use multiple disks to build **faster, bigger, and more reliable** disk system.
- ▣ RAID is arranged into six different levels.
 - ◆ RAID Level 0: Striping multiple disks
 - ◆ RAID Level 1: Use mirroring
 - ◆ RAID Level 4, level 5: Parity based redundancy

□ Capacity

- ◆ N disks, B blocks per disk
- ◆ $N*B$ blocks in total → How much useful capacity is available to the clients of RAID?

□ Reliability

- ◆ How many disk faults can the RAID tolerate

□ Performance

- ◆ Read
- ◆ write

RAID Level 0

- RAID Level 0 is the simplest form as **striping** blocks.
 - ◆ Spread the blocks across the disks in a round-robin fashion.

Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

RAID-0: Simple Striping

RAID Level 0 (Cont.)

- ▣ Chunk size
 - ◆ Small chunk:
 - more intra-file parallelism
 - Larger positioning time: positioning time is the max positioning time of the disks
 - ◆ Large chunk:
 - Reduced intra-file parallelism
 - Smaller positioning time
- ▣ An example of RAID Label 0 with a bigger chunk size
 - ◆ Chunk size : 2 blocks (8KB)

Disk 0	Disk 1	Disk 2	Disk 3	
0	2	4	6	chunk size: 2 blocks
1	3	5	7	
5	10	12	14	
9	11	13	15	

Striping with a Bigger Chunk Size

RAID Level 0 Analysis

- ▣ Evaluate the capacity, reliability, performance of striping.
 - ◆ First way: single request latency
 - How much parallelism can exist during a single I/O operation.
 - ◆ Second way: steady-state throughput of the RAID:
 - Total bandwidth of many concurrent requests.

RAID Level 0 Analysis (Cont.)

▣ Single Disk

- ◆ Average seek time: 7 ms
- ◆ Average rotational delay: 3 ms
- ◆ Transfer rate of disk: 50 MB/s

▣ Single Disk Performance

- ◆ 10 Mbyte seq. IO, $S = \frac{\text{Amount of Data}}{\text{Time to access}} = \frac{10 \text{ MB}}{(7+3+200)=210 \text{ ms}} = 47.62 \text{ MB /s}$

- ◆ 10 Kbyte Random IO, $R = \frac{\text{Amount of Data}}{\text{Time to access}} = \frac{10 \text{ KB}}{(7+3+0.195)=10.195 \text{ ms}} = 0.981 \text{ MB /s}$

▣ RAID 0

- ◆ Random write, random read = $N \cdot R$
- ◆ Sequential write, sequential read = $N \cdot S$

RAID Level 1

- ▣ RAID Level 1 is mirroring
 - ◆ Copy more than one of each block in the system.
 - ◆ Copy block places on a separate disk to tolerate the disk failures.

Disk 0	Disk 1	Disk 2	Disk 3
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

Simple RAID-1: Mirroring

RAID level 1

- ▣ Capacity $N \cdot B / 2$
- ▣ Reliability
 - ◆ From one to upto $N/2$ depending upon the failure disk
- ▣ Performance
 - ◆ Sequential write: $N \cdot S / 2$
 - ◆ Sequential read: $N \cdot S / 2$
 - ◆ Random write: $N \cdot R / 2$
 - ◆ Random Read: $N \cdot R$

RAID Level 4

- RAID Level 4 is to add redundancy to a disk array as **parity**.

* P: Parity

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

Simple RAID-4 with parity

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	0	1	1	xor(0,0,1,1)
0	1	0	0	Xor(0,1,0,0)

RAID Level 4 (Cont.)

- ▣ The simple RAID Level 4 optimization known as a **Full-stripe write**.
 - ◆ Calculate the new value of P0 (Parity 0)
 - ◆ Write all of the blocks to the five disks above in parallel
 - ◆ Full-stripe writes are the most efficient way

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

Full-stripe Writes In RAID-4

Anlaysia

- Capacity: $(N-1)*B$
- Sequential read: $(N-1)*S$
- Sequential write: $(N-1)*S$ for full stripe write

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
4	5	6	7	P1
8	9	10	11	P2
12	13	14	15	P3

- Random read: $(N-1)*R$

Analysis

□ Random write:

- ◆ Additive Parity update: read all blocks, update the block, compute the new parity and write the updated block and the updated parity.
- ◆ Subtractive parity update: read the parity, write (new xor old) xor (old parity). (read on parity disk)
- ◆ For each write, the RAID perform **4 physical I/O**. (two read and writes)

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
*4	5	6	7	+P1
8	9	10	11	P2
12	*13	14	15	+P3

Random write performance: $(R/2)$ MB/sec

Small write problem happens

RAID Level 5

- ▣ RAID Level 5 is solution of small write problem.
 - ◆ small write problem cause parity-disk bottleneck of RAID Level 4.
 - ◆ works almost identically to RAID-4, except that it rotates the parity blocks across drives.
- ▣ RAID Level 5's Each stripe is now rotated across the disks.

Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

RAID-5 with Rotated Parity

Analysis

- ▣ Capacity: $(N-1)*B$
- ▣ Reliability: 1
- ▣ Performance
 - ◆ Sequential read, sequential write: $(N-1)S$
 - ◆ Random read: $N*R$
 - ◆ Random write: single write can cause 4 IO's (two read, two write), All N disks can work in parallel: $(N*R)/4$

Summary

	RAID-0	RAID-1	RAID-4	RAID-5
Capacity	N	N/2	N-1	N-1
Reliability	0	1 (for sure) N/2 (if lucky)	1	1
Throughput				
Sequential Read	NS	(N/2)S	(N-1)S	(N-1)S
Sequential Write	NS	(N/2) S	(N-1)S	(N-1)S
Random Read	NR	NR	(N-1)R	NR
Random Write	NR	(N/2)R	R/2	(N/4)R
Latency				
Read	D	D	D	D
Write	D	D	2D	2D

Operating Systems

Youjip Won

KAIST



39. File and Directories

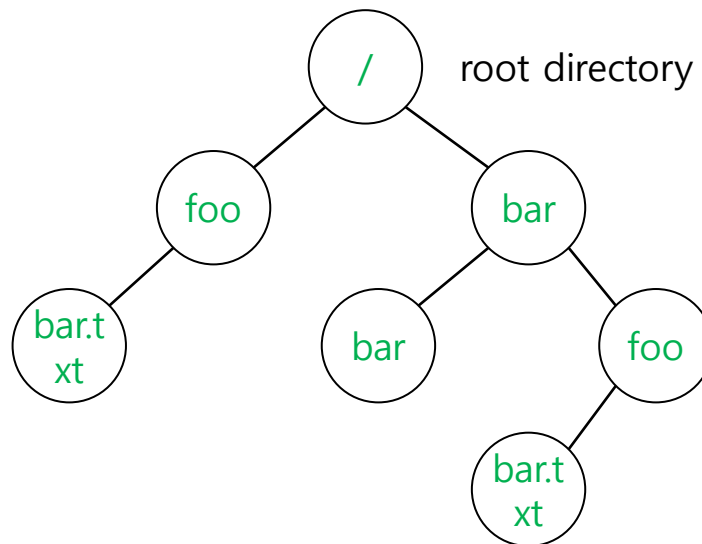
Concepts

□ File

- ◆ File is simply a linear array of bytes.
- ◆ Each file has low-level name as 'inode number'

□ Directory

- ◆ A file
- ◆ A list of <user-readable filename, low-level filename> pairs



An Example Directory Tree

vaild files :
/foo/bar.txt
/bar/foo/bar.txt

vaild directory :
/
/foo
/bar
/bar/bar
/bar/foo/

Interface: Creating a file

- Use `open` system call with `O_CREAT` flag.

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC, S_IRUSR|S_IWUSR);
```

- ◆ `O_CREAT` : create file.
- ◆ `O_WRONLY` : only write to that file while opened.
- ◆ `O_TRUNC` : make the file size zero (remove any existing content).

- `open` system call returns `file descriptor`.

- ◆ file descriptor is an integer, is used to access files.
- ◆ Ex) `read (file descriptor)`
- ◆ File descriptor table

```
struct proc {  
    ...  
    struct file *ofile[NOFILE]; // Open files  
    ...  
};
```

Interface: Reading and Writing Files

- An Example of reading and writing 'foo' file.

```
prompt> echo hello > foo //save the output to the file foo
prompt> cat foo           //dump the contents to the screen
hello
prompt>
```

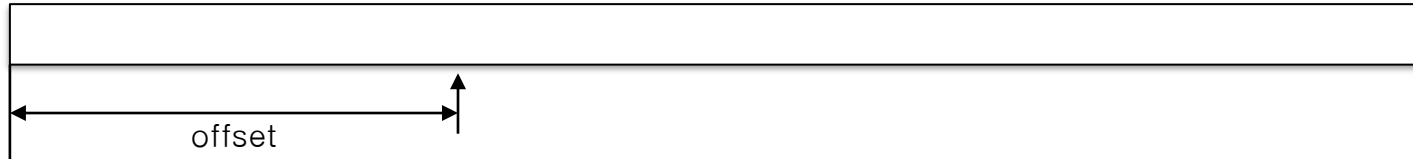
- The result of strace to figure out cat is doing.

```
prompt> strace cat foo //strace to figure out what cat is doing
...
open("foo", O_RDONLY|O_LARGEFILE) = 3
read(3, "hello\n", 4096)           = 6 /*the number of bytes to read*/
write(1, "hello\n", 6)              = 6
hello
read(3, "hello\n", 4096)           = 0
write(1, "hello\n", 6)              = 0
..
prompt>
```

- ◆ `open()`: open file for reading with `O_RDONLY` and `O_LARGEFILE` flags.
 - returns file descriptor 3 (0,1,2, is for standard input/output/error)
- ◆ `read()`: read bytes from the file.
- ◆ `write()`: write buffer to standard output.

Reading and Writing Files (Cont.)

▣ OFFSET



- ◆ The position of the file where we start read and write.
- ◆ When a file is open, "an offset" is allocated.
- ◆ Updated after read/write

▣ How to read or write to a specific offset within a file ?

```
off_t lseek(int fd, off_t offset /*location */, int whence);
```

- ◆ Third argument is how the seek is performed.
 - SEEK_SET : to offset bytes.
 - SEEK_CUR: to its current location plus offset bytes.
 - SEEK_END: to the size of the file plus offset bytes.

abstractions

```
struct file {  
    int ref;  
    char readable;  
    char writable;  
    struct inode *ip;  
    uint off;  
};
```

```
struct {  
    struct spinlock lock;  
    struct file file[NFILE];  
} ftable;
```

System Calls	Return Code	Current Offset
fd = open("file", O_RDONLY);	3	0
read(fd, buffer, 100);	100	100
read(fd, buffer, 100);	100	200
read(fd, buffer, 100);	100	300
read(fd, buffer, 100);	0	300
close(fd);	0	—

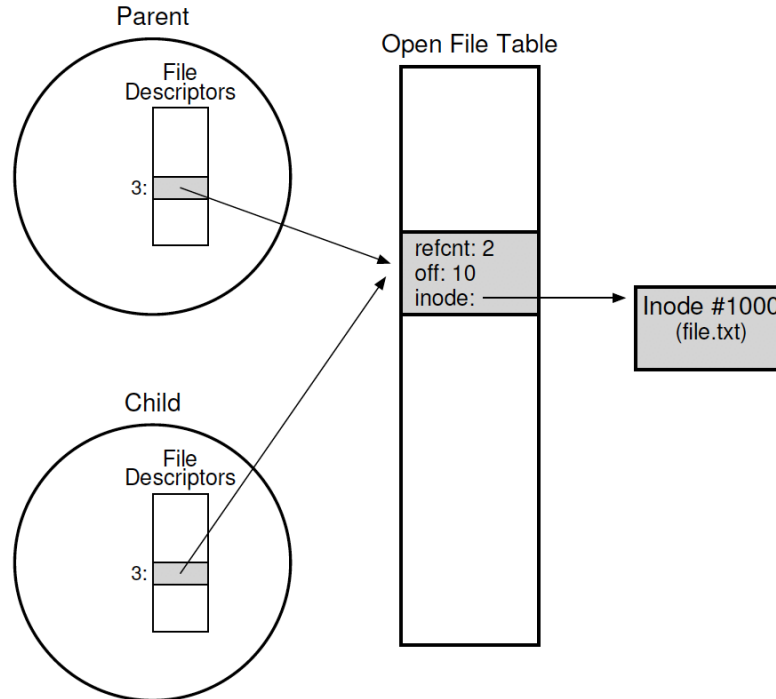
Sample traces

System Calls	Return Code	OFT[10] Current Offset	OFT[11] Current Offset
fd1 = open("file", O_RDONLY);	3	0	–
fd2 = open("file", O_RDONLY);	4	0	0
read(fd1, buffer1, 100);	100	100	0
read(fd2, buffer2, 100);	100	100	100
close(fd1);	0	–	100
close(fd2);	0	–	–

System Calls	Return Code	Current Offset
fd = open("file", O_RDONLY);	3	0
lseek(fd, 200, SEEK_SET);	200	200
read(fd, buffer, 50);	50	250
close(fd);	0	–

fork() and dup()

- Child process inherits the file descriptor table of the parent.



- Duplicating a file descriptor

```
int main(int argc, char *argv[]) {
    int fd = open("README", O_RDONLY);
    assert(fd >= 0);
    int fd2 = dup(fd);
    // now fd and fd2 can be used interchangeably
    return 0;
}
```

fsync()

❑ Persistency

- ♦ `write()` : write data to the buffer. Later, save it to the storage.
- ♦ some applications require more than eventual guarantee. Ex) DBMS

❑ `fsync()` : the writes are forced immediately to disk.

```
off_t fsync(int fd /*for the file referred to by the specified file*/)
```

❑ An Example of `fsync()`.

```
1  int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);  
2  int rc = write(fd, buffer, size);  
3  rc = fsync(fd);
```

- ♦ If a file is created, it needs to be durably a part of the directory.
 - Above code requires `fsync()` to directory also.

Renaming Files

- ▣ `rename()` : rename a file to different name.
 - ◆ It implemented as an atomic call.
 - ◆ Ex) change from foo to bar

```
prompt > mv foo bar
```

- ▣ Saving a file in an editor

```
1  int fd = open("foo.txt.tmp", O_WRONLY|O_CREAT|O_TRUNC);
2  write(fd, buffer, size); // write out new version of file
3  fsync(fd);
4  close(fd);
5  rename("foo.txt.tmp", "foo.txt");
```

Getting Information About Files

- ▣ `stat()` : Show the File metadata

- ◆ metadata is information about each file, ex: size, permission, ..

- ▣ `stat` structure is below:

```
1  struct stat {
2      dev_t st_dev; /* ID of device containing file */
3      ino_t st_ino; /* inode number */
4      mode_t st_mode; /* protection */
5      nlink_t st_nlink; /* number of hard links */
6      uid_t st_uid; /* user ID of owner */
7      gid_t st_gid; /* group ID of owner */
8      dev_t st_rdev; /* device ID (if special file) */
9      off_t st_size; /* total size, in bytes */
10     blksize_t st_blksize; /* blocksize for filesystem I/O */
11     blkcnt_t st_blocks; /* number of blocks allocated */
12     time_t st_atime; /* time of last access */
13     time_t st_mtime; /* time of last modification */
14     time_t st_ctime; /* time of last status change */
15 };
```

Getting Information About Files (Cont.)

- ▣ An example of `stat()`
 - ◆ All information is in a inode

```
prompt> echo hello > file  
prompt> stat file
```

```
File: `file'  
Size: 6 Blocks: 8 IO Block: 4096 regular file  
Device: 811h/2065d Inode: 67158084 Links: 1  
Access: (0640/-rw-r-----) Uid: (30686/ root) Gid: (30686/ remzi)  
Access: 2011-05-03 15:50:20.157594748 -0500  
Modify: 2011-05-03 15:50:20.157594748 -0500  
Change: 2011-05-03 15:50:20.157594748 -0500
```

Removing Files

- ▣ The result of `strace` to figure out what `rm` is doing.
 - ◆ `rm` is Linux command to remove a file
 - ◆ `rm` calls `unlink()` to remove a file.

```
1  prompt> strace rm foo
2
3  unlink("foo")
4  ...
5  prompt>
```

Making Directories

▣ `mkdir()`: Make a directory

- ◆ When a directory is created, it is **empty**.
- ◆ Empty directory have two entries: `.` (itself), `..`(parent)

```
prompt> strace mkdir foo
...
mkdir("foo", 0777)                = 0
...
prompt>
```

```
1 prompt> ls -al
2 total 8
3 drwxr-x--- 2 roo root 6 Apr 30 16:17 ./
4 drwxr-x--- 26 root root 4096 Apr 30 16:17 ../
```


Reading Directories

▣ readdir()

- ◆ Directory is a file, but with a specific structure.
- ◆ When reading a directory, we use specific system call other than read().
- ◆ A sample code to read directory entries.

```
1  int main(int argc, char *argv[]) {
2      DIR *dp = opendir("."); /* open current directory */
3      assert(dp != NULL);
4      struct dirent *d;
5      while ((d = readdir(dp)) != NULL) { /* read one directory entry */
6          printf("%d %s\n", (int) d->d_ino, d->d_name);
7      }
8      closedir(dp); /*close current directory */
9      return 0;
10 }
```

▣ Structure of the directory entry

```
struct dirent {  
    char d_name[256]; /* filename */  
    ino_t d_ino; /* inode number */  
    off_t d_off; /* offset to the next dirent */  
    unsigned short d_reclen; /* length of this record */  
    unsigned char d_type; /* type of file */  
};
```

Deleting Directories

- ▣ `rmdir()`: Delete a directory.
 - ◆ `rmdir()` requires directory be empty before it deleted.
 - ◆ If you call `rmdir()` to a non-empty directory, it will fail.

Hard Links

- ▣ `link()`: Link old file and a new file.

- ◆ Create hard link named file2.

```
prompt> echo hello > file
prompt> cat file
hello
prompt> ln file file2 /* create a hard link, link file to file2 */
prompt> cat file2
hello
```

- ▣ The result of `link()`

- ◆ Two files have same inode number, but two human name(file, file2)

```
prompt> ls -i file file2
67158084 file /* inode value is 67158084 */
67158084 file2 /* inode value is 67158084 */
prompt>
```

Hard Links (Cont.)

- ❑ How to create hard link file?
 - ◆ **Step1.** Make an **inode**, track all information about the file.
 - ◆ **Step2.** **Link** a human-readable name to file.
 - ◆ **Step3.** Put link file into a current directory.
- ❑ After creating a hard link to file, old and new files have no difference.
- ❑ Thus, to remove a file, we call **unlink()** .

- ▣ What `unlink()` is doing ?
 - ◆ Check reference count within the inode number.
 - ◆ Remove link between human-readable name and inode number.
 - ◆ Decrease reference count.
 - When only it reaches zero, It delete a file (free the inode and related blocks)

unlink Hard Links (Cont.)

▣ The result of unlink()

```
prompt> echo hello > file          /* create file*/
prompt> stat file
... Inode: 67158084 Links: 1 ...    /* Link count is 1 */
prompt> ln file file2              /* hard link file2 */
prompt> stat file
... Inode: 67158084 Links: 2 ...    /* Link count is 2 */
prompt> stat file2
... Inode: 67158084 Links: 2 ...    /* Link count is 2 */
prompt> ln file2 file3             /* hard link file3 */
prompt> stat file
... Inode: 67158084 Links: 3 ...    /* Link count is 3 */
prompt> rm file                    /* remove file */
prompt> stat file2
... Inode: 67158084 Links: 2 ...    /* Link count is 2 */
prompt> rm file2                   /* remove file2 */
prompt> stat file3
... Inode: 67158084 Links: 1 ...    /* Link count is 1 */
prompt> rm file3
```

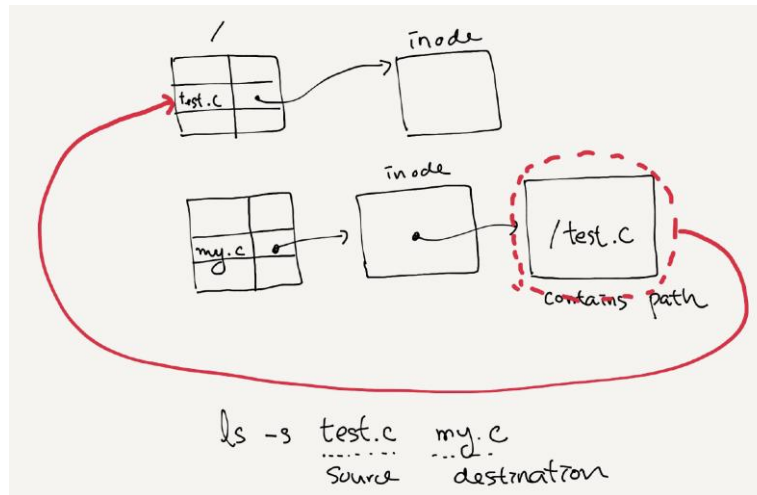
Symbolic Links

▣ Symbolic link

- ◆ Special file that contains path to the source directory.
- ◆ Hard Link cannot create to a directory.
- ◆ Hard Link cannot create to a file to other partition.

▣ An example of symbolic link

```
prompt> echo hello > file
prompt> ln -s file file2 /* option -s : create a symbolic link, */
prompt> cat file2
hello
```



Symbolic Links (Cont.)

- ▣ Symbolic link is different file type.

```
prompt> ls -al
drwxr-x--- 2 remzi remzi 29 May 3 19:10 ./
drwxr-x--- 27 remzi remzi 4096 May 3 15:14 ../          /* directory */
-rw-r----- 1 remzi remzi 6 May 3 19:10 file          /* regular file */
lrwxrwxrwx 1 remzi remzi 4 May 3 19:10 file2 -> file /* symbolic link */
```

- ▣ Symbolic link is subject to the dangling reference.

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
prompt> rm file
prompt> cat file2
cat: file2: No such file or directory
```

Summary

- ❑ Create file
- ❑ read/write/lseek
- ❑ mkdir/readdir
- ❑ fsync
- ❑ hardlink/softlink

Operating Systems

Youjip Won

KAIST



40. Filesystem Implementation

Overview

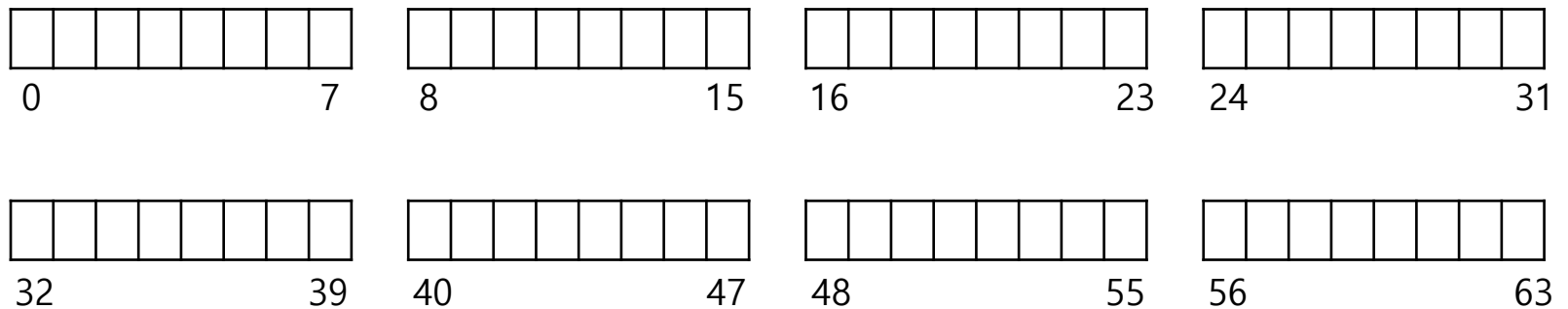
- ▣ In this chapter, we study very simple file system (vsfs)
 - ◆ Basic on-disk structures, access methods, and various policies of vsfs
- ▣ We will study...
 - ◆ How can we build a simple file system?
 - ◆ What structures are needed on the disk?
 - ◆ What do they need to track?
 - ◆ How are they accessed?

File system Implementation

- ▣ What types of **data structures** are utilized by the file system?
- ▣ How file system organize its data and metadata?
- ▣ Understand access methods of a file system.
 - ◆ `open()`, `read()`, `write()`, etc.

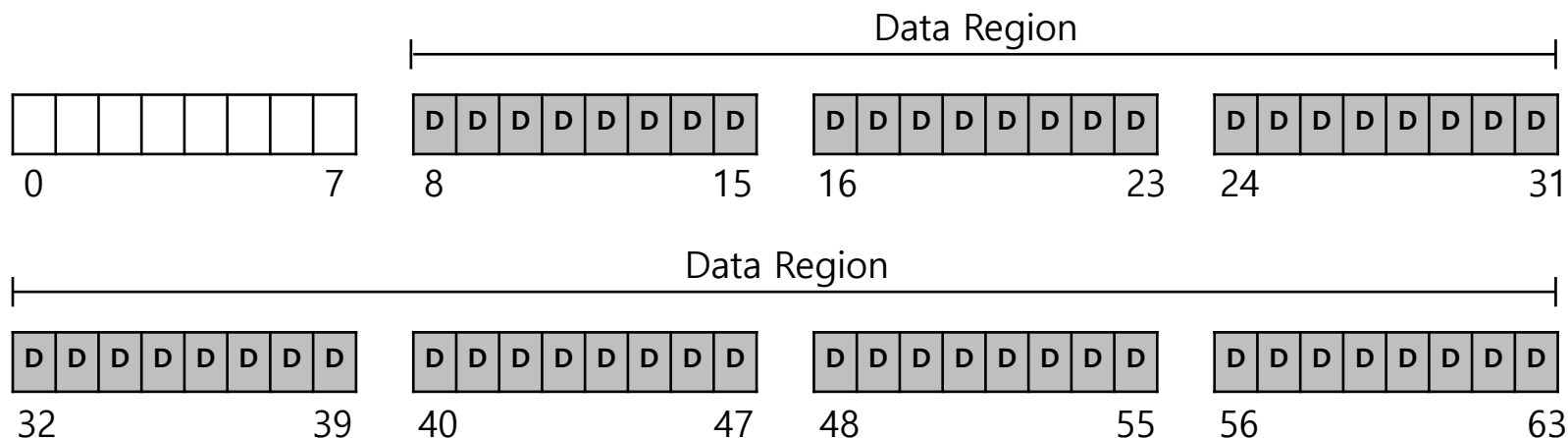
Overall Organization

- ▣ Let's develop the overall organization of the file system data structure.
- ▣ Divide the disk into **blocks**.
 - ◆ Block size is 4 KB.
 - ◆ The blocks are addressed from 0 to $N - 1$.



Data region in file system

- Reserve **data region** to store user data

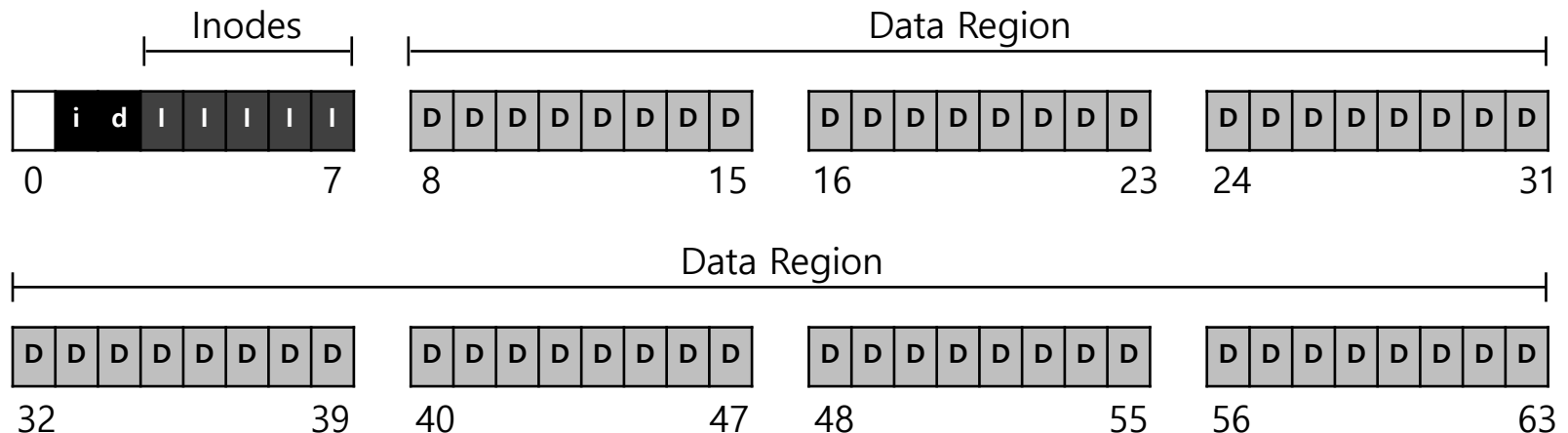


- File system has to track which data block comprise a file, the size of the file, its owner, etc.

How we store these inodes in file system?

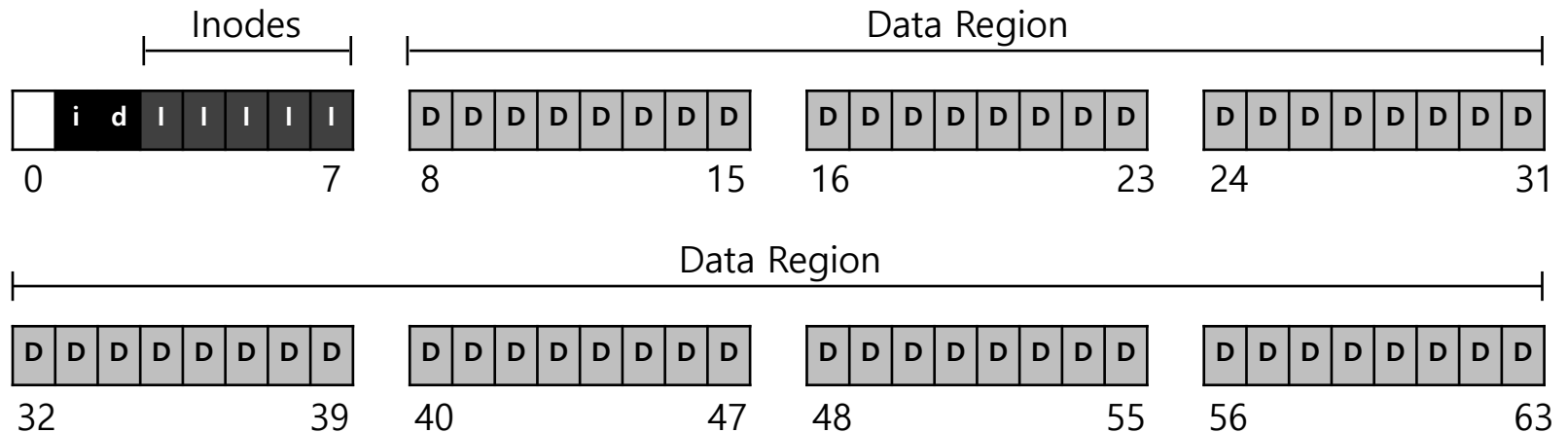
Inode table in file system

- ▣ Reserve some space for **inode table**
 - ◆ This holds an array of on-disk inodes.
 - ◆ Ex) inode tables : 3 ~ 7, inode size : 256 bytes
 - 4-KB block can hold 16 inodes.
 - The file system contains 80 inodes. (maximum number of files)



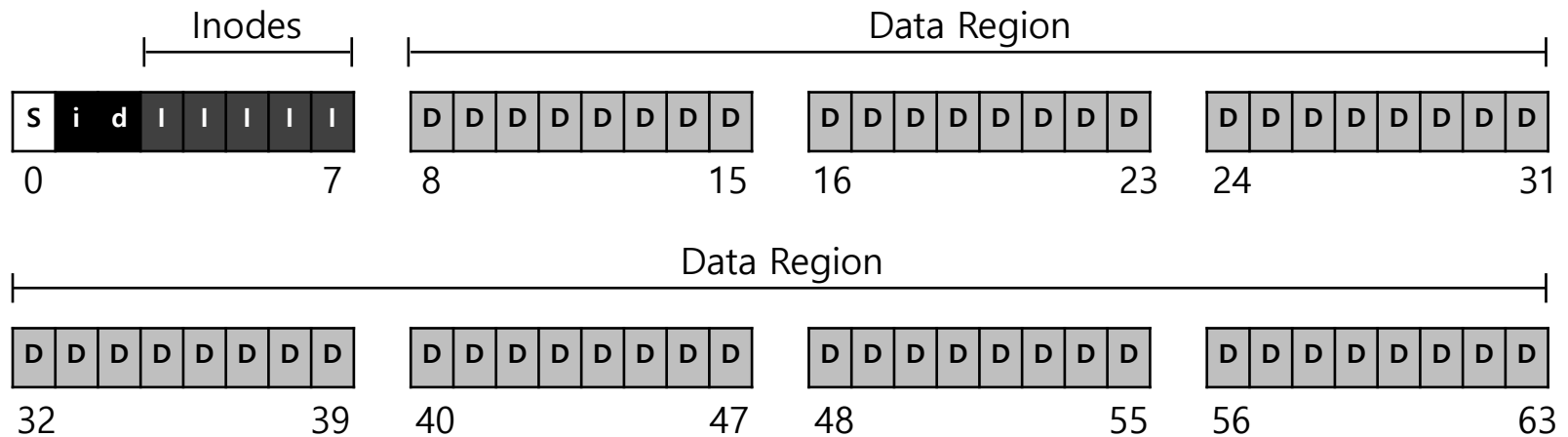
allocation structures

- ▣ This is to track whether inodes or data blocks are free or allocated.
- ▣ Use **bitmap**, each bit indicates free(0) or in-use(1)
 - ◆ data bitmap: for data region for data region
 - ◆ inode bitmap: for inode table



super block

- Super block contains this information for particular file system
 - Ex) The number of inodes, begin location of inode table. etc



- Thus, when mounting a file system, OS will read the superblock first, to initialize various information.

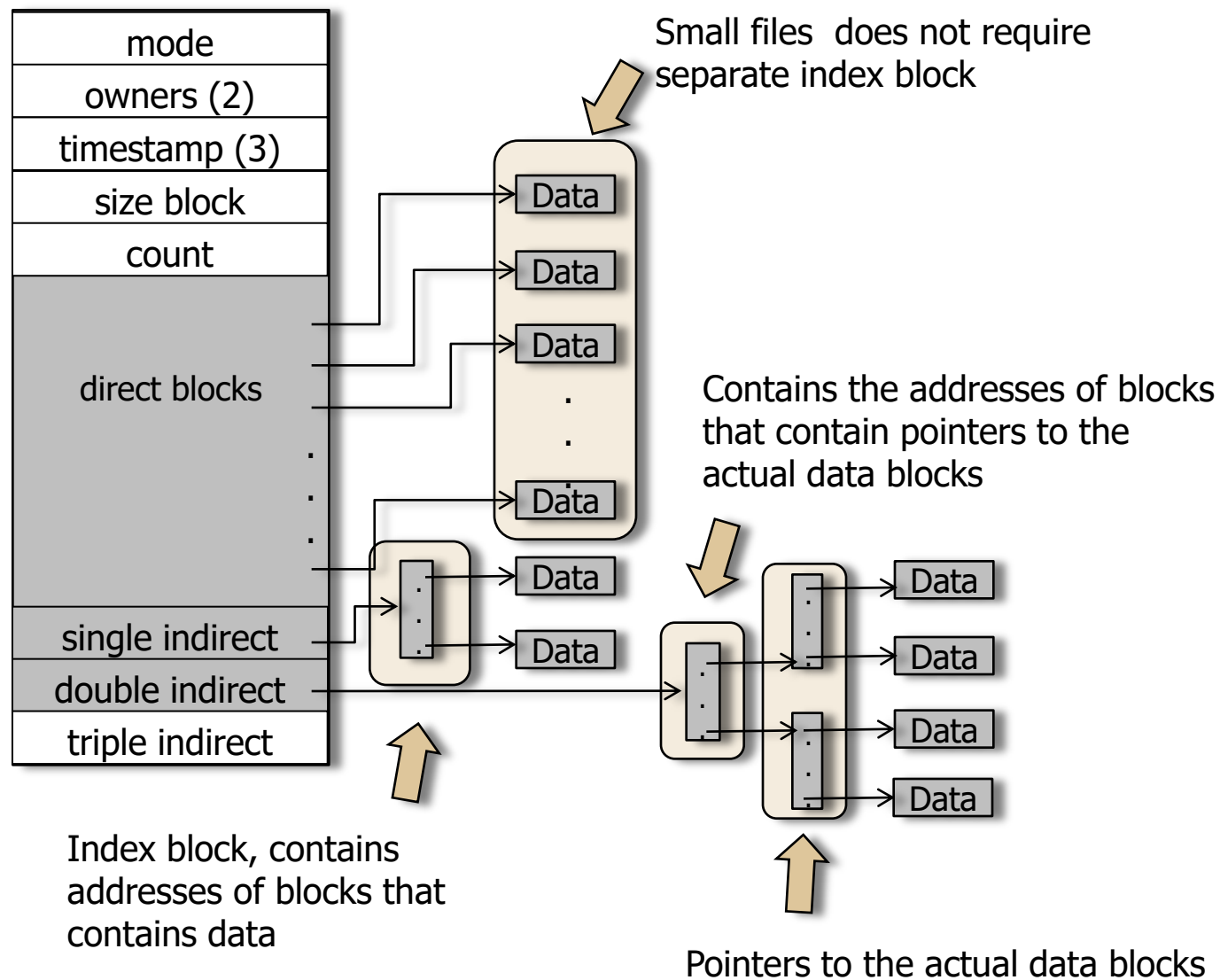
File Organization: The inode

- Each inode is referred to by inode number.
 - by inode number, File system calculate where the inode is on the disk.
 - Ex) inode number: 32
 - Calculate the offset into the inode region $(32 \times \text{sizeof}(\text{inode}) (256 \text{ bytes}) = 8192$
 - Add start address of the inode table(12 KB) + inode region(8 KB) = 20 KB

The Inode table

				iblock 0				iblock 1				iblock 2				iblock 3				iblock 4			
Super	i-bmap	d-bmap	0	1	2	3	16	17	18	19	32	33	34	35	48	49	50	51	64	65	66	67	
			4	5	6	7	20	21	22	23	36	37	38	39	52	53	54	55	68	69	70	71	
			8	9	10	11	24	25	26	27	40	41	42	43	56	57	58	59	72	73	74	75	
			12	13	14	15	28	29	30	31	44	45	46	47	60	61	62	63	76	77	78	79	
0KB	4KB	8KB	12KB	16KB				20KB				24KB				28KB				32KB			

File Structure: Indexed Allocation



Directory Structure

VSFS

inum | reflen | strlen | name

5	4	2	.
2	4	3	..
12	4	4	foo
13	4	4	bar
24	8	7	foobar

EXT4

	inode	reflen	file_type	name_len	name
0	21	12	1	2	· \0 \0 \0
12	22	12	2	2	· · \0 \0
24	53	16	5	2	h o m e 1 \0 \0 \0
40	67	28	3	2	u s r \0
52	0	16	7	1	o l d f i l e \0
68	34	12	4	2	s b i n

File Read

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
open(bar)			read	read	read	read	read			
read()					read			read		
read()					read				read	
read()					read					
read()					read					read
					write					

File Creation

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
create (/foo/bar)		read write	read	read		read	read			
					read write		write			
write()	read write			write	read			write		
					write					

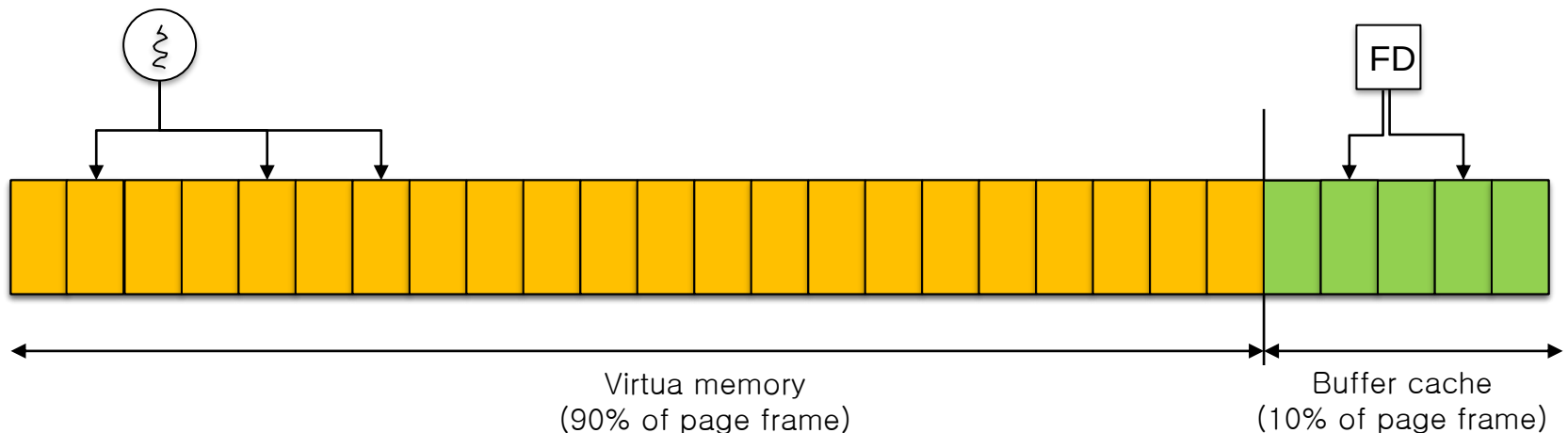
...

File Creation (Cont.)

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
...										
write()	read write				read					
					write	write				
write()	read write				read					
					write	write				

Caching and Buffering

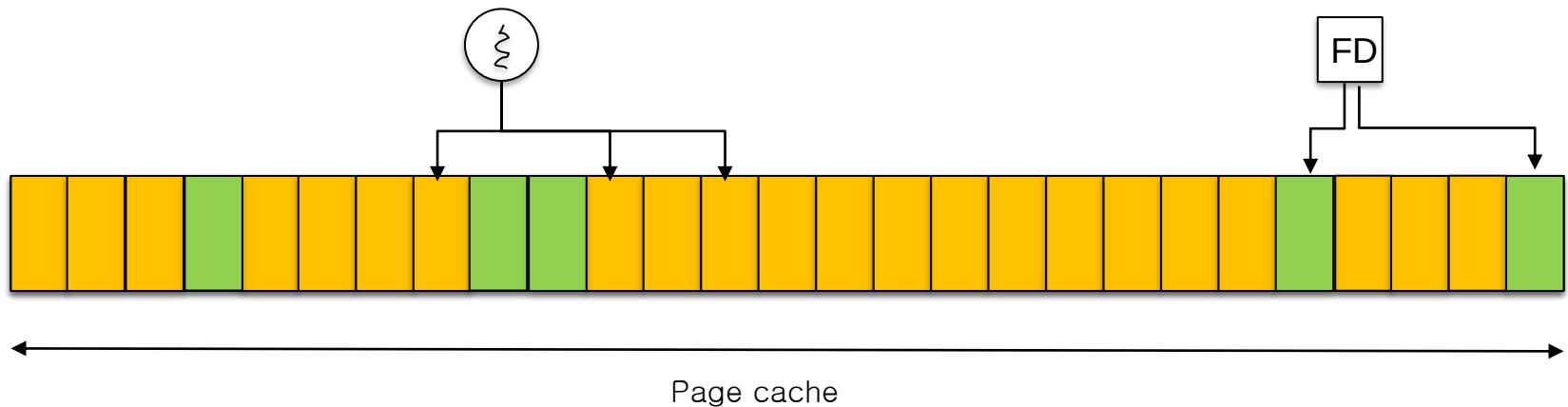
- ▣ Reading and writing can very IO intensive.
 - ◆ File open: two IO for each directory component and one read for the data.
- ▣ Buffer Cache
 - ◆ cache the disk blocks to reduce the IO.
 - ◆ LRU replacement
 - ◆ Static partitioning: 10% of DRAM, inefficient usage



Caching and Buffering

□ Page Cache

- ◆ Merge virtual memory and buffer cache
 - Process uses page table to map a virtual page to a page frame.
 - A file IO uses "address_space"(Linux) to map a file block to a physical page frame.
- ◆ Dynamic partitioning



Summary

- ▣ Requirements for building filesystem
 - ◆ File information: inode
 - ◆ File structure: indexed file
 - ◆ Directory (name→inode-number): array of <inode #, name>'s
 - ◆ Free block information: Bitmap
- ▣ All are flexible.

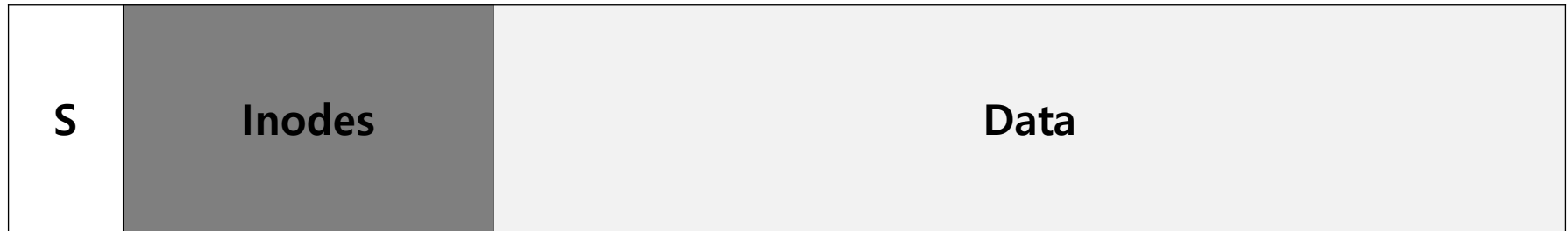
Operating Systems

Youjip Won



41. Locality and The Fast File System

- ▣ The First UNIX File system:



Data structures

- ▣ Simple and supports the basic abstractions.
- ▣ Easy to use file system.

However, Poor Performance !

- ▣ In this chapter, we study how the first unix filesystem can become fast.

Problem of Unix operating system

- ❑ Unix file system treated the disk as a **random-access memory**.
- ❑ Example of random-access blocks with Four files.
 - ◆ Data blocks for each file can accessed by going back and forth the disk, because they are are **contiguous**.

A1	A2	B1	B2	C1	C2	D1	D2
----	----	----	----	----	----	----	----

- ◆ File b and d is deleted.

A1	A2			C1	C2		
----	----	--	--	----	----	--	--

- ◆ File E is created with free blocks. (**spread across** the block)

A1	A2	E1	E2	C1	C2	E3	E4
----	----	----	----	----	----	----	----

FFS: Disk Awareness is the solution

- ▣ FFS is **Fast File system** designed by a group at Berkeley.
- ▣ The design of FFS is that file system structures and allocation policies to be “disk aware” and improve performance.
 - ◆ Keep same API with file system. (`open()`, `read()`, `write()`, etc)
 - ◆ Changing the internal implementation.

Cylinder Group

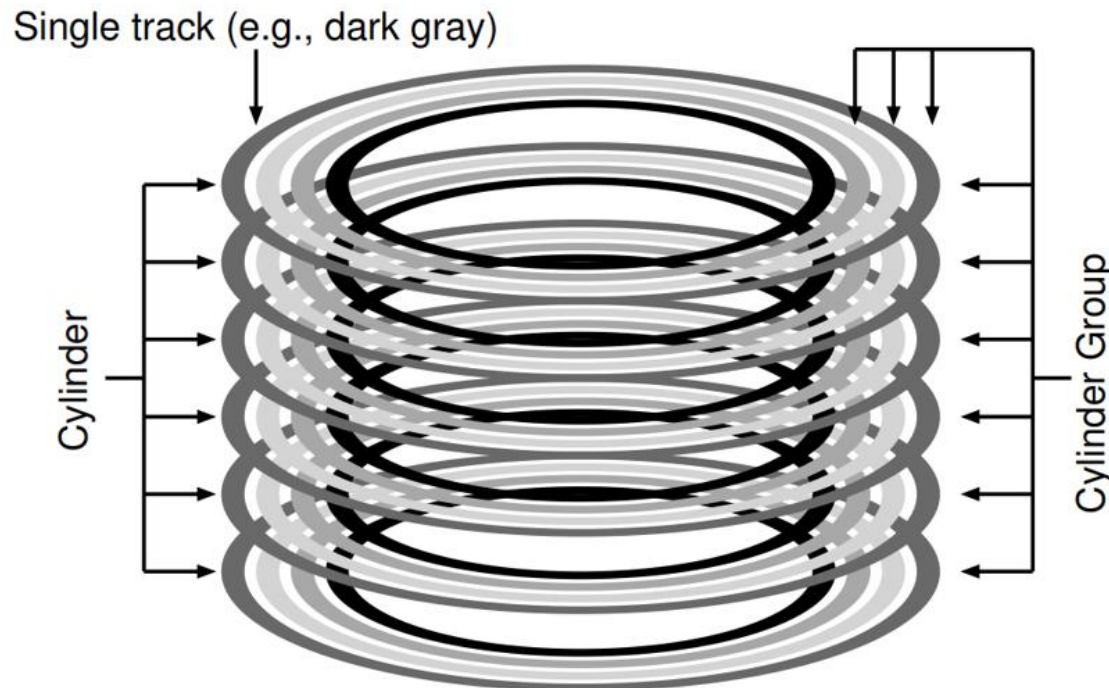
- ▣ FFS divides the disk into a bunch of groups. **(Cylinder Group)**
 - ◆ Modern file system call cylinder group as block group.
- ▣ These groups are used to improve seek performance.
 - ◆ By placing two files within the same group.
 - ◆ Accessing one after the other **will not be long seeks** across the disk.
 - ◆ FFS needs to allocate the files and directories within each of these groups.



- ▣ Data structure for each cylinder group.
 - ◆ A copy of the super block for reliability reason.
 - ◆ inode bitmap and data bitmap to track free inode and data block.
 - ◆ inodes and data block

Cylinder Group (Cont.)

- ▣ **Cylinder:** Tracks at same distance from center of drive across different surfaces.
 - ◆ All tracks with same color
- ▣ **Cylinder Group:** Set of N consecutive cylinders
 - ◆ if $N=3$, first group does not include black track



How To Allocate Files and Directories?

- ▣ Policy is “**keep related stuff together**”
- ▣ The placement of directories
 - ◆ Find the cylinder group with a low number of allocated directories and a high number of free inodes.
 - ◆ Put the directory data and inode in that group.
- ▣ The placement of files.
 - ◆ Allocate data blocks of a file in the same group as its inode
 - ◆ It places all files in the same group as their directory

How To Allocate Files and Directories? (Example)

■ In the example

- ♦ there are 10 groups, and each group have 10 inodes and 10 data blocks.
- ♦ there are three directories: '/', '/a', and '/b'
- ♦ there are four files: '/a/c', '/a/d', '/a/e', and '/b/f'

Filesystem is described as below:

<Table representing Filesystem>

group	inodes	data blocks
0	/-----	/-----
1	acde-----	acde-----
2	bf-----	bff-----
3	-----	-----
4	-----	-----
5	-----	-----
6	-----	-----
7	-----	-----
8	-----	-----

<Description of Table>

In group 0, an inode and a data block for '/' are allocated
There are nine free inodes and data blocks

In group 1, inodes for file 'a', 'c', 'd', and 'e' is allocated,
one data block for file 'a', 'c', 'd', and 'e' are allocated.
There are six free inodes and three free data blocks

In group 2, inodes for file 'b' and 'f' are allocated, one
data block for file 'b', two data blocks for file 'f' are allocated.
There are eight free inodes and seven free data blocks

How To Allocate Files and Directories? (Example)

group	inodes	data blocks
0	/-----	/-----
1	acde-----	accddee---
2	bf-----	bff-----
3	-----	-----
4	-----	-----
5	-----	-----
6	-----	-----
7	-----	-----
8	-----	-----

< With FFS policies >

group	inodes	data blocks
0	/-----	/-----
1	a-----	a-----
2	b-----	b-----
3	c-----	cc-----
4	d-----	dd-----
5	e-----	ee-----
6	f-----	ff-----
7	-----	-----
8	-----	-----

< Without FFS policies >

▣ With FFS policies

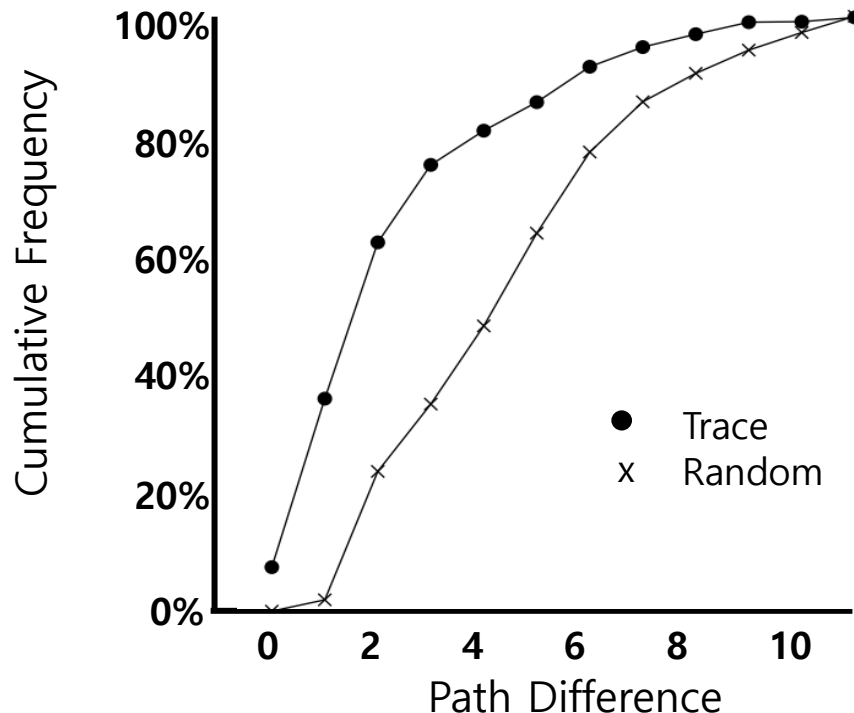
- ◆ the data blocks of each file are near each file's inode
- ◆ files in the same directory are near one another.

▣ Without FFS policies (Simply spread directories and files across groups)

- ◆ Access to fiels '/a/c', '/a/d', and '/a/e' now spans three groups instead of one as per the FFS approach.

FFS Locality for SEER Traces.

- SEER traces is analysis about how "far away" file accesses were from one another in the directory tree.
- Path difference: how far up the directory tree you have to travel to find the common ancestor of two files



The Large-File Exception

- FFS without the large file exception handling.

G: block group

G0	G1	G2	G3	G4	G5	G6	G7	G8	G9	G10
----	----	----	----	----	----	----	----	----	----	-----

0 1 2 3 4
5 6 7 9

- FFS with the large file exception handling.
 - ◆ Spread blocks of a file across the disk.
 - ◆ Consider the time transferring big data from disk and block seek time.

G0	G1	G2	G3	G4	G5	G6	G7	G8	G9	G10
----	----	----	----	----	----	----	----	----	----	-----

8 9

0 1

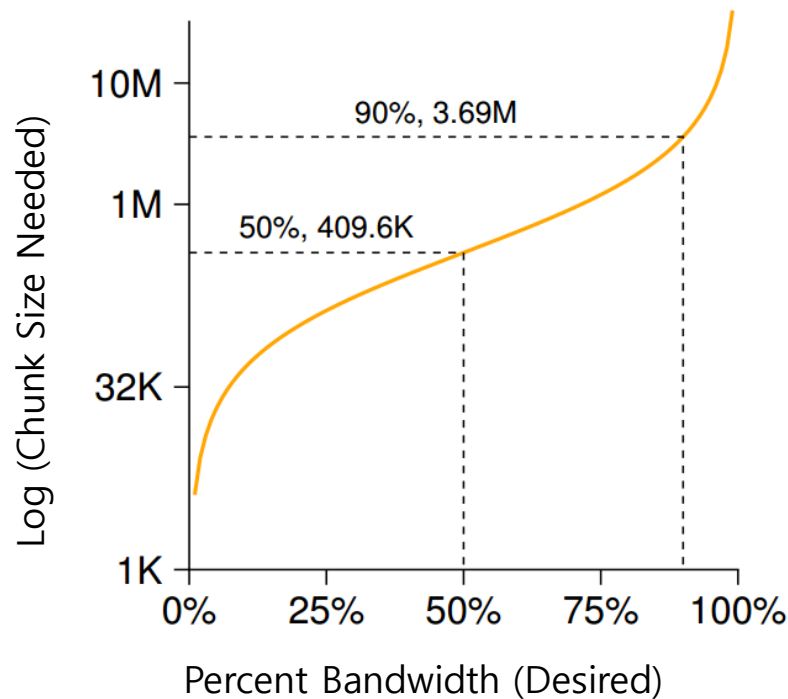
2 3

4 5

6 7

The Large-File Exception (Cont.)

- ▣ How big do chunks have to be?
 - ◆ First twelve direct blocks places in same group as the inode
 - ◆ Indirect block, and all blocks it point places in a different group.



A few other things about FFS

- ▣ Internal fragmentation
- ▣ Sub-blocks
 - ◆ Ex) Create a file with 1 KB : use two sub-blocks, not an entire 4-KB blocks
- ▣ Parameterization
- ▣ Track buffer
- ▣ Long file names
 - ◆ Enabling more expressive names in the file system
- ▣ Symbolic link

Summary

- ▣ The introduction of fast file system (FFS)
 - ◆ It finds that the problem of file management is one of the most. important issue within an operating system.
 - ◆ It makes file system fast, considering characteristic of disk.
- ▣ Many file systems take cues from FFS.
 - ◆ ex) ext4, ext3, ext4

Operating Systems

Youjip Won



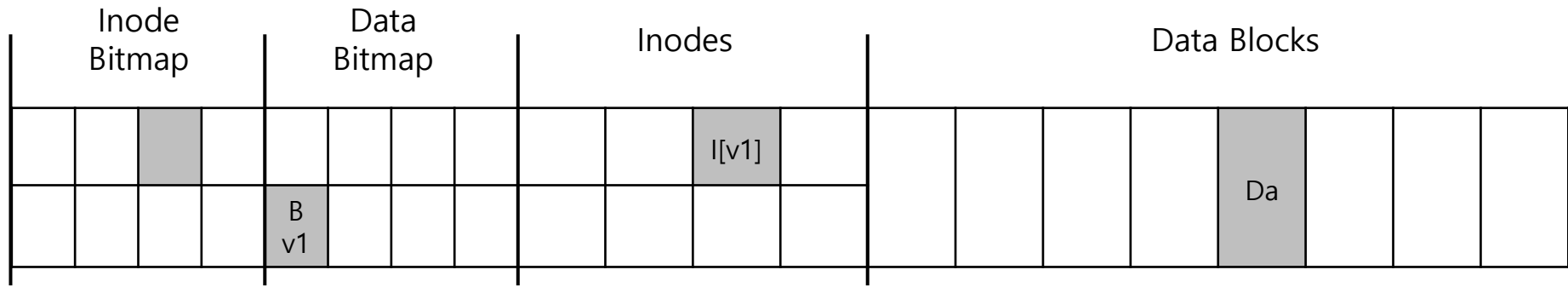
42. Crash Consistency: FSCK and Journaling

- File system data structures must **persist**.
 - ◆ files, directories, all of the other metadata ,etc
- How to update persistent data structure?
 - ◆ If the system crashes or loses power, on-disk structure will be in **inconsistent** state.
- In this chapter, we describe how to update file system consistently

An Example of Crash Consistency

▣ Scenario

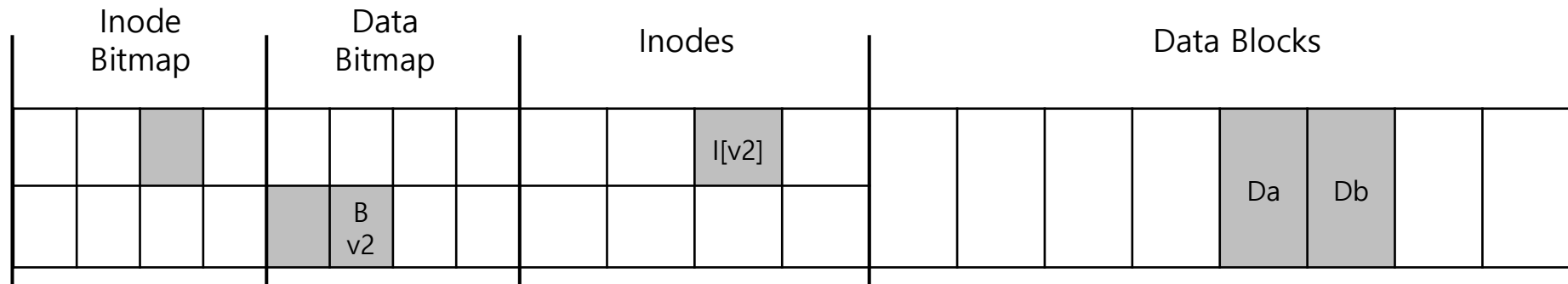
- ◆ Append of a single data block to an existing file.



Before Append a single data block

An Example of Crash Consistency (Cont.)

- File system perform three writes to the disk.
 - inode $I[v2]$
 - Data bitmap $B[v2]$
 - Data block (Db)



After Append a single data block

Crash Scenario

- ▣ Only one of the below block is written to disk.
 - ◆ Data block (Db): lost update
 - ◆ Update inode (I[v2]) block: garbage, consistency problem
 - ◆ Updated bitmap (B[v2]): space leak

- ▣ Two writes succeed and the last one fails.
 - ◆ The inode(I[v2]) and bitmap (B[v2]), but not data (Db): consistent
 - ◆ The inode(I[v2]) and data block (Db), but not bitmap(B[v2]): inconsistent
 - ◆ The bitmap(B[v2]) and data block (Db), but not the inode(I[v2]): inconsistent

Crash-consistency problem (consistent- update problem)

▣ The File System Checker (**fsck**)

- ◆ `fsck` is a Unix tool for finding inconsistencies and repairing them.
- ◆ super block: if the number of blocks in the filesystem is larger than the filesystem size
- ◆ free blocks: find all the blocks accessible from the root directory and see if it matches the block bitmap.
- ◆ inode state: check if the state of each inode is valid
- ◆ inode link: check if the reference count for each inode is consistent
- ◆ check if a block is shared by the two inodes.
- ◆ check for "bad" block pointer: "bad" block pointer is the one that points to the location that lies outside the filesystem partition.
- ◆ directory: Check if `.` and `..` are properly set up. Make sure that there are only one hardlink for a directory.

▣ **Journaling** (or Write-Ahead Logging)

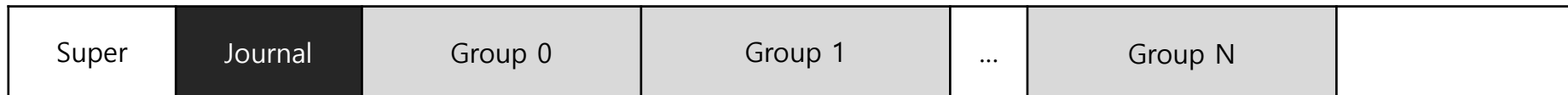
- ◆ Before overwriting the on-disk structures in place, write down a little note on the disk, describing what you are to do.
- ◆ Writing this note is the “write ahead”. The structure that is the destination of the “write ahead” is called log. hence, This is Write-Ahead Logging.

Journaling

- File system reserves some small amount of space within the partition or on another device.



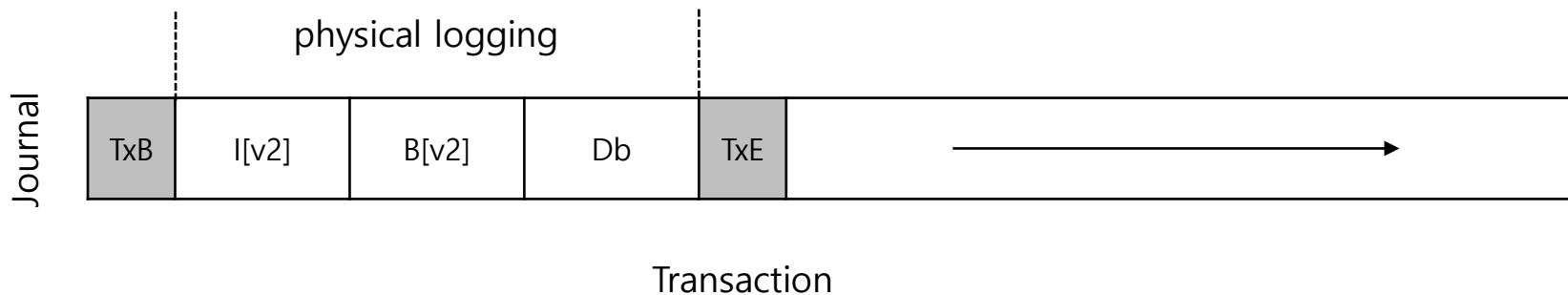
without journaling



with journaling

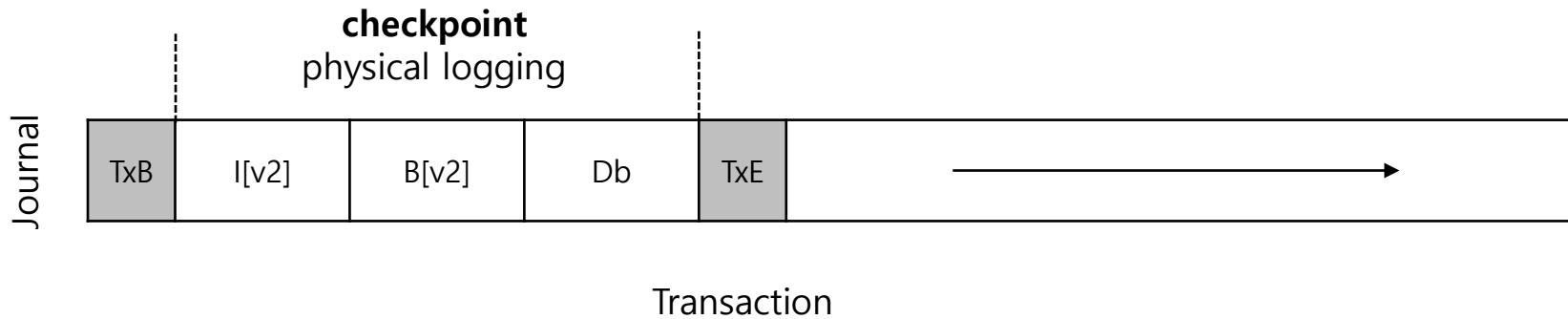
Data Journaling

- Lets' update a file (appending a data block to a file). Following structures are updated.
 - ◆ inode ($I[v2]$), bitmap ($B[v2]$), and data block (Db)
- First, **Journal write**: write the transaction as below.
 - ◆ TxB: Transaction begin block (including transaction identifier)
 - ◆ TxE: Transaction end block
 - ◆ others: contain the exact contents of the blocks



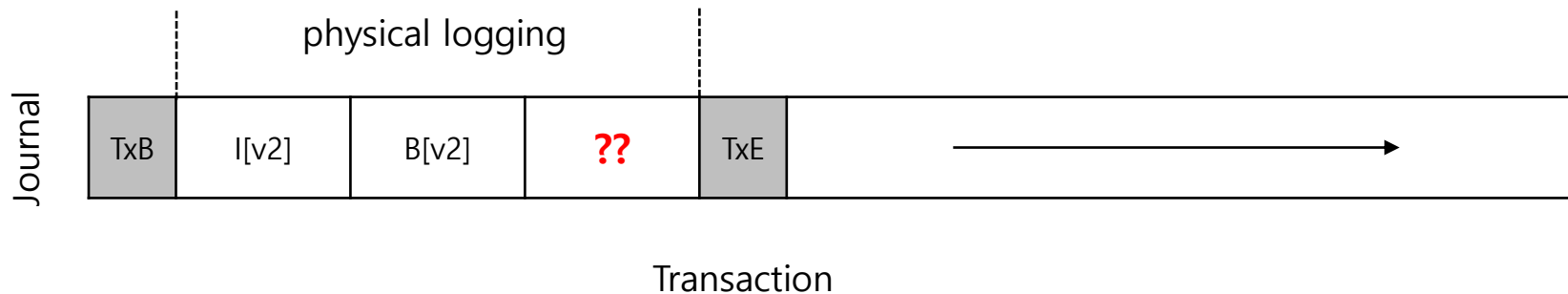
Data Journaling (Cont.)

- Second, **Checkpoint**: Write the physical log to their original disk locations.



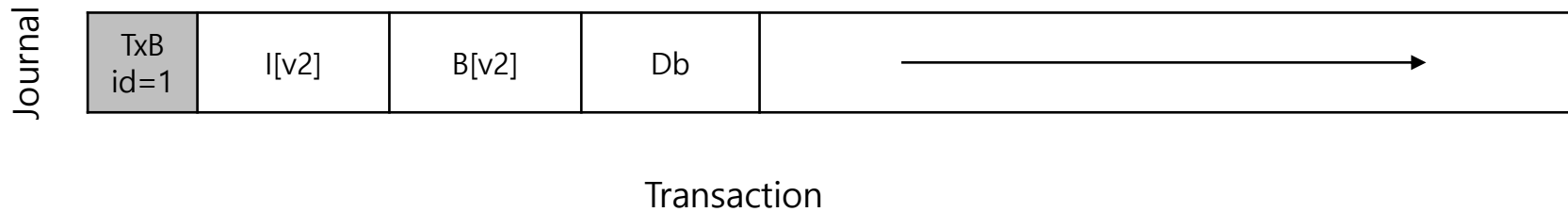
Crash during Data Journaling

- Wat if a *crash occurs* during the writes to the journal?

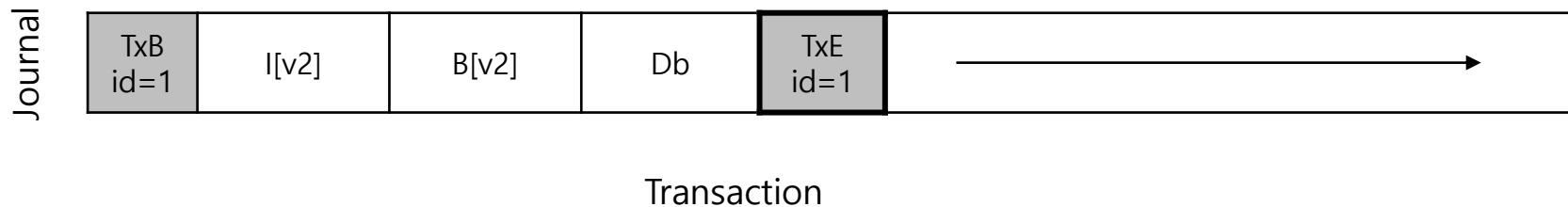


To avoid data being inconsistent

- First, write all blokes **except the TxE block** to journal.



- Second, The file system issues the write of the TxE.



To avoid data being inconsistent (Cont.)

- ▣ **Journal write:** write the contents of the transaction to the log
- ▣ **Journal commit:** write the transaction commit block
- ▣ **Checkpoint:** write the contents of the update to their locations.

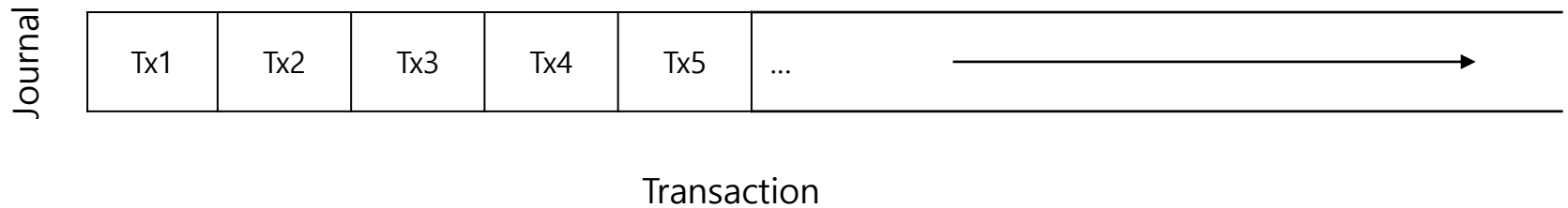
- ❑ If the crash happens, before the transactions is written to the log
 - ◆ The pending update is skipped.
- ❑ If the crash happens, after the transactions is written to the log, but before the checkpoint.
 - ◆ **Recover** the update as follow:
 - Scan the log and lock for transactions that have committed to the disk.
 - Transactions are replayed.

Batching Log Updates

- ▣ If we create two files in same directory, the same inode and the directory entry block is to the log and committed twice.
- ▣ To reduce excessive write traffic to disk, journaling manage the **global transaction**.
 - ◆ Write the content of the global transaction forced by synchronous request.
 - ◆ Write the content of the global transaction after timeout of 5 seconds.

Making the log finite

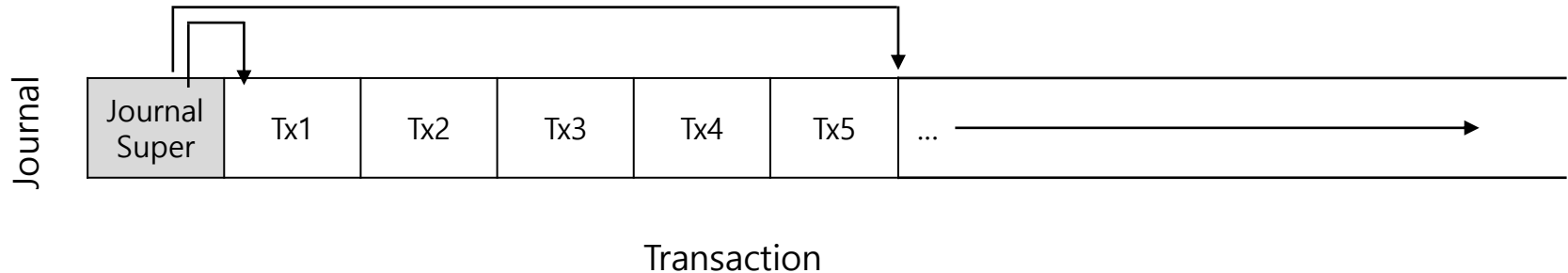
- ▣ The log is of a finite size (**circular log**).
 - ◆ To re-using it over and over



Making The log Finite (Cont.)

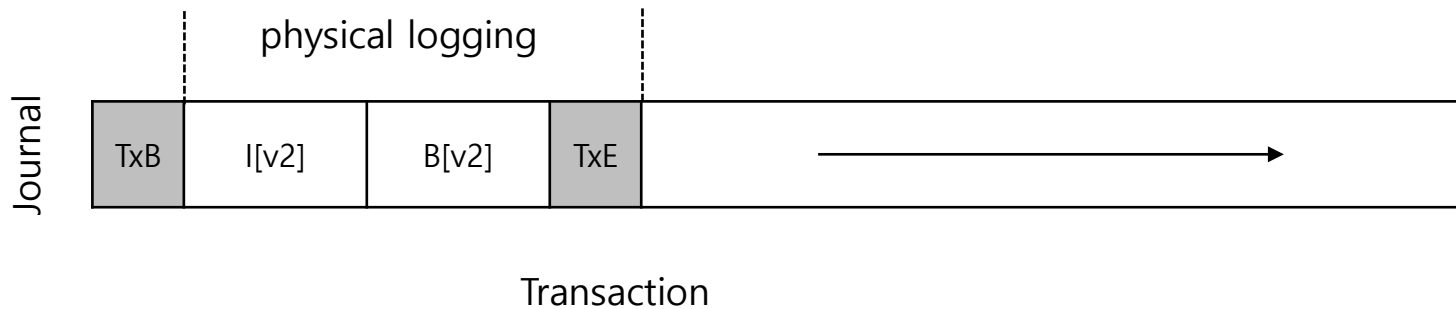
▣ journal super block

- ◆ Mark the oldest and newest transactions in the log.
- ◆ The journaling system records which transactions have not been check pointed.



Metadata Journaling

- Because of the high cost of writing every data block to disk twice
 - ◆ commit to log (journal)
 - ◆ checkpoint to on-disk location.
- Filesystem uses ordered journaling (metadata journaling).

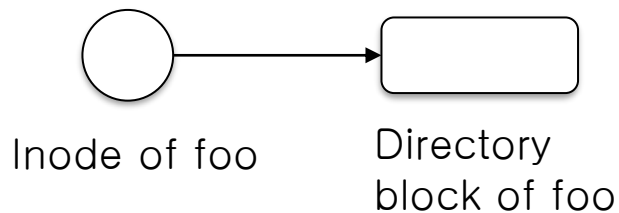
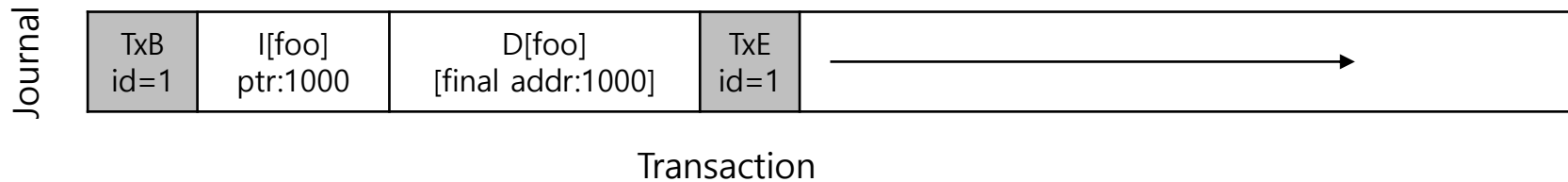


Metadata Journaling (Cont.)

- ▣ **Data Write:** Write data to final location
- ▣ **Journal metadata write:** Write the begin and metadata to the log
- ▣ **Journal commit:** Write the transaction commit block to the log
- ▣ **Checkpoint metadata:** Write the contents of the metadata to the disk
- ▣ **Free:** Later, mark the transaction free in journal super block

Tricky case: Block Reuse

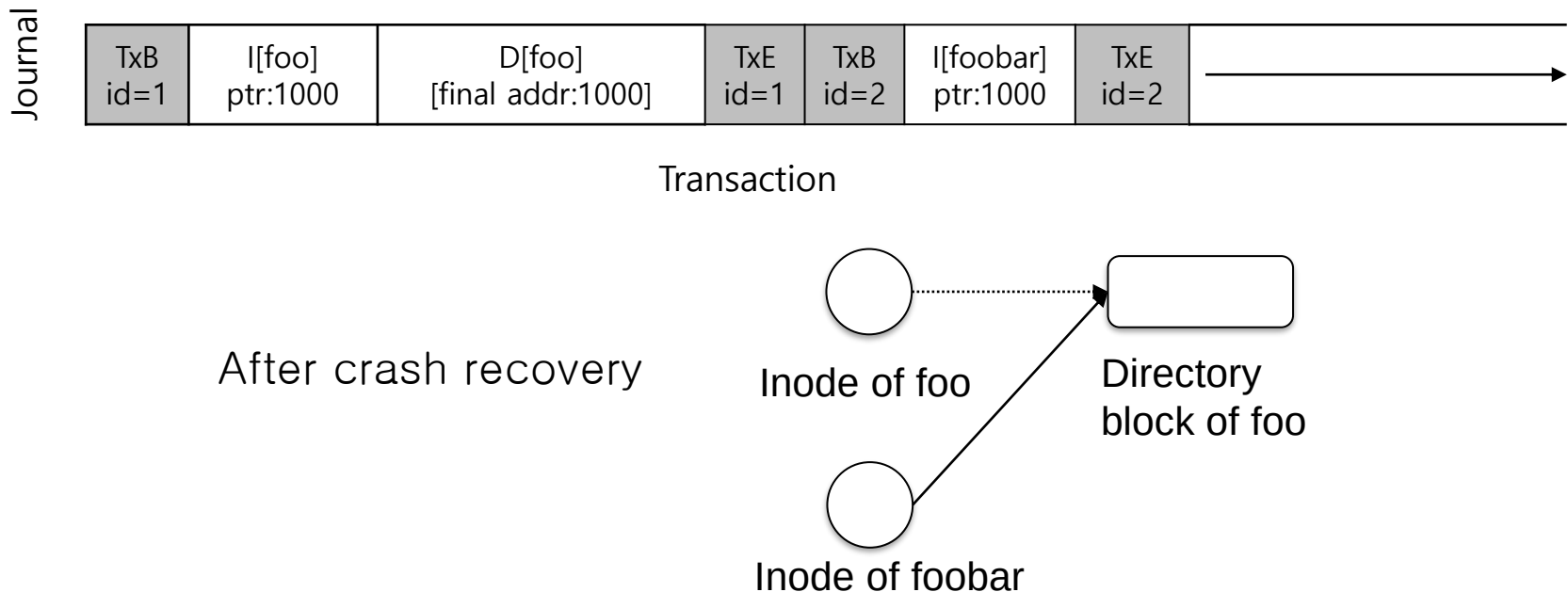
- Revoke record: some metadata should not be replayed.
- Scenario.
 - Directory "foo" is updated.



Tricky case: Block Reuse

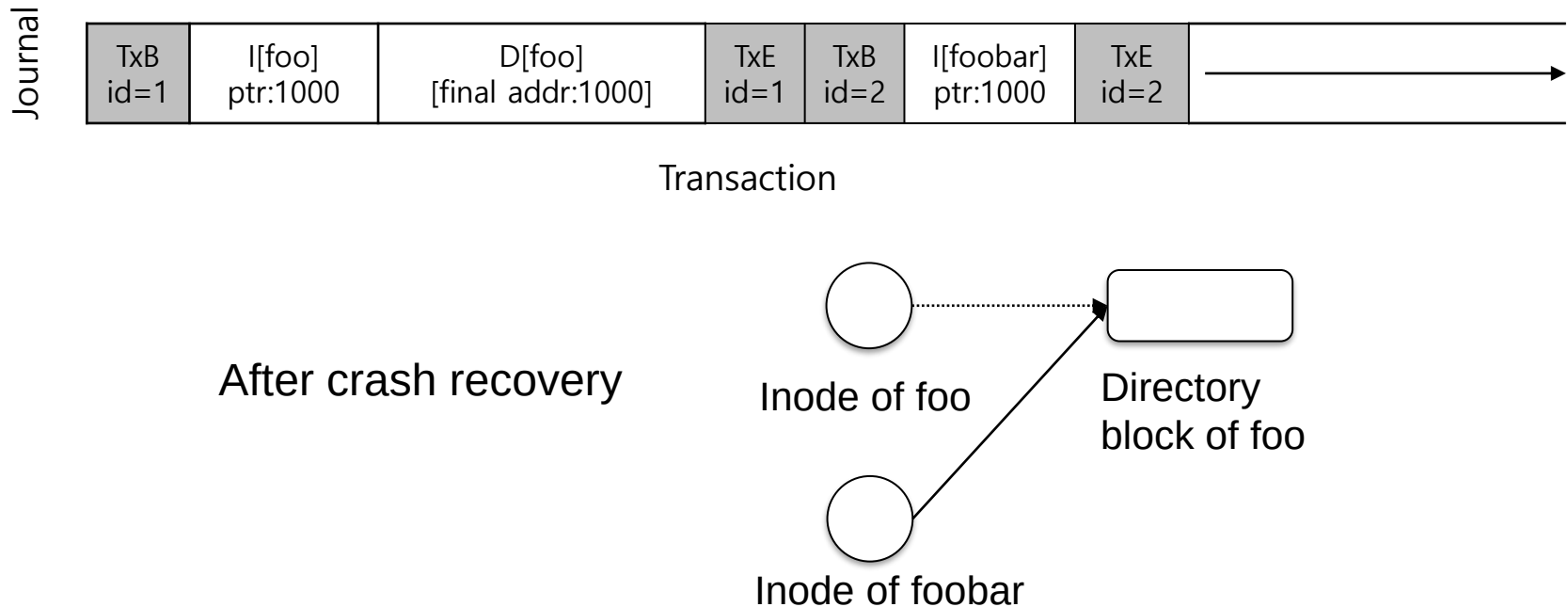
Scenario.

1. Delete "foo" directory, freeing up block 1000 for reuse
2. Create a file "foobar", reusing block 1000 for data



Tricky case: Block Reuse

- Problem: allocate the block that was deleted but was not checkpointed.



- Solution1: Do not use the deleted block until it is checkpointed.
- Solution 2: When deleting a directory, record "revoke" at the journal.

Metadata Journaling Timeline

TxB	Journal contents	TxE	File System	
	(metadata)		Metadata	Data
issue complete	issue			issue
	complete			complete
		issue complete		
			issue complete	

Metadata Journaling Timeline

Data Journaling Timeline

Journal contents				File System	
TxB	(metadata)	(data)	TxE	Metadata	Data
issue	issue	issue			
complete					
	complete	complete			
			issue		
			complete		
				issue	issue
				complete	complete

Data Journaling Timeline

Summary

- ▣ The problem of crash consistency
 - ◆ Filesystem becomes inconsistent so users cannot use it
- ▣ Approaches to attacking this problem
 - ◆ **fsck**: it works but is likely too slow to recover on modern systems.
 - ◆ **Journaling**: It reduces recovery time from $O(\text{size-of-the-disk-volume})$ to $O(\text{size-of-the-log})$
- ▣ Two modes of journaling
 - ◆ Metadata journaling(Ordered journaling): Log only metadata only after data written
 - ◆ Data journaling: Log both of metadata and data

Operating Systems

Youjip Won



43. Log-structured File Systems

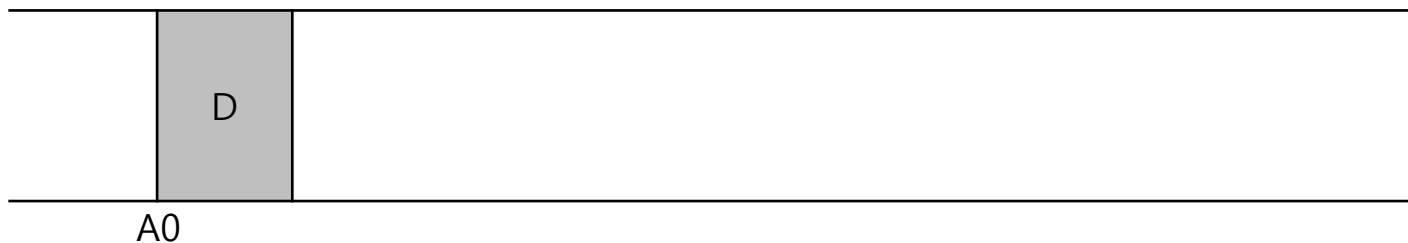
Overview

- ▣ In the early 90`s, a new file system known as the log-structured file system(LFS) was developed.
- ▣ The Motivation ...
 - ◆ Memory sizes were growing.
 - ◆ Large gap between random IO and sequential IO performance.
 - ◆ Existing File System perform poorly on common workloads.
 - ◆ File System were not RAID-aware. There exists small write problem.
- ▣ In this chapter, we study Log-Structured Filesystem(LFS).
 - ◆ How can a file system **transform all writes into sequential writes**?

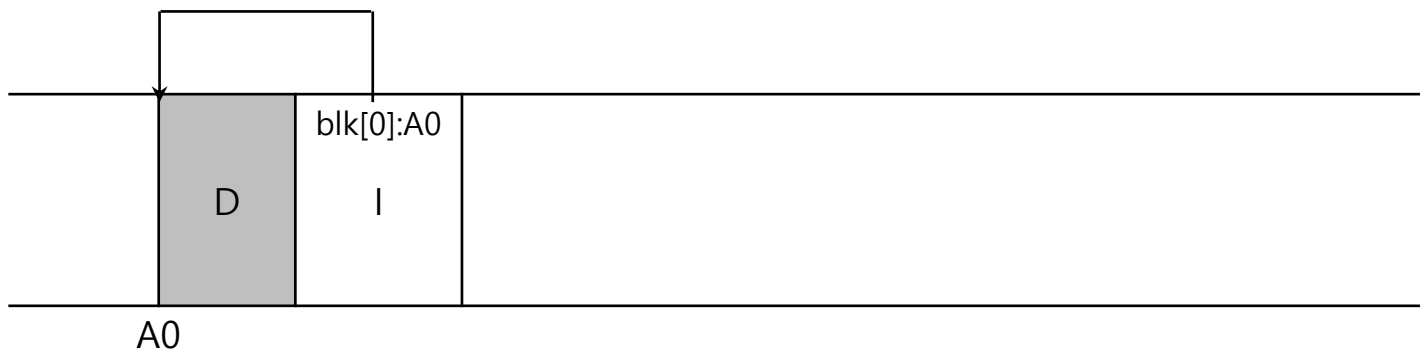
Writing to Disk Sequentially

- How do we transform all updates to file-system state into a series of sequential writes to disk?

- ◆ data update

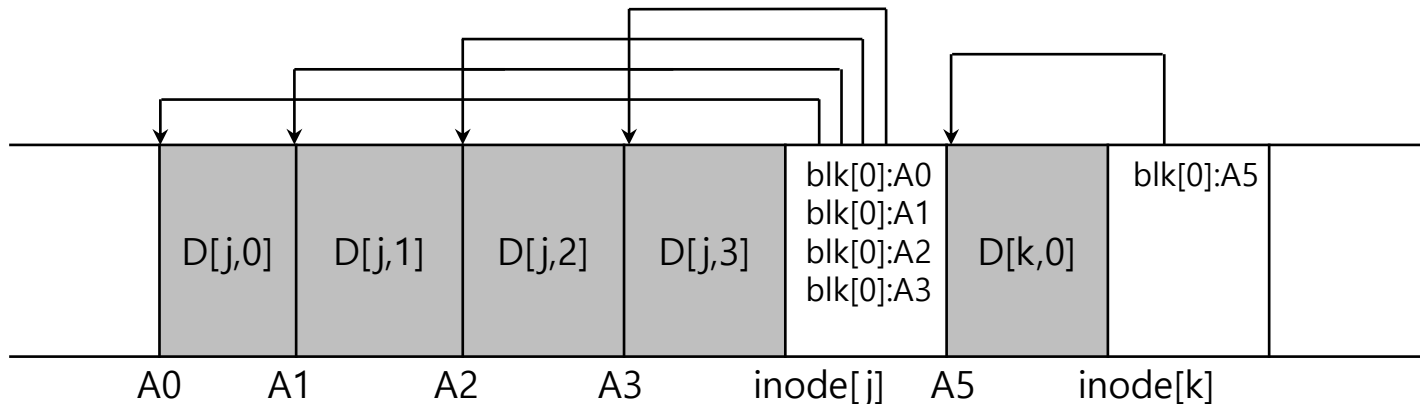


- ◆ metadata needs to be updated too. (Ex. inode)



Segment

- Writing to the disk sequentially is not enough to guarantee the efficient writes.
 - Disk may rotate between the writes. → lose a single revolution between the writes.
- Write buffering.
 - Segment: a set of sequential writes that are written to the disk with a single unit.
 - Keep track of updates in **memory buffer**. (a few Mbyte)
 - Write them to disk all at once, when it has sufficient number of updates.



Right segment size

- Time to write D Mbyte

$$T_{write} = T_{position} + \frac{D}{R_{peak}}$$

- Effective write bandwidth

$$R_{effective} = \frac{D}{T_{write}} = \frac{D}{T_{position} + \frac{D}{R_{peak}}} \quad (43.2)$$

- We like to make the effective write bandwidth close to peak bandwidth with some fraction F ($0 < F < 1$)

$$R_{effective} = \frac{D}{T_{position} + \frac{D}{R_{peak}}} = F \times R_{peak}$$

Right segment size

- Then, D can be computed as follows.

$$D = F \times R_{peak} \times (T_{position} + \frac{D}{R_{peak}})$$

$$D = (F \times R_{peak} \times T_{position}) + \left(F \times R_{peak} \times \frac{D}{R_{peak}} \right)$$

$$D = \frac{F}{1-F} \times R_{peak} \times T_{position}$$

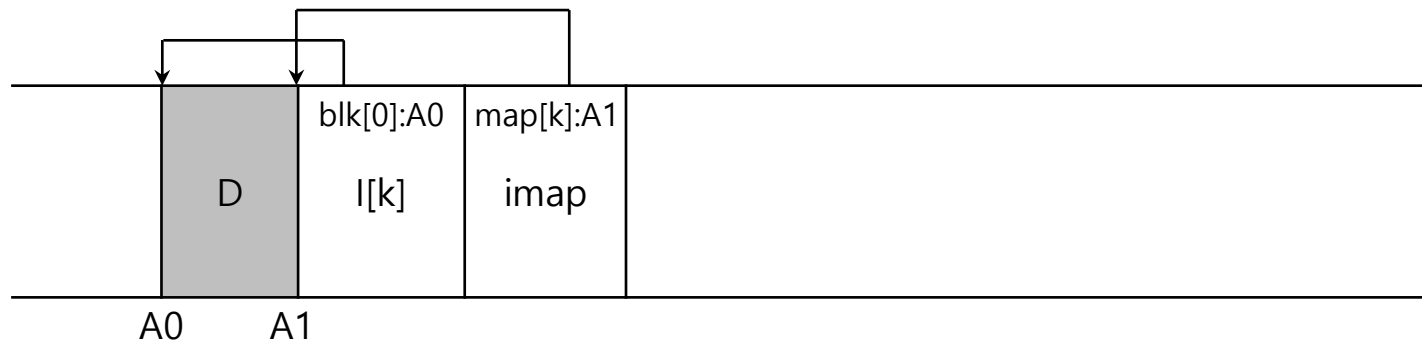
- Example: Positioning time 10 msec, peak transfer rate 100MByte/sec, we like to achieve 90% of the peak rate

$$D = 0.9 \times 0.1 \times 100 \text{ Mbyte/sec} \times 0.01 \text{ secs} = 9 \text{ Mbyte}$$

- What is D if F = 0.95?

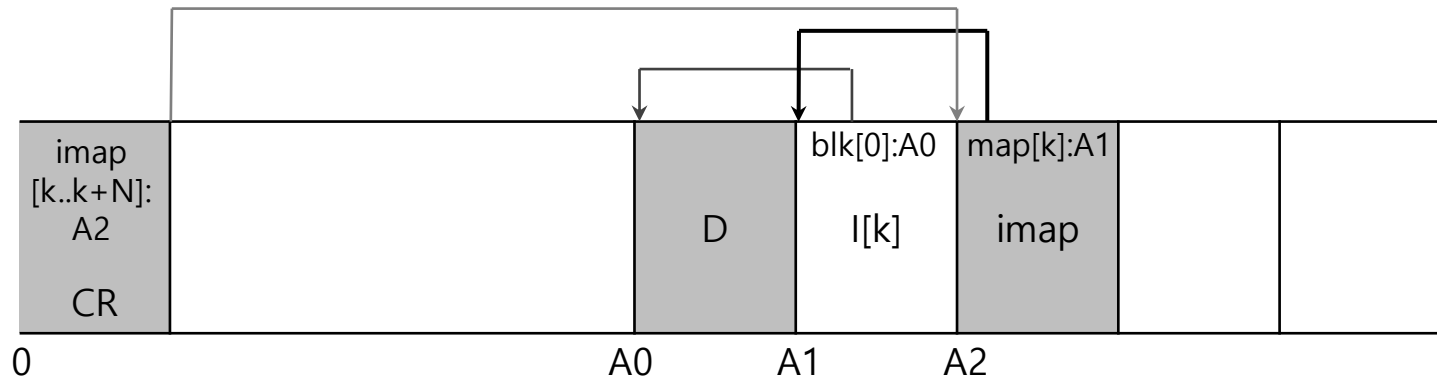
Finding Inode

- ▣ The position of the inodes keep changing.
- ▣ The Inode Map
 - ◆ A data structure that contains the location of the most recent inode for a given inode number.
 - ◆ Places the chunk of updated inode map next to the updated inode.
 - ◆ Where to find the inode map?



The Checkpoint Region

- How to find the inode map spread across the disk?
 - The LFS File system must have fixed location on disk to begin a file lookup.
- Checkpoint Region**
 - fixed location in the LFS partition.
 - Contain the pointers to the latest of the inode map.



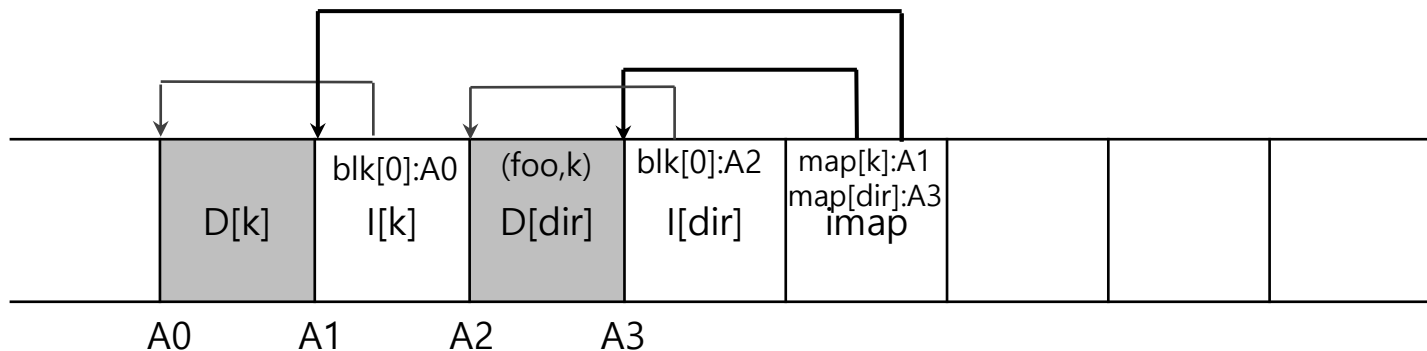
Reading a file from the disk

- ▣ Reading a file block
 - ◆ Read a checkpoint region
 - ◆ Read inode map
 - ◆ Read inode
 - ◆ Read data block
- ▣ What about sequential read?
 - ◆ It may become random read.

LFS is optimized for the write operation.

What About Directories?

- Directory: a set of <inode, filename>
- How does LFS store directory data?
- Creating a file: `foo`
 - ◆ Update the directory inode. (inode #: dir)
 - ◆ Update the directory entry. (`foo`, `k`)
 - ◆ Update inode for the created file. (inode #: `k`)
 - ◆ Update the data block for the created file.

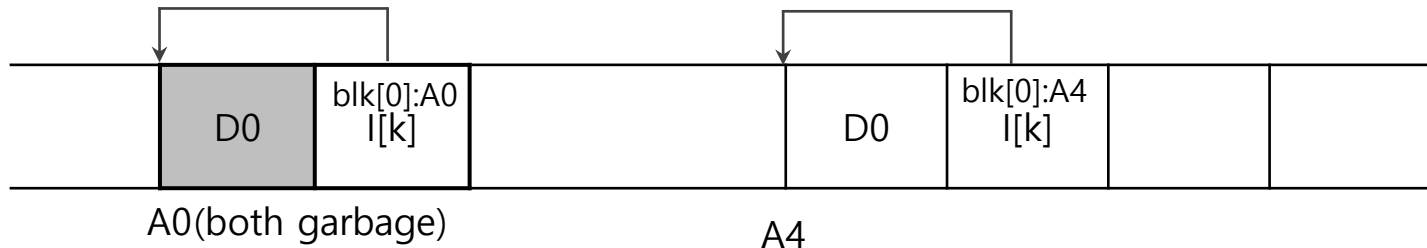


- Recursive update (cascade update issue): the location of the inode keeps changing. → the associated directory entry can be updated as well. → solved by inode map.

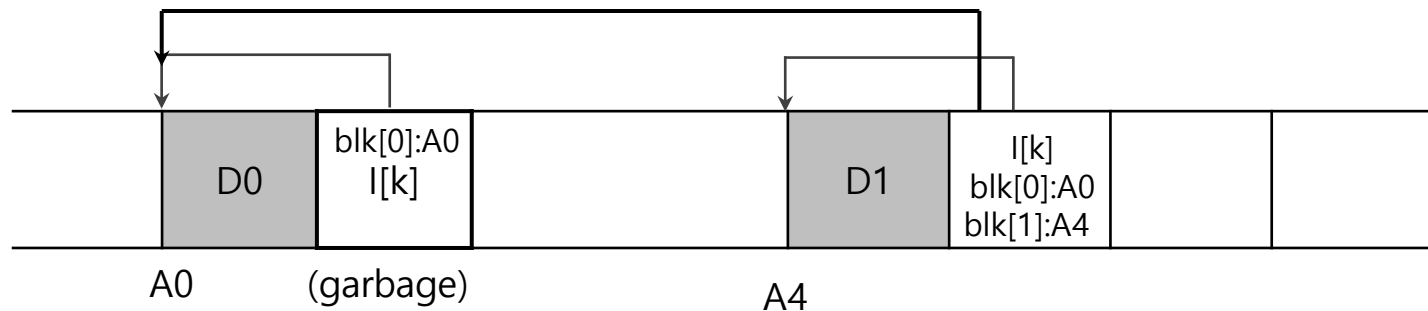
Garbage

- ❑ LFS keeps writing newer version of file.
- ❑ Garbage: LFS leaves the older versions of file structures all over the disk.
- ❑ An example of garbage

- ◆ Overwrite the data block:



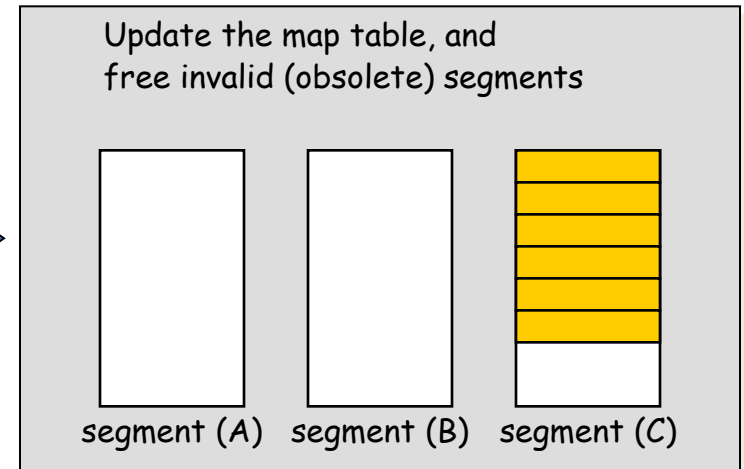
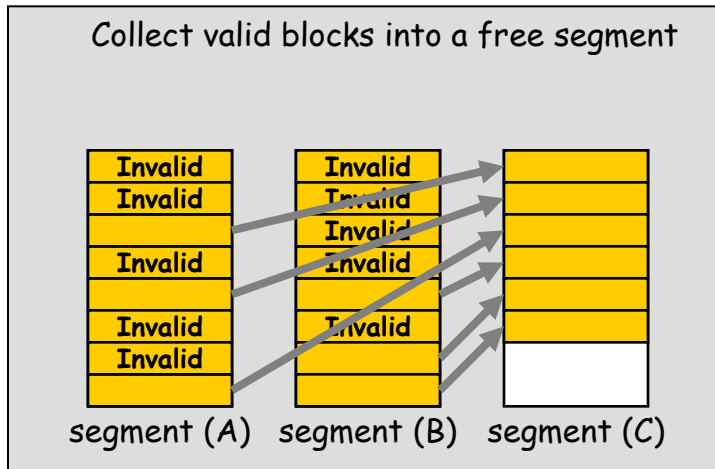
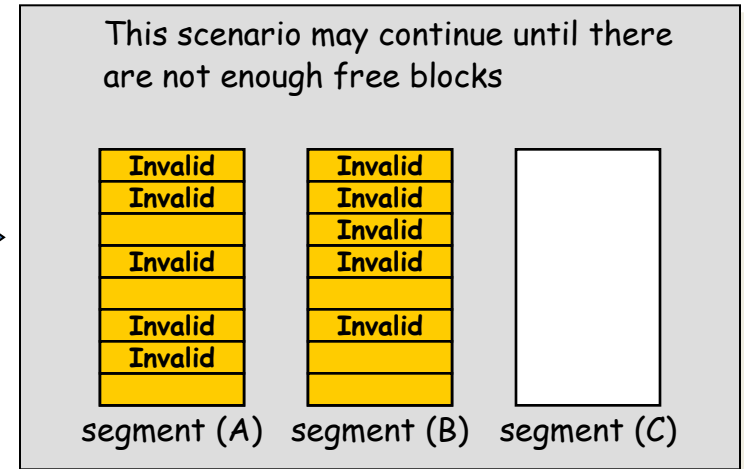
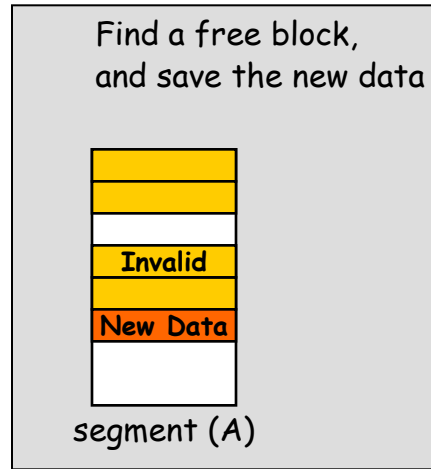
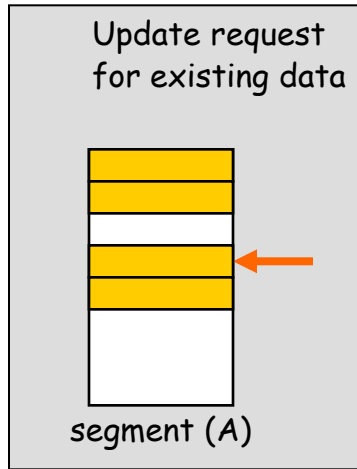
- ◆ Append a block to that original file k:



Garbage Collection (segment cleaning)

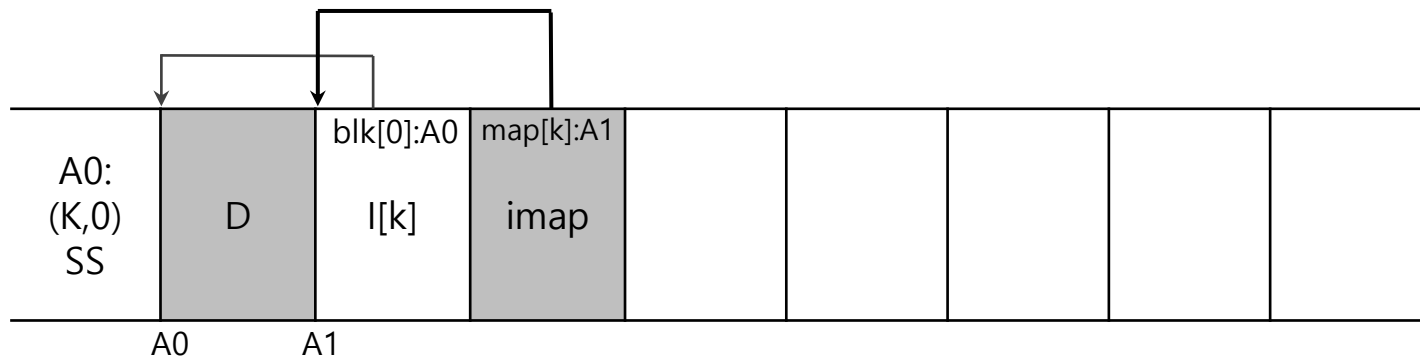
- ▣ What to do with the older versions of the block
 - ◆ Versioning filesystem: keep the old blocks and allow the users to restore to the older version of the filesystem status.
 - ◆ LFS: periodically clean the older versions of the file data, inodes and other structures.
- ▣ Unit of garbage collection: Segment
 - ◆ Reads a number of old segments, M segments.
 - ◆ Identify the valid blocks.
 - ◆ Write them to a number of new segments (in memory), N segments.
 - ◆ Write N segments to the disk.
 - ◆ Then, $N < M$.

Garbage collection



Segment Summary Block

- ▣ Store the inode and the file offset for each data block in it.
- ▣ In garbage collection, we need to identify the obsolete blocks.
- ▣ Compare the block address of file K offset 0 based upon the Segment Summary and based upon the in-memory imap. If they coincide, the block is alive. → expensive

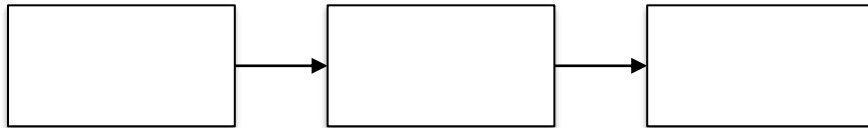


Issues in garbage collection

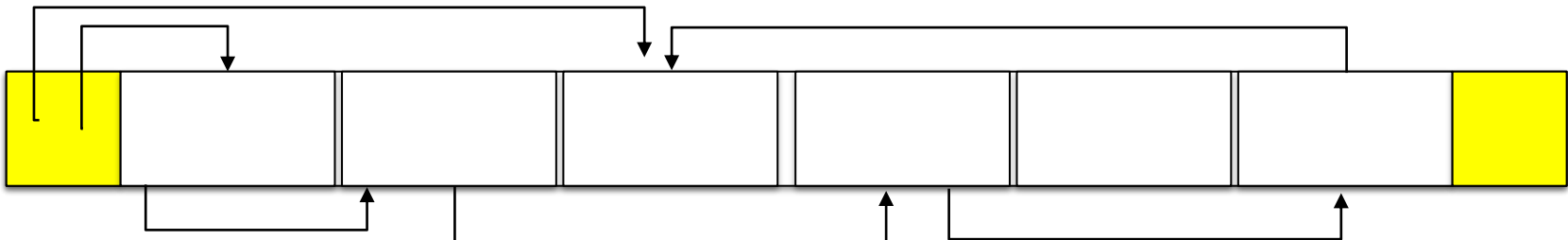
- ▣ When to clean
 - ◆ Periodically
 - ◆ When a system is idle
 - ◆ When the disk is full
- ▣ Which block to consolidate?
 - ◆ Hot segment: the blocks are updated periodically
 - ◆ Cold segment: the blocks are not updated.
 - ◆ Hot segment: clean later.
 - ◆ Cold segment: clean sooner.

Crash recovery

- What if the crash happens when the LFS is in the middle of writing the segment to the disk?
- LFS maintains a set of segments as a linked list in memory



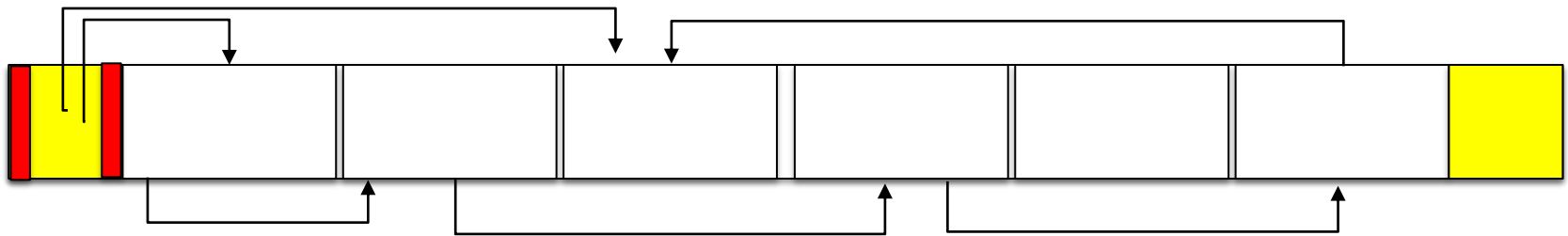
- LFS organizes the filesystem partition as a log (a linked list of the segments).
 - Two checkpoint regions: one at the beginning and the one at the end.



Crash recovery

Consistent Update on CR

- ◆ Write timestamp at the beginning of CR.
- ◆ Write CR body.
- ◆ Write time stamp at the end of the CR.
- ◆ When crash occurs, chooses the most recent CR with valid consistent time stamps.



Crash recovery

- ◆ Read the CR and rebuild imap.
- ◆ Perform roll-forward.
 - Start from the first segment in CR.
 - Scan the valid segment following the "next segment" pointer and update the imap.

Summary

- ▣ Introduce a new approach to updating the disk.
 - ◆ **Shadow paging** in database system, **Copy-on-Write** in file system.
- ▣ Gather all updates into an in-memory segment.
 - ◆ Write them out together sequentially.
- ▣ LFS-style is excellent for performance on many different devices.
 - ◆ Hard drives, parity-based RAIDs, even Flash-based SSDs.
- ▣ Some modern commercial filesystems adopt a similar copy-on-write approach even though it generates garbage.
 - ◆ NetApp's **WAFL**, Sun's **ZFS** and Linux **btrfs**
 - ◆ In particular, WAFL turns cleaning problem into a feature, by providing old version of the file system via **snapshot**.

KERNEL MEMORY ALLOCATORS, PART 1

CS124 – Operating Systems
Winter 2016-2017, Lecture 16

Kernel Memory Allocators

- Kernels often keep complex data structures in memory...
 - Process control blocks, file descriptors, filesystem inodes, ...
- Also, kernels need buffer space for device interactions...
- The kernel must include a dynamic memory allocator to service various allocation and deallocation requests
- Two kinds of memory allocation in the kernel:
 1. Add pages to the virtual address space of the kernel, or to the virtual address space of a specific user process
 2. Allocate a chunk of memory for use by the kernel (e.g. for a process control block, for use by a system call, etc.)
- Second kind is handled by the **kernel memory allocator**

Kernel Memory Allocators (2)

- Most parts of the kernel use the kernel allocator, so a primary requirement is that it must be fast
- Kernel memory allocator is called from interrupt context as well as process context
- Example: kernel allocator called from interrupt context, when the system has little available memory
 - System can page out some unnecessary pages to free up room...
 - ...but that would involve putting the requestor to sleep ☹
- Generally, kernel allocators have at least two modes:
 - Allocation request is in interrupt context – cannot sleep; allocator may fail to satisfy request
 - Kernels try to avoid failing in this situation by having extra space around
 - Allocation request is in process context – can sleep; allocator will eventually succeed (barring disk-full errors, etc.)

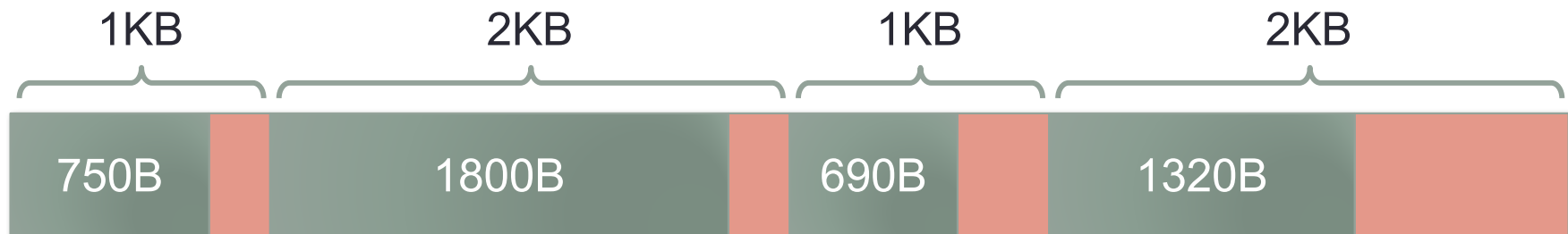
Memory Fragmentation

- Kernels need to use memory as efficiently as possible
 - Don't want the OS to impose a substantial resource overhead on the overall system
- Another primary concern is **memory fragmentation**
- Two kinds of fragmentation issues can occur:
- **External fragmentation** is when allocated blocks and free blocks become interspersed
- Example: allocate blocks of varying sizes
 - Then the two 1KB blocks are freed
 - 2KB is free, but not in a contiguous memory region
 - A 2KB allocation request will fail



Memory Fragmentation (2)

- **Internal fragmentation** is caused by constraints imposed by the allocation strategy itself
- Example: allocator only hands out memory in blocks that are multiples of 1KB in size
- Allocation requests:
 - 750 bytes (block size = 1024 bytes; waste = 274 bytes)
 - 1800 bytes (block size = 2048 bytes; waste = 248 bytes)
 - 690 bytes (block size = 1024 bytes; waste = 334 bytes)
 - 1320 bytes (block size = 2048 bytes; waste = 728 bytes)
- Total loss due to internal fragmentation: 1584 bytes



Kernel Allocator and Virtual Memory

- When the kernel needs to increase its available memory size, it receives memory in units of virtual memory pages
 - e.g. on IA32, virtual pages are 4KiB in size
- A common scenario:
 - Allocator tries to satisfy request from existing free space, but can't
 - Allocator requests one or more new pages from the virtual memory system
- Want to maximize the use of each virtual page to minimize paging, and also to minimize internal fragmentation
- Good kernel memory allocators are always tuned to work efficiently with the virtual memory system

Kernel Allocation Characteristics

- Kernel allocation patterns are generally well understood by kernel developers
 - Can design the allocator to satisfy requests efficiently
- Generally, allocation requests have a maximum size
 - e.g. for larger I/O buffers
 - Larger requests tend to be less common; smaller requests are extremely common
- Also common to have many requests for a particular size
 - e.g. for process control blocks, file descriptors, etc.
 - Leads to an **allocate-free-reallocate** usage pattern:
 - A specific kind of object is allocated, used, then released
 - Soon, the OS will need to reallocate that kind of object again
- Better kernel memory allocators are designed to maximize performance in the context of these patterns

Resource Map Allocator

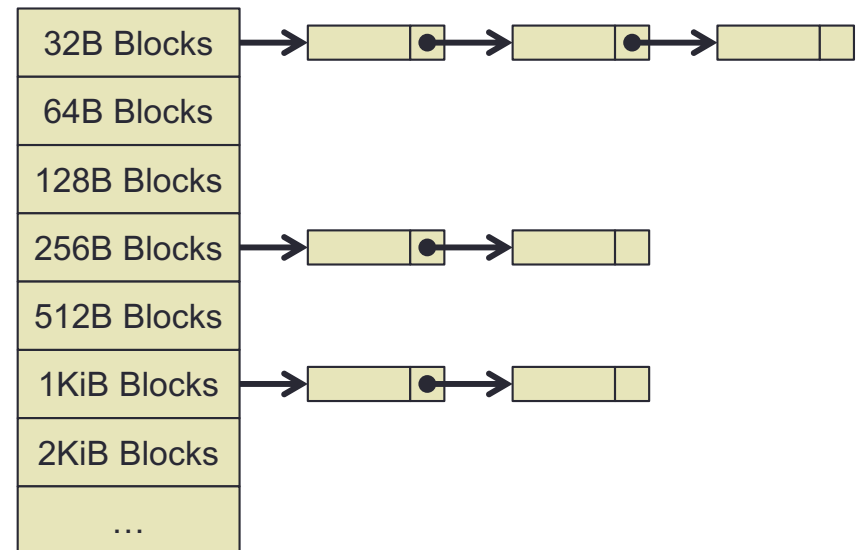
- Simplest kernel allocator is the **resource map allocator**
 - (Also called **sequential fits** or **explicit free list** allocator)
- Maintains a simple list of free regions in the memory pool
 - $\langle base, size \rangle$ pairs, where *base* is the starting address
 - (Allocated regions have a header to record their size)
- If list is maintained in order of increasing *base* address:
 - Easy to coalesce adjacent free regions
- If list is maintained in order of increasing *size* value:
 - Faster to find a region of the appropriate size for an allocation
- Benefits:
 - Very simple to implement
 - Almost no internal fragmentation at all
 - Can implement first-fit, best-fit, next-fit policies very easily

Resource Map Allocator (2)

- Drawbacks:
- Very prone to external fragmentation issues
 - External fragmentation becomes an issue as a program runs for a longer time...
 - The kernel runs as long as the system is in use
- Much slower than other kernel allocators
 - Even with optimizations, allocation/deallocation can be very slow
 - (e.g. if resource map is maintained in some particular order...)
- Resource map allocator doesn't take virtual memory system into account hardly at all
 - Some systems have used resource map allocator at the page level, not at the level of individual allocations

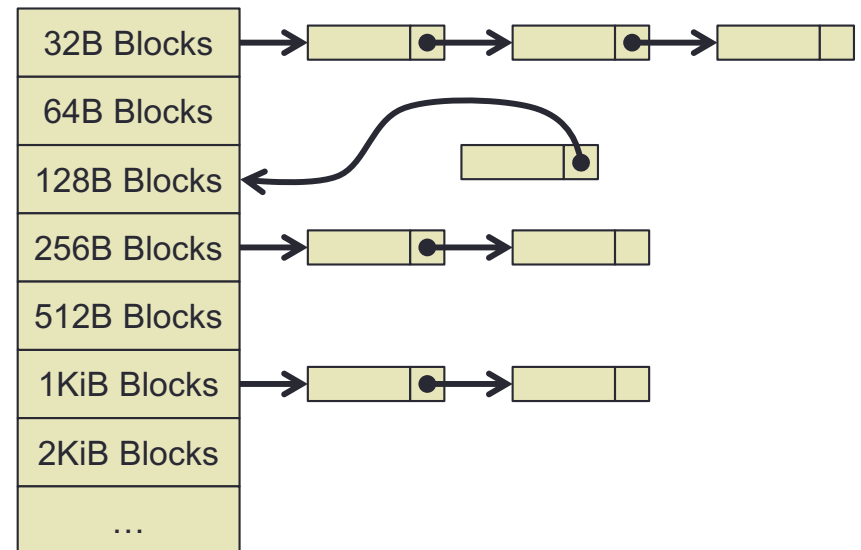
Power-of-Two Free List Allocators

- **Power-of-two free list allocators** round up allocation requests to the next power of two
 - Or, if a request spans multiple virtual pages, rounds up to a whole number of virtual pages
 - e.g. a request for 58 bytes is stored in a block of 64 bytes
- Instead of maintaining one free list, keep an array of free lists, each for a different size
 - Free blocks in each list hold a pointer to next block in the list
- Allocation is usually very fast:
 - Round up request size to nearest power of 2 (or whole # of pages)
 - Go to corresponding list and get the first block of that size



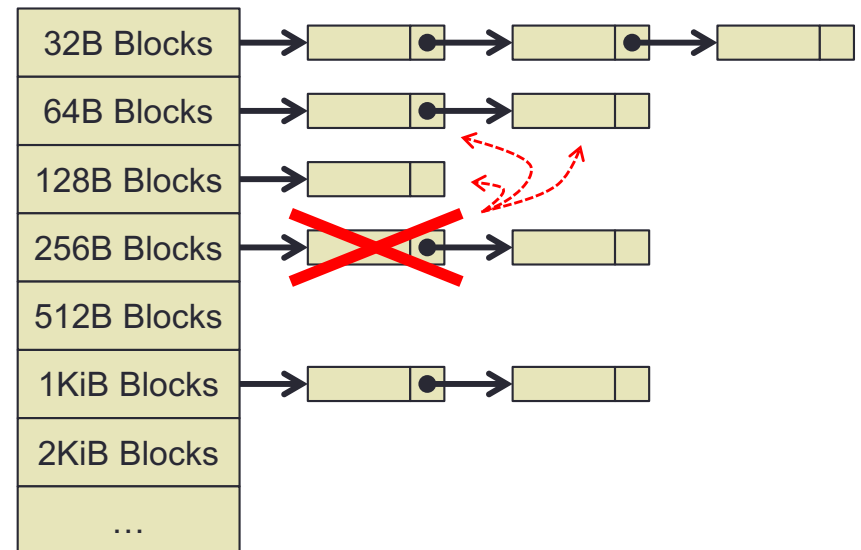
Power-of-Two Free List Allocators (2)

- Freeing is usually equally fast:
 - Find the free list that the block being deallocated should be stored into, and add it to that list
- Problem: how do we tell what list corresponds to the allocated block? (i.e. how do we know the block's size?)
- A simple solution:
 - One word of allocated blocks is used to store the free-list that the block corresponds to
- Problem: now blocks can only hold slightly less than their size
 - If kernel requests sizes that are powers of 2, 50% of the block's space will be lost!
 - (In practice, this happens often ☹️)



Power-of-Two Free List Allocators (3)

- Many options to handle the case when blocks of the required size aren't available
 - e.g. a request of 48 bytes, requiring a 64 byte block
- A very common solution: take a larger block, and split it repeatedly into smaller blocks
 - e.g. to get a 64B block, split a 256B block into a 128B block and two 64B blocks
- Problem:
 - Splitting large blocks makes it more difficult to coalesce smaller blocks into larger ones
 - Similarly, reclaiming entire pages becomes difficult; page may have blocks across multiple free lists

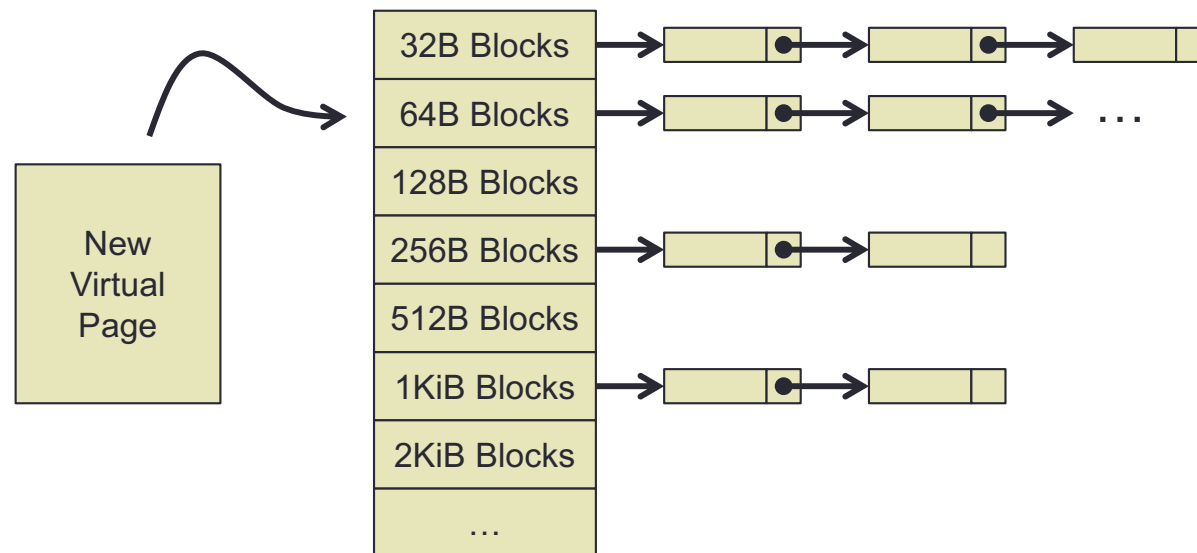


Power-of-Two Free List Allocators (4)

- Benefits:
 - Very fast allocation and deallocation
 - Very good at reusing previous allocations of a given size
 - Very resilient against external fragmentation – power-of-two blocks pack together into pages extremely well
- Drawbacks:
 - Suffers badly from internal fragmentation issues
 - Research results show an expected loss from internal fragmentation around 28% (e.g. Buddy Systems by Peterson and Norman, 1977)
 - Internal fragmentation is magnified by the fact that allocated blocks lose a word to remember their corresponding free-list
 - Coalescing and page reclamation tends to be very difficult without imposing additional time or space overhead
 - e.g. sort free-lists by address to facilitate coalescing, or use boundary tags, etc.

Power-of-Two Free List Allocators (5)

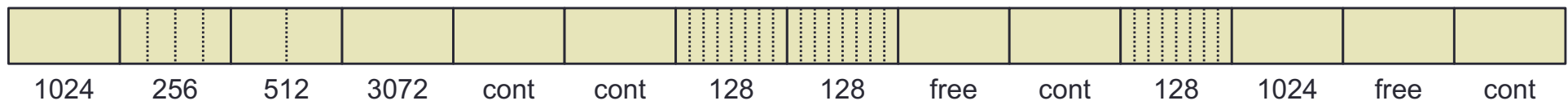
- Another option when more free blocks are needed:
 - Request a new virtual page, and split the entire page into blocks of the same size
 - Example: for 48-byte request, get a new 4KiB virtual page, split it into 64 64B blocks, and add all blocks to the 64B free-list



- If all blocks in a virtual page are the same size, why waste space recording what free-list each block corresponds to?
 - Can record this detail at the page level instead

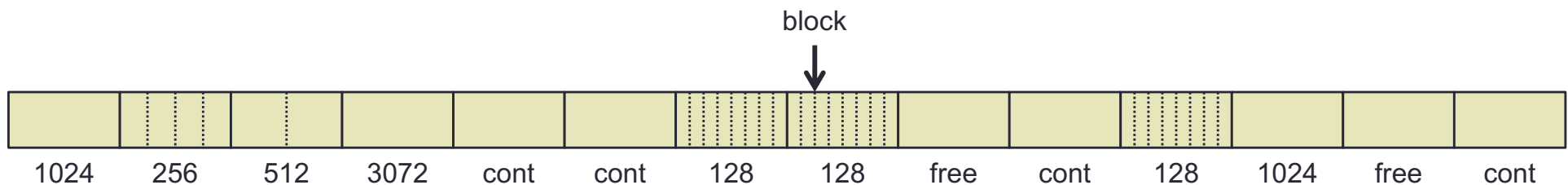
The McKusick-Karels Allocator

- The McKusick-Karels allocator is a variant of power-of-two allocator that maintains block sizes at the page level
 - Used in the 4.3BSD UNIX kernel
- Major benefit: allocated blocks no longer lose a word to the block size (or associated free-list pointer)
- Instead, the kernel allocator maintains an array of details for every page in the allocator's pool
 - 4.3BSD allocator calls this **kmemsizes**
 - Allocator also manages a free-list per block size, as before
- Array entries specify the allocation block size, or “free,” or a “continuation of the previous allocation”
- Example from McKusick/Karels paper (1KiB page size):



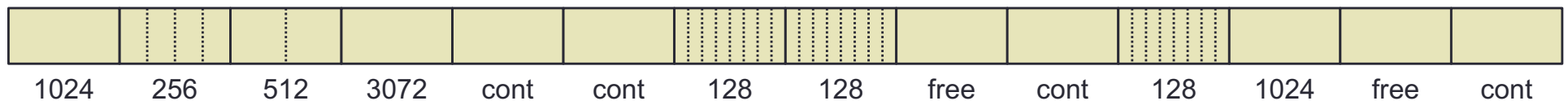
The McKusick-Karels Allocator (2)

- Easy to find a block's size from its address: just access array-entry corresponding to page number of block
 - $entry = address \text{ div } pagesize$
 - (since *pagesize* is a power of 2, can do this with a shift-right)
- Example: block address = 7424, page size = 1024
 - $7424 \text{ div } 1024 = 7$
 - `kmemsizes[7] = 128`, so the block size is 128 bytes
- Can store other details in page-level metadata, e.g. how many blocks in a page are allocated
 - Allows pages to be released very easily



The McKusick-Karels Allocator (3)

- The McKusick-Karels allocator is better than the simple power-of-two allocator, but still not great
- Internal fragmentation is reduced by factoring block-size details out of individual blocks, and into page-level details
 - But, still high due to the constraint of power-of-two block sizes



MODERN OPERATING SYSTEMS

Third Edition
ANDREW S. TANENBAUM

Chapter 8

Multiple Processor Systems

Multiple Processor Systems

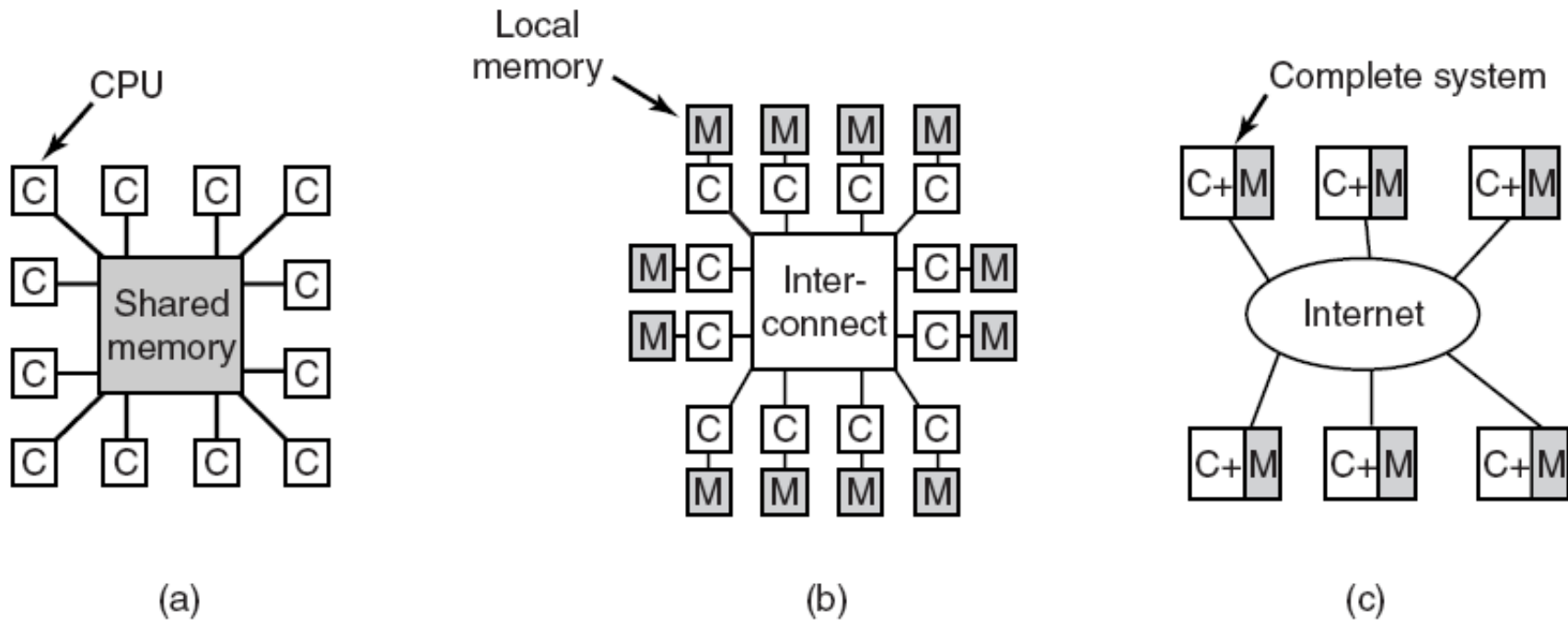


Figure 8-1. (a) A shared-memory multiprocessor. (b) A message-passing multicomputer. (c) A wide area distributed system.

UMA Multiprocessors with Bus-Based Architectures

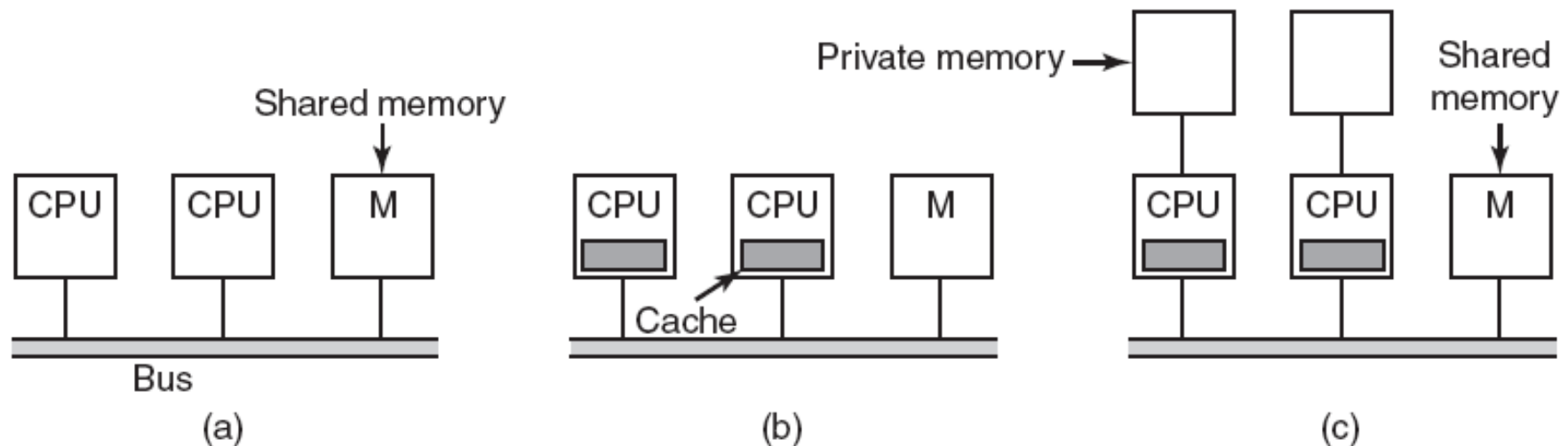


Figure 8-2. Three bus-based multiprocessors. (a) Without caching. (b) With caching. (c) With caching and private memories.

UMA Multiprocessors Using Crossbar Switches

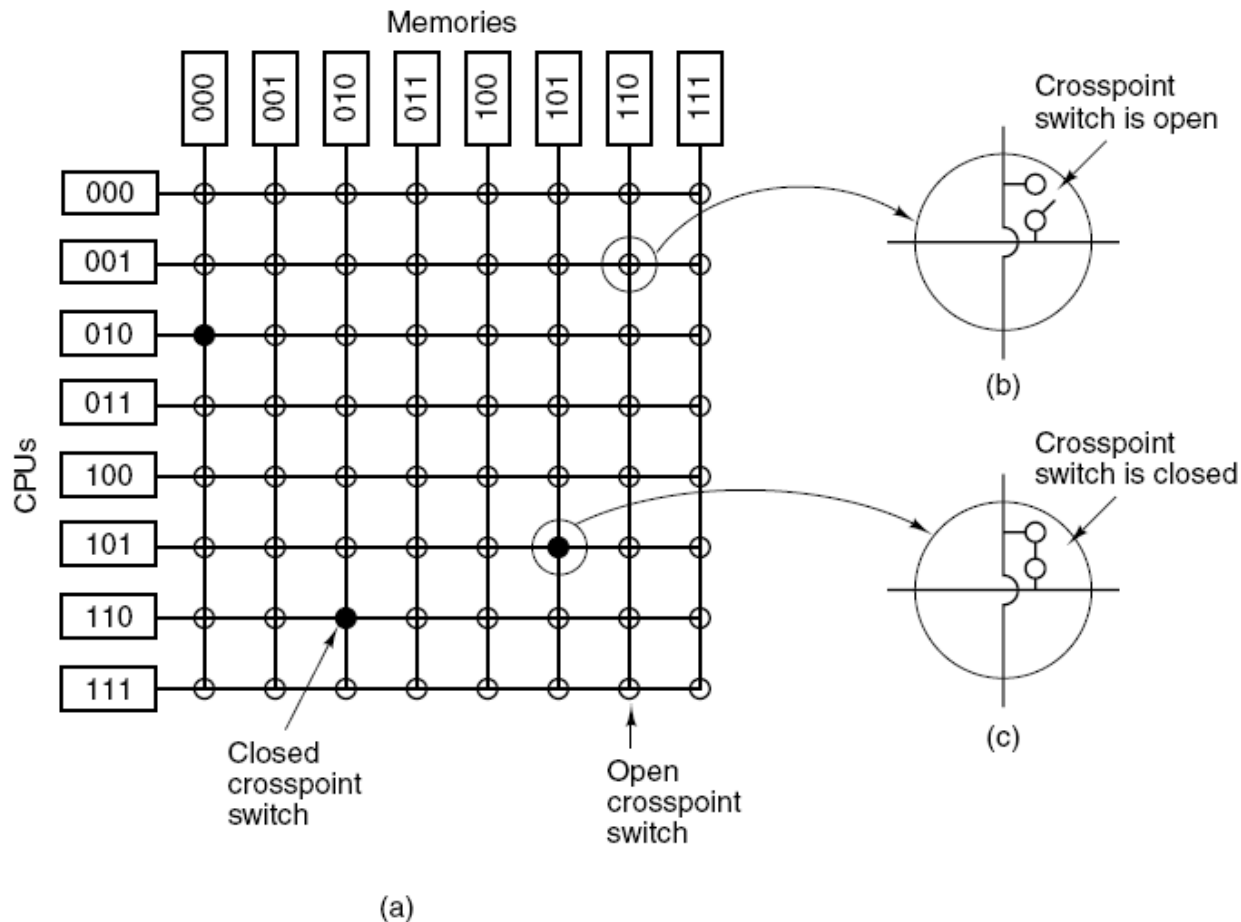
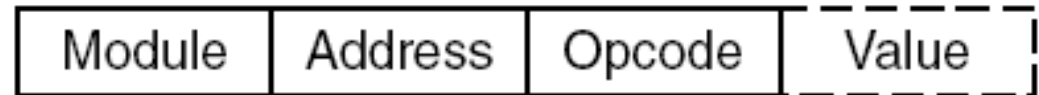


Figure 8-3. (a) An 8×8 crossbar switch. (b) An open crosspoint.
(c) A closed crosspoint.

UMA Multiprocessors Using Multistage Switching Networks (1)



(a)



(b)

Figure 8-4. (a) A 2×2 switch with two input lines, A and B, and two output lines, X and Y. (b) A message format.

UMA Multiprocessors Using Multistage Switching Networks (2)

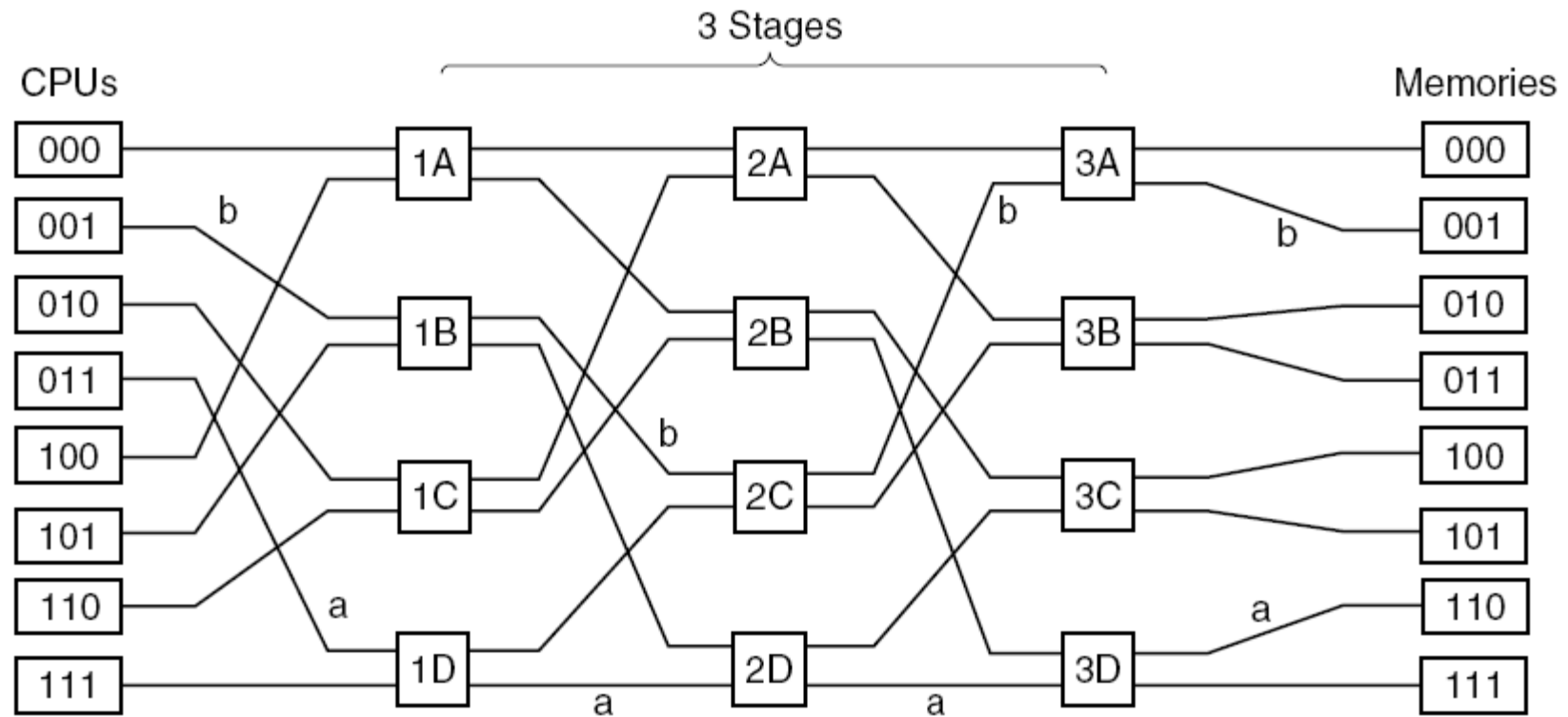


Figure 8-5. An omega switching network.

NUMA Multiprocessors (1)

Characteristics of NUMA machines:

1. There is a single address space visible to all CPUs.
2. Access to remote memory is via LOAD and STORE instructions.
3. Access to remote memory is slower than access to local memory.

NUMA Multiprocessors (2)

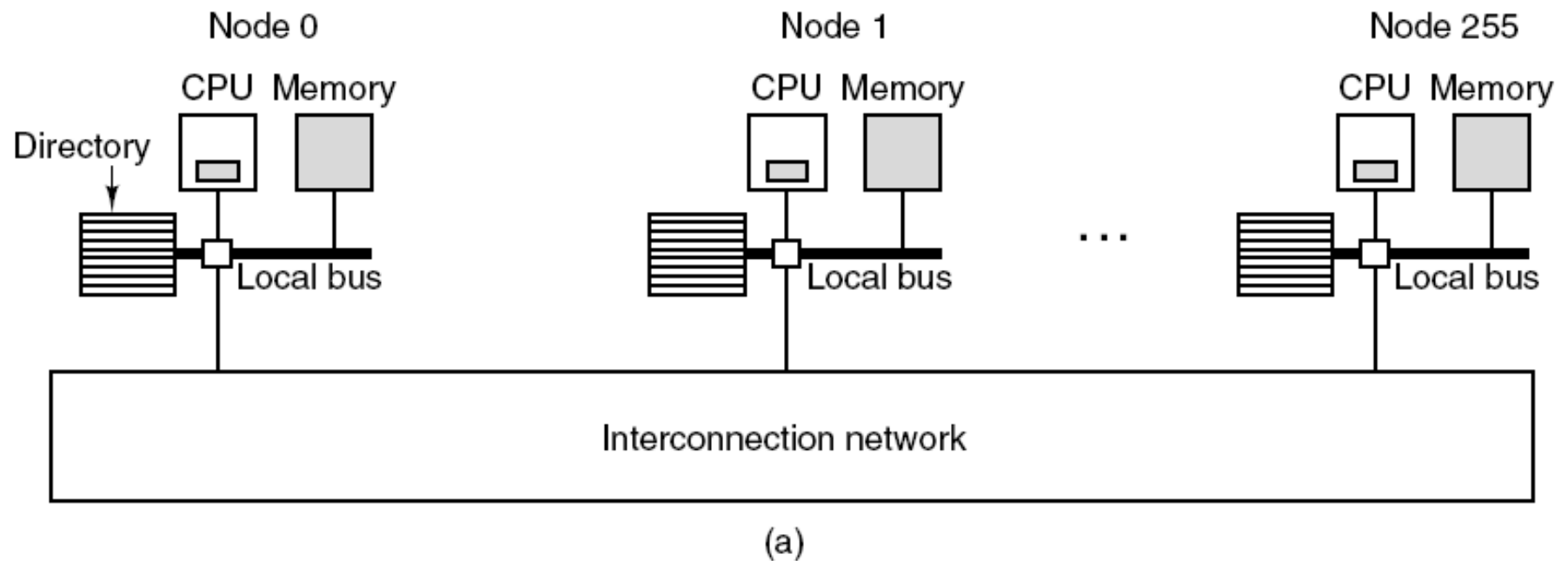
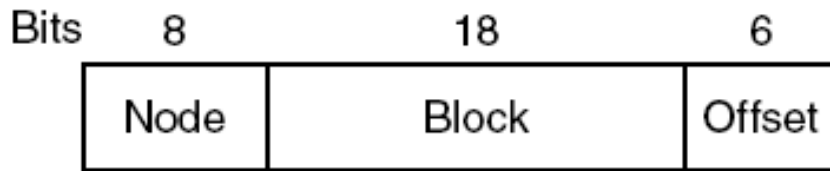
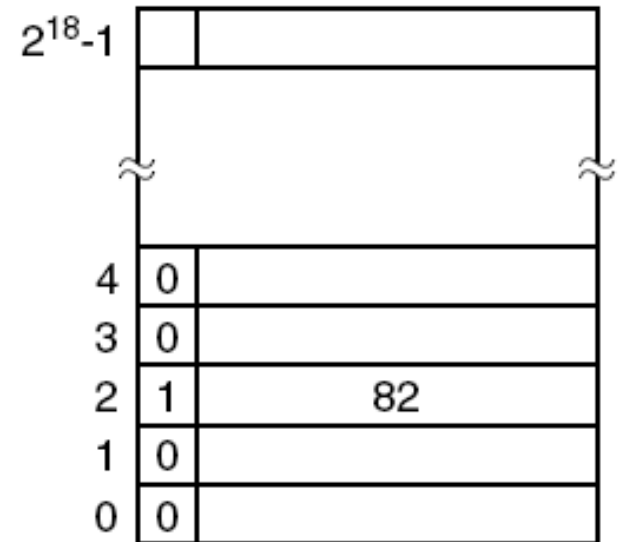


Figure 8-6. (a) A 256-node directory-based multiprocessor.

NUMA Multiprocessors (3)



(b)



(c)

Figure 8-6. (b) Division of a 32-bit memory address into fields.
(c) The directory at node 36.

Each CPU Has Its Own Operating System

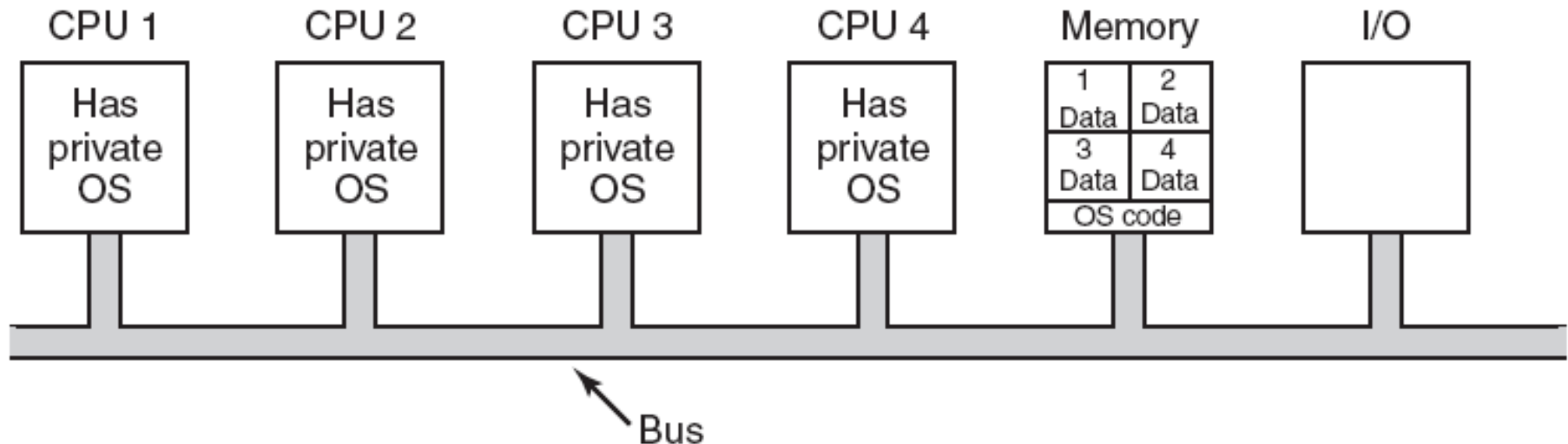


Figure 8-7. Partitioning multiprocessor memory among four CPUs, but sharing a single copy of the operating system code. The boxes marked Data are the operating system's private data for each CPU.

Master-Slave Multiprocessors

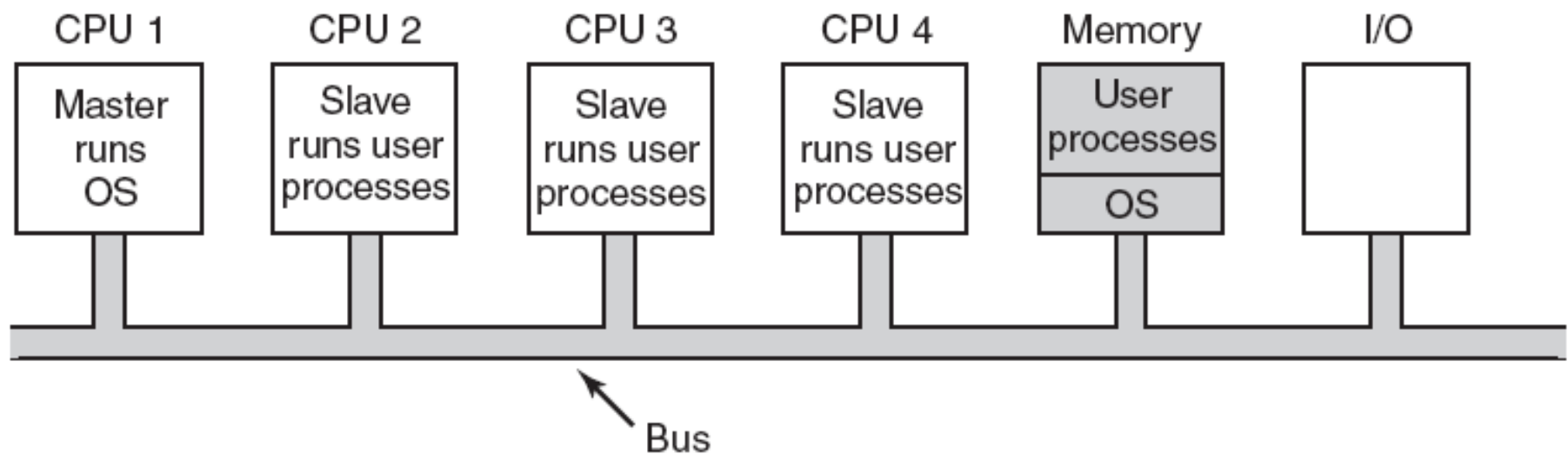


Figure 8-8. A master-slave multiprocessor model.

Symmetric Multiprocessors

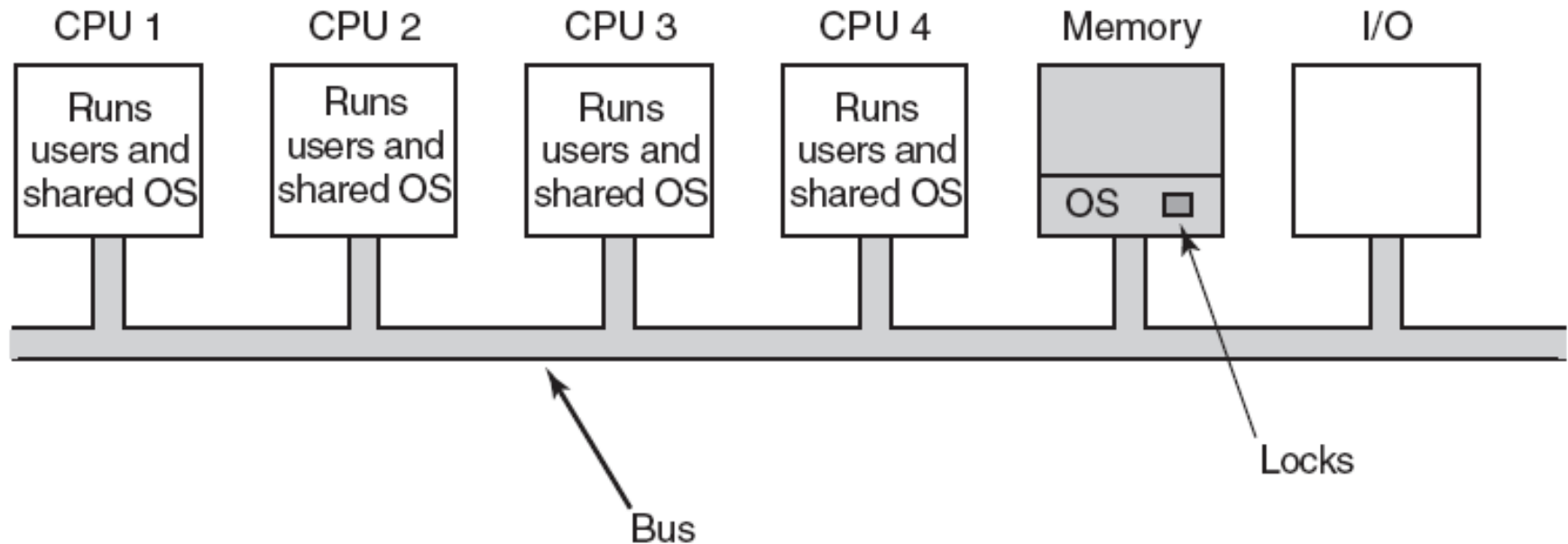


Figure 8-9. The SMP multiprocessor model.

Multiprocessor Synchronization (1)

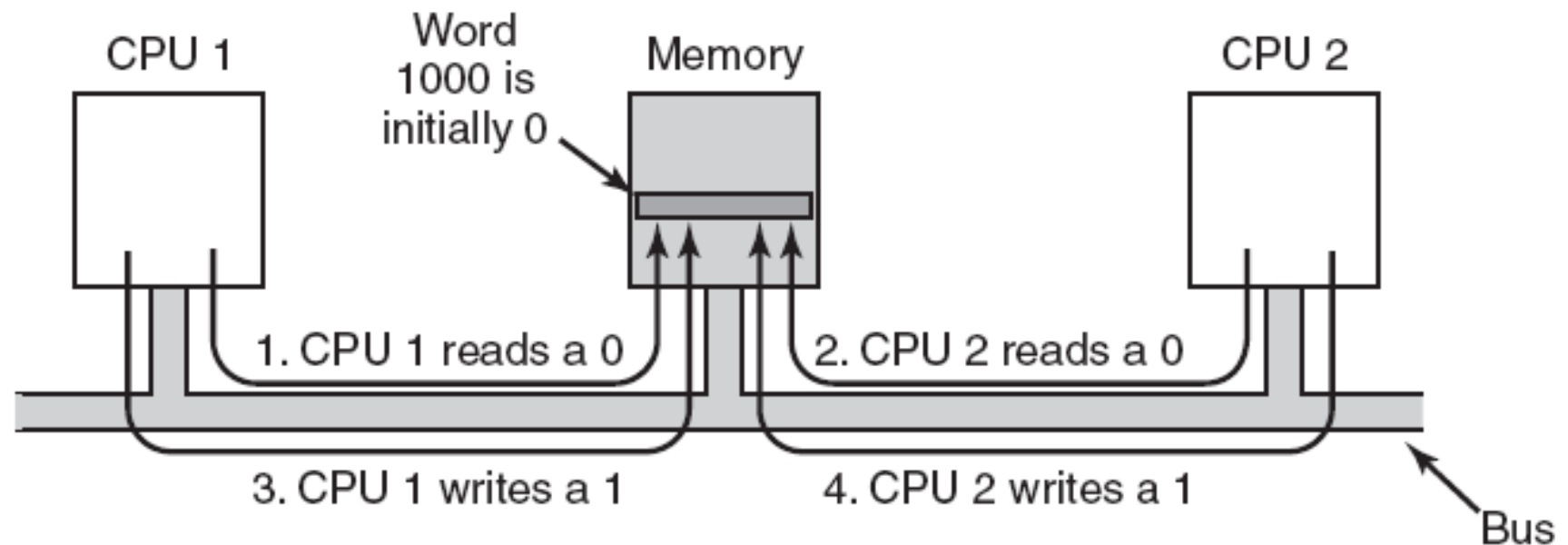


Figure 8-10. The TSL instruction can fail if the bus cannot be locked. These four steps show a sequence of events where the failure is demonstrated.

Multiprocessor Synchronization (2)

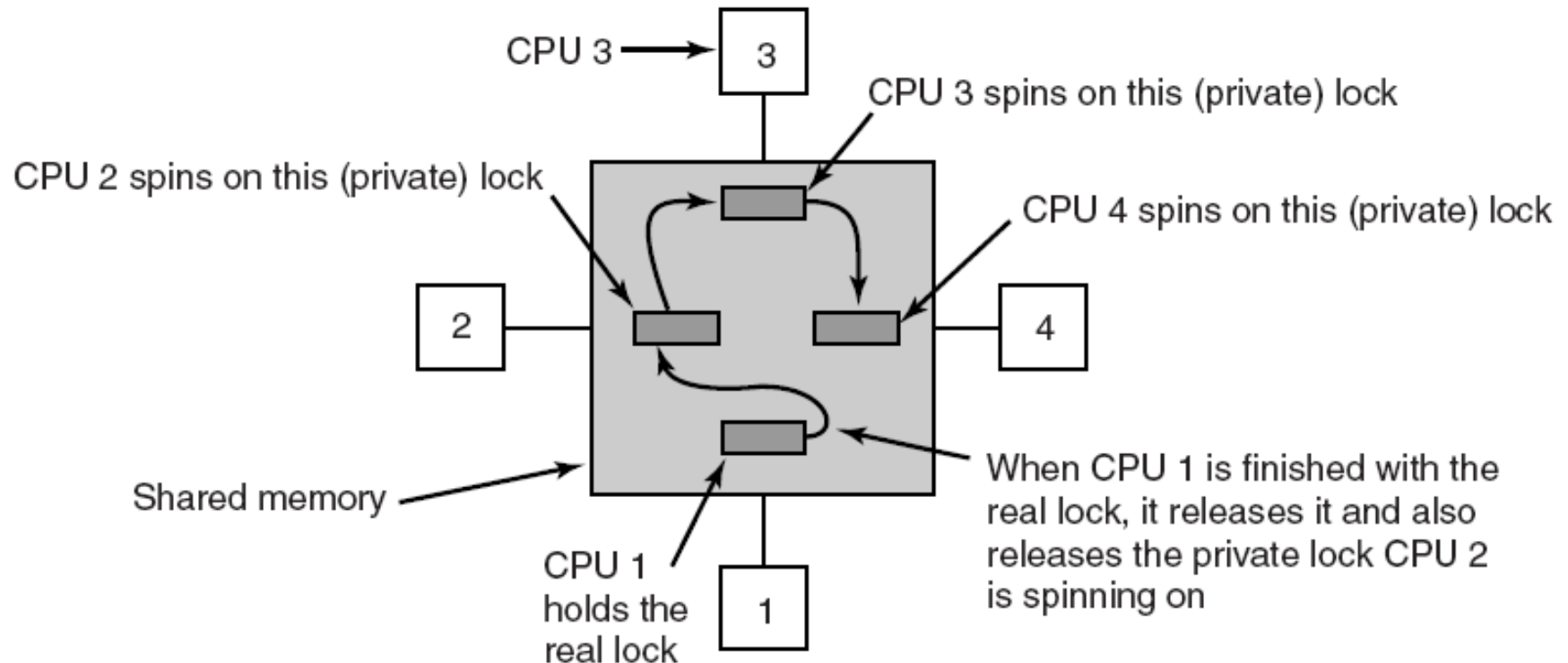


Figure 8-11. Use of multiple locks to avoid cache thrashing.

MODERN OPERATING SYSTEMS

Third Edition
ANDREW S. TANENBAUM

Chapter 10 Case Study 1: LINUX

History of UNIX and Linux

- UNICS
- PDP-11 UNIX
- Portable UNIX
- Berkeley UNIX
- Standard UNIX
- MINIX
- Linux

UNIX/Linux Goals

- Designed by programmers, for programmers
- Designed to be:
 - Simple
 - Elegant
 - Consistent
 - Powerful
 - Flexible

Interfaces to Linux

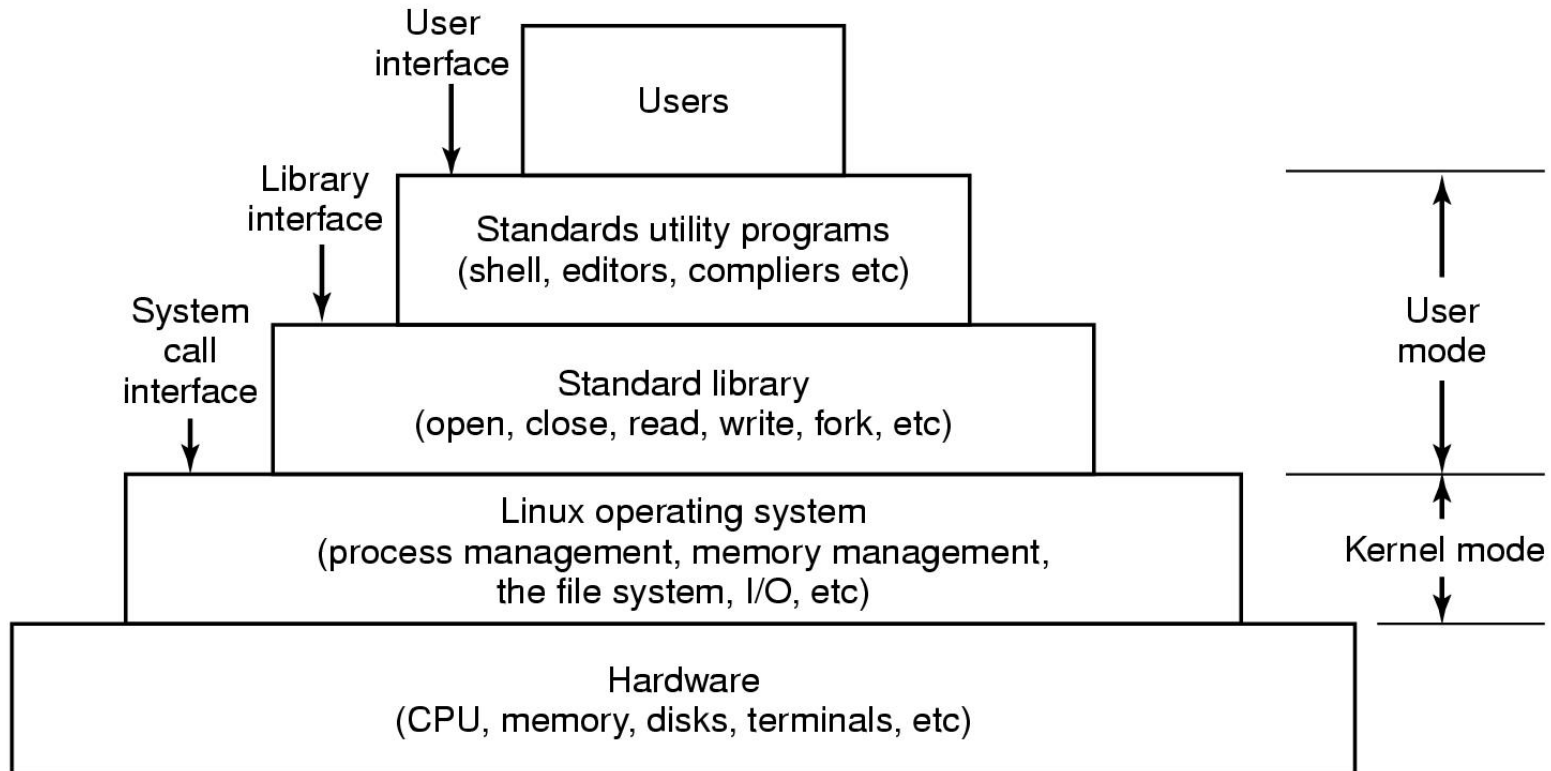


Figure 10-1. The layers in a Linux system.

Linux Utility Programs (1)

Categories of utility programs:

- File and directory manipulation commands.
- Filters.
- Program development tools, such as editors and compilers.
- Text processing.
- System administration.
- Miscellaneous.

Linux Utility Programs (2)

Program	Typical use
cat	Concatenate multiple files to standard output
chmod	Change file protection mode
cp	Copy one or more files
cut	Cut columns of text from a file
grep	Search a file for some pattern
head	Extract the first lines of a file
ls	List directory
make	Compile files to build a binary
mkdir	Make a directory
od	Octal dump a file
paste	Paste columns of text into a file
pr	Format a file for printing
ps	List running processes
rm	Remove one or more files
rmdir	Remove a directory
sort	Sort a file of lines alphabetically
tail	Extract the last lines of a file
tr	Translate between character sets

Figure 10-2. A few of the common Linux utility programs required by POSIX.

Kernel Structure

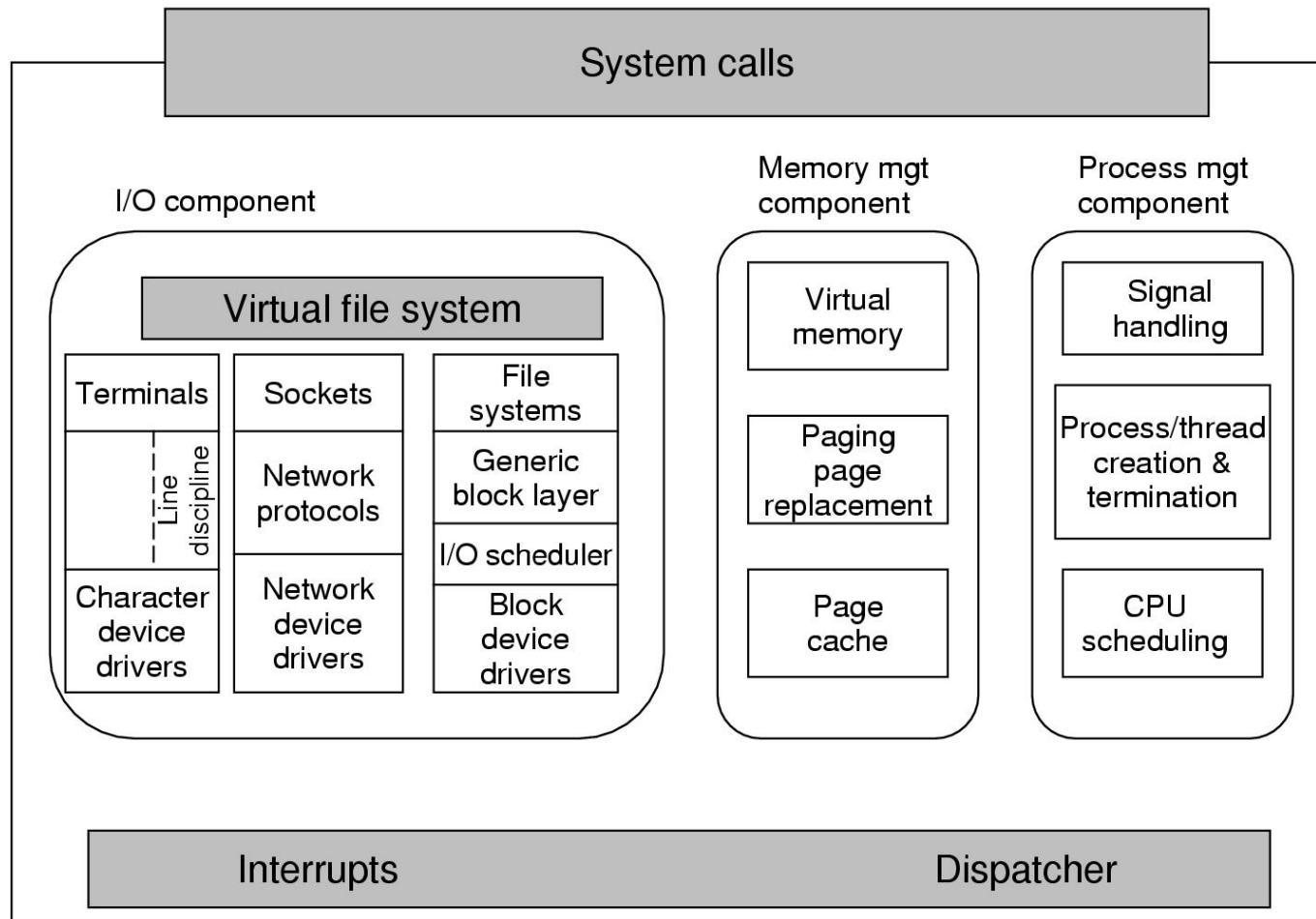


Figure 10-3. Structure of the Linux kernel

Processes in Linux

```
pid = fork( );           /* if the fork succeeds, pid > 0 in the parent */
if (pid < 0) {           /* fork failed (e.g., memory or some table is full) */
    handle_error( );
} else if (pid > 0) {
    /* parent code goes here. */
} else {
    /* child code goes here. */
}
```

Figure 10-4. Process creation in Linux.

Communications Overview

Raw messages: UDP

Reliable messages: TCP

Remote procedure call: RPC

RPC

Remote **P**rocedure **C**all

What could be easier than calling a function?

Strategy: create wrappers so calling a function on another machine feels just like calling a local function

Very common abstraction

RPC

Machine

Machine

^A

```
int main(...) {  
    int x =  
    foo("hello");  
}  
  
int foo(char *msg) {  
    send msg to B  
    recv msg from B  
}
```

^B

```
int foo(char *msg) {  
    ...  
}  
  
void foo_listener() {  
    while(1) {  
        recv, call  
        foo  
    }  
}
```

What it feels like for
programmer

RPC

Machine

Machine

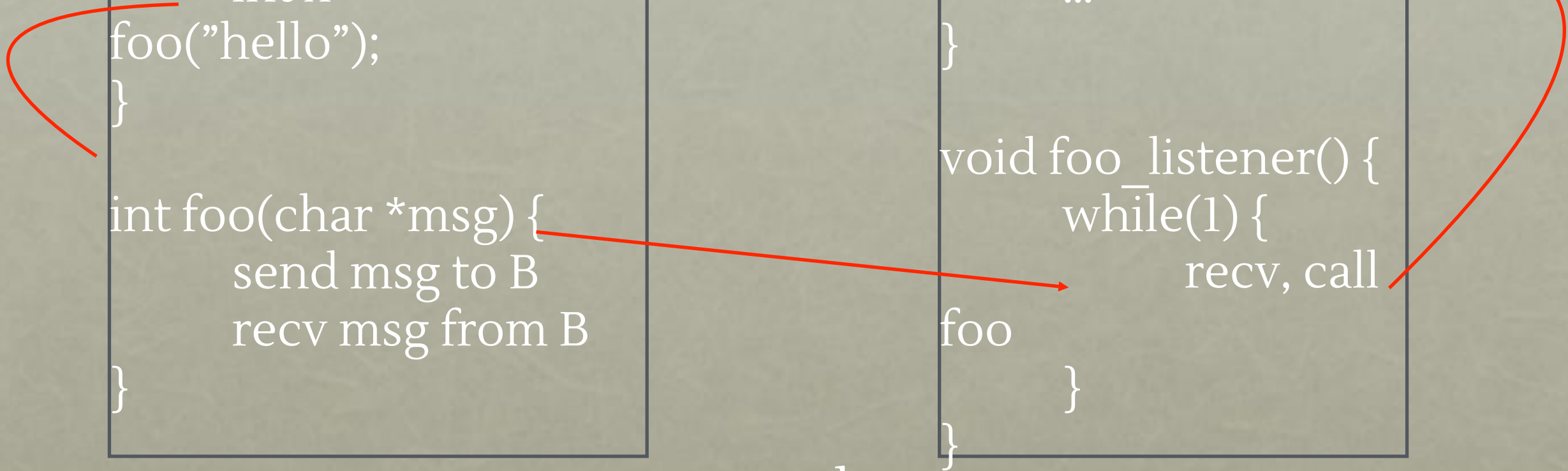
A

```
int main(...) {  
    int x =  
    foo("hello");  
}  
  
int foo(char *msg) {  
    send msg to B  
    recv msg from B  
}
```

B

```
int foo(char *msg) {  
    ...  
}  
  
void foo_listener() {  
    while(1) {  
        recv, call  
        foo  
    }  
}
```

Actual
calls



RPC

Machine

Machine

^A
`int main(...) {
 int x =
 foo("hello");
}`

^B
`int foo(char *msg) {
 ...
}`

client
wrapper

```
int foo(char *msg) {  
    send msg to B  
    recv msg from B  
}
```

server
wrapper

```
void foo_listener() {  
    while(1) {  
        recv, call  
        foo  
    }  
}
```

Wrapper
s

RPC Tools

RPC packages help with two components

(1) Runtime library

- Thread pool
- Socket listeners call functions on server

(2) Stub generation

- Create wrappers automatically
- Many tools available (rpcgen, thrift, protobufs)

Wrapper Generation

Wrappers must do conversions:

- client arguments to message
- message to server arguments
- convert server return value to message
- convert message to client return value

Need uniform endianness (wrappers do this)

Conversion is called marshaling/unmarshaling, or serializing/deserializing

Wrapper Generation: Pointers

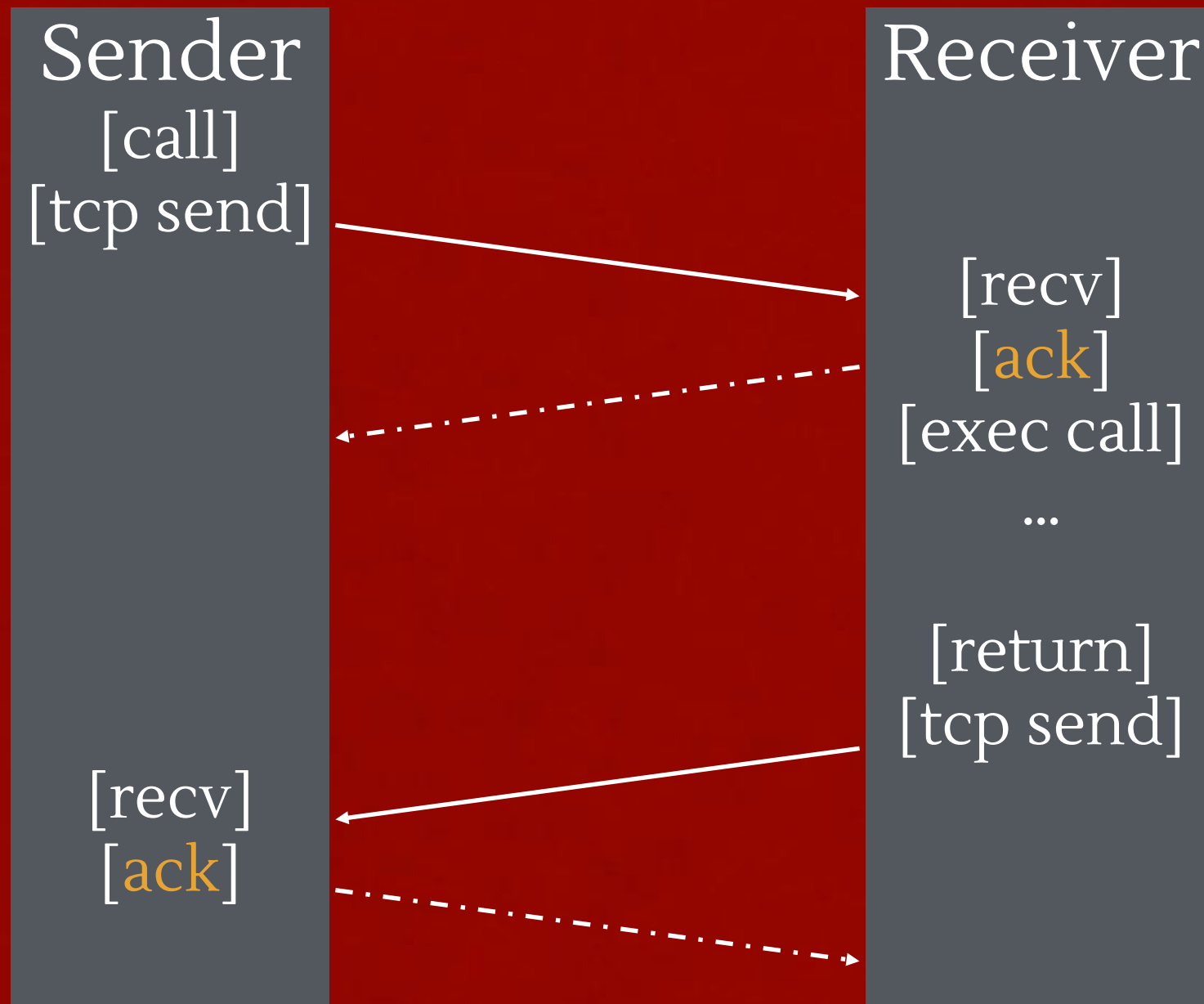
Why are pointers problematic?

Address passed from client not valid on server

Solutions?

- smart RPC package: follow pointers and copy data

RPC over TCP?



Why
wasteful?

RPC over UDP

Strategy: use function return as implicit ACK

Piggybacking technique

What if function takes a long time

- then send a separate ACK

